

Complete AI Agents Cheat Sheet

A comprehensive guide to building production-ready AI agents with LangChain, LangGraph, LangSmith, and related tools

Table of Contents

- [1. Core Concepts](#)
- [2. LangChain Fundamentals](#)
- [3. LangGraph - Advanced Agent Workflows](#)
- [4. LangSmith - Monitoring & Debugging](#)
- [5. Agent Architectures](#)
- [6. Memory Systems](#)
- [7. Tools & Function Calling](#)
- [8. Retrieval Systems \(RAG\)](#)
- [9. Evaluation & Testing](#)
- [10. Production Deployment](#)
- [11. Best Practices](#)

Core Concepts

What is an AI Agent?

An AI agent is an autonomous system that:

- Perceives** its environment through inputs
- Reasons** about what actions to take
- Acts** using tools and APIs
- Learns** from feedback and experience

Agent Types

Type	Description	Use Case
ReAct	Reasoning + Acting in iterative loops	General problem-solving
Plan-and-Execute	Plans first, then executes steps	Complex multi-step tasks
Reflexion	Self-reflects and improves	Learning from failures

Type	Description	Use Case
Tool-using	Calls external APIs/functions	Data retrieval, calculations
Conversational	Maintains context in dialogue	Chatbots, assistants
Multi-agent	Multiple agents collaborate	Complex workflows

LangChain Fundamentals

Installation

```
bash

# Core package
pip install langchain

# Community integrations
pip install langchain-community

# Specific integrations
pip install langchain-openai
pip install langchain-anthropic
pip install langchain-google-genai
pip install langchain-cohere

# Vector stores
pip install langchain-chroma
pip install langchain-pinecone
pip install langchain-weaviate
```

Core Components

1. Language Models

```
python
```

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from langchain_google_genai import ChatGoogleGenerativeAI

# Initialize LLM
llm = ChatOpenAI(
    model="gpt-4",
    temperature=0.7,
    max_tokens=1000,
    timeout=30,
    max_retries=2
)

# Streaming
for chunk in llm.stream("Tell me a joke"):
    print(chunk.content, end="", flush=True)

# Async
response = await llm.ainvoke("What is AI?")
```

2. Prompts & Prompt Templates

```
python
```

```

from langchain.prompts import (
    PromptTemplate,
    ChatPromptTemplate,
    MessagesPlaceholder,
    FewShotPromptTemplate
)

# Simple template
prompt = PromptTemplate(
    input_variables=["topic"],
    template="Write a poem about {topic}"
)

# Chat template
chat_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful AI assistant."),
    ("human", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad")
])

# Few-shot template
examples = [
    {"input": "happy", "output": "joyful"},
    {"input": "sad", "output": "melancholic"}
]

few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=PromptTemplate(
        input_variables=["input", "output"],
        template="Input: {input}\nOutput: {output}"
    ),
    prefix="Translate emotions to synonyms:",
    suffix="Input: {input}\nOutput:",
    input_variables=["input"]
)

```

3. Chains (LCEL - LangChain Expression Language)

python


```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
```

```
# Basic chain
```

```
chain = prompt | llm | StrOutputParser()
result = chain.invoke({"topic": "AI"})
```

```
# Chain with multiple steps
```

```
chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
```

```
# Parallel chains
```

```
from langchain_core.runnables import RunnableParallel
```

```
chain = RunnableParallel(
    summary=summary_chain,
    keywords=keyword_chain
)
```

```
# Conditional routing
```

```
from langchain_core.runnables import RunnableBranch
```

```
branch = RunnableBranch(
    (lambda x: "code" in x["input"], code_chain),
    (lambda x: "math" in x["input"], math_chain),
    general_chain # default
)
```

4. Output Parsers

```
python
```

```
from langchain.output_parsers import (
    PydanticOutputParser,
    StructuredOutputParser,
    CommaSeparatedListOutputParser
)
from pydantic import BaseModel, Field

# Pydantic parser
class Person(BaseModel):
    name: str = Field(description="Person's name")
    age: int = Field(description="Person's age")

parser = PydanticOutputParser(pydantic_object=Person)

prompt = PromptTemplate(
    template="Extract person info.\n{format_instructions}\n{query}",
    input_variables=["query"],
    partial_variables={"format_instructions": parser.get_format_instructions()}
)

# Structured parser
response_schemas = [
    ResponseSchema(name="answer", description="answer to question"),
    ResponseSchema(name="confidence", description="confidence score 0-1")
]
parser = StructuredOutputParser.from_response_schemas(response_schemas)
```

LangGraph

Installation

```
bash

pip install langgraph
pip install langgraph-checkpoint-sqlite # For persistence
```

Core Concepts

LangGraph enables building **stateful, cyclic workflows** with:

- **Nodes:** Functions that process state
- **Edges:** Connections between nodes
- **State:** Shared data structure
- **Conditional edges:** Dynamic routing

- **Persistence:** Save/resume workflows

Basic Graph Structure

python

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, Annotated
import operator

# Define state
class AgentState(TypedDict):
    messages: Annotated[list, operator.add]
    next_action: str
    iterations: int

# Create graph
workflow = StateGraph(AgentState)

# Add nodes
def agent_node(state: AgentState):
    # Your logic here
    return {
        "messages": [AIMessage(content="Response")],
        "iterations": state["iterations"] + 1
    }

def tool_node(state: AgentState):
    # Execute tools
    return {"messages": [ToolMessage(content="Result")]}

workflow.add_node("agent", agent_node)
workflow.add_node("tools", tool_node)

# Add edges
workflow.set_entry_point("agent")

# Conditional edge
def should_continue(state: AgentState):
    if state["iterations"] >= 5:
        return "end"
    return "continue"

workflow.add_conditional_edges(
    "agent",
    should_continue,
    {
        "continue": "tools",
        "end": END
    }
)
```

```
workflow.add_edge("tools", "agent")
```

```
# Compile
```

```
app = workflow.compile()
```

Advanced Graph Patterns

1. ReAct Agent with LangGraph

```
python
```

```
from langgraph.prebuilt import create_react_agent
```

```
from langchain_core.tools import tool
```

```
@tool
```

```
def search(query: str) -> str:
```

```
    """Search the web for information."""
```

```
    return f'Results for: {query}'
```

```
@tool
```

```
def calculator(expression: str) -> str:
```

```
    """Evaluate mathematical expressions."""
```

```
    return str(eval(expression))
```

```
tools = [search, calculator]
```

```
agent = create_react_agent(llm, tools)
```

```
# Run agent
```

```
for chunk in agent.stream(
```

```
    {"messages": [("human", "What's 25 * 4?")]},
```

```
    stream_mode="values"
```

```
):
```

```
    chunk["messages"][-1].pretty_print()
```

2. Multi-Agent System

```
python
```

```

from langgraph.graph import StateGraph, END

class MultiAgentState(TypedDict):
    messages: Annotated[list, operator.add]
    next_agent: str
    final_answer: str

# Define specialized agents
def researcher(state):
    response = research_llm.invoke(state["messages"])
    return {
        "messages": [response],
        "next_agent": "writer"
    }

def writer(state):
    response = writer_llm.invoke(state["messages"])
    return {
        "messages": [response],
        "next_agent": "critic"
    }

def critic(state):
    response = critic_llm.invoke(state["messages"])
    if "approved" in response.content.lower():
        return {"final_answer": response.content, "next_agent": END}
    return {"messages": [response], "next_agent": "writer"}

# Build graph
graph = StateGraph(MultiAgentState)
graph.add_node("researcher", researcher)
graph.add_node("writer", writer)
graph.add_node("critic", critic)

graph.set_entry_point("researcher")
graph.add_edge("researcher", "writer")
graph.add_edge("writer", "critic")
graph.add_conditional_edges(
    "critic",
    lambda x: x["next_agent"],
    {"writer": "writer", END: END}
)

app = graph.compile()

```

3. Human-in-the-Loop

```
python

from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph

memory = MemorySaver()

def human_approval_node(state):
    # Pause for human input
    return state

workflow.add_node("human_approval", human_approval_node)
workflow.add_edge("agent", "human_approval")
workflow.add_edge("human_approval", "execute")

app = workflow.compile(
    checkpointer=memory,
    interrupt_before=["human_approval"]
)

# Run until interrupt
config = {"configurable": {"thread_id": "1"}}
for event in app.stream({"messages": [("human", "Task")]}), config):
    print(event)

# Resume after human input
app.update_state(
    config,
    {"approved": True},
    as_node="human_approval"
)

# Continue
for event in app.stream(None, config):
    print(event)
```

4. Persistence & Checkpointing

```
python
```

```

from langgraph.checkpoint.sqlite import SqliteSaver

# SQLite persistence
with SqliteSaver.from_conn_string("checkpoints.db") as checkpointer:
    app = workflow.compile(checkpointer=checkpointer)

    config = {"configurable": {"thread_id": "user-123"}}

    # Run and save state
    app.invoke({"messages": [("human", "Hello")]}, config)

    # Resume later
    app.invoke({"messages": [("human", "Continue")]}, config)

    # Get state history
    for state in app.get_state_history(config):
        print(state)

```

LangGraph Built-in Agents

```

python

from langgraph.prebuilt import create_react_agent, ToolNode

# Quick ReAct agent
agent = create_react_agent(llm, tools)

# Custom with ToolNode
workflow = StateGraph(AgentState)
workflow.add_node("agent", agent_node)
workflow.add_node("tools", ToolNode(tools))

```

LangSmith

Setup

```

bash

pip install langsmith

# Set environment variables
export LANGCHAIN_TRACING_V2=true
export LANGCHAIN_API_KEY=your_api_key
export LANGCHAIN_PROJECT=my-project

```


Tracing & Monitoring

```
python

from langsmith import Client
from langchain.callbacks import LangChainTracer

# Initialize client
client = Client()

# Automatic tracing (just run your chains)
chain = prompt | llm
result = chain.invoke({"input": "Hello"})

# Manual tracing
from langsmith import traceable

@traceable(run_type="chain", name="my_custom_chain")
def my_function(input_text):
    response = llm.invoke(input_text)
    return response

# Add metadata and tags
chain.invoke(
    {"input": "Hello"},
    config={
        "metadata": {"user_id": "123", "environment": "prod"},
        "tags": ["production", "customer-support"]
    }
)
```

Evaluation & Testing

```
python
```

```

from langsmith import evaluate

# Create dataset
examples = [
    ("What is AI?", "AI is artificial intelligence..."),
    ("Explain ML", "Machine learning is...")
]

dataset_name = "qa-dataset"
client.create_dataset(dataset_name, description="QA pairs")

for input_text, output_text in examples:
    client.create_example(
        inputs={"question": input_text},
        outputs={"answer": output_text},
        dataset_name=dataset_name
    )

# Evaluator function
def qa_evaluator(run, example):
    prediction = run.outputs["answer"]
    reference = example.outputs["answer"]

    # Custom evaluation logic
    score = similarity(prediction, reference)

    return {"score": score}

# Run evaluation
results = evaluate(
    lambda inputs: chain.invoke(inputs),
    data=dataset_name,
    evaluators=[qa_evaluator],
    experiment_prefix="qa-eval"
)

# Built-in evaluators
from langchain.evaluation import load_evaluator

criteria_evaluator = load_evaluator("criteria", criteria="conciseness")
string_distance = load_evaluator("string_distance")
embedding_distance = load_evaluator("embedding_distance")

```

Online Evaluation

python

```
from langsmith import evaluate_existing

# Evaluate production runs
evaluate_existing(
    project_name="production-runs",
    evaluators=[qa_evaluator],
    filters='eq(metadata.environment, "prod")'
)
```

Feedback Collection

```
python

# Submit feedback
client.create_feedback(
    run_id="run-uuid",
    key="user-rating",
    score=0.9,
    comment="Great response!"
)

# Query feedback
feedback = client.list_feedback(run_ids=["run-uuid"])
```

Datasets & Few-shot Examples

```
python

# Upload dataset from file
import json

with open("examples.json") as f:
    examples = json.load(f)

dataset = client.create_dataset("my-dataset")
for ex in examples:
    client.create_example(
        inputs=ex["input"],
        outputs=ex["output"],
        dataset_id=dataset.id
    )

# Use for few-shot learning
examples = client.list_examples(dataset_name="my-dataset")
```

Agent Architectures

1. ReAct (Reasoning + Acting)

```
python

from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub

# Get ReAct prompt
prompt = hub.pull("hwchase17/react")

agent = create_react_agent(llm, tools, prompt)

agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True,
    max_iterations=10,
    handle_parsing_errors=True
)

result = agent_executor.invoke({"input": "What's the weather in Paris?"})
```

2. Plan-and-Execute

```
python

from langchain.agents import create_plan_and_execute_agent

agent = create_plan_and_execute_agent(
    llm=llm,
    tools=tools,
    verbose=True
)

# Agent first creates a plan, then executes each step
result = agent.invoke({"input": "Research AI trends and write a report"})
```

3. Structured Chat Agent

```
python
```

```
from langchain.agents import create_structured_chat_agent

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant"),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad")
])

agent = create_structured_chat_agent(llm, tools, prompt)
```

4. OpenAI Functions Agent

```
python

from langchain.agents import create_openai_functions_agent

agent = create_openai_functions_agent(llm, tools, prompt)

agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    return_intermediate_steps=True
)
```

5. Custom Agent with LangGraph

```
python
```

```
from langgraph.graph import StateGraph, END
from langchain_core.messages import HumanMessage, AIMessage
```

```
class CustomAgentState(TypedDict):
    messages: Annotated[list, operator.add]
    plan: list[str]
    current_step: int
    completed: bool
```

```
def planner(state):
    """Create execution plan"""
    messages = state["messages"]
    response = planner_llm.invoke([
        SystemMessage(content="Create a step-by-step plan"),
        *messages
    ])
    plan = response.content.split("\n")
    return {"plan": plan, "current_step": 0}
```

```
def executor(state):
    """Execute current step"""
    step = state["plan"][state["current_step"]]
    response = executor_llm.invoke([
        SystemMessage(content=f"Execute: {step}"),
        *state["messages"]
    ])
    return {
        "messages": [AIMessage(content=response.content)],
        "current_step": state["current_step"] + 1
    }
```

```
def should_continue(state):
    if state["current_step"] >= len(state["plan"]):
        return "end"
    return "continue"
```

```
workflow = StateGraph(CustomAgentState)
workflow.add_node("planner", planner)
workflow.add_node("executor", executor)
workflow.set_entry_point("planner")
workflow.add_edge("planner", "executor")
workflow.add_conditional_edges(
    "executor",
    should_continue,
    {"continue": "executor", "end": END}
)
```

```
agent = workflow.compile()
```

Memory Systems

1. Conversation Buffer Memory

```
python

from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True
)

# Add to conversation
memory.save_context(
    {"input": "Hello"},
    {"output": "Hi! How can I help?"}
)

# Load memory
history = memory.load_memory_variables({})
```

2. Conversation Buffer Window Memory

```
python

from langchain.memory import ConversationBufferWindowMemory

# Keep only last k interactions
memory = ConversationBufferWindowMemory(
    k=5, # Last 5 exchanges
    return_messages=True
)
```

3. Conversation Summary Memory

```
python
```

```
from langchain.memory import ConversationSummaryMemory

memory = ConversationSummaryMemory(
    llm=llm,
    return_messages=True
)

# Automatically summarizes conversation
```

4. Conversation Summary Buffer Memory

```
python

from langchain.memory import ConversationSummaryBufferMemory

memory = ConversationSummaryBufferMemory(
    llm=llm,
    max_token_limit=1000, # Summarize when exceeds
    return_messages=True
)
```

5. Vector Store-Backed Memory

```
python

from langchain.memory import VectorStoreRetrieverMemory
from langchain_community.vectorstores import Chroma

vectorstore = Chroma(embedding_function=embeddings)

retriever = vectorstore.as_retriever(search_kwargs={"k": 5})

memory = VectorStoreRetrieverMemory(
    retriever=retriever,
    memory_key="history"
)
```

6. Entity Memory

```
python
```



```
from langchain.memory import ConversationEntityMemory

memory = ConversationEntityMemory(
    llm=llm,
    return_messages=True
)

# Tracks entities (people, places, things) mentioned
```

7. LangGraph Persistence (Recommended)

```
python

from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph

class ConversationState(TypedDict):
    messages: Annotated[list, operator.add]
    user_info: dict
    conversation_summary: str

memory = MemorySaver()

def conversation_node(state):
    # Access full history from state
    messages = state["messages"]
    response = llm.invoke(messages)
    return {"messages": [response]}

workflow = StateGraph(ConversationState)
workflow.add_node("chat", conversation_node)
workflow.set_entry_point("chat")
workflow.add_edge("chat", END)

app = workflow.compile(checkpointer=memory)

# Each thread maintains separate memory
config = {"configurable": {"thread_id": "user-123"}}
app.invoke({"messages": [HumanMessage(content="Hi")]}, config)
```

Tools & Function Calling

Creating Tools

1. @tool Decorator

```
python

from langchain_core.tools import tool

@tool
def search_api(query: str, max_results: int = 5) -> str:
    """Search the web for information.

    Args:
        query: The search query
        max_results: Maximum number of results to return
    """
    # Implementation
    return f"Results for {query}"

@tool
def calculator(expression: str) -> float:
    """Evaluate mathematical expressions.

    Args:
        expression: Math expression like "2 + 2" or "sqrt(16)"
    """
    return eval(expression)
```

2. StructuredTool

```
python
```

```

from langchain.tools import StructuredTool
from pydantic import BaseModel, Field

class SearchInput(BaseModel):
    query: str = Field(description="Search query")
    domain: str = Field(description="Domain to search in")

def search_function(query: str, domain: str) -> str:
    return f"Searching {domain} for {query}"

search_tool = StructuredTool.from_function(
    func=search_function,
    name="domain_search",
    description="Search within a specific domain",
    args_schema=SearchInput
)

```

3. BaseTool Class

```

python

from langchain.tools import BaseTool
from typing import Optional, Type

class CustomTool(BaseTool):
    name: str = "custom_calculator"
    description: str = "Performs custom calculations"

    def _run(self, expression: str) -> str:
        """Synchronous implementation"""
        result = eval(expression)
        return f"Result: {result}"

    async def _arun(self, expression: str) -> str:
        """Async implementation"""
        result = eval(expression)
        return f"Result: {result}"

```

Tool Collections

```

python

```

```
from langchain_community.tools import (
    WikipediaQueryRun,
    DuckDuckGoSearchRun,
    ArxivQueryRun,
    PubmedQueryRun,
    YouTubeSearchTool
)
from langchain_community.utilities import (
    WikipediaAPIWrapper,
    DuckDuckGoSearchAPIWrapper
)

# Wikipedia
wikipedia = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())

# Web Search
search = DuckDuckGoSearchRun(api_wrapper=DuckDuckGoSearchAPIWrapper())

# Scientific papers
arxiv = ArxivQueryRun()
pubmed = PubmedQueryRun()

# YouTube
youtube = YouTubeSearchTool()

tools = [wikipedia, search, arxiv, pubmed, youtube]
```

Toolkits

```
python
```

```
from langchain_community.agent_toolkits import (
    FileManagementToolkit,
    SQLDatabaseToolkit,
    create_python_agent
)
from langchain_community.utilities import SQLDatabase

# File management
file_toolkit = FileManagementToolkit(
    root_dir="./workspace",
    selected_tools=["read_file", "write_file", "list_directory"]
)

# SQL Database
db = SQLDatabase.from_uri("sqlite:///database.db")
sql_toolkit = SQLDatabaseToolkit(db=db, llm=llm)

# Python REPL
python_agent = create_python_agent(
    llm=llm,
    tool=PythonREPLTool(),
    verbose=True
)
```

Custom API Tools

```
python
```

```
import requests
```

```
@tool
```

```
def get_weather(city: str) -> str:
    """Get current weather for a city."""
    api_key = "your_key"
    url = f"https://api.openweathermap.org/data/2.5/weather"
    params = {"q": city, "appid": api_key}
    response = requests.get(url, params=params)
    data = response.json()
    return f"Weather in {city}: {data['weather'][0]['description']}"
```

```
@tool
```

```
def send_email(to: str, subject: str, body: str) -> str:
    """Send an email."""
    # Email sending logic
    return f"Email sent to {to}"
```

```
@tool
```

```
def create_calendar_event(title: str, date: str, time: str) -> str:
    """Create a calendar event."""
    # Calendar API logic
    return f"Event '{title}' created for {date} at {time}"
```

Tool Error Handling

```
python
```

```
from langchain_core.tools import ToolException

@tool
def risky_operation(value: str) -> str:
    """Operation that might fail."""
    try:
        result = perform_operation(value)
        return result
    except Exception as e:
        raise ToolException(f"Operation failed: {str(e)}")

# In agent
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    handle_parsing_errors=True,
    max_iterations=5
)
```

Retrieval Systems (RAG)

Document Loading

```
python
```

```
from langchain_community.document_loaders import (
    PyPDFLoader,
    TextLoader,
    CSVLoader,
    UnstructuredMarkdownLoader,
    WebBaseLoader,
    DirectoryLoader,
    GitLoader,
    NotionDirectoryLoader
)
```

PDF

```
pdf_loader = PyPDFLoader("document.pdf")
pages = pdf_loader.load()
```

Web

```
web_loader = WebBaseLoader("https://example.com")
docs = web_loader.load()
```

Directory

```
loader = DirectoryLoader(
    "./docs",
    glob="**/*.md",
    loader_cls=UnstructuredMarkdownLoader
)
docs = loader.load()
```

Multiple sources

```
from langchain_community.document_loaders import UnstructuredFileLoader

loader = UnstructuredFileLoader("mixed_content.docx")
```

Text Splitting

```
python
```



```
from langchain.text_splitter import (
    RecursiveCharacterTextSplitter,
    CharacterTextSplitter,
    TokenTextSplitter,
    MarkdownTextSplitter,
    PythonCodeTextSplitter
)

# Recursive (recommended)
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    length_function=len,
    separators=["\n\n", "\n", " ", ""]
)

docs = text_splitter.split_documents(loaded_docs)

# Code-aware splitting
python_splitter = PythonCodeTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

# Semantic splitting
from langchain_experimental.text_splitter import SemanticChunker

semantic_splitter = SemanticChunker(
    embeddings,
    breakpoint_threshold_type="percentile"
)
```

Embeddings

```
python
```

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.embeddings import (
    HuggingFaceEmbeddings,
    CohereEmbeddings,
    BedrockEmbeddings
)

# OpenAI
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-large",
    dimensions=1024
)

# HuggingFace (local)
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

# Cohere
embeddings = CohereEmbeddings(model="embed-english-v3.0")

# Embed documents
embedded_docs = embeddings.embed_documents([doc.page_content for doc in docs])
# Embed query
embedded_query = embeddings.embed_query("What is AI?")
```

Vector Stores

```
python
```

```

from langchain_community.vectorstores import (
    Chroma,
    Pinecone,
    Weaviate,
    FAISS,
    Qdrant
)

# Chroma (local)
vectorstore = Chroma.from_documents(
    documents=docs,
    embedding=embeddings,
    persist_directory="./chroma_db"
)

# Pinecone (cloud)
from pinecone import Pinecone as PineconeClient

pc = PineconeClient(api_key="your_key")
index = pc.Index("index-name")

vectorstore = Pinecone.from_documents(
    docs, embeddings, index_name="index-name"
)

# FAISS (fast local)
vectorstore = FAISS.from_documents(docs, embeddings)
vectorstore.save_local("faiss_index")

# Load
vectorstore = FAISS.load_local("faiss_index", embeddings)

# Weaviate (scalable)
vectorstore = Weaviate.from_documents(
    docs,
    embeddings,
    client=weaviate_client,
    index_name="Document"
)

```

Retrieval Strategies

```
python
```

```
# Basic similarity search
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4}
)

# MMR (Maximum Marginal Relevance)
retriever = vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 4, "fetch_k": 20, "lambda_mult": 0.5}
)

# Similarity with score threshold
retriever = vectorstore.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"score_threshold": 0.7, "k": 4}
)
```

Advanced Retrieval

Multi-Query Retriever

```
python

from langchain.retrievers.multi_query import MultiQueryRetriever

retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(),
    llm=llm
)

# Generates multiple search queries automatically
docs = retriever.get_relevant_documents("What is machine learning?")
```

Contextual Compression

```
python
```

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor

compressor = LLMChainExtractor.from_llm(llm)

compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=vectorstore.as_retriever()
)

# Returns only relevant parts of documents
docs = compression_retriever.get_relevant_documents("query")
```

Ensemble Retriever

```
python

from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever

# Combine vector search with keyword search
bm25_retriever = BM25Retriever.from_documents(docs)
bm25_retriever.k = 2

ensemble_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, vectorstore.as_retriever()],
    weights=[0.5, 0.5]
)
```

Parent Document Retriever

```
python
```

```
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore

# Retrieve small chunks, return full documents
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=2000)
child_splitter = RecursiveCharacterTextSplitter(chunk_size=400)

store = InMemoryStore()

retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=store,
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)

retriever.add_documents(docs)
```

RAG Chain

```
python
```

```

from langchain.chains import RetrievalQA
from langchain.chains.combine_documents import create_stuff_documents_chain

# Simple RAG
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever,
    return_source_documents=True
)

result = qa_chain({"query": "What is AI?"})

# With custom prompt (LCEL)
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough

template = """Answer based on context:

Context: {context}

Question: {question}

Answer: """

prompt = ChatPromptTemplate.from_template(template)

def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

answer = rag_chain.invoke("What is machine learning?")

```

RAG with Citations

```
python
```

```
from langchain.chains import create_citation_fuzzy_match_chain
```

```
chain = create_citation_fuzzy_match_chain(llm)
```

```
context = "\n".join([doc.page_content for doc in docs])
```

```
answer = chain.run(question="What is AI?", context=context)
```

```
# Returns answer with citation markers [1], [2], etc.
```

Conversational RAG

```
python
```



```
from langchain.chains import create_history_aware_retriever
from langchain_core.prompts import MessagesPlaceholder

# Contextualize question based on history
contextualize_prompt = ChatPromptTemplate.from_messages([
    ("system", "Reformulate the question given chat history"),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{input}")
])

history_aware_retriever = create_history_aware_retriever(
    llm, retriever, contextualize_prompt
)

# QA chain with history
qa_prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer based on context:\n\n{context}"),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{input}")
])

from langchain.chains import create_retrieval_chain

rag_chain = create_retrieval_chain(
    history_aware_retriever,
    create_stuff_documents_chain(llm, qa_prompt)
)

# Use with memory
chat_history = []
response = rag_chain.invoke({
    "input": "What is AI?",
    "chat_history": chat_history
})

chat_history.extend([
    HumanMessage(content="What is AI?"),
    AIMessage(content=response["answer"])
])
```

Evaluation & Testing

Unit Testing

```
python

import pytest
from langchain.schema import HumanMessage

def test_agent_response():
    response = agent.invoke({"input": "Hello"})
    assert "messages" in response
    assert len(response["messages"]) > 0

def test_tool_execution():
    result = search_tool.run("AI news")
    assert len(result) > 0
    assert "AI" in result

@pytest.mark.asyncio
async def test_async_agent():
    response = await agent.ainvoke({"input": "Hello"})
    assert response is not None
```

Evaluation Metrics

```
python
```

```

from langchain.evaluation import (
    load_evaluator,
    EvaluatorType
)

# Criteria-based
criteria_eval = load_evaluator("criteria", criteria="conciseness")
result = criteria_eval.evaluate_strings(
    prediction="Long rambling answer...",
    input="What is 2+2?"
)

# String distance
distance_eval = load_evaluator("string_distance", distance="levenshtein")
result = distance_eval.evaluate_strings(
    prediction="The capital is Paris",
    reference="Paris is the capital"
)

# Embedding distance
embedding_eval = load_evaluator(
    "embedding_distance",
    embeddings=embeddings
)
result = embedding_eval.evaluate_strings(
    prediction="Machine learning uses algorithms",
    reference="ML utilizes algorithms"
)

# QA correctness
qa_eval = load_evaluator("qa")
result = qa_eval.evaluate_strings(
    input="What is the capital of France?",
    prediction="Paris",
    reference="The capital of France is Paris"
)

```

Custom Evaluators

```
python
```

```
from langchain.evaluation import StringEvaluator

class CustomEvaluator(StringEvaluator):
    @property
    def requires_input(self) -> bool:
        return True

    @property
    def requires_reference(self) -> bool:
        return True

    def _evaluate_strings(
        self,
        prediction: str,
        input: str = None,
        reference: str = None,
        **kwargs
    ) -> dict:
        # Your evaluation logic
        score = calculate_score(prediction, reference)
        return {
            "score": score,
            "reasoning": "Evaluation reasoning"
        }

evaluator = CustomEvaluator()
```

Batch Evaluation

```
python
```

```
from langsmith import evaluate

# Prepare test dataset
test_cases = [
    {"input": "What is 2+2?", "expected": "4"},
    {"input": "Capital of France?", "expected": "Paris"},
]

# Evaluator
def accuracy_evaluator(run, example):
    prediction = run.outputs.get("output", "")
    expected = example.outputs.get("expected", "")
    return {"score": 1.0 if prediction.strip() == expected.strip() else 0.0}

# Run evaluation
results = evaluate(
    lambda x: agent.invoke(x),
    data=test_cases,
    evaluators=[accuracy_evaluator],
    experiment_prefix="agent-accuracy-test"
)

print(f"Average accuracy: {results['results']['score'].mean()}")
```

A/B Testing

```
python
```

```
from langsmith import Client

client = Client()

# Test two different prompts
results_a = evaluate(
    lambda x: chain_a.invoke(x),
    data=dataset_name,
    experiment_prefix="prompt-a"
)

results_b = evaluate(
    lambda x: chain_b.invoke(x),
    data=dataset_name,
    experiment_prefix="prompt-b"
)

# Compare results in LangSmith UI
```

Synthetic Data Generation

```
python

from langchain.evaluation import QAGenerateChain

# Generate QA pairs from documents
qa_generator = QAGenerateChain.from_llm(llm)

qa_pairs = qa_generator.apply_and_parse(
    [{"doc": doc} for doc in docs]
)

# Use for testing
for pair in qa_pairs:
    print(f'Q: {pair['question']}')
    print(f'A: {pair['answer']}')
```

Production Deployment

API Deployment with FastAPI

```
python
```

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from langchain.callbacks import AsyncIteratorCallbackHandler
import asyncio

app = FastAPI()

class QueryRequest(BaseModel):
    message: str
    thread_id: str = "default"

class QueryResponse(BaseModel):
    response: str
    sources: list = []

@app.post("/chat", response_model=QueryResponse)
async def chat(request: QueryRequest):
    try:
        config = {"configurable": {"thread_id": request.thread_id}}

        result = await agent.ainvoke(
            {"messages": [HumanMessage(content=request.message)]},
            config=config
        )

        return QueryResponse(
            response=result["messages"][-1].content,
            sources=result.get("sources", [])
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# Streaming endpoint
@app.post("/chat/stream")
async def chat_stream(request: QueryRequest):
    async def event_generator():
        config = {"configurable": {"thread_id": request.thread_id}}

        async for chunk in agent.astream(
            {"messages": [HumanMessage(content=request.message)]},
            config=config
        ):
            if "messages" in chunk:
                yield f"data: {chunk['messages'][-1].content}\n\n"

    return StreamingResponse(

```

```
        event_generator(),
        media_type="text/event-stream"
    )

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Rate Limiting

```
python

from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

@app.post("/chat")
@limiter.limit("10/minute")
async def chat(request: QueryRequest):
    # Implementation
    pass
```

Caching

```
python
```



```
from langchain.cache import InMemoryCache, SQLiteCache
from langchain.globals import set_llm_cache
import redis
from langchain.cache import RedisCache

# In-memory cache
set_llm_cache(InMemoryCache())

# SQLite cache
set_llm_cache(SQLiteCache(database_path=".langchain.db"))

# Redis cache
redis_client = redis.Redis(host='localhost', port=6379)
set_llm_cache(RedisCache(redis_client))

# Semantic cache
from langchain.cache import RedisSemanticCache

set_llm_cache(
    RedisSemanticCache(
        redis_url="redis://localhost:6379",
        embedding=embeddings,
        score_threshold=0.9
    )
)
```

Error Handling & Retry Logic

```
python
```

```
from tenacity import (
    retry,
    stop_after_attempt,
    wait_exponential,
    retry_if_exception_type
)
from langchain.schema import OutputParserException

@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=1, min=4, max=10),
    retry=retry_if_exception_type(Exception)
)
async def invoke_with_retry(agent, input_data, config):
    try:
        return await agent.ainvoke(input_data, config)
    except OutputParserException:
        # Handle parsing errors
        pass
    except Exception as e:
        logger.error(f'Agent error: {e}')
        raise
```

Monitoring & Logging

python

```

import logging
from langchain.callbacks import get_openai_callback

# Setup logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Track costs
with get_openai_callback() as cb:
    result = agent.invoke({"input": "Query"})
    logger.info(f'Tokens: {cb.total_tokens}, Cost: ${cb.total_cost}')

# Custom callback
from langchain.callbacks.base import BaseCallbackHandler

class CustomHandler(BaseCallbackHandler):
    def on_llm_start(self, serialized, prompts, **kwargs):
        logger.info(f'LLM started with prompts: {prompts}')

    def on_llm_end(self, response, **kwargs):
        logger.info(f'LLM finished: {response}')

    def on_tool_start(self, serialized, input_str, **kwargs):
        logger.info(f'Tool {serialized['name']} started')

    def on_agent_action(self, action, **kwargs):
        logger.info(f'Agent action: {action}')

agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    callbacks=[CustomHandler()]
)

```

Docker Deployment

dockerfile

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

yml

docker-compose.yml

version: '3.8'

services:

api:

build: .

ports:

- "8000:8000"

environment:

- OPENAI_API_KEY=\${OPENAI_API_KEY}

- LANGCHAIN_API_KEY=\${LANGCHAIN_API_KEY}

- LANGCHAIN_TRACING_V2=true

volumes:

- ./data:/app/data

depends_on:

- redis

redis:

image: redis:alpine

ports:

- "6379:6379"

Environment Management

python

```
from pydantic_settings import BaseSettings
```

```
class Settings(BaseSettings):
```

```
    openai_api_key: str
```

```
    anthropic_api_key: str
```

```
    langchain_api_key: str
```

```
    langchain_project: str = "production"
```

```
    redis_url: str = "redis://localhost:6379"
```

```
class Config:
```

```
    env_file = ".env"
```

```
settings = Settings()
```

Best Practices

1. Prompt Engineering

```
python
```

```
#  Good: Clear, specific instructions
```

```
prompt = """You are a helpful research assistant.
```

Task: Summarize the key findings from the provided research papers.

Guidelines:

- Focus on methodology and results
- Keep summaries under 200 words
- Include statistical significance when mentioned

Papers:

```
{documents}
```

```
Summary: """
```

```
#  Bad: Vague instructions
```

```
prompt = "Summarize these papers: {documents}"
```

2. Error Handling

```
python
```

```
#  Good: Comprehensive error handling
```

```
try:
```

```
    result = agent.invoke({"input": user_query})
```

```
except OutputParserException as e:
```

```
    logger.error(f"Parsing error: {e}")
```

```
    result = {"output": "I had trouble understanding the response. Please try again."}
```

```
except RateLimitError as e:
```

```
    logger.error(f"Rate limit: {e}")
```

```
    result = {"output": "System is busy. Please wait a moment."}
```

```
except Exception as e:
```

```
    logger.error(f"Unexpected error: {e}")
```

```
    result = {"output": "An error occurred. Our team has been notified."}
```

```
#  Bad: No error handling
```

```
result = agent.invoke({"input": user_query})
```

3. Token Management

```
python
```

```
#  Good: Track and limit tokens
```

```
from langchain.callbacks import get_openai_callback
```

```
with get_openai_callback() as cb:
```

```
    result = chain.invoke({"input": text})
```

```
    if cb.total_tokens > 50000:
```

```
        logger.warning(f"High token usage: {cb.total_tokens}")
```

```
    logger.info(f"Cost: ${cb.total_cost:.4f}")
```

```
# Use truncation for long texts
```

```
from langchain.text_splitter import TokenTextSplitter
```

```
splitter = TokenTextSplitter(chunk_size=3000, chunk_overlap=100)
```

```
chunks = splitter.split_text(long_text)
```

4. Security

```
python
```

 *Good: Input validation and sanitization*

```
from pydantic import BaseModel, validator
```

```
class UserQuery(BaseModel):
```

```
    text: str
```

```
    @validator('text')
```

```
    def validate_text(cls, v):
```

```
        if len(v) > 5000:
```

```
            raise ValueError("Query too long")
```

```
        if any(word in v.lower() for word in ["<script>", "DROP TABLE"]):
```

```
            raise ValueError("Invalid input")
```

```
        return v
```

API key management

```
import os
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
api_key = os.getenv("OPENAI_API_KEY")
```

Never hardcode keys

 *Bad*

```
api_key = "sk-abc123..."
```

5. Testing Strategy

```
python
```

```
# Unit tests
```

```
def test_agent_tool_calling():  
    response = agent.invoke({"input": "What's 5 + 3?"})  
    assert "8" in response["output"]
```

```
# Integration tests
```

```
def test_rag_pipeline():  
    docs = loader.load()  
    vectorstore = Chroma.from_documents(docs, embeddings)  
    retriever = vectorstore.as_retriever()  
    response = rag_chain.invoke("Test query")  
    assert len(response) > 0
```

```
# Evaluation tests
```

```
def test_response_quality():  
    evaluator = load_evaluator("criteria", criteria="helpfulness")  
    result = evaluator.evaluate_strings(  
        prediction=agent_response,  
        input=user_query  
    )  
    assert result["score"] > 0.7
```

6. Performance Optimization

```
python
```

```
#  Use async for concurrent operations
```

```
async def process_multiple_queries(queries):  
    tasks = [agent.ainvoke({"input": q}) for q in queries]  
    results = await asyncio.gather(*tasks)  
    return results
```

```
#  Batch processing
```

```
from langchain.chains import LLMChain
```

```
chain = LLMChain(llm=llm, prompt=prompt)  
results = await chain.abatch([{"input": q} for q in queries])
```

```
#  Streaming for better UX
```

```
async for chunk in agent.astream({"input": query}):  
    print(chunk, end="", flush=True)
```

7. Memory Management

```
python
```



```
#  Good: Clean up resources
from langchain.vectorstores import Chroma

vectorstore = Chroma(persist_directory="./db")
# Use vectorstore
vectorstore._client.persist() # Persist before closing
del vectorstore # Clean up

# Limit conversation history
def trim_messages(messages, max_tokens=4000):
    """Keep only recent messages within token limit"""
    total_tokens = 0
    trimmed = []

    for msg in reversed(messages):
        msg_tokens = len(msg.content) // 4 # Rough estimate
        if total_tokens + msg_tokens > max_tokens:
            break
        trimmed.insert(0, msg)
        total_tokens += msg_tokens

    return trimmed
```

8. Observability

```
python
```

 *Good: Comprehensive logging and tracing*

```
import structlog
```

```
logger = structlog.get_logger()
```

```
@traceable(name="process_query")
```

```
async def process_query(query: str, user_id: str):
```

```
    logger.info("query_started", user_id=user_id, query=query)
```

```
    try:
```

```
        result = await agent.ainvoke(
```

```
            {"input": query},
```

```
            config={
```

```
                "metadata": {
```

```
                    "user_id": user_id,
```

```
                    "timestamp": datetime.now().isoformat()
```

```
                },
```

```
                "tags": ["production", "user-query"]
```

```
            }
```

```
        )
```

```
    logger.info("query_completed", user_id=user_id, success=True)
```

```
    return result
```

```
except Exception as e:
```

```
    logger.error("query_failed", user_id=user_id, error=str(e))
```

```
    raise
```

9. Cost Optimization

python

```

#  Use smaller models when appropriate
from langchain_openai import ChatOpenAI

# For simple tasks
cheap_llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

# For complex reasoning
expensive_llm = ChatOpenAI(model="gpt-4", temperature=0)

# Route based on complexity
def get_llm_for_task(task_complexity: str):
    if task_complexity == "simple":
        return cheap_llm
    return expensive_llm

#  Implement caching
from langchain.cache import SQLiteCache
set_llm_cache(SQLiteCache(database_path=".cache.db"))

#  Use prompt compression
from langchain.prompts import PromptTemplate

# Compress context
compressed_prompt = PromptTemplate(
    template="Key points: {key_points}\nQuestion: {question}",
    input_variables=["key_points", "question"]
)

```

10. Version Control for Prompts

```
python
```

```
#  Good: Version and track prompts
```

```
PROMPTS = {  
    "v1": {  
        "template": "Answer: {input}",  
        "description": "Initial version"  
    },  
    "v2": {  
        "template": "Think step by step:\n{input}",  
        "description": "Added reasoning"  
    }  
}
```

```
def get_prompt(version="v2"):  
    return PromptTemplate(  
        template=PROMPTS[version]["template"],  
        input_variables=["input"]  
    )
```

```
# Track in LangSmith
```

```
chain = get_prompt("v2") | llm
```

```
result = chain.invoke(  
    {"input": query},  
    config={"tags": ["prompt-v2"]}  
)
```

Quick Reference Commands

Installation

```
bash
```

Core

```
pip install langchain langchain-community langchain-openai
```

Agents

```
pip install langgraph
```

Monitoring

```
pip install langsmith
```

Vector stores

```
pip install chromadb faiss-cpu pinecone-client
```

Utilities

```
pip install python-dotenv tiktoken
```

Environment Setup

```
bash
```

```
export OPENAI_API_KEY="sk-..."
```

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

```
export LANGCHAIN_TRACING_V2=true
```

```
export LANGCHAIN_API_KEY="ls-..."
```

```
export LANGCHAIN_PROJECT="my-project"
```

Common Imports

```
python
```

LLMs

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
```

Agents

```
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import create_react_agent
from langchain.agents import AgentExecutor, create_openai_functions_agent
```

Tools

```
from langchain_core.tools import tool
from langchain.tools import StructuredTool
```

Memory

```
from langgraph.checkpoint.memory import MemorySaver
from langchain.memory import ConversationBufferMemory
```

RAG

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings
```

Monitoring

```
from langsmith import traceable, Client
```

Additional Resources

Official Documentation

- **LangChain:** <https://python.langchain.com>
- **LangGraph:** <https://langchain-ai.github.io/langgraph/>
- **LangSmith:** <https://docs.smith.langchain.com>

Community

- **Discord:** <https://discord.gg/langchain>
- **GitHub:** <https://github.com/langchain-ai/langchain>
- **Twitter:** @LangChainAI

Learning Resources

- **LangChain Academy:** <https://academy.langchain.com>

- LangChain Blog: <https://blog.langchain.dev>
 - Example Projects: <https://github.com/langchain-ai/langchain/tree/master/templates>
-

Common Patterns Cheatsheet

Task	Solution
Build simple chatbot	Use <code>ConversationChain</code> with <code>ConversationBufferMemory</code>
Build tool-using agent	Use <code>create_react_agent</code> with custom tools
Implement RAG	Load docs → Split → Embed → Store → Retrieve → Generate
Add memory to agent	Use LangGraph with <code>MemorySaver</code> checkpointer
Multi-step workflow	Use LangGraph <code>StateGraph</code> with conditional edges
Human approval needed	Use LangGraph with <code>interrupt_before</code>
Monitor production	Enable LangSmith tracing with metadata and tags
Evaluate performance	Use LangSmith <code>evaluate()</code> with custom evaluators
Handle long conversations	Use <code>ConversationSummaryMemory</code> or trim messages
Optimize costs	Cache results, use smaller models, compress prompts
Stream responses	Use <code>astream()</code> or <code>stream()</code> methods
Parallel execution	Use <code>RunnableParallel</code> or <code>asyncio.gather()</code>

Created with  for AI Agent Builders

Last Updated: February 2026

Python Complete Theory Guide

From Basics to Advanced Concepts

Table of Contents

1. [Python Basics](#)
 2. [Object-Oriented Programming \(OOP\)](#)
 3. [Multithreading](#)
 4. [Multiprocessing](#)
 5. [Dynamic Programming](#)
 6. [Design Patterns](#)
-

1. Python Basics

1.1 Data Types and Variables

Theory

Python is dynamically typed, meaning you don't need to declare variable types explicitly. The interpreter infers the type at runtime.

Built-in Data Types:

- **Numeric:** int, float, complex
- **Sequence:** str, list, tuple, range
- **Mapping:** dict
- **Set:** set, frozenset
- **Boolean:** bool
- **None:** NoneType

Key Concepts:

- Variables are references to objects in memory
- Python uses automatic memory management (garbage collection)
- Everything in Python is an object
- Mutable vs Immutable objects

Mutable Objects (can be changed after creation):

- list, dict, set, bytearray

Immutable Objects (cannot be changed after creation):

- int, float, str, tuple, frozenset, bool

Type Conversion

Python supports both implicit (automatic) and explicit (manual) type conversion.

1.2 Control Flow

Conditional Statements

Python uses indentation (typically 4 spaces) to define code blocks instead of braces.

if-elif-else Structure:

- Executes code based on boolean conditions
- `elif` is short for "else if"
- Conditions are evaluated top-to-bottom
- First true condition executes, rest are skipped

Ternary Operator: Shorthand for simple if-else: `value_if_true if condition else value_if_false`

Loops

For Loop:

- Iterates over sequences (lists, strings, ranges, etc.)
- Uses iterator protocol under the hood
- `range(start, stop, step)` generates sequences efficiently

While Loop:

- Continues while condition is True
- Risk of infinite loops if condition never becomes False
- Useful when number of iterations is unknown

Loop Control:

- `break`: Exit loop immediately
- `continue`: Skip to next iteration
- `else`: Executes if loop completes without break

1.3 Functions

Theory

Functions are first-class objects in Python, meaning they can be:

- Assigned to variables
- Passed as arguments
- Returned from other functions
- Stored in data structures

Function Components:

1. **def keyword**: Defines function
2. **Function name**: Identifier
3. **Parameters**: Input placeholders
4. **Docstring**: Documentation (optional but recommended)
5. **Function body**: Code to execute
6. **return statement**: Output (optional, defaults to None)

Parameter Types

Positional Arguments:

- Matched by position
- Must be provided in correct order

Keyword Arguments:

- Matched by name
- Order doesn't matter
- More readable for functions with many parameters

Default Parameters:

- Provide default values
- Must come after non-default parameters
- **Warning**: Don't use mutable defaults (like lists)

Variable Arguments:

- `*args`: Tuple of positional arguments

- `**kwargs`: Dictionary of keyword arguments
- Allows flexible function signatures

Scope and Namespaces

LEGB Rule (name resolution order):

1. **Local**: Inside current function
2. **Enclosing**: In enclosing functions (for nested functions)
3. **Global**: Module level
4. **Built-in**: Python's built-in names

Keywords:

- `global`: Modify global variable from inside function
- `nonlocal`: Modify enclosing function's variable (for closures)

Lambda Functions

- Anonymous functions (no name)
 - Single expression only
 - Syntax: `lambda parameters: expression`
 - Commonly used with `map()`, `filter()`, `sorted()`
 - Less readable for complex logic
-

1.4 Data Structures

Lists

Theory:

- Dynamic arrays (can grow/shrink)
- Ordered and indexed (0-based)
- Allow duplicate elements
- Mutable (can be modified)
- Heterogeneous (can contain different types)

Time Complexity:

- Access by index: $O(1)$
- Append: $O(1)$ amortized
- Insert/Delete at beginning: $O(n)$

- Search: $O(n)$

Important Methods:

- `append()`: Add to end - $O(1)$
- `insert()`: Add at position - $O(n)$
- `pop()`: Remove from end - $O(1)$
- `remove()`: Remove by value - $O(n)$
- `extend()`: Add multiple items - $O(k)$ where k is items added
- `sort()`: In-place sort - $O(n \log n)$

List Comprehensions:

- Concise way to create lists
- Syntax: `[expression for item in iterable if condition]`
- More Pythonic and often faster than loops
- Can be nested but readability decreases

Tuples

Theory:

- Immutable sequences
- Ordered and indexed
- Can contain duplicates
- Faster than lists for read operations
- Used for heterogeneous data
- Can be used as dictionary keys (if all elements are immutable)

Use Cases:

- Returning multiple values from functions
- Protecting data from modification
- Dictionary keys
- Performance optimization

Dictionaries

Theory:

- Hash tables (key-value pairs)
- Unordered (before Python 3.7) / Insertion-ordered (Python 3.7+)

- Keys must be immutable and unique
- Values can be any type
- Extremely fast lookups

Time Complexity:

- Access/Insert/Delete by key: $O(1)$ average case
- Search by value: $O(n)$

Important Methods:

- `get()`: Safe access with default
- `keys()`: View of keys
- `values()`: View of values
- `items()`: View of (key, value) pairs
- `update()`: Merge dictionaries
- `pop()`: Remove and return value
- `setdefault()`: Get or set default

Dictionary Comprehensions:

- Syntax: `{key_expr: value_expr for item in iterable}`

Sets

Theory:

- Unordered collection of unique elements
- Implemented using hash tables
- Fast membership testing
- Support mathematical set operations

Time Complexity:

- Add/Remove/Contains: $O(1)$ average
- Union/Intersection: $O(\text{len}(s) + \text{len}(t))$

Set Operations:

- Union: `|` or `union()`
- Intersection: `&` or `intersection()`
- Difference: `-` or `difference()`
- Symmetric Difference: `^` or `symmetric_difference()`

1.5 String Operations

Theory

Strings in Python are:

- Immutable sequences of Unicode characters
- Enclosed in single ("), double (""), or triple ('' or ''') quotes
- Triple quotes allow multi-line strings
- Support indexing and slicing
- Rich set of built-in methods

String Immutability:

- Every string operation creates a new string
- Original string is never modified
- Can impact performance for many concatenations

String Methods

Case Conversion:

- `upper()`, `lower()`, `capitalize()`, `title()`, `swapcase()`

Search/Test:

- `find()`, `index()`, `count()`, `startswith()`, `endswith()`
- `isalpha()`, `isdigit()`, `isalnum()`, `isspace()`

Modification:

- `replace()`, `strip()`, `lstrip()`, `rstrip()`
- `split()`, `join()`

String Formatting

f-strings (Python 3.6+):

- Most modern and readable
- Embedded expressions with `{}`
- Support format specifications: `{value:width.precision}`

`format()` method:

- More flexible than % formatting
- Positional and keyword arguments
- Reusable format strings

% formatting (old style):

- C-style formatting
- Less recommended but still common in legacy code

Slicing

Syntax: `string[start:stop:step]`

- start: inclusive (default 0)
 - stop: exclusive (default len)
 - step: increment (default 1)
 - Negative indices count from end
 - Negative step reverses direction
-

1.6 File Handling

Theory

File operations in Python follow:

1. **Open:** Create file object
2. **Read/Write:** Perform operations
3. **Close:** Release resources

File Modes:

- `'r'`: Read (default)
- `'w'`: Write (truncates file)
- `'a'`: Append
- `'x'`: Exclusive creation (fails if exists)
- `'b'`: Binary mode
- `'t'`: Text mode (default)
- `'+'`: Read and write

Best Practice: Use context managers (`with` statement) to ensure files are properly closed even if errors occur.

Reading Files

Methods:

- `read()`: Entire file as string
- `readline()`: Single line
- `readlines()`: List of lines
- Iteration: Most memory-efficient for large files

Writing Files

Methods:

- `write()`: Write string
- `writelines()`: Write list of strings (no newlines added)

File Pointer:

- `seek(offset, whence)`: Move file pointer
 - `tell()`: Get current position
-

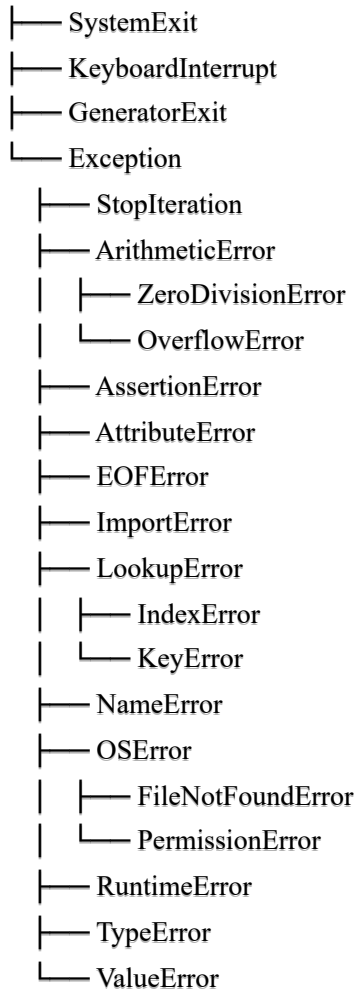
1.7 Exception Handling

Theory

Exceptions are runtime errors that disrupt normal program flow. Python uses exception objects to represent errors.

Exception Hierarchy:

BaseException



Exception Handling Blocks

try-except:

- Catch and handle specific exceptions
- Multiple except blocks for different exceptions
- Generic Exception catches all (use cautiously)

else:

- Executes if no exception occurred
- Runs after try, before finally
- Good for code that should only run on success

finally:

- Always executes (even if exception or return)
- Used for cleanup (close files, release resources)
- Executes even if exception is re-raised

Raising Exceptions

raise statement:

- Trigger exceptions programmatically
- Can raise built-in or custom exceptions
- Can re-raise caught exceptions

Custom Exceptions:

- Inherit from Exception class
 - Add custom attributes/methods
 - Make error handling more specific
-

1.8 Comprehensions and Generators

List Comprehensions

Theory:

- Concise way to create lists
- More readable than traditional loops
- Often faster than equivalent for loops
- Should fit on one line for readability

Structure: `[expression for item in iterable if condition]`

Dictionary Comprehensions

Structure: `{key_expr: value_expr for item in iterable if condition}`

Set Comprehensions

Structure: `{expression for item in iterable if condition}`

Generator Expressions

Theory:

- Like list comprehensions but return generator objects
- Lazy evaluation (compute values on-demand)
- Memory efficient for large datasets
- Single-use (can't iterate twice)

Structure: `(expression for item in iterable if condition)`

Generator Functions

Theory:

- Functions that use `yield` instead of `return`
- Maintain state between calls
- Produce values one at a time
- Memory efficient for sequences
- Can be infinite

Key Differences from Regular Functions:

- Don't compute all values upfront
 - State is preserved between yields
 - Can receive values via `send()`
 - Can be closed with `close()`
-

1.9 Decorators

Theory

Decorators are functions that modify the behavior of other functions or classes without changing their source code.

Core Concepts:

- Functions are first-class objects
- Functions can be nested
- Functions can be passed as arguments
- Closures capture enclosing scope

How Decorators Work:

1. Take a function as input
2. Define a wrapper function with modified behavior
3. Return the wrapper function
4. Replace original function with wrapper

Syntax:

```
python
```

```
@decorator
def function():
    pass

# Equivalent to:
function = decorator(function)
```

Types of Decorators

Function Decorators:

- Modify function behavior
- Add functionality (logging, timing, caching)
- Enforce access control

Class Decorators:

- Modify class behavior
- Add methods/attributes
- Implement patterns (Singleton, etc.)

Decorators with Arguments:

- Need an extra layer of nesting
- Outer function takes decorator arguments
- Returns actual decorator

Built-in Decorators:

- `@property`: Convert method to attribute
- `@staticmethod`: Method doesn't access instance/class
- `@classmethod`: Method receives class as first argument
- `@functools.wraps`: Preserve original function metadata

1.10 Iterators and Iteration

Theory

Iterable:

- Object that can return an iterator
- Implements `__iter__()` method

- Examples: list, tuple, dict, set, str

Iterator:

- Object representing stream of data
- Implements `__iter__()` and `__next__()`
- Raises `StopIteration` when exhausted
- Maintains state (current position)

Iteration Protocol:

1. Call `iter(obj)` to get iterator
2. Call `next(iterator)` repeatedly
3. Catch `StopIteration` exception

for Loop Internals:

```
python

#for item in iterable:
#    process(item)

iterator = iter(iterable)
while True:
    try:
        item = next(iterator)
        process(item)
    except StopIteration:
        break
```

2. Object-Oriented Programming (OOP)

2.1 Classes and Objects

Theory

Object-Oriented Programming is a paradigm based on "objects" that contain data (attributes) and code (methods).

Key Concepts:

- **Class:** Blueprint for creating objects
- **Object:** Instance of a class
- **Attribute:** Data stored in object

- **Method:** Function defined in class

Class Definition:

- Use `class` keyword
- Class names use PascalCase convention
- Can inherit from other classes
- Contains attributes and methods

Instance vs Class Attributes:

- **Instance attributes:** Unique to each object
- **Class attributes:** Shared by all instances
- Define instance attributes in `__init__()`
- Define class attributes in class body

Methods:

- **Instance methods:** Operate on instance (`self`)
- **Class methods:** Operate on class (`@classmethod, cls`)
- **Static methods:** Independent utility functions (`@staticmethod`)

The init Method

- Constructor-like method (initializer)
- Called automatically when object is created
- First parameter is always `self` (instance reference)
- Used to set initial state

The self Parameter

- Reference to current instance
 - Must be first parameter of instance methods
 - Name is convention (could be anything)
 - Python doesn't pass it automatically (explicit)
-

2.2 The Four Pillars of OOP

Encapsulation

Theory: Bundling data and methods that operate on that data within a single unit (class), and restricting access

to some components.

Access Levels in Python:

- **Public:** Accessible everywhere (normal naming)
- **Protected:** By convention, single underscore prefix `_attribute`
- **Private:** Name mangling, double underscore prefix `__attribute`

Name Mangling:

- `_attribute` becomes `_ClassName__attribute`
- Prevents accidental access/override in subclasses
- Not true privacy (can still access if needed)

Properties:

- Python's way of implementing getters/setters
- Use `@property` decorator
- Make attribute access look like direct access
- Allows validation, computation, lazy loading

Benefits:

- Data hiding and protection
- Flexibility to change implementation
- Validation and error checking
- Controlled access

Inheritance

Theory: Mechanism where a new class (child/derived) inherits attributes and methods from existing class (parent/base).

Types of Inheritance:

1. Single Inheritance:

- One parent class
- Simple hierarchy
- Most common type

2. Multiple Inheritance:

- Multiple parent classes
- Diamond problem possible

- Method Resolution Order (MRO) critical
- Python uses C3 linearization

3. Multilevel Inheritance:

- Chain of inheritance ($A \rightarrow B \rightarrow C$)
- Each level adds functionality
- Deep hierarchies can be complex

4. Hierarchical Inheritance:

- Multiple children from one parent
- Common base, specialized children

Method Resolution Order (MRO):

- Order in which Python searches for methods
- Use `ClassName.__mro__` or `ClassName.mro()`
- C3 linearization algorithm
- Left-to-right, depth-first

super() Function:

- Call parent class methods
- Respects MRO in multiple inheritance
- Don't need to name parent class
- Cooperative multiple inheritance

Method Overriding:

- Child class redefines parent method
- Same name, different implementation
- Can call parent method with `super()`

Benefits:

- Code reusability
- Extensibility
- Hierarchical classification
- Polymorphism support

Polymorphism

Theory: Ability of objects of different classes to be treated as objects of a common base class. "Many forms" - same interface, different implementations.

Types:

1. Compile-time Polymorphism (Method Overloading):

- Python doesn't support true overloading
- Can simulate with default arguments
- Can use `*args` and `**kwargs`
- Can check argument types manually

2. Runtime Polymorphism (Method Overriding):

- Child class overrides parent method
- Determined at runtime
- Basis of polymorphic behavior

Duck Typing:

- "If it walks like a duck and quacks like a duck, it's a duck"
- Type determined by behavior, not inheritance
- Very Pythonic approach
- Flexible but less safe

Benefits:

- Flexibility and extensibility
- Simplified code
- Easy to add new types
- Interface-based programming

Abstraction

Theory: Hiding complex implementation details and showing only necessary features. Focus on "what" it does, not "how" it does it.

Abstract Base Classes (ABC):

- Cannot be instantiated
- Define interface contract
- Enforce implementation in subclasses
- Use `abc` module

Abstract Methods:

- Decorated with `@abstractmethod`
- Must be implemented by concrete subclasses
- Subclass can't be instantiated if method not implemented

Benefits:

- Reduces complexity
 - Enforces structure
 - Separates interface from implementation
 - Supports code maintenance
-

2.3 Special Methods (Magic Methods)

Theory

Special methods (dunder methods) allow your classes to implement Python's built-in behaviors.

Common Categories:

Object Lifecycle:

- `__init__(self)`: Constructor
- `__new__(cls)`: Object creation (before **init**)
- `__del__(self)`: Destructor (rarely used)

String Representation:

- `__str__(self)`: Informal string (for users) - `str()`
- `__repr__(self)`: Official representation (for developers) - `repr()`
- Should return string

Comparison Operators:

- `__eq__(self, other)`: `==` operator
- `__ne__(self, other)`: `!=` operator
- `__lt__(self, other)`: `<` operator
- `__le__(self, other)`: `<=` operator
- `__gt__(self, other)`: `>` operator
- `__ge__(self, other)`: `>=` operator

Arithmetic Operators:

- `__add__(self, other)`: + operator
- `__sub__(self, other)`: - operator
- `__mul__(self, other)`: * operator
- `__truediv__(self, other)`: / operator
- `__floordiv__(self, other)`: // operator
- `__mod__(self, other)`: % operator
- `__pow__(self, other)`: ** operator

Container Methods:

- `__len__(self)`: len() function
- `__getitem__(self, key)`: indexing []
- `__setitem__(self, key, value)`: assignment []
- `__delitem__(self, key)`: deletion del []
- `__contains__(self, item)`: in operator
- `__iter__(self)`: for loop support

Callable:

- `__call__(self, ...)`: Make instance callable like function

Context Manager:

- `__enter__(self)`: with statement entry
 - `__exit__(self, exc_type, exc_val, exc_tb)`: with statement exit
-

2.4 Composition vs Inheritance

Theory

Inheritance: "is-a" relationship

- Car is a Vehicle
- Dog is an Animal
- Creates tight coupling

Composition: "has-a" relationship

- Car has an Engine

- Person has an Address
- Creates loose coupling

When to Use Each:

Use Inheritance When:

- Clear hierarchical relationship exists
- Subclass truly "is-a" type of parent
- Need to override parent behavior
- Polymorphism is important

Use Composition When:

- Need functionality from multiple sources
- Relationship is "has-a" not "is-a"
- Want flexibility to change implementation
- Avoid deep inheritance hierarchies

Problems with Deep Inheritance:

- Fragile base class problem
- Difficulty understanding behavior
- Hard to modify
- Tight coupling

Benefits of Composition:

- More flexible
 - Easier to test
 - Reduces coupling
 - Follows "favor composition over inheritance" principle
-

2.5 Class Design Principles

SOLID Principles

S - Single Responsibility Principle:

- Class should have one reason to change
- One responsibility per class
- Improves maintainability

O - Open/Closed Principle:

- Open for extension, closed for modification
- Add new functionality without changing existing code
- Use inheritance and composition

L - Liskov Substitution Principle:

- Subclasses should be substitutable for base classes
- Subclass can't weaken base class behavior
- Maintains polymorphism

I - Interface Segregation Principle:

- Many specific interfaces better than one general
- Clients shouldn't depend on unused methods
- Use ABC with focused interfaces

D - Dependency Inversion Principle:

- Depend on abstractions, not concretions
 - High-level modules independent of low-level
 - Both depend on abstractions
-

3. Multithreading

3.1 Threading Fundamentals

Theory

Process vs Thread:

Process:

- Independent execution unit
- Own memory space
- Heavy-weight
- Inter-process communication is complex
- Python: multiprocessing module

Thread:

- Lightweight execution unit within process
- Shares memory space with other threads
- Light-weight
- Easier communication (shared memory)
- Python: threading module

Concurrency vs Parallelism:

Concurrency:

- Multiple tasks making progress (not necessarily simultaneous)
- Task switching gives illusion of simultaneity
- Single-core CPU can be concurrent

Parallelism:

- Multiple tasks executing simultaneously
- Requires multiple cores
- Subset of concurrency

Python Threading

threading Module:

- Built-in module for thread management
- Two main ways to create threads:
 1. Create Thread object with target function
 2. Subclass Thread and override run()

Thread Lifecycle:

1. **New:** Thread object created
2. **Runnable:** start() called
3. **Running:** Executing
4. **Blocked:** Waiting for resource
5. **Dead:** Execution completed

Main Thread:

- Program starts with main thread
- Child threads can be created
- Program ends when all threads complete (daemon=False)

3.2 The Global Interpreter Lock (GIL)

Theory

What is GIL:

- Mutex that protects Python objects
- Only one thread executes Python bytecode at a time
- Even on multi-core systems
- Specific to CPython implementation

Why GIL Exists:

- Simplifies memory management
- Makes C extensions easier
- Historical design decision
- Protects reference counting

Impact:

CPU-bound tasks:

- GIL limits to single core
- No true parallelism
- Multithreading can be slower (context switching overhead)
- Use multiprocessing instead

I/O-bound tasks:

- Threads release GIL during I/O operations
- Can benefit from multithreading
- Waiting for network, disk, user input
- GIL not a bottleneck

Workarounds:

1. Use multiprocessing for CPU-bound tasks
 2. Use async/await for I/O-bound tasks
 3. Use alternative Python implementations (Jython, IronPython)
 4. Write C extensions that release GIL
-

3.3 Thread Synchronization

Theory

Race Condition:

- Multiple threads access shared data
- At least one modifies it
- Outcome depends on timing
- Results are unpredictable
- Leads to data corruption

Critical Section:

- Code that accesses shared resources
- Must be protected from concurrent access
- Only one thread at a time

Synchronization Primitives

Lock (Mutex):

- Most basic synchronization primitive
- Binary: locked or unlocked
- acquire() and release()
- Only thread that acquired can release
- Can cause deadlocks

RLock (Reentrant Lock):

- Can be acquired multiple times by same thread
- Must be released equal number of times
- Tracks owning thread and recursion level
- Useful for recursive functions

Semaphore:

- Counter-based synchronization
- Allows N threads to access resource
- Useful for resource pools (connection pools, etc.)
- acquire() decrements, release() increments

BoundedSemaphore:

- Like Semaphore but prevents over-release
- Raises error if released too many times
- Safer than regular Semaphore

Event:

- Thread communication mechanism
- Boolean flag with signaling
- wait() blocks until set
- set() wakes all waiting threads
- clear() resets flag

Condition:

- Complex synchronization scenarios
- Associated with a lock
- wait() releases lock and blocks
- notify() or notify_all() wakes threads
- Used for producer-consumer patterns

Barrier:

- Synchronization point for fixed number of threads
 - All threads must reach barrier before continuing
 - Useful for multi-phase algorithms
 - wait() blocks until all threads arrive
-

3.4 Thread Safety

Theory**Thread-Safe:**

- Can be safely used by multiple threads concurrently
- No race conditions
- Correct behavior guaranteed

Thread-Unsafe:

- Not safe for concurrent access
- Can cause race conditions
- Needs external synchronization

Python Built-in Thread Safety:

Thread-Safe:

- List operations (mostly)
- Dict operations (mostly)
- print() function
- Most built-in types

Not Thread-Safe:

- Compound operations (check-then-act)
- Multiple operations together
- Custom data structures (without locks)

Making Code Thread-Safe:

1. Use locks around critical sections
 2. Use thread-safe data structures (queue.Queue)
 3. Avoid shared mutable state
 4. Use immutable objects
 5. Use thread-local storage
-

3.5 Thread Communication

Theory

Queue Module:

- Thread-safe queues
- Blocks on empty get() or full put()
- Perfect for producer-consumer pattern
- No manual locking needed

Types:

- **Queue:** FIFO (First In First Out)

- **LifoQueue:** LIFO (Last In First Out) - Stack
- **PriorityQueue:** Items retrieved by priority

Queue Methods:

- `put(item, block=True, timeout=None)`: Add item
- `get(block=True, timeout=None)`: Remove and return item
- `task_done()`: Mark task complete
- `join()`: Block until all tasks done
- `qsize()`: Approximate size
- `empty()`: Check if empty
- `full()`: Check if full

Thread-Local Data:

- Data specific to each thread
 - Use `threading.local()`
 - Like global but per-thread
 - No synchronization needed
-

3.6 Threading Best Practices

When to Use Threads:

- I/O-bound operations
- Network requests
- File I/O
- Database operations
- User interfaces (keep responsive)

When NOT to Use Threads:

- CPU-bound operations (use multiprocessing)
- Simple sequential tasks
- When shared state is complex

Best Practices:

1. Minimize shared state
2. Use high-level primitives (Queue, not Lock)

3. Avoid daemon threads for important work
 4. Always handle exceptions in threads
 5. Use context managers for locks (with statement)
 6. Document thread-safety expectations
 7. Test with thread sanitizers
 8. Keep critical sections small
 9. Avoid nested locks (deadlock risk)
 10. Use thread pools (ThreadPoolExecutor)
-

3.7 ThreadPoolExecutor

Theory

Executor Pattern:

- High-level interface for async execution
- Manages thread pool
- Submits tasks and gets futures
- Handles thread lifecycle

concurrent.futures Module:

- Modern threading/multiprocessing interface
- ThreadPoolExecutor for threads
- ProcessPoolExecutor for processes
- Similar API for both

Benefits:

- Automatic thread management
- Reuses threads (no creation overhead)
- Clean shutdown
- Future objects for results
- Exception handling built-in

Future Objects:

- Represents future result
- Methods: result(), done(), cancel()
- Can add callbacks

- Exception propagation

Methods:

- `submit(fn, *args, **kwargs)`: Submit single task
 - `map(fn, *iterables)`: Submit multiple tasks
 - `shutdown(wait=True)`: Clean shutdown
-

4. Multiprocessing

4.1 Multiprocessing Fundamentals

Theory

Why Multiprocessing:

- Bypass GIL limitation
- True parallelism
- Utilize multiple CPU cores
- Better for CPU-bound tasks

Process Creation:

- Each process has own memory space
- Separate Python interpreter
- Higher overhead than threads
- More isolated (safer)

multiprocessing Module:

- Similar API to threading
- Process class instead of Thread
- Process creation methods:
 1. Target function
 2. Subclass Process

Start Methods:

- **spawn**: New Python interpreter (Windows default)
- **fork**: Copy parent process (Unix default)
- **forkserver**: Server creates new processes

Considerations:

- Windows always uses spawn
 - fork faster but can have issues
 - spawn safer, slower startup
 - Set with `set_start_method()`
-

4.2 Inter-Process Communication (IPC)

Theory

Processes don't share memory, so need special mechanisms to communicate.

Queue

multiprocessing.Queue:

- Process-safe queue
- Uses pipes and locks
- Similar to `threading.Queue`
- Can transfer picklable objects

Key Differences from `threading.Queue`:

- Uses serialization (pickle)
- Higher overhead
- All objects must be picklable
- Can be slower

Pipe

Theory:

- Two-way communication channel
- Returns two connection objects
- Faster than Queue for two processes
- No built-in locking (can corrupt)

Methods:

- `send(obj)`: Send object
- `recv()`: Receive object

- `close()`: Close connection
- `poll([timeout])`: Check if data available

Duplex vs Simplex:

- Duplex: Both ends can send/receive (default)
- Simplex: One-way communication

Shared Memory

Value and Array:

- Shared memory primitives
- Synchronized access built-in
- Faster than Queue/Pipe
- Limited to simple types

Manager:

- Proxy objects for shared state
- Supports complex types (list, dict)
- Higher overhead than Value/Array
- Process-safe automatically

Manager Types:

- list, dict, Namespace, Lock, RLock, Semaphore, etc.
 - Proxies handle synchronization
 - Can be used across network
-

4.3 Process Synchronization

Theory

Similar to threading but for processes.

Lock:

- Mutual exclusion
- Prevent simultaneous access
- Similar to threading.Lock
- Uses OS primitives

RLock:

- Reentrant lock
- Same process can acquire multiple times

Semaphore:

- Counter-based
- Limit concurrent access

Event:

- Signal between processes
- set(), clear(), wait()

Condition:

- Complex synchronization
- Wait for specific conditions

Barrier:

- Synchronization point
 - Fixed number of processes
-

4.4 Process Pools

Theory**Pool Class:**

- Manage worker processes
- Distribute tasks automatically
- Reuse processes
- High-level interface

Methods:**map():**

- Parallel version of built-in map()
- Blocks until complete
- Returns results in order

- Chunks tasks automatically

imap():

- Lazy version of map()
- Returns iterator
- Memory efficient
- Results as they complete

imap_unordered():

- Like imap but results in completion order
- Faster than imap
- Order not preserved

apply():

- Execute function once
- Blocks until complete
- Deprecated, use apply_async

apply_async():

- Asynchronous version
- Returns AsyncResult object
- Can add callback

starmap():

- Like map but unpacks arguments
- Useful for multiple arguments

starmap_async():

- Asynchronous starmap

Context Manager:

- Use with statement
 - Automatic cleanup
 - Ensures termination
-

4.5 ProcessPoolExecutor

Theory

concurrent.futures:

- Modern interface
- Consistent with ThreadPoolExecutor
- Higher-level than Pool

Advantages:

- Future objects
- Exception handling
- Clean API
- Interchangeable with ThreadPoolExecutor

Methods: Same as ThreadPoolExecutor:

- `submit()`
- `map()`
- `shutdown()`

Choosing Pool vs Executor:

- **Pool:** More control, more methods
 - **Executor:** Simpler, modern, futures
-

4.6 Best Practices

When to Use Multiprocessing:

- CPU-bound tasks
- Number crunching
- Image/video processing
- Data analysis
- Scientific computing

When NOT to Use:

- I/O-bound tasks (use threading/async)
- Simple tasks (overhead not worth it)

- Requires shared complex state

Best Practices:

1. Minimize data transfer between processes
2. Use pools for many small tasks
3. Keep processes independent
4. Pickle overhead consideration
5. Use appropriate start method
6. Handle exceptions properly
7. Clean shutdown (terminate if needed)
8. Profile before optimizing
9. Consider memory usage
10. Use chunksize for map operations

Common Pitfalls:

- Pickling limitations (lambda, local functions)
 - Fork safety issues
 - Resource leaks
 - Not joining processes
 - Race conditions in shared memory
-

5. Dynamic Programming

5.1 Dynamic Programming Fundamentals

Theory

What is Dynamic Programming: Dynamic Programming (DP) is an optimization technique that solves complex problems by breaking them down into simpler subproblems and storing the results to avoid redundant calculations.

Key Characteristics:

1. **Optimal Substructure:** Optimal solution contains optimal solutions to subproblems
2. **Overlapping Subproblems:** Same subproblems are solved multiple times

When to Use DP:

- Problem can be divided into overlapping subproblems
- Optimal solution can be constructed from optimal solutions to subproblems

- Brute force has repeated calculations
- Counting problems
- Optimization problems (min/max)

DP vs Divide and Conquer:

- **Divide and Conquer:** Subproblems are independent (Merge Sort, Quick Sort)
 - **DP:** Subproblems overlap (Fibonacci, Shortest Path)
-

5.2 Approaches to Dynamic Programming

Top-Down (Memoization)

Theory:

- Start with original problem
- Recursively break into subproblems
- Store (memoize) results of subproblems
- Check cache before computing

Characteristics:

- Natural recursive structure
- Easy to implement from recursive solution
- Only computes needed subproblems
- Call stack overhead
- Risk of stack overflow for deep recursion

Implementation Techniques:

1. Dictionary for memoization
2. LRU cache decorator
3. Manual recursion with cache

When to Use:

- Natural recursive formulation
- Not all subproblems needed
- Easier to understand
- Proof of concept

Bottom-Up (Tabulation)

Theory:

- Start with smallest subproblems
- Build up to original problem
- Fill table iteratively
- No recursion

Characteristics:

- Iterative implementation
- No call stack overhead
- Computes all subproblems
- More space efficient (can optimize)
- Often faster in practice

Implementation Techniques:

1. 1D array/list
2. 2D array/matrix
3. Space optimization (rolling arrays)

When to Use:

- All subproblems needed
 - Space optimization important
 - Avoid recursion depth issues
 - Production code
-

5.3 Common DP Patterns

1D DP

Pattern:

- Single dimension problem
- $\boxed{\text{dp}[i]}$ represents solution for position i
- Typically linear recurrence relation

Examples:

- Fibonacci sequence
- Climbing stairs
- House robber
- Coin change (min coins)

General Structure:

1. Define dp array
2. Initialize base cases
3. Fill array using recurrence relation
4. Return $dp[n]$ or final value

2D DP

Pattern:

- Two dimensions
- $dp[i][j]$ represents solution for (i,j)
- Grid or two-sequence problems

Examples:

- Longest common subsequence
- Edit distance
- Matrix chain multiplication
- 0/1 knapsack

General Structure:

1. Create 2D table
2. Initialize base cases (first row/column)
3. Fill table row by row or column by column
4. Return $dp[m][n]$ or final cell

Knapsack Pattern

0/1 Knapsack:

- Item included or not (binary choice)
- $dp[i][w]$ = max value with items $0..i$ and weight w

Unbounded Knapsack:

- Unlimited quantity of each item

- Simpler recurrence relation

Variations:

- Subset sum
- Partition problem
- Coin change

Subsequence/Substring Pattern

Subsequence:

- Elements in order but not consecutive
- Skip elements allowed

Substring:

- Consecutive elements
- No skipping

Common Problems:

- Longest common subsequence (LCS)
- Longest increasing subsequence (LIS)
- Palindromic subsequence
- Edit distance

Grid/Path Pattern

Characteristics:

- 2D grid navigation
- Move constraints (right, down, etc.)
- Count paths or find optimal path

Examples:

- Unique paths
 - Minimum path sum
 - Dungeon game
 - Triangle minimum path
-

5.4 DP Optimization Techniques

Space Optimization

Rolling Array:

- Use only necessary rows/columns
- Reduce 2D to 1D
- Reduce space from $O(n^2)$ to $O(n)$

In-place Modification:

- Use input array for DP table
- Only when input can be modified

Two Variable Technique:

- For Fibonacci-like sequences
- Keep only last two values
- $O(1)$ space

Time Optimization

State Reduction:

- Eliminate unnecessary states
- Combine related states
- Simplify recurrence relation

Greedy Observation:

- Some DP problems have greedy property
- Can reduce from DP to greedy
- Much faster

Binary Search Integration:

- Longest increasing subsequence
 - Reduces $O(n^2)$ to $O(n \log n)$
-

5.5 Steps to Solve DP Problems

Step 1: Identify if it's DP

- Optimal substructure?
- Overlapping subproblems?
- Asking for optimal/count/possible?

Step 2: Define State

- What do you need to track?
- What are the parameters?
- What does $dp[i]$ or $dp[i][j]$ represent?

Step 3: Formulate Recurrence Relation

- How to compute $dp[i]$ from previous states?
- What are the choices at each step?
- Base cases?

Step 4: Decide Top-Down or Bottom-Up

- Complexity of recursion?
- All subproblems needed?
- Space constraints?

Step 5: Implement and Test

- Start with memoization (easier)
- Convert to tabulation for optimization
- Test with examples
- Check edge cases

Step 6: Optimize

- Can space be reduced?
- Can states be combined?
- Is there a greedy solution?

5.6 Common DP Problems Categories

Linear DP

- Single sequence
- Fibonacci, climbing stairs

- House robber variants
- Decode ways

Interval DP

- Range-based problems
- Burst balloons
- Matrix chain multiplication
- Palindrome partitioning

Grid DP

- 2D navigation
- Unique paths
- Minimum path sum
- Maximal rectangle

Knapsack DP

- Subset selection
- 0/1 knapsack
- Unbounded knapsack
- Subset sum
- Partition problems

String DP

- Two strings comparison
- Longest common subsequence
- Edit distance
- Wildcard matching
- Regular expression

Tree DP

- Tree structure problems
- House robber III
- Binary tree cameras
- Maximum path sum

State Machine DP

- Multiple states at each step
 - Best time to buy/sell stock
 - Paint house
 - State transitions
-

5.7 Memoization Decorators

`functools.lru_cache`

Theory:

- Least Recently Used cache
- Decorator for automatic memoization
- Maximum cache size (maxsize parameter)
- Thread-safe
- Only works with hashable arguments

Parameters:

- `maxsize`: Maximum cache entries (None = unlimited)
- `typed`: Separate cache for different types

Cache Info:

- `cache_info()`: Statistics
- `cache_clear()`: Clear cache

Limitations:

- Arguments must be hashable
 - Not suitable for mutable arguments
 - Memory usage with large cache
-

6. Design Patterns

6.1 Introduction to Design Patterns

Theory

What are Design Patterns: Reusable solutions to commonly occurring problems in software design. Not code,

but templates for solving problems.

Benefits:

- Proven solutions
- Common vocabulary
- Best practices
- Faster development
- Easier maintenance
- Better communication

Gang of Four (GoF): Published "Design Patterns: Elements of Reusable Object-Oriented Software" (1994)
Defined 23 classic patterns in 3 categories

Categories:

1. **Creational:** Object creation mechanisms
2. **Structural:** Object composition and relationships
3. **Behavioral:** Object interaction and responsibility

When to Use Patterns:

- Solving common problems
- Need flexibility and maintainability
- Team communication
- Large codebases

When NOT to Use:

- Over-engineering simple problems
 - Forcing patterns where they don't fit
 - Premature optimization
-

6.2 Creational Patterns

Singleton Pattern

Intent: Ensure a class has only one instance and provide global access point to it.

Use Cases:

- Database connections
- Configuration managers

- Logging
- Thread pools
- Caches

Implementation Approaches:

1. Class Variable:

- Simple implementation
- Not thread-safe

2. new Method:

- Controls instance creation
- More Pythonic

3. Decorator:

- Reusable across classes
- Clean syntax

4. Metaclass:

- Most robust
- Controls class creation

Pros:

- Controlled access to sole instance
- Global access point
- Lazy initialization possible

Cons:

- Global state (testing issues)
- Can violate Single Responsibility Principle
- Difficult to subclass
- Hidden dependencies

Thread Safety:

- Add locks for thread-safe creation
- Use double-checked locking
- Or rely on module-level instance

Factory Pattern

Intent: Define interface for creating objects, but let subclasses decide which class to instantiate.

Types:

Simple Factory:

- Not a true GoF pattern
- Function/method creates objects
- Encapsulates creation logic

Factory Method:

- Defines interface for creating objects
- Subclasses override to specify type
- Template Method pattern for creation

Abstract Factory:

- Creates families of related objects
- Multiple factory methods
- Ensures consistency

Use Cases:

- Object type determined at runtime
- Complex creation logic
- Multiple related objects
- Decoupling object creation from usage

Pros:

- Loose coupling
- Single Responsibility Principle
- Open/Closed Principle
- Easy to extend

Cons:

- Can introduce complexity
- Many subclasses

Builder Pattern

Intent: Separate construction of complex object from its representation. Build object step by step.

Use Cases:

- Complex objects with many parameters
- Different representations of same data
- Step-by-step construction
- Immutable objects

Components:

- **Product:** Object being built
- **Builder:** Abstract interface
- **Concrete Builder:** Implements construction steps
- **Director:** Constructs using Builder (optional)

Pros:

- Construct complex objects step-by-step
- Same construction process, different representations
- Isolates complex construction code
- Fluent interface (method chaining)

Cons:

- More code and classes
- Increased complexity

Prototype Pattern

Intent: Create new objects by copying existing objects (prototypes).

Use Cases:

- Creating objects is expensive
- Many similar objects needed
- Avoid subclass explosion
- Runtime configuration

Implementation:

- Use `copy` module
- Shallow copy: `copy.copy()`

- Deep copy: `copy.deepcopy()`

Pros:

- Avoids expensive creation
- Reduces subclassing
- Can add/remove products at runtime
- Configure objects with varying values

Cons:

- Deep copying complex objects can be tricky
 - Circular references issues
-

6.3 Structural Patterns

Adapter Pattern

Intent: Convert interface of class into another interface clients expect. Allows incompatible interfaces to work together.

Use Cases:

- Legacy code integration
- Third-party library incompatibility
- Reusing existing classes with incompatible interfaces

Types:

- **Class Adapter:** Uses inheritance (multiple inheritance)
- **Object Adapter:** Uses composition (more flexible)

Components:

- **Target:** Interface client expects
- **Adaptee:** Existing incompatible interface
- **Adapter:** Converts Adaptee to Target

Pros:

- Reuse existing code
- Single Responsibility Principle
- Open/Closed Principle

- Flexibility

Cons:

- Increased complexity
- Sometimes easier to change existing class

Decorator Pattern

Intent: Attach additional responsibilities to object dynamically. Flexible alternative to subclassing.

Use Cases:

- Add functionality without modifying class
- Combine multiple behaviors
- Runtime decoration
- Avoid explosion of subclasses

Python Implementation:

- Function decorators (using @)
- Class decorators
- Wrapper classes

Components:

- **Component:** Abstract interface
- **Concrete Component:** Object to decorate
- **Decorator:** Wraps component
- **Concrete Decorator:** Adds responsibilities

Pros:

- More flexible than inheritance
- Responsibilities added/removed at runtime
- Combine multiple decorators
- Single Responsibility Principle

Cons:

- Many small objects
- Order matters
- Can be hard to debug

Facade Pattern

Intent: Provide unified interface to set of interfaces in subsystem. Higher-level interface makes subsystem easier to use.

Use Cases:

- Simplify complex systems
- Layer architecture
- Library wrappers
- API design

Components:

- **Facade:** Simple interface to complex subsystem
- **Subsystem Classes:** Complex functionality

Pros:

- Simplifies usage
- Loose coupling
- Layered architecture
- Easier testing

Cons:

- Can become god object
- May limit functionality

Proxy Pattern

Intent: Provide surrogate or placeholder for another object to control access to it.

Types:

Virtual Proxy:

- Lazy initialization
- Create expensive object on demand

Protection Proxy:

- Access control
- Check permissions

Remote Proxy:

- Represents object in different address space
- Network communication

Cache Proxy:

- Store results
- Avoid expensive operations

Use Cases:

- Lazy loading
- Access control
- Caching
- Logging
- Remote objects

Pros:

- Control object access
- Lazy initialization
- Additional functionality transparently
- Open/Closed Principle

Cons:

- Added complexity
- Slower response (sometimes)

Composite Pattern

Intent: Compose objects into tree structures to represent part-whole hierarchies. Treat individual objects and compositions uniformly.

Use Cases:

- Tree structures
- File systems
- GUI components
- Organization hierarchies

Components:

- **Component:** Abstract interface for all objects
- **Leaf:** Individual object (no children)

- **Composite:** Container of components

Pros:

- Treat single objects and compositions uniformly
- Easy to add new component types
- Open/Closed Principle

Cons:

- Can be overly general
 - Hard to restrict components
-

6.4 Behavioral Patterns

Observer Pattern

Intent: Define one-to-many dependency between objects. When one object changes state, all dependents are notified and updated automatically.

Also Known As:

- Publish-Subscribe
- Event-Listener

Use Cases:

- Event handling systems
- MVC architecture (Model notifies View)
- Distributed event handling
- Real-time updates

Components:

- **Subject:** Maintains list of observers, notifies on change
- **Observer:** Interface for objects to be notified
- **Concrete Subject:** Stores state, sends notifications
- **Concrete Observer:** Maintains reference to subject, implements update

Pros:

- Loose coupling
- Dynamic relationships (add/remove observers at runtime)

- Broadcast communication
- Open/Closed Principle

Cons:

- Observers notified in random order
- Memory leaks if not unsubscribed
- Can cause unexpected updates

Strategy Pattern

Intent: Define family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets algorithm vary independently from clients.

Use Cases:

- Multiple algorithms for same task
- Avoid conditional statements
- Different variants of algorithm
- Runtime algorithm selection

Components:

- **Strategy:** Interface for algorithm
- **Concrete Strategy:** Implements algorithm
- **Context:** Uses Strategy interface

Pros:

- Swap algorithms at runtime
- Isolate algorithm implementation
- Replace inheritance with composition
- Open/Closed Principle

Cons:

- Increased number of classes
- Clients must be aware of strategies
- Communication overhead

Command Pattern

Intent: Encapsulate request as object. Allows parameterization of clients with different requests, queue requests, and support undoable operations.

Use Cases:

- Undo/Redo functionality
- Macro recording
- Transaction systems
- Job queues
- GUI buttons/menu items

Components:

- **Command:** Interface for executing operation
- **Concrete Command:** Implements execute, binds receiver and action
- **Invoker:** Asks command to execute
- **Receiver:** Performs actual work
- **Client:** Creates and configures commands

Pros:

- Decouples invoker from receiver
- Easy to add new commands (Open/Closed)
- Supports undo/redo
- Supports command composition
- Deferred execution

Cons:

- Increased number of classes
- Complexity for simple operations

Template Method Pattern

Intent: Define skeleton of algorithm in base class, but let subclasses override specific steps without changing algorithm structure.

Use Cases:

- Common algorithm structure, varying steps
- Frameworks (define extension points)
- Code reuse
- Inversion of control

Components:

- **Abstract Class:** Defines template method and abstract steps
- **Concrete Class:** Implements abstract steps

Template Method:

- Final method in base class
- Calls abstract and concrete methods
- Defines algorithm structure

Pros:

- Code reuse
- Control over algorithm structure
- Hooks for customization

Cons:

- Limited flexibility
- Violates Liskov Substitution (sometimes)
- Template can become complex

Iterator Pattern

Intent: Provide way to access elements of aggregate object sequentially without exposing underlying representation.

Use Cases:

- Traverse collections
- Hide internal structure
- Support multiple traversals
- Uniform interface for different structures

Python Built-in Support:

- `iter__()` method
- `next__()` method
- `for` loop uses iterator protocol
- `iter()` and `next()` built-in functions

Components:

- **Iterator:** Interface with `next()` and `has_next()`
- **Concrete Iterator:** Implements traversal

- **Aggregate:** Interface for creating iterator
- **Concrete Aggregate:** Returns concrete iterator

Pros:

- Clean collection traversal
- Multiple traversals simultaneously
- Uniform interface
- Single Responsibility Principle

Cons:

- Overkill for simple collections
- Less efficient than direct access (sometimes)

State Pattern

Intent: Allow object to alter behavior when internal state changes. Object appears to change class.

Use Cases:

- Objects with state-dependent behavior
- Large conditional statements based on state
- State machines
- Game AI

Components:

- **Context:** Maintains current state, delegates to state object
- **State:** Interface for state-specific behavior
- **Concrete State:** Implements behavior for specific state

Pros:

- Organizes state-specific code
- Makes state transitions explicit
- Single Responsibility Principle
- Open/Closed Principle

Cons:

- Overkill for few states
- Increases number of classes

Chain of Responsibility Pattern

Intent: Avoid coupling sender of request to receiver by giving multiple objects a chance to handle request. Chain receiving objects and pass request along chain.

Use Cases:

- Multiple objects can handle request
- Handler not known beforehand
- Logging frameworks
- Event bubbling
- Middleware

Components:

- **Handler:** Interface for handling requests
- **Concrete Handler:** Handles request or passes to next
- **Client:** Initiates request to chain

Pros:

- Reduced coupling
- Flexibility in assigning responsibilities
- Can add/remove handlers dynamically

Cons:

- Request might go unhandled
 - Can be hard to debug
 - Performance concerns (long chain)
-

6.5 Advanced Patterns

Dependency Injection

Intent: Provide dependencies to object from outside rather than object creating them. Inversion of Control (IoC).

Types:

- **Constructor Injection:** Pass dependencies via constructor
- **Setter Injection:** Pass via setter methods
- **Interface Injection:** Client implements interface for injection

Benefits:

- Loose coupling
- Easy testing (mock dependencies)
- Better code organization
- Flexibility

Implementation in Python:

- Manual injection
- Dependency injection frameworks (injector, dependency_injector)

Repository Pattern

Intent: Mediate between domain and data mapping layers. Collection-like interface for accessing domain objects.

Use Cases:

- Data access abstraction
- Centralize data logic
- Testing (mock repositories)
- Multiple data sources

Benefits:

- Separation of concerns
- Centralized data access logic
- Easy to test
- Swap data sources

Unit of Work Pattern

Intent: Maintain list of objects affected by business transaction and coordinates writing changes. Manages transactions.

Use Cases:

- Database transactions
- Multiple repository operations
- Ensure atomicity
- Track changes

Benefits:

- Transaction management
 - Batch database updates
 - Consistency
 - Rollback on failure
-

6.6 Anti-Patterns to Avoid

God Object:

- Object knows/does too much
- Violates Single Responsibility
- Hard to maintain

Spaghetti Code:

- Unstructured, hard to follow
- Tangled dependencies
- Difficult to modify

Golden Hammer:

- Using same solution for all problems
- "If all you have is a hammer..."
- Inappropriate pattern usage

Premature Optimization:

- Optimizing before needed
- Adds complexity
- Profile first!

Cargo Cult Programming:

- Using code without understanding
 - Copy-paste without thinking
 - Ritual behavior
-

6.7 Pattern Selection Guide

Problem: Need single instance → Singleton

Problem: Create object without specifying exact class → Factory Method, Abstract Factory

Problem: Build complex object step-by-step → Builder

Problem: Clone existing object → Prototype

Problem: Incompatible interfaces → Adapter

Problem: Add functionality dynamically → Decorator

Problem: Simplify complex system → Facade

Problem: Control access to object → Proxy

Problem: Part-whole hierarchies → Composite

Problem: Notify dependent objects of changes → Observer

Problem: Swap algorithms at runtime → Strategy

Problem: Encapsulate requests → Command

Problem: Define algorithm skeleton → Template Method

Problem: Traverse collection → Iterator

Problem: State-dependent behavior → State

Problem: Pass request along chain → Chain of Responsibility

6.8 Pythonic Patterns

Duck Typing

"If it walks like a duck and quacks like a duck..."

- Don't check types, check behavior
- More flexible than inheritance
- Very Pythonic

EAFP vs LBYL

EAFP (Easier to Ask Forgiveness than Permission):

- Try operation, handle exception
- Pythonic approach

LBYL (Look Before You Leap):

- Check before operation
- More defensive

- Not Pythonic

Context Managers

Theory:

- Manage resources (files, locks, connections)
- Use `with` statement
- Implement `__enter__` and `__exit__`
- Or use `@contextmanager` decorator

Benefits:

- Guaranteed cleanup
- Exception safe
- Clean syntax

Descriptors

Theory:

- Define `__get__`, `__set__`, `__delete__`
- Control attribute access
- Power behind properties
- Advanced technique

Use Cases:

- Validation
- Lazy properties
- Type checking
- Caching

Conclusion

This comprehensive guide covers:

1. **Python Basics:** Core language features and data structures
2. **OOP:** Classes, inheritance, polymorphism, abstraction, encapsulation
3. **Multithreading:** Concurrent execution, synchronization, GIL
4. **Multiprocessing:** True parallelism, IPC, process pools

5. **Dynamic Programming:** Optimization technique, memoization, patterns
6. **Design Patterns:** Reusable solutions to common problems

Key Takeaways:

- **Master the basics** before moving to advanced topics
- **Understand theory** but focus on practical application
- **Choose the right tool** for the problem (threads vs processes)
- **Optimize when needed**, not prematurely
- **Learn patterns** but don't force them
- **Write Pythonic code** - use language idioms
- **Test your code** - especially concurrent code
- **Profile before optimizing** - measure don't guess
- **Document your code** - explain why, not what
- **Keep learning** - Python ecosystem is vast

Practice Resources:

- LeetCode / HackerRank for algorithms and DP
- Real projects for OOP and design patterns
- Concurrent applications for threading/multiprocessing
- Open source contributions
- Code reviews
- Reading others' code

Next Steps:

1. Implement examples from each section
2. Build projects using these concepts
3. Read others' code using these patterns
4. Refactor existing code applying learned principles
5. Explore advanced topics (async/await, metaclasses, etc.)

Happy coding! 

Complete NLP Cheat Sheet

Table of Contents

1. [NLP Fundamentals](#)
 2. [Text Preprocessing](#)
 3. [Text Representation](#)
 4. [Classical NLP Techniques](#)
 5. [Deep Learning for NLP](#)
 6. [Transformer Architecture](#)
 7. [BERT Model](#)
 8. [T5 Model](#)
 9. [GPT Models](#)
 10. [Advanced Architectures](#)
 11. [Common NLP Tasks](#)
 12. [Evaluation Metrics](#)
 13. [Prompt Engineering](#)
 14. [Fine-tuning Strategies](#)
 15. [NLP Challenges](#)
 16. [Tools and Libraries](#)
-

NLP Fundamentals

Natural Language Processing (NLP) is the field of AI focused on enabling computers to understand, interpret, and generate human language.

Core Concepts

- **Corpus:** A large collection of text documents
- **Token:** Individual unit of text (word, subword, character)
- **Vocabulary:** Set of all unique tokens in a corpus
- **Embeddings:** Dense vector representations of text
- **Context:** Surrounding words that give meaning to a token
- **Semantic Similarity:** Measure of meaning similarity between texts
- **Syntactic Analysis:** Understanding grammatical structure
- **Pragmatics:** Understanding intended meaning in context

NLP Pipeline Stages

1. **Data Collection:** Gathering text data from various sources
2. **Text Preprocessing:** Cleaning and normalizing text
3. **Feature Extraction:** Converting text to numerical representations
4. **Model Training:** Learning patterns from data
5. **Evaluation:** Measuring model performance
6. **Deployment:** Putting model into production
7. **Monitoring:** Tracking performance over time

Types of NLP

Rule-Based NLP:

- Uses hand-crafted linguistic rules
- Regular expressions, grammar rules
- Deterministic, interpretable
- Limited scalability

Statistical NLP:

- Uses probabilistic models
- N-grams, Hidden Markov Models
- Learns from data
- Better generalization

Neural NLP:

- Uses deep learning models
- Learns complex patterns
- State-of-the-art performance
- Requires large datasets

Hybrid Approaches:

- Combines multiple methods
 - Leverages strengths of each
 - Common in production systems
-

Text Preprocessing

Tokenization

Breaking text into smaller units (tokens).

Types:

- **Word Tokenization:** "Hello world" → ["Hello", "world"]
- **Subword Tokenization:** "unhappiness" → ["un", "happiness"]
- **Character Tokenization:** "cat" → ["c", "a", "t"]
- **Sentence Tokenization:** Splitting text into sentences

Common Preprocessing Steps

1. **Lowercasing:** Convert all text to lowercase
2. **Removing punctuation:** Strip special characters
3. **Removing stop words:** Filter common words (the, is, at)
4. **Stemming:** Reduce words to root form (running → run)
5. **Lemmatization:** Reduce to dictionary form (better → good)
6. **Removing numbers/URLs/emails:** Clean irrelevant data
7. **Handling contractions:** can't → cannot
8. **Unicode normalization:** Handle different character encodings
9. **Spell correction:** Fix typos and misspellings
10. **HTML/XML tag removal:** Clean web-scraped data

Advanced Tokenization Methods

- **Byte Pair Encoding (BPE):** Merges frequent character pairs iteratively
- **WordPiece:** Used by BERT, similar to BPE
- **SentencePiece:** Language-independent tokenization
- **Unigram:** Probabilistic tokenization method
- **BPE-Dropout:** Adds randomness for robustness

Text Normalization Techniques

Case Folding:

- **Lowercase:** Standard approach
- **Truecasing:** Preserve proper nouns
- **Mixed:** Task-dependent

Noise Removal:

- HTML tags and entities
- Special characters
- Repeated characters (sooo → so)
- Emojis (keep or remove based on task)

Standardization:

- Date formats
 - Number formats
 - Abbreviations expansion
 - Slang normalization
-

Text Representation

Traditional Methods

1. One-Hot Encoding

- Binary vector with 1 at token's index
- Size = vocabulary size
- No semantic information
- Sparse representation
- Example: "cat" = [0,0,1,0,0,...,0]

2. Bag of Words (BoW)

- Count frequency of each word
- Ignores word order and grammar
- Sparse representation
- Simple but effective baseline

3. TF-IDF (Term Frequency-Inverse Document Frequency)

- Weights words by importance
- $TF = (\text{word count in document}) / (\text{total words in document})$
- $IDF = \log(\text{total documents} / \text{documents containing word})$
- $TF-IDF = TF \times IDF$

- Reduces weight of common words
- Increases weight of rare, distinctive words

4. N-gram Features

- Captures local word order
- Bigrams, trigrams, etc.
- Better than BoW for some tasks
- Vocabulary size grows exponentially

Word Embeddings

Word2Vec (2013)

Two architectures:

- **CBOW (Continuous Bag of Words)**: Predict word from context
- **Skip-gram**: Predict context from word

Training Methods:

- Negative sampling
- Hierarchical softmax

Properties:

- Creates dense vectors (100-300 dimensions)
- Captures semantic meaning
- Linear relationships: $\text{king} - \text{man} + \text{woman} \approx \text{queen}$
- Similar words have similar vectors
- Trained on large corpora (Google News, Wikipedia)

GloVe (Global Vectors, 2014)

- Based on word co-occurrence statistics
- Combines global matrix factorization with local context
- Pre-trained vectors available (6B, 42B, 840B tokens)
- Captures both semantic and syntactic relationships

FastText (2016)

- Extension of Word2Vec
- Represents words as bag of character n-grams

- Handles out-of-vocabulary words better
- Good for morphologically rich languages
- Can generate embeddings for unseen words

Contextual Embeddings

ELMo (Embeddings from Language Models, 2018)

- Bidirectional LSTM-based
- Character-level inputs
- Word representations vary by context
- Pre-trained on large corpus
- Can be used as features in downstream tasks

CoVe (Contextualized Word Vectors, 2017)

- Trained using machine translation
- Transfer learning from MT to other tasks

BERT, GPT, and Transformer-based:

- Deep contextual representations
- Pre-trained on massive datasets
- Fine-tunable for specific tasks
- State-of-the-art across many benchmarks

Sentence and Document Embeddings

Doc2Vec (Paragraph Vectors)

- Extension of Word2Vec to documents
- Two variants: PV-DM and PV-DBOW
- Fixed-length vector for variable-length text

Sentence-BERT (SBERT)

- Uses Siamese/triplet networks
- Efficient semantic similarity computation
- Good for clustering and retrieval

Universal Sentence Encoder

- Trained on multiple tasks

- Transformer or DAN architecture
- Available via TensorFlow Hub

InferSent

- Trained on natural language inference
 - Captures sentence semantics
-

Classical NLP Techniques

N-grams and Language Models

Sequence of N consecutive tokens.

- **Unigram:** Single word ("dog")
- **Bigram:** Two words ("happy birthday")
- **Trigram:** Three words ("New York City")
- **4-gram, 5-gram:** Higher order n-grams

Language Models:

- Assign probability to sequences
- $P(\text{sentence}) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1, w_2) \times \dots$
- Markov assumption for simplification
- Used in speech recognition, MT, text generation

Smoothing Techniques:

- Laplace (add-one) smoothing
- Good-Turing smoothing
- Kneser-Ney smoothing
- Handle unseen n-grams

Part-of-Speech (POS) Tagging

Labeling words with grammatical categories.

Common Tags:

- NN (noun), NNS (plural noun)
- VB (verb), VBD (past tense verb)

- JJ (adjective), RB (adverb)
- DT (determiner), IN (preposition)

Methods:

- Rule-based taggers
- HMM (Hidden Markov Model) taggers
- Maximum Entropy Markov Models (MEMM)
- CRF (Conditional Random Fields)
- Neural taggers (BiLSTM-CRF)

Named Entity Recognition (NER)

Identifying and classifying named entities.

Common Entities:

- PERSON: Names of people
- ORGANIZATION: Companies, agencies
- LOCATION: Cities, countries, landmarks
- DATE: Dates and times
- MONEY: Monetary values
- PERCENT: Percentages
- PRODUCT: Products and services
- EVENT: Named events

Approaches:

- Rule-based (gazetteers, patterns)
- Statistical (CRF, HMM)
- Neural (BiLSTM-CRF, BERT-based)

Annotation Schemes:

- IOB (Inside, Outside, Beginning)
- IOBES (IOB + End, Single)

Syntactic Parsing

Constituency Parsing:

- Breaks sentence into nested constituents

- Creates parse trees with phrase structure
- NP (Noun Phrase), VP (Verb Phrase), etc.

Dependency Parsing:

- Shows grammatical relationships between words
- Creates directed graph
- Relations: nsubj (subject), dobj (object), amod (modifier)

Methods:

- Probabilistic Context-Free Grammars (PCFG)
- Transition-based parsing
- Graph-based parsing
- Neural parsers (BiLSTM, Transformers)

Semantic Analysis

Word Sense Disambiguation (WSD):

- Determine correct meaning of polysemous words
- "bank" → financial institution vs. river bank
- Knowledge-based or supervised learning

Semantic Role Labeling (SRL):

- Identifies predicate-argument structure
- Who did what to whom, when, where, why
- Frame-based representations

Coreference Resolution:

- Identify when different expressions refer to same entity
- "John went to store. He bought milk." (He = John)
- Mention detection + mention clustering

Topic Modeling

Latent Dirichlet Allocation (LDA):

- Probabilistic generative model
- Documents as mixtures of topics
- Topics as distributions over words

- Unsupervised learning

Non-negative Matrix Factorization (NMF):

- Matrix factorization approach
- Document-term matrix $\rightarrow$ document-topic $\times$ topic-term
- Interpretable topics

Latent Semantic Analysis (LSA):

- SVD on document-term matrix
 - Dimensionality reduction
 - Captures latent semantic structure
-

Deep Learning for NLP

Recurrent Neural Networks (RNN)

- Process sequential data one token at a time
- Maintain hidden state to capture context
- Share parameters across time steps
- **Problem:** Vanishing/exploding gradients
- Difficulty learning long-term dependencies

Equations:

- $h_t = \tanh(W_{hh} \times h_{t-1} + W_{xh} \times x_t + b_h)$
- $y_t = W_{hy} \times h_t + b_y$

Long Short-Term Memory (LSTM)

- Special RNN architecture with gates
- **Forget gate:** Decides what to discard from cell state
- **Input gate:** Decides what new information to add
- **Output gate:** Decides what to output
- Better at capturing long-term dependencies
- Solves vanishing gradient problem

Bidirectional LSTM:

- Processes sequence in both directions
- Forward LSTM + Backward LSTM
- Combines information from past and future
- Better context understanding

Gated Recurrent Unit (GRU)

- Simpler variant of LSTM
- Two gates: reset and update
- Fewer parameters than LSTM
- Faster training
- Often similar performance to LSTM

Convolutional Neural Networks (CNN) for Text

Architecture:

- 1D convolutions over text
- Multiple filter sizes (3, 4, 5 grams)
- Max pooling to extract features
- Fully connected layers for classification

Advantages:

- Captures local patterns
- Parallel computation (faster than RNNs)
- Good for classification tasks
- Effective feature extraction

Applications:

- Sentence classification
- Text categorization
- Relation extraction

Attention Mechanism

Key Concepts:

- Allows model to focus on relevant parts of input
- Computes weighted sum of input representations

- Weights learned during training
- Dynamic context vectors

Types:

1. Additive Attention (Bahdanau):

- $\text{score}(h_t, s_i) = v^T \tanh(W_1 h_t + W_2 s_i)$

2. Multiplicative Attention (Luong):

- $\text{score}(h_t, s_i) = h_t^T W s_i$

3. Scaled Dot-Product Attention:

- $\text{score}(Q, K) = (Q \times K^T) / \sqrt{d_k}$
- Used in Transformers

Self-Attention:

- Attention over same sequence
- Each element attends to all elements
- Captures dependencies within sequence

Sequence-to-Sequence Models

Encoder-Decoder Architecture:

- Encoder: Processes input sequence $\rightarrow$ context vector
- Decoder: Generates output sequence from context
- Used for translation, summarization, etc.

With Attention:

- Decoder attends to encoder hidden states
 - Different parts of input for each output
 - Solves bottleneck problem
 - Significantly improves performance
-

Transformer Architecture

Overview

The Transformer (2017, "Attention Is All You Need") revolutionized NLP by relying entirely on attention mechanisms, eliminating recurrence.

Key Components

1. Self-Attention Mechanism

Allows each token to attend to all other tokens in the sequence.

Process:

1. Create Query (Q), Key (K), Value (V) matrices from input embeddings
2. Compute attention scores: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$
3. Each token gets context-aware representation

Intuition:

- Query: What am I looking for?
- Key: What do I contain?
- Value: What do I actually communicate?

Why scaling by $\sqrt{d_k}$?

- Prevents dot products from becoming too large
- Keeps gradients stable
- Improves training dynamics

Why it works:

- Captures long-range dependencies efficiently
- Parallel computation (unlike RNNs)
- Direct connections between all positions
- No gradient vanishing across long distances

2. Multi-Head Attention

Runs multiple self-attention operations in parallel.

Benefits:

- Capture different types of relationships

- Focus on different representation subspaces
- Different heads learn different patterns
- Richer representations

Process:

1. Project Q, K, V into h different subspaces
2. Apply attention in each subspace
3. Concatenate results
4. Project back to original dimension

Formula: $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$ where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

3. Positional Encoding

Since Transformers process all tokens simultaneously, they need explicit position information.

Sinusoidal Positional Encoding:

- $\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$
- $\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$
- Deterministic, no parameters
- Can extrapolate to longer sequences

Learned Positional Embeddings:

- Trained parameters
- Fixed maximum sequence length
- Often used in practice

Relative Position Encodings:

- Encode relative distances between tokens
- More flexible
- Better generalization

4. Feed-Forward Networks

Applied to each position independently and identically.

Architecture:

- Two linear transformations with activation

- $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$
- Usually: $d_{\text{model}} \rightarrow 4 \times d_{\text{model}} \rightarrow d_{\text{model}}$
- ReLU or GELU activation

Purpose:

- Add non-linearity
- Process each position's representation
- Increase model capacity

5. Layer Normalization & Residual Connections

Layer Normalization:

- Normalize across features for each example
- $\text{LayerNorm}(x) = \gamma(x - \mu) / \sigma + \beta$
- Stabilizes training
- Reduces internal covariate shift

Residual Connections:

- Add input to output: $x + \text{Sublayer}(x)$
- Enables gradient flow
- Allows training very deep networks
- Identity mapping when needed

Combined (Add & Norm):

- $\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$
- Standard in Transformers
- Two variants: Pre-LN and Post-LN

Encoder-Decoder Architecture

Encoder:

- Stack of N identical layers (typically 6-12)
- Each layer has two sub-layers:
 1. Multi-head self-attention
 2. Position-wise feed-forward network
- Residual connections and layer normalization
- Processes entire input in parallel

- Outputs contextualized representations

Decoder:

- Stack of N identical layers
- Each layer has three sub-layers:
 1. Masked multi-head self-attention
 2. Multi-head attention over encoder output
 3. Position-wise feed-forward network
- Masked attention prevents looking at future positions
- Autoregressive generation
- Cross-attention connects to encoder

Information Flow:

1. Input → Encoder → Context representations
2. Previous outputs → Decoder self-attention
3. Decoder attends to encoder outputs
4. Generate next token
5. Repeat autoregressively

Transformer Variants by Architecture

Encoder-only (Autoencoding):

- BERT, RoBERTa, ALBERT, DeBERTa
- Bidirectional context
- Best for understanding tasks
- Classification, NER, QA (extractive)

Decoder-only (Autoregressive):

- GPT series, LLaMA, Mistral, Falcon
- Unidirectional (left-to-right)
- Best for generation tasks
- Text completion, creative writing, code

Encoder-Decoder (Sequence-to-Sequence):

- T5, BART, mBART, mT5
- Both bidirectional and autoregressive

- Best for transformation tasks
- Translation, summarization, QA (generative)

Advanced Transformer Variants

Longformer:

- Efficient attention for long documents
- Local + global attention
- Linear complexity in sequence length

BigBird:

- Sparse attention patterns
- Random, window, and global attention
- Handles sequences up to 4096 tokens

Reformer:

- Locality-sensitive hashing for attention
- Reversible layers
- Reduces memory requirements

Linformer:

- Low-rank approximation of attention
- Linear complexity
- Maintains performance

Performers:

- Fast attention via orthogonal random features
 - Linear time and space complexity
 - FAVOR+ algorithm
-

BERT Model

BERT: Bidirectional Encoder Representations from Transformers (2018)

Architecture:

- Encoder-only Transformer
- BERT-Base: 12 layers, 768 hidden size, 12 attention heads, 110M parameters
- BERT-Large: 24 layers, 1024 hidden size, 16 attention heads, 340M parameters
- Max sequence length: 512 tokens

Key Innovation: Bidirectional Context

Unlike previous models (like GPT) that read text left-to-right or right-to-left, BERT reads in both directions simultaneously, creating deeper understanding of context.

Comparison:

- GPT: Unidirectional (left-to-right)
- ELMo: Shallow bidirectional (concatenation)
- BERT: Deep bidirectional (joint conditioning)

Pre-training Tasks

1. Masked Language Modeling (MLM)

Process:

- Randomly mask 15% of tokens in input
- Predict masked tokens using bidirectional context
- Example: "The [MASK] sat on the mat" → predict "cat"
- Forces model to understand context from both directions

Masking Strategy:

- 80%: Replace with [MASK] token
- 10%: Replace with random word
- 10%: Keep unchanged

Why mixed strategy?

- [MASK] token doesn't appear during fine-tuning
- Random words force model to rely on context
- Unchanged words reduce mismatch

2. Next Sentence Prediction (NSP)

Process:

- Given sentence pairs (A, B)

- Predict if B actually follows A (IsNext vs NotNext)
- 50% positive pairs, 50% random pairs

Purpose:

- Understand sentence relationships
- Useful for QA and NLI tasks
- Later shown to be less important (RoBERTa removed it)

Special Tokens

- **[CLS]**: Classification token (start of sequence)
 - Final state used for classification
 - Aggregates sequence information
- **[SEP]**: Separator between sentences
- **[MASK]**: Masked token placeholder
- **[PAD]**: Padding for batch processing

Input Representation

BERT combines three types of embeddings:

1. **Token Embeddings**: WordPiece vocabulary (30,000 tokens)
2. **Segment Embeddings**: Sentence A vs Sentence B
3. **Position Embeddings**: Absolute position (0-511)

Total Input = Token Embedding + Segment Embedding + Position Embedding

Pre-training Details

Data:

- BooksCorpus (800M words)
- English Wikipedia (2,500M words)
- Total: 3.3 billion words

Training:

- 1 million steps
- Batch size: 256 sequences
- Sequence length: 512 tokens
- Learning rate: 1e-4

- Warmup: 10,000 steps
- Hardware: 4-16 Cloud TPUs

Fine-tuning BERT

BERT uses a two-stage process:

1. **Pre-training:** Learn general language understanding
2. **Fine-tuning:** Adapt to specific task

Single Sentence Classification:

- Input: [CLS] sentence [SEP]
- Use [CLS] representation
- Add classification layer
- Tasks: Sentiment analysis, spam detection

Sentence Pair Classification:

- Input: [CLS] sentence A [SEP] sentence B [SEP]
- Use [CLS] representation
- Tasks: Natural Language Inference, paraphrase detection

Question Answering (SQuAD):

- Input: [CLS] question [SEP] passage [SEP]
- Predict start and end positions of answer
- Two output vectors: start scores, end scores
- Answer = span from start to end

Named Entity Recognition:

- Token-level classification
- Each token gets a label
- Labels: B-PER, I-PER, B-ORG, I-ORG, O, etc.
- Use representation of each token

Fine-tuning Strategy:

- Usually 2-4 epochs
- Small learning rate (2e-5, 3e-5, 5e-5)
- Batch size: 16 or 32

- Fast convergence

BERT Variants and Improvements

RoBERTa (Robustly Optimized BERT, 2019):

- Removed NSP task
- Dynamic masking (different each epoch)
- Larger batches and learning rate
- More training data (160GB text)
- Trained longer
- Consistently better than BERT

ALBERT (A Lite BERT, 2019):

- Factorized embedding parameterization
- Cross-layer parameter sharing
- Sentence-order prediction (SOP) instead of NSP
- 18x fewer parameters than BERT-large
- Similar or better performance

DistilBERT (2019):

- Knowledge distillation from BERT
- 40% fewer parameters
- 60% faster
- Retains 97% of BERT's performance
- Good for resource-constrained environments

ELECTRA (2020):

- Replaced token detection instead of MLM
- Discriminator vs generator setup
- More sample-efficient
- Better performance with less compute
- Particularly good for smaller models

DeBERTa (Decoding-enhanced BERT, 2020):

- Disentangled attention mechanism
- Enhanced mask decoder

- Virtual adversarial training
- State-of-the-art on SuperGLUE

SpanBERT (2019):

- Masks contiguous random spans
- Span boundary objective
- Better for span-based tasks
- Improved on QA and coreference

ERNIE (Enhanced Representation through Knowledge):

- Masks entities and phrases
- Incorporates knowledge graphs
- Better world knowledge

Multilingual BERT

mBERT (Multilingual BERT):

- Trained on 104 languages
- Shared vocabulary (110k WordPieces)
- Zero-shot cross-lingual transfer
- Not explicitly aligned across languages

XLNet-RoBERTa:

- Trained on 100 languages
- 2.5TB CommonCrawl data
- Better cross-lingual performance
- No language embeddings needed

Advantages of BERT

- Deep bidirectional context understanding
- Transfer learning: pre-train once, fine-tune many times
- State-of-the-art on 11 NLP tasks at release
- Handles polysemy (context-dependent meanings)
- Relatively easy to fine-tune
- Strong zero-shot and few-shot capabilities

Limitations

- Computationally expensive (large models)
- Fixed maximum sequence length (512 tokens)
- Cannot generate text fluently (encoder-only)
- Pre-training is very resource-intensive
- Static vocabulary (can't handle new words well)
- Independence assumption in MLM

Applications

- Search engines (Google Search uses BERT)
 - Question answering systems
 - Sentiment analysis
 - Text classification
 - Named entity recognition
 - Information extraction
 - Document ranking
 - Chatbots and dialogue systems
-

T5 Model

T5: Text-to-Text Transfer Transformer (2019)

Core Philosophy: Every NLP task is framed as a text-to-text problem.

Key Insight: Unified framework allows using the same model, loss function, and hyperparameters for all tasks.

Unified Text-to-Text Framework

T5 converts all tasks to the same format:

- **Input:** Text prompt describing the task + input
- **Output:** Text response

Examples:

- **Translation:** "translate English to German: Hello" → "Hallo"
- **Summarization:** "summarize: [article text]" → "[summary]"
- **Classification:** "sst2 sentence: I love this movie!" → "positive"

- **Question Answering:** "question: What is NLP? context: [text]" → "[answer]"
- **Regression:** "stsb sentence1: X sentence2: Y" → "4.5"
- **Coreference:** "wsc: [text with pronouns]" → "True" or "False"

Architecture

Encoder-Decoder Transformer:

- Based on original Transformer architecture
- Relative position embeddings instead of absolute
- Simplified layer normalization (no bias, no subtraction)
- Encoder and decoder have same structure

Model Sizes:

- **T5-Small:** 60M parameters (8 layers, 512 hidden)
- **T5-Base:** 220M parameters (12 layers, 768 hidden)
- **T5-Large:** 770M parameters (24 layers, 1024 hidden)
- **T5-3B:** 3 billion parameters
- **T5-11B:** 11 billion parameters

Scaling: Performance improves consistently with model size.

Pre-training Approach

Unsupervised Objective: Span Corruption

Process:

1. Corrupt random spans of text (avg. length 3 tokens)
2. Replace each span with unique sentinel token
3. Model predicts all corrupted spans

Example:

- Original: "Thank you for inviting me to your party last week"
- Corrupted: "Thank you <X> to <Y> last week"
- Target: "<X> for inviting me <Y> your party <Z>"

Why span corruption?

- More challenging than single token prediction
- Forces understanding of longer context

- Better than random token replacement
- Balances between reconstruction and generation

C4 Dataset (Colossal Clean Crawled Corpus):

- 750 GB of clean English web text
- Filtered from Common Crawl
- Removed duplicates, incomplete sentences
- Cleaned markup and offensive content

Multi-Task Training

Supervised Multi-Task Learning:

- Trains on diverse tasks simultaneously
- Tasks share parameters
- Improves generalization

Task Mixture:

- Translation (WMT datasets)
- Summarization (CNN/Daily Mail, XSum)
- Question Answering (SQuAD, Natural Questions)
- Classification (GLUE tasks)
- Common sense reasoning (COPA, WSC)

Mixing Strategy:

- Examples-proportional mixing
- Temperature-based sampling
- Equal mixing across tasks
- Best: proportional with temperature

Key Innovations

1. Comprehensive Ablation Study

T5 paper systematically compared:

- Architectures (encoder-decoder vs decoder-only vs prefix-LM)
- Pre-training objectives (MLM, prefix-LM, deshuffling)
- Datasets (C4, Wikipedia, WebText)

- Transfer approaches (fine-tuning, adapter layers)
- Scaling (model size, data size, training steps)

Findings:

- Encoder-decoder works best for most tasks
- Denoising objectives (like span corruption) are effective
- Scaling model and data improves performance
- Pre-training on in-domain data helps

2. Task Prefixes for Multi-Task Learning

Examples:

- "translate English to French:"
- "summarize:"
- "cola sentence:"
- "stsb sentence1: ... sentence2: ..."

Benefits:

- Explicit task identification
- No task-specific parameters needed
- Easy to add new tasks
- Supports zero-shot learning

3. Transfer Learning Framework

Pre-training:

- Unsupervised learning on massive corpus
- Learn general language understanding

Multi-task fine-tuning:

- Supervised learning on multiple tasks
- Improves generalization

Task-specific fine-tuning:

- Adapt to specific downstream task
- Often achieves best performance

Fine-tuning Strategies

Supervised Fine-tuning:

- Provide task-specific training examples
- Model learns task from examples
- Standard approach for best performance

Multi-task Fine-tuning:

- Fine-tune on multiple tasks together
- Better generalization
- Useful when tasks are related

Few-shot Learning:

- Provide few examples in prompt
- No parameter updates
- Limited by context window
- Example: "translate: cat → chat, dog → chien, bird → ?"

Zero-shot Learning:

- Describe task in natural language
- No task examples
- Relies on pre-training knowledge
- Performance varies by task

Variants and Improvements

mT5 (Multilingual T5, 2020):

- Trained on 101 languages
- Uses mC4 (multilingual C4)
- 71 TB of web text
- Cross-lingual transfer learning
- Single model for all languages

ByT5 (Byte-level T5, 2021):

- Works directly with UTF-8 bytes
- No tokenization needed

- Better for noisy text
- Good for multiple languages
- More robust to typos

UL2 (Unified Language Learner, 2022):

- Combines different denoising strategies
- X-denoising (extreme spans)
- S-denoising (short spans)
- R-denoising (prefix-LM)
- Better generalization across tasks

Flan-T5 (2022):

- Instruction-tuned T5
- Fine-tuned on 1800+ tasks
- Tasks formatted as instructions
- Significantly better zero-shot performance
- Better instruction following
- Widely used for practical applications

LongT5 (2021):

- Efficient attention for long documents
- Transient global attention
- Handles sequences up to 16,384 tokens
- Good for document summarization

Advantages of T5

- Unified framework simplifies training and deployment
- Strong transfer learning across tasks
- Flexible: handles any text-to-text task
- Excellent for both generation and understanding
- Scalable: performance improves with size
- Multi-task learning improves generalization
- Easy to add new tasks

Limitations

- Computationally expensive for large variants
- Requires careful prompt engineering
- Fixed context window (512-1024 tokens typically)
- Pre-training requires massive compute
- Autoregressive decoding can be slow

Applications

- Machine translation
 - Text summarization
 - Question answering
 - Text classification
 - Paraphrasing
 - Grammar correction
 - Data-to-text generation
 - Conversational AI
-

GPT Models

GPT: Generative Pre-trained Transformer

Core Idea: Pre-train on language modeling, then fine-tune for downstream tasks.

GPT-1 (2018)

Architecture:

- Decoder-only Transformer
- 12 layers, 768 hidden size
- 117M parameters
- Unidirectional (left-to-right)

Pre-training:

- Objective: Predict next token
- Dataset: BooksCorpus (7,000 books)
- Autoregressive language modeling

Fine-tuning:

- Task-specific input transformations
- Minimal architecture changes
- Demonstrated transfer learning effectiveness

GPT-2 (2019)**Architecture:**

- Larger decoder-only Transformer
- 4 sizes: Small (117M), Medium (345M), Large (762M), XL (1.5B)
- 48 layers, 1600 hidden size (XL)
- Layer normalization moved to input of sub-blocks

Pre-training:

- Dataset: WebText (40GB, 8M web pages)
- Higher quality dataset
- Longer training

Zero-Shot Learning:

- No fine-tuning needed for many tasks
- Task conditioning via prompts
- Demonstrated emergence of capabilities

Controversial Release:

- Initially withheld due to misuse concerns
- Staged release of models
- Sparked AI safety discussions

GPT-3 (2020)**Architecture:**

- 175 billion parameters
- 96 layers, 12,288 hidden size
- 96 attention heads
- Context window: 2048 tokens

Pre-training:

- Dataset: 570GB text (filtered Common Crawl, WebText2, Books, Wikipedia)
- 300 billion tokens
- Massive computational requirements

Few-Shot Learning:

- In-context learning without gradient updates
- Provide task examples in prompt
- No fine-tuning needed

Emergent Capabilities:

- Arithmetic, code generation
- Translation, summarization
- Creative writing, reasoning
- Common sense knowledge

API-Only:

- Not open-sourced
- Available through OpenAI API
- Commercial applications

GPT-3.5 (2022)

Improvements:

- Base model + instruction tuning
- RLHF (Reinforcement Learning from Human Feedback)
- Better instruction following
- More helpful, harmless, honest

Variants:

- text-davinci-003: Instruction-tuned
- gpt-3.5-turbo: Optimized for chat
- Code-capable versions

GPT-4 (2023)

Architecture:

- Multimodal (text and images)
- Exact size not disclosed
- Rumored mixture of experts

Capabilities:

- Significantly better reasoning
- Advanced math and coding
- Image understanding
- Longer context (8K, 32K variants)
- More factual and aligned

Safety:

- Extensive RLHF training
- Red teaming for risks
- Improved refusal behavior

Key Concepts in GPT Series

Autoregressive Generation:

- Predict one token at a time
- Condition on all previous tokens
- Left-to-right generation

In-Context Learning:

- Learn tasks from examples in prompt
- No parameter updates
- Few-shot, one-shot, zero-shot

Prompt Engineering:

- Craft prompts to elicit desired behavior
- Critical for performance
- Includes instructions, examples, context

Scaling Laws:

- Performance scales with model size, data, compute
- Predictable improvements
- Motivates larger models

Comparison: BERT vs GPT

Feature	BERT	GPT
Architecture	Encoder-only	Decoder-only
Attention	Bidirectional	Unidirectional
Pre-training	MLM + NSP	Next token prediction
Best for	Understanding	Generation
Fine-tuning	Task-specific heads	Prompt-based
Generation	Poor	Excellent
Classification	Excellent	Good with prompting

Advanced Architectures

Encoder-Decoder Models

BART (Bidirectional and Auto-Regressive Transformers, 2019):

- Combines BERT and GPT ideas
- Pre-training: Corrupt document, reconstruct it
- Multiple noise functions (masking, deletion, permutation)
- Strong for generation tasks
- Excellent for summarization

mBART (Multilingual BART):

- BART for 25+ languages
- Good cross-lingual transfer
- Machine translation

PEGASUS (2020):

- Pre-trained for abstractive summarization
- Gap sentence generation objective
- Masks entire sentences
- State-of-the-art summarization

Efficient Transformers

ALBERT (A Lite BERT):

- Factorized embeddings
- Cross-layer parameter sharing
- 89% fewer parameters than BERT-large

MobileBERT:

- Compact BERT for mobile devices
- 4.3x smaller, 5.5x faster
- Knowledge distillation

TinyBERT:

- Extremely small (14.5M parameters)
- 7.5x smaller, 9.4x faster
- Two-stage distillation

Domain-Specific Models

BioBERT:

- BERT pre-trained on biomedical text
- PubMed abstracts, PMC full-text
- Better for biomedical NER, relation extraction

SciBERT:

- BERT for scientific text
- Trained on 1.14M papers
- Custom vocabulary

ClinicalBERT:

- BERT for clinical notes
- MIMIC-III dataset
- Medical concept extraction

FinBERT:

- BERT for financial text
- Sentiment analysis for finance
- Risk classification

LegalBERT:

- BERT for legal documents
- Case law, legislation
- Legal NER and classification

Retrieval-Augmented Models

RAG (Retrieval-Augmented Generation):

- Combines retrieval with generation
- Retrieve relevant documents
- Use as additional context
- Better factuality

REALM (Retrieval-Augmented Language Model):

- Neural retrieval integrated into pre-training
- End-to-end differentiable
- Improved knowledge-intensive tasks

DPR (Dense Passage Retrieval):

- Dense embeddings for retrieval
- BERT-based encoders
- Better than sparse methods (BM25)

Vision-Language Models

CLIP (Contrastive Language-Image Pre-training):

- Joint text-image embeddings

- Zero-shot image classification
- Learned from 400M image-text pairs

ALIGN:

- Similar to CLIP
- Larger scale (1.8B pairs)
- Noisy data, simple approach

DALL-E:

- Text-to-image generation
- Autoregressive Transformer
- Creative image synthesis

Code Models

Codex (GPT for Code):

- GPT-3 fine-tuned on code
- Powers GitHub Copilot
- Multi-language code generation

CodeBERT:

- BERT for programming languages
- Pre-trained on code-comment pairs
- Code search, documentation

CodeT5:

- T5 for code understanding and generation
 - Identifier-aware pre-training
 - Multiple programming languages
-

Common NLP Tasks

1. Text Classification

Sentiment Analysis:

- Classify text by sentiment (positive, negative, neutral)

- Fine-grained (1-5 stars) or coarse (binary)
- Applications: Product reviews, social media monitoring

Topic Classification:

- Assign topics/categories to documents
- Multi-class or multi-label
- Applications: News categorization, email routing

Intent Detection:

- Identify user's intent in text
- Critical for chatbots, voice assistants
- Often combined with slot filling

Spam Detection:

- Classify messages as spam or not
- Email filtering, comment moderation

Emotion Detection:

- Classify emotions (joy, anger, sadness, fear, etc.)
- More fine-grained than sentiment

2. Sequence Labeling

Named Entity Recognition (NER):

- Identify and classify named entities
- Person, organization, location, date, etc.
- IOB tagging scheme

Part-of-Speech (POS) Tagging:

- Label each word with grammatical role
- Noun, verb, adjective, etc.

Chunking:

- Identify phrases (noun phrases, verb phrases)
- Shallow parsing

Slot Filling:

- Extract structured information
- Common in dialogue systems
- Example: Extract date, time, location from booking request

3. Sequence-to-Sequence Tasks

Machine Translation:

- Translate text between languages
- Neural MT (Transformer-based) is state-of-the-art
- Challenges: low-resource languages, domain adaptation

Text Summarization:

- **Extractive:** Select important sentences
- **Abstractive:** Generate new summary
- Single-document or multi-document

Paraphrasing:

- Rewrite text with same meaning
- Style transfer (formal ↔ casual)
- Simplification

Data-to-Text Generation:

- Generate text from structured data
- Tables, knowledge graphs, databases
- Applications: Report generation, descriptions

4. Question Answering

Extractive QA:

- Answer is span in given text
- SQuAD-style tasks
- BERT-based models excel

Generative QA:

- Generate answer from knowledge

- May not be in source text
- GPT-style models

Open-Domain QA:

- Answer questions about anything
- Requires retrieval + reading
- RAG models

Multi-Hop QA:

- Requires reasoning over multiple facts
- HotpotQA, ComplexWebQuestions
- Challenging for current models

Conversational QA:

- QA in dialogue context
- Coreference resolution needed
- Follow-up questions

5. Text Generation

Story Generation:

- Creative writing, narratives
- GPT models excel
- Coherence, creativity challenges

Dialogue Generation:

- Chatbots, virtual assistants
- Context tracking, personality
- Open-domain vs task-oriented

Code Generation:

- Generate code from natural language
- Codex, CodeT5
- Applications: Programming assistants

Poetry Generation:

- Generate poems with style, meter, rhyme
- GPT-2/3 show surprising creativity

6. Information Extraction

Relation Extraction:

- Identify relationships between entities
- "John works at Microsoft" → (John, works_at, Microsoft)
- Knowledge base construction

Event Extraction:

- Identify events and participants
- Who did what, when, where, why
- Temporal reasoning

Knowledge Graph Construction:

- Build structured knowledge from text
- Entity linking, relation extraction
- Applications: Search, QA

7. Information Retrieval

Document Ranking:

- Rank documents by relevance
- Search engines
- BM25, BERT-based rankers

Semantic Search:

- Find semantically similar content
- Dense retrieval (SBERT, DPR)
- Beyond keyword matching

Passage Retrieval:

- Find relevant passages for QA
- First stage of open-domain QA
- Dense vs sparse retrieval

8. Text Matching

Natural Language Inference (NLI):

- Determine relationship between sentences
- Entailment, contradiction, neutral
- MNLI, SNLI datasets

Semantic Similarity:

- Measure meaning similarity
- Regression or classification
- STS-B dataset

Paraphrase Detection:

- Determine if sentences have same meaning
- Binary classification
- QQP, MRPC datasets

9. Other Tasks

Text Simplification:

- Rewrite complex text to be simpler
- Preserve meaning, reduce complexity
- Applications: Accessibility, education

Grammatical Error Correction:

- Identify and correct grammar errors
- Spelling, syntax, style
- Seq2seq models

Text Style Transfer:

- Change style while preserving content
- Formal ↔ informal
- Sentiment flipping

Coreference Resolution:

- Identify when expressions refer to same entity

- Crucial for understanding
- Difficult task

Temporal Reasoning:

- Understand and reason about time
- Event ordering, duration
- Challenging for current models

Common Sense Reasoning:

- Tasks requiring world knowledge
 - COPA, HellaSwag, WinoGrande
 - Still challenging despite large models
-

Evaluation Metrics

Classification Metrics

Accuracy: $(TP + TN) / (TP + TN + FP + FN)$

- Simple, intuitive
- Misleading for imbalanced data

Precision: $TP / (TP + FP)$

- Of predicted positives, how many correct?
- Important when false positives costly

Recall (Sensitivity): $TP / (TP + FN)$

- Of actual positives, how many found?
- Important when false negatives costly

F1 Score: $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

- Harmonic mean of precision and recall
- Balanced measure

F-beta Score: $(1 + \beta^2) \times (\text{Precision} \times \text{Recall}) / (\beta^2 \times \text{Precision} + \text{Recall})$

- $\beta > 1$: Favor recall

- $\beta < 1$: Favor precision

Macro F1: Average F1 across all classes

- Equal weight to each class
- Good for imbalanced datasets

Micro F1: Calculate F1 from total TP, FP, FN

- Weighted by class frequency
- Same as accuracy for multi-class

Matthews Correlation Coefficient (MCC):

- Correlation between predictions and truth
- Range: -1 to 1
- Good for imbalanced data

ROC-AUC:

- Area under ROC curve
- Measures classifier performance across thresholds
- Good for comparing models

Sequence Labeling Metrics

Token-Level Accuracy:

- Percentage of correctly labeled tokens
- May not reflect entity-level performance

Entity-Level F1:

- Precision/recall for complete entities
- Strict: All tokens must match
- Relaxed: Partial credit for overlap

Span-Level F1:

- For NER, chunking
- Counts exact span matches

Language Generation Metrics

BLEU (Bilingual Evaluation Understudy):

- Measures n-gram overlap with references
- BLEU-1, BLEU-2, ..., BLEU-4
- Range: 0-100 (higher better)
- Common for machine translation
- **Limitation:** Doesn't capture meaning, rewards literal matches

ROUGE (Recall-Oriented Understudy for Gisting Evaluation):

- ROUGE-N: N-gram overlap
- ROUGE-L: Longest common subsequence
- ROUGE-W: Weighted LCS
- Common for summarization
- Recall-focused

METEOR:

- Considers synonyms, stemming, paraphrasing
- Better correlation with human judgment
- Precision-recall harmonic mean

CIDEr (Consensus-based Image Description Evaluation):

- TF-IDF weighted n-gram matching
- Multiple references
- Common for image captioning

SPICE:

- Semantic propositional content
- Graph-based matching
- Captures objects, attributes, relations

BERTScore:

- Uses BERT embeddings to compare texts
- Token-level semantic similarity
- Better correlation with human judgment

- More robust than n-gram metrics

BLEURT:

- Learned metric using BERT
- Trained on human ratings
- Better than BLEU for correlation

Perplexity:

- How well model predicts text
- $PPL = \exp(\text{cross-entropy loss})$
- Lower is better
- Common for language models

Question Answering Metrics

Exact Match (EM):

- 1 if predicted answer exactly matches
- 0 otherwise
- Strict, may underestimate performance

F1 Score:

- Token-level F1 between prediction and reference
- More forgiving than EM
- Partial credit for overlaps

Mean Reciprocal Rank (MRR):

- For ranking tasks
- $MRR = \text{avg}(1 / \text{rank of correct answer})$
- Higher is better

Retrieval Metrics

Precision@K:

- Precision in top K results
- $P@10$ common

Recall@K:

- Recall in top K results

Mean Average Precision (MAP):

- Average precision across queries
- Considers ranking

Normalized Discounted Cumulative Gain (NDCG):

- Considers relevance levels (not just binary)
- Position-weighted
- Gold standard for ranking

Hit Rate:

- Percentage of queries with relevant result in top K

Human Evaluation

Often necessary for:

- Generation quality
- Coherence, fluency
- Factual accuracy
- Helpfulness

Methods:

- Likert scales (1-5 ratings)
- Pairwise comparison (A vs B)
- Error analysis
- A/B testing

Metrics:

- Fluency, coherence, relevance
 - Informativeness
 - Factual correctness
 - Human preference
-

Prompt Engineering

What is Prompt Engineering?

The art and science of designing inputs to language models to elicit desired outputs.

Basic Prompting Techniques

Zero-Shot Prompting:

- No examples provided
- Just task description
- Example: "Translate to French: Hello"

Few-Shot Prompting:

- Provide examples in prompt
- Model learns pattern
- Example:

Translate to French:
Hello → Bonjour
Goodbye → Au revoir
Thank you → ?

One-Shot Prompting:

- Single example
- Middle ground

Advanced Prompting Strategies

Chain-of-Thought (CoT) Prompting:

- Include reasoning steps in examples
- "Let's think step by step"
- Dramatically improves reasoning
- Especially effective for math, logic

Self-Consistency:

- Generate multiple reasoning paths
- Take majority vote

- Improves accuracy

Tree of Thoughts:

- Explore multiple reasoning branches
- Backtrack and prune
- More thorough exploration

ReAct (Reasoning + Acting):

- Interleave reasoning and actions
- For interactive tasks
- "Thought: ..., Action: ..., Observation: ..."

Least-to-Most Prompting:

- Break problem into subproblems
- Solve sequentially
- Build on previous solutions

Prompt Design Principles

Be Specific:

- Clear, detailed instructions
- Specify format, style, length
- Example: "Write a 3-sentence summary"

Provide Context:

- Background information
- Relevant details
- Constraints

Use Delimiters:

- Separate instruction from input
- Example: ""text"", ###, ---
- Helps model parse structure

Request Structured Output:

- JSON, XML, tables

- Easier to parse
- Example: "Return as JSON with keys: sentiment, confidence"

Iterative Refinement:

- Start simple, add complexity
- Test and refine
- A/B test different phrasings

Role Prompting

Assign a role to the model:

- "You are an expert Python programmer"
- "Act as a creative writing teacher"
- Influences tone, depth, perspective

Instruction Tuning

Flan, InstructGPT approach:

- Train model on instruction-following tasks
- Better zero-shot performance
- More helpful, harmless, honest

Prompt Templates

Reusable patterns:

```
Task: {task_description}  
Input: {input_text}  
Output format: {format}  
Additional constraints: {constraints}
```

Common Pitfalls

Ambiguity:

- Vague instructions lead to poor results
- Be explicit

Information Overload:

- Too much context confuses model

- Keep prompts focused

Biased Examples:

- Examples can introduce bias
- Diverse, representative examples

Prompt Injection:

- Security risk: malicious prompts
 - Sanitize user inputs
 - Use system prompts
-

Fine-tuning Strategies

Transfer Learning Paradigm

Pre-training → Fine-tuning:

1. Pre-train on large unlabeled corpus
2. Fine-tune on task-specific labeled data
3. Achieves strong performance with less data

Full Fine-tuning

Process:

- Update all model parameters
- Requires task-specific dataset
- Generally best performance

Considerations:

- Requires significant compute
- Risk of catastrophic forgetting
- One model per task

Best practices:

- Lower learning rate than pre-training (1e-5 to 5e-5)
- Few epochs (2-4 typically)
- Small batch size (8-32)

- Gradient clipping
- Warmup steps
- Early stopping

Parameter-Efficient Fine-tuning

Motivation:

- Full fine-tuning expensive
- Multiple task-specific models costly
- Share base model, swap task modules

Adapter Layers:

- Small bottleneck layers inserted in Transformer
- Only train adapters (1-4% of parameters)
- Base model frozen
- Competitive performance

LoRA (Low-Rank Adaptation):

- Decompose weight updates as low-rank matrices
- $W' = W + BA$ (B and A are low-rank)
- Train B and A only
- 0.1-1% trainable parameters
- No inference latency

Prefix Tuning:

- Prepend trainable vectors to input
- Only optimize prefix parameters
- Base model frozen
- Good for generation tasks

Prompt Tuning:

- Similar to prefix tuning
- Soft prompts (continuous vectors)
- Very few parameters

BitFit:

- Only fine-tune bias terms
- Surprisingly effective
- 0.1% of parameters

Multi-Task Learning

Joint Training:

- Train on multiple tasks simultaneously
- Shared representations
- Improved generalization
- Task interference possible

Sequential Fine-tuning:

- Fine-tune on tasks sequentially
- Intermediate task transfer
- Order matters

Curriculum Learning:

- Order tasks by difficulty
- Start easy, progress to hard
- Can improve final performance

Continual Learning

Challenge:

- Catastrophic forgetting
- Model forgets previous tasks

Strategies:

- Elastic Weight Consolidation (EWC)
- Progressive Neural Networks
- Experience Replay
- Parameter isolation

Data Augmentation

Back Translation:

- Translate to another language and back
- Generates paraphrases
- Increases diversity

Synonym Replacement:

- Replace words with synonyms
- Maintains meaning

Random Insertion/Deletion:

- Add/remove words
- Makes model more robust

Mixup:

- Interpolate examples
- Regularization effect

Contextual Word Embeddings:

- Replace words with similar contextual embeddings
- BERT-based augmentation

Regularization Techniques

Dropout:

- Randomly drop units during training
- Prevents overfitting

Weight Decay:

- L2 regularization on weights
- Prevents large weights

Label Smoothing:

- Soften target distributions
- Prevents overconfidence

Mixup/Manifold Mixup:

- Interpolate examples and labels

- Encourages smoothness

Domain Adaptation

Challenge:

- Model trained on source domain
- Deploy on target domain
- Distribution shift

Strategies:

- Continued pre-training on target domain
- Gradual unfreezing
- Domain-adversarial training
- Self-training on unlabeled target data

Few-Shot and Zero-Shot Learning

Few-Shot:

- Limited labeled examples (5-100)
- Meta-learning
- Prototypical networks
- GPT-3 style in-context learning

Zero-Shot:

- No task-specific training data
- Rely on pre-training
- Natural language task descriptions
- T5, GPT-3 approaches

NLP Challenges

Ambiguity

Lexical Ambiguity:

- Words with multiple meanings
- "bank" → financial institution vs. river bank

- Context needed to resolve

Syntactic Ambiguity:

- Multiple valid parse trees
- "I saw the man with the telescope"
- Who has the telescope?

Semantic Ambiguity:

- Unclear meaning even with syntax
- Metaphors, idioms
- "It's raining cats and dogs"

Pragmatic Ambiguity:

- Context-dependent meaning
- Sarcasm, irony
- "Great, another meeting"

Context Understanding

Long-Range Dependencies:

- Information far apart in text
- Transformers help but still limited
- Context window constraints

Coreference Resolution:

- Identifying referents
- "John went to store. He bought milk."
- Challenging across sentences

World Knowledge:

- Facts not in text
- Common sense reasoning
- Background assumptions

Multilinguality

Challenges:

- 7,000+ languages
- Most are low-resource
- Cross-lingual transfer
- Code-switching (mixing languages)

Approaches:

- Multilingual models (mBERT, XLM-R)
- Zero-shot cross-lingual transfer
- Translation-based methods

Data Challenges

Data Scarcity:

- Many domains/languages lack data
- Annotation expensive
- Transfer learning helps

Data Quality:

- Noisy web data
- Annotation errors
- Biased datasets

Data Privacy:

- Sensitive information
- GDPR, compliance
- Federated learning

Bias and Fairness

Types of Bias:

- Gender bias (associations, stereotypes)
- Racial bias
- Cultural bias
- Selection bias in data

Mitigation:

- Diverse training data
- Bias detection and measurement
- Debiasing techniques
- Fairness constraints

Explainability

Challenge:

- Neural models are black boxes
- Hard to explain predictions
- Trust, debugging, compliance

Approaches:

- Attention visualization
- Feature importance (LIME, SHAP)
- Probing tasks
- Adversarial examples

Robustness

Adversarial Examples:

- Small perturbations cause errors
- Deliberate or natural noise
- Spelling errors, typos

Out-of-Distribution:

- Data different from training
- Performance degradation
- Domain shift

Mitigation:

- Data augmentation
- Adversarial training
- Robust architectures

Evaluation

Metrics Don't Capture All:

- BLEU high doesn't mean good translation
- Human evaluation expensive
- Inter-annotator agreement

Dataset Artifacts:

- Models exploit dataset biases
- Don't generalize to real world
- Adversarial test sets needed

Computational Cost

Large Models:

- Training expensive (millions of dollars)
- Environmental impact
- Inference latency

Solutions:

- Model compression
- Distillation
- Quantization
- Efficient architectures

Common Sense and Reasoning

Challenges:

- Implicit knowledge
- Causal reasoning
- Physical reasoning
- Theory of mind

Current Limitations:

- Models lack true understanding
- Pattern matching vs. reasoning
- Brittleness on edge cases

Tools and Libraries

Python Libraries

Transformers (Hugging Face):

- Pre-trained models (BERT, GPT, T5, etc.)
- Easy fine-tuning
- Tokenizers, training, inference
- 100,000+ models on Hub
- Essential tool for modern NLP

SpaCy:

- Industrial-strength NLP
- Fast tokenization, POS, NER, parsing
- Pre-trained pipelines
- Production-ready

NLTK (Natural Language Toolkit):

- Classic NLP library
- Tokenizers, stemmers, corpora
- Educational, less efficient

Gensim:

- Topic modeling (LDA)
- Word2Vec, Doc2Vec
- Similarity queries
- Document clustering

AllenNLP:

- Research library built on PyTorch
- Pre-built models and components
- Interpretability tools

Flair:

- Simple NLP framework
- State-of-the-art models
- Easy to use
- Sequence labeling focus

FastText:

- Efficient text classification
- Word embeddings
- Subword information

Deep Learning Frameworks

PyTorch:

- Dynamic computation graphs
- Pythonic, flexible
- Preferred for research
- Strong NLP ecosystem

TensorFlow:

- Static graphs (TF 1.x), eager execution (TF 2.x)
- Production-focused
- TensorFlow Hub for models

JAX:

- NumPy + autograd + GPU/TPU
- Functional programming
- Very fast
- Growing NLP adoption

Training and Deployment

Ray:

- Distributed training
- Hyperparameter tuning
- Model serving

DeepSpeed:

- Microsoft's training optimization
- ZeRO optimizer
- Train huge models efficiently

Triton Inference Server:

- NVIDIA's serving solution
- Multi-framework support
- Optimized inference

ONNX Runtime:

- Cross-platform inference
- Model optimization
- Hardware acceleration

Datasets and Benchmarks

GLUE (General Language Understanding Evaluation):

- 9 NLU tasks
- CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, RTE, WNLI
- Leaderboard

SuperGLUE:

- Harder version of GLUE
- 8 tasks
- BoolQ, CB, COPA, MultiRC, ReCoRD, RTE, WiC, WSC

SQuAD (Stanford Question Answering Dataset):

- Extractive QA
- SQuAD 1.1: 100K questions
- SQuAD 2.0: Unanswerable questions

Natural Questions:

- Google search queries
- Wikipedia passages
- Short and long answers

CommonGen, CommonsenseQA:

- Common sense reasoning
- Challenging for models

Annotation Tools

Label Studio:

- Multi-type annotation
- NER, classification, etc.
- ML-assisted labeling

Prodigy:

- Active learning
- Efficient annotation
- SpaCy integration

Doccano:

- Open-source
- Text classification, sequence labeling
- Collaborative

Model Hubs

Hugging Face Hub:

- 100,000+ models
- Datasets, spaces (demos)
- Community-driven

TensorFlow Hub:

- Google's model repository
- Pre-trained models
- Easy integration

Papers With Code:

- Research + implementations
- Benchmarks, leaderboards

- Reproducibility

Quick Reference: Model Comparison

Model	Type	Best For	Params	Context	Released
BERT-Base	Encoder	Understanding	110M	512	2018
RoBERTa	Encoder	Understanding	125M	512	2019
GPT-2	Decoder	Generation	1.5B	1024	2019
GPT-3	Decoder	Few-shot	175B	2048	2020
T5-Base	Enc-Dec	Versatile	220M	512	2019
BART	Enc-Dec	Summarization	406M	1024	2019
XLNet	AR	Understanding	340M	512	2019
ELECTRA	Encoder	Efficient	335M	512	2020
DeBERTa	Encoder	SOTA	1.5B	512	2020

Learning Resources and Best Practices

Recommended Learning Path

Beginner:

1. Start with fundamentals (tokenization, embeddings)
2. Implement simple models (Naive Bayes, Logistic Regression)
3. Learn PyTorch/TensorFlow basics
4. Use pre-trained models (BERT for classification)
5. Practice on Kaggle competitions

Intermediate:

1. Deep dive into Transformers
2. Fine-tune models for specific tasks
3. Understand attention mechanisms
4. Experiment with different architectures

5. Read seminal papers

Advanced:

1. Implement models from scratch
2. Research novel architectures
3. Optimize for production
4. Contribute to open source
5. Stay current with latest research

Key Papers to Read

Foundational:

- "Attention Is All You Need" (Transformer, 2017)
- "BERT: Pre-training of Deep Bidirectional Transformers" (2018)
- "Language Models are Unsupervised Multitask Learners" (GPT-2, 2019)
- "Exploring the Limits of Transfer Learning with T5" (2019)
- "Language Models are Few-Shot Learners" (GPT-3, 2020)

Word Embeddings:

- "Efficient Estimation of Word Representations" (Word2Vec, 2013)
- "GloVe: Global Vectors for Word Representation" (2014)
- "Enriching Word Vectors with Subword Information" (FastText, 2016)

Important Variants:

- "RoBERTa: A Robustly Optimized BERT Pretraining Approach" (2019)
- "ALBERT: A Lite BERT for Self-supervised Learning" (2019)
- "DeBERTa: Decoding-enhanced BERT with Disentangled Attention" (2020)
- "ELECTRA: Pre-training Text Encoders as Discriminators" (2020)

Best Practices

Data Preparation:

- Clean and normalize text appropriately
- Use proper train/val/test splits
- Balance datasets when needed
- Augment data strategically

- Version control datasets

Model Selection:

- Start with pre-trained models
- Choose architecture based on task
- Consider computational constraints
- Benchmark multiple approaches
- Document decisions

Training:

- Monitor training/validation loss
- Use early stopping
- Save checkpoints frequently
- Track experiments (Weights & Biases, MLflow)
- Use appropriate learning rate schedules

Evaluation:

- Use multiple metrics
- Error analysis is crucial
- Test on out-of-distribution data
- Human evaluation when possible
- A/B testing in production

Deployment:

- Optimize for latency (quantization, distillation)
- Monitor model performance
- Version models properly
- Plan for updates
- Handle edge cases gracefully

Common Mistakes to Avoid

1. **Data Leakage:** Test data in training
2. **Ignoring Class Imbalance:** Leads to poor minority class performance
3. **Over-reliance on Accuracy:** Use appropriate metrics
4. **Not Checking for Bias:** Can perpetuate harmful stereotypes

5. **Insufficient Error Analysis:** Misses improvement opportunities
 6. **Overfitting to Test Set:** Hyperparameter tuning on test data
 7. **Ignoring Computational Costs:** Unsustainable in production
 8. **Poor Documentation:** Makes reproduction impossible
 9. **Not Validating Assumptions:** Context window limits, tokenization issues
 10. **Neglecting Edge Cases:** Real-world data is messy
-

Future Directions in NLP

Emerging Trends

Multimodal Models:

- Vision + Language (CLIP, Flamingo)
- Audio + Text
- Unified representations
- Better grounding of language

Efficient Models:

- Smaller models with comparable performance
- Distillation, pruning, quantization
- Edge deployment
- Environmental sustainability

Retrieval-Augmented Generation:

- Combining retrieval with generation
- Better factuality
- Access to external knowledge
- More controllable outputs

Instruction Tuning:

- Better instruction following
- Natural language interfaces
- Few-shot capabilities
- Alignment with human intent

Long-Context Models:

- Handling thousands of tokens
- Document-level understanding
- Better memory and recall
- Efficient attention mechanisms

Constitutional AI:

- Self-improvement through feedback
- Harmlessness, helpfulness, honesty
- Reduced human supervision
- More aligned systems

Open Problems

True Understanding:

- Models lack genuine comprehension
- Statistical patterns vs. meaning
- Grounding in real world
- Causality and reasoning

Data Efficiency:

- Humans learn from much less data
- Few-shot learning improvements
- Meta-learning advances
- Self-supervised innovations

Robustness:

- Adversarial examples
- Out-of-distribution generalization
- Systematic generalization
- Compositional understanding

Interpretability:

- Understanding model decisions
- Debugging failures
- Building trust

- Meeting regulations

Ethical AI:

- Mitigating bias
- Privacy preservation
- Fairness guarantees
- Responsible deployment

Multilingual Equity:

- Low-resource languages
 - Cross-lingual transfer
 - Cultural sensitivity
 - Linguistic diversity
-

Glossary of Terms

Attention: Mechanism allowing models to focus on relevant parts of input

Autoencoding: Reconstruction-based pre-training (like BERT's MLM)

Autoregressive: Predicting next token given previous ones (like GPT)

BLEU: Metric for evaluating generation based on n-gram overlap

Contextualized Embeddings: Word representations that change based on context

Curriculum Learning: Training on progressively harder examples

Distillation: Transferring knowledge from large model to small one

Encoder-Decoder: Architecture with separate encoding and decoding components

Few-Shot Learning: Learning from few examples

Fine-tuning: Adapting pre-trained model to specific task

Hallucination: Model generating plausible but false information

In-Context Learning: Learning from examples in prompt without parameter updates

Knowledge Distillation: Training small model to mimic large one

LoRA: Low-rank adaptation for efficient fine-tuning

Masked Language Modeling: Predicting masked tokens (BERT's pre-training)

Multi-Head Attention: Multiple attention mechanisms in parallel

Perplexity: Measure of how well model predicts text (lower is better)

Positional Encoding: Adding position information to embeddings

Pre-training: Initial training on large corpus before fine-tuning

Prompt Engineering: Designing inputs to elicit desired model behavior

RAG: Retrieval-Augmented Generation

RLHF: Reinforcement Learning from Human Feedback

Self-Attention: Attention over same sequence

Sequence-to-Sequence: Mapping input sequence to output sequence

Softmax: Function converting scores to probability distribution

Subword Tokenization: Breaking words into smaller units

Temperature: Controls randomness in generation (higher = more random)

Transfer Learning: Using knowledge from one task for another

Transformer: Architecture based on self-attention

Zero-Shot Learning: Performing task without task-specific training

Key Takeaways

1. **Transformers revolutionized NLP** through self-attention and parallel processing, enabling better modeling of long-range dependencies
2. **Pre-training + fine-tuning** is the dominant paradigm - learn general language understanding, then specialize for tasks
3. **BERT excels at understanding** through deep bidirectional context and masked language modeling
4. **GPT excels at generation** through autoregressive modeling and massive scale
5. **T5 unifies all tasks** into text-to-text format for maximum flexibility and simplicity
6. **Contextual embeddings** capture word meaning based on context, far superior to static embeddings
7. **Attention mechanisms** allow models to dynamically focus on relevant information
8. **Transfer learning** enables solving new tasks with limited labeled data
9. **Prompt engineering** is crucial for getting best performance from large language models
10. **Scaling laws** show that larger models, more data, and more compute lead to better performance
11. **Evaluation is multifaceted** - no single metric captures all aspects of performance
12. **Practical considerations** matter: efficiency, robustness, fairness, and interpretability

13. **NLP is rapidly evolving** - stay current with research, but fundamentals remain important
 14. **Understanding architectures** helps choose the right tool for each task
 15. **Ethical considerations** are paramount - bias, privacy, environmental impact, and responsible deployment
-

Quick Command Reference

Using Hugging Face Transformers

```
python
```

```
# Load pre-trained model
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")

# Tokenize text
inputs = tokenizer("Hello world", return_tensors="pt")

# Get embeddings
outputs = model(**inputs)
embeddings = outputs.last_hidden_state

# Text classification
from transformers import pipeline
classifier = pipeline("sentiment-analysis")
result = classifier("I love NLP!")

# Question answering
qa = pipeline("question-answering")
result = qa(question="What is NLP?", context="NLP is...")

# Text generation
generator = pipeline("text-generation", model="gpt2")
result = generator("Once upon a time", max_length=50)

# Named Entity Recognition
ner = pipeline("ner")
result = ner("John works at Microsoft in Seattle")

# Translation
translator = pipeline("translation_en_to_fr")
result = translator("Hello, how are you?")

# Summarization
summarizer = pipeline("summarization")
result = summarizer(long_article, max_length=130)
```

Fine-tuning Example

```
python
```

```
from transformers import AutoModelForSequenceClassification, Trainer, TrainingArguments
```

```
# Load model
```

```
model = AutoModelForSequenceClassification.from_pretrained(  
    "bert-base-uncased", num_labels=2  
)
```

```
# Define training arguments
```

```
training_args = TrainingArguments(  
    output_dir="./results",  
    num_train_epochs=3,  
    per_device_train_batch_size=16,  
    per_device_eval_batch_size=64,  
    warmup_steps=500,  
    weight_decay=0.01,  
    logging_dir="./logs",  
    evaluation_strategy="epoch",  
)
```

```
# Create trainer
```

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_dataset,  
    eval_dataset=eval_dataset,  
)
```

```
# Train
```

```
trainer.train()
```

```
# Evaluate
```

```
results = trainer.evaluate()
```

SpaCy Example

```
python
```

```
import spacy

# Load model
nlp = spacy.load("en_core_web_sm")

# Process text
doc = nlp("Apple is looking at buying U.K. startup for $1 billion")

# Tokenization
tokens = [token.text for token in doc]

# POS tagging
pos = [(token.text, token.pos_) for token in doc]

# Named Entity Recognition
entities = [(ent.text, ent.label_) for ent in doc.ents]

# Dependency parsing
deps = [(token.text, token.dep_, token.head.text) for token in doc]

# Sentence segmentation
sentences = [sent.text for sent in doc.sents]
```

This comprehensive cheat sheet covers the fundamentals through advanced topics in NLP, providing a solid foundation for both learning and practical application. Keep it handy as a reference guide!

Mathematics for Machine Learning - Complete Cheat Sheet

1. LINEAR ALGEBRA

Vectors

Definition: Ordered array of numbers representing magnitude and direction

```
python

import numpy as np

# Vector creation
v = np.array([1, 2, 3])
w = np.array([4, 5, 6])

# Vector operations
addition = v + w          # [5, 7, 9]
scalar_mult = 3 * v       # [3, 6, 9]
dot_product = np.dot(v, w) # 1*4 + 2*5 + 3*6 = 32
magnitude = np.linalg.norm(v) #  $\sqrt{1^2 + 2^2 + 3^2} = \sqrt{14}$ 
```

Key Formulas:

- Dot Product: $\mathbf{v} \cdot \mathbf{w} = \sum(\mathbf{v}_i \times \mathbf{w}_i) = |\mathbf{v}||\mathbf{w}|\cos(\theta)$
- Magnitude: $\|\mathbf{v}\| = \sqrt{\sum \mathbf{v}_i^2}$
- Unit Vector: $\hat{\mathbf{v}} = \mathbf{v} / \|\mathbf{v}\|$

ML Use Cases: Feature vectors, embeddings, word2vec

Matrices

Definition: 2D array of numbers ($m \times n$)

```
python

# Matrix creation
A = np.array([[1, 2], [3, 4], [5, 6]]) # 3x2 matrix
B = np.array([[7, 8], [9, 10]])       # 2x2 matrix

# Matrix operations
transpose = A.T          # Swap rows/columns
matmul = A @ B           # Matrix multiplication
element_wise = A * 2      # Scalar multiplication
identity = np.eye(3)     # 3x3 identity matrix
```


Key Operations:

- **Matrix Multiplication:** $C = AB$ where $C[i,j] = \sum(A[i,k] \times B[k,j])$
 - Only valid when A is $(m \times n)$ and B is $(n \times p)$
- **Transpose:** A^T where $A^T[i,j] = A[j,i]$
- **Inverse:** A^{-1} where $AA^{-1} = I$ (only for square matrices)

```
python

# Matrix inverse and determinant
A_square = np.array([[1, 2], [3, 4]])
inverse = np.linalg.inv(A_square)
determinant = np.linalg.det(A_square) # ad - bc for 2x2
```

ML Use Cases: Neural network weights, data transformations, image processing

Eigenvalues & Eigenvectors

Definition: $Av = \lambda v$ (eigenvector v , eigenvalue λ)

```
python

A = np.array([[4, 2], [1, 3]])
eigenvalues, eigenvectors = np.linalg.eig(A)

# eigenvalues: scaling factors
# eigenvectors: directions that don't change under transformation
```

ML Use Cases: PCA (dimensionality reduction), PageRank, covariance matrices

Matrix Decompositions

Singular Value Decomposition (SVD):

```
python

#  $A = U\Sigma V^T$ 
A = np.array([[1, 2, 3], [4, 5, 6]])
U, S, Vt = np.linalg.svd(A)

# U: left singular vectors (m x m)
# S: singular values (diagonal)
# Vt: right singular vectors (n x n)
```

ML Use Cases: Recommender systems, image compression, LSA

2. CALCULUS

Derivatives

Definition: Rate of change of a function

Single Variable:

```
python
```

```
#  $f(x) = x^2$ 
```

```
#  $f'(x) = 2x$ 
```

```
from scipy.misc import derivative
```

```
f = lambda x: x**2
```

```
df = derivative(f, 3.0, dx=1e-6) #  $\approx 6$ 
```

Common Derivatives:

- $(x^n)' = n \cdot x^{n-1}$
- $(e^x)' = e^x$
- $(\ln x)' = 1/x$
- $(\sin x)' = \cos x$
- $(\cos x)' = -\sin x$

Chain Rule: $(f(g(x)))' = f'(g(x)) \cdot g'(x)$

```
python
```

```
# Example:  $f(x) = (2x + 1)^3$ 
```

```
#  $f'(x) = 3(2x + 1)^2 \cdot 2 = 6(2x + 1)^2$ 
```

Partial Derivatives

Definition: Derivative with respect to one variable, holding others constant

```
python
```

```
import sympy as sp

x, y = sp.symbols('x y')
f = x**2 + 3*x*y + y**2

#  $\partial f / \partial x$ 
df_dx = sp.diff(f, x) # 2x + 3y

#  $\partial f / \partial y$ 
df_dy = sp.diff(f, y) # 3x + 2y
```

ML Use Cases: Gradient descent, backpropagation

Gradient

Definition: Vector of all partial derivatives

$$\nabla f = [\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n]$$

```
python

def gradient(f, point):
    """Numerical gradient computation"""
    grad = np.zeros_like(point)
    h = 1e-5
    for i in range(len(point)):
        point_plus = point.copy()
        point_plus[i] += h
        point_minus = point.copy()
        point_minus[i] -= h
        grad[i] = (f(point_plus) - f(point_minus)) / (2*h)
    return grad

# Loss function
def loss(w):
    return np.sum(w**2)

w = np.array([3.0, 4.0])
grad = gradient(loss, w) # [6.0, 8.0]
```

Gradient Descent

Core Algorithm for ML Optimization:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \alpha \nabla L(\mathbf{w})$$

```
python
```

```

def gradient_descent(f, grad_f, initial_w, learning_rate=0.01, iterations=1000):
    w = initial_w.copy()
    history = [w.copy()]

    for i in range(iterations):
        grad = grad_f(w)
        w = w - learning_rate * grad
        history.append(w.copy())

    return w, history

# Example: Minimize  $f(x) = x^2 + y^2$ 
def f(w):
    return w[0]**2 + w[1]**2

def grad_f(w):
    return 2 * w

initial = np.array([10.0, 10.0])
optimal, history = gradient_descent(f, grad_f, initial)

```

ML Use Cases: Training neural networks, linear regression, logistic regression

3. PROBABILITY & STATISTICS

Probability Basics

Key Formulas:

- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ (Union)
- $P(A \cap B) = P(A) \cdot P(B)$ (Independent events)
- $P(A|B) = P(A \cap B) / P(B)$ (Conditional probability)

Bayes' Theorem: $P(A|B) = P(B|A) \cdot P(A) / P(B)$

python

```
# Example: Email spam classification
```

```
#  $P(\text{Spam} | \text{"lottery"}) = P(\text{"lottery"} | \text{Spam}) \cdot P(\text{Spam}) / P(\text{"lottery"})$ 
```

```
p_spam = 0.3 # Prior
```

```
p_lottery_given_spam = 0.8
```

```
p_lottery_given_ham = 0.05
```

```
p_lottery = p_lottery_given_spam * p_spam + p_lottery_given_ham * (1 - p_spam)
```

```
p_spam_given_lottery = (p_lottery_given_spam * p_spam) / p_lottery
```

```
#  $\approx 0.77$ 
```

Random Variables & Distributions

Expected Value: $E[X] = \sum x_i \cdot P(x_i)$ (discrete) or $\int x \cdot f(x) dx$ (continuous)

Variance: $\text{Var}(X) = E[(X - \mu)^2] = E[X^2] - (E[X])^2$

Standard Deviation: $\sigma = \sqrt{\text{Var}(X)}$

```
python
```

```
from scipy import stats
```

```
# Normal/Gaussian Distribution
```

```
mu, sigma = 0, 1
```

```
x = np.random.normal(mu, sigma, 1000)
```

```
# PDF:  $f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot e^{-(x-\mu)^2/(2\sigma^2)}$ 
```

```
pdf_value = stats.norm.pdf(0, mu, sigma)
```

```
# CDF
```

```
cdf_value = stats.norm.cdf(1.96, mu, sigma) #  $\approx 0.975$ 
```

Common Distributions:

```
python
```

```
# Bernoulli (coin flip)
bernoulli = stats.bernoulli.rvs(p=0.5, size=100)

# Binomial (n coin flips)
binomial = stats.binom.rvs(n=10, p=0.5, size=100)

# Uniform
uniform = stats.uniform.rvs(loc=0, scale=1, size=100)

# Exponential
exponential = stats.expon.rvs(scale=1, size=100)
```

Statistical Measures

```
python

data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Central tendency
mean = np.mean(data)      # 5.5
median = np.median(data)  # 5.5
mode = stats.mode(data, keepdims=True).mode[0]

# Spread
variance = np.var(data)    # Population variance
std = np.std(data)         # Population std
sample_var = np.var(data, ddof=1) # Sample variance (unbiased)

# Percentiles
q25 = np.percentile(data, 25) # 3.25
q75 = np.percentile(data, 75) # 7.75
```

ML Use Cases: Feature normalization, uncertainty quantification, Bayesian ML

4. INFORMATION THEORY

Entropy

Definition: Measure of uncertainty/information content

$$H(X) = -\sum P(x_i) \cdot \log_2(P(x_i))$$

```
python
```

```
def entropy(probabilities):
    """Shannon entropy"""
    probabilities = np.array(probabilities)
    probabilities = probabilities[probabilities > 0] # Avoid log(0)
    return -np.sum(probabilities * np.log2(probabilities))

# Fair coin: H = 1 bit
fair_coin = [0.5, 0.5]
print(entropy(fair_coin)) # 1.0

# Biased coin: H < 1 bit
biased_coin = [0.9, 0.1]
print(entropy(biased_coin)) # 0.469
```

Cross-Entropy

Definition: Measures difference between two probability distributions

$$H(p, q) = -\sum p(x) \cdot \log(q(x))$$

```
python

def cross_entropy(y_true, y_pred):
    """Cross-entropy loss"""
    epsilon = 1e-15 # Avoid log(0)
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    return -np.sum(y_true * np.log(y_pred))

# Classification example
y_true = np.array([1, 0, 0]) # One-hot encoded
y_pred = np.array([0.7, 0.2, 0.1]) # Model prediction
loss = cross_entropy(y_true, y_pred)
```

ML Use Cases: Classification loss function, neural network training

KL Divergence

Definition: Measures how one distribution differs from another

$$D_{KL}(P||Q) = \sum P(x) \cdot \log(P(x)/Q(x))$$

```
python
```

```
def kl_divergence(p, q):
    """KL divergence from Q to P"""
    p = np.array(p)
    q = np.array(q)
    return np.sum(p * np.log(p / q))

p = np.array([0.4, 0.6])
q = np.array([0.5, 0.5])
kl = kl_divergence(p, q) # Not symmetric: D_KL(P||Q) ≠ D_KL(Q||P)
```

ML Use Cases: Variational autoencoders, reinforcement learning

5. OPTIMIZATION

Convex Optimization

Definition: Finding minimum of convex function

Convex Function: $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$ for $\lambda \in [0,1]$

```
python

# Example: Linear regression (convex)
def mse_loss(w, X, y):
    """Mean Squared Error (convex)"""
    predictions = X @ w
    return np.mean((predictions - y)**2)

# Closed-form solution:  $w^* = (X^T X)^{-1} X^T y$ 
def solve_linear_regression(X, y):
    return np.linalg.inv(X.T @ X) @ X.T @ y
```

Optimization Algorithms

Batch Gradient Descent:

```
python
```



```
def batch_gradient_descent(X, y, w, lr=0.01, epochs=100):
    m = len(y)
    for epoch in range(epochs):
        predictions = X @ w
        error = predictions - y
        gradient = (1/m) * X.T @ error
        w = w - lr * gradient
    return w
```

Stochastic Gradient Descent (SGD):

```
python

def sgd(X, y, w, lr=0.01, epochs=100):
    m = len(y)
    for epoch in range(epochs):
        for i in range(m):
            xi = X[i:i+1]
            yi = y[i:i+1]
            prediction = xi @ w
            error = prediction - yi
            gradient = xi.T @ error
            w = w - lr * gradient
    return w
```

Mini-batch Gradient Descent:

```
python

def mini_batch_gd(X, y, w, lr=0.01, batch_size=32, epochs=100):
    m = len(y)
    for epoch in range(epochs):
        indices = np.random.permutation(m)
        X_shuffled = X[indices]
        y_shuffled = y[indices]

        for i in range(0, m, batch_size):
            X_batch = X_shuffled[i:i+batch_size]
            y_batch = y_shuffled[i:i+batch_size]

            predictions = X_batch @ w
            error = predictions - y_batch
            gradient = (1/len(y_batch)) * X_batch.T @ error
            w = w - lr * gradient
    return w
```

Adam Optimizer (Adaptive Moment Estimation):

```
python

def adam_optimizer(grad_fn, w, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8, iterations=1000):
    m = np.zeros_like(w) # First moment
    v = np.zeros_like(w) # Second moment

    for t in range(1, iterations + 1):
        grad = grad_fn(w)

        m = beta1 * m + (1 - beta1) * grad
        v = beta2 * v + (1 - beta2) * (grad ** 2)

        m_hat = m / (1 - beta1 ** t) # Bias correction
        v_hat = v / (1 - beta2 ** t)

        w = w - lr * m_hat / (np.sqrt(v_hat) + eps)

    return w
```

6. REAL-WORLD ML APPLICATIONS

Linear Regression

```
python

from sklearn.linear_model import LinearRegression

#  $y = Xw + b$ 
# Minimize:  $L(w) = (1/2m) \sum (\hat{y}_i - y_i)^2$ 

X = np.array([[1], [2], [3], [4]])
y = np.array([2, 4, 6, 8])

model = LinearRegression()
model.fit(X, y)

# Manual implementation
X_bias = np.c_[np.ones(len(X)), X] # Add bias term
w = np.linalg.inv(X_bias.T @ X_bias) @ X_bias.T @ y
```

Logistic Regression

```
python

from sklearn.linear_model import LogisticRegression

# Sigmoid:  $\sigma(z) = 1 / (1 + e^{-z})$ 
# Loss:  $L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$ 

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def logistic_loss(w, X, y):
    z = X @ w
    predictions = sigmoid(z)
    epsilon = 1e-15
    predictions = np.clip(predictions, epsilon, 1 - epsilon)
    return -np.mean(y * np.log(predictions) + (1 - y) * np.log(1 - predictions))
```

Neural Network Forward Pass

```
python

def relu(x):
    return np.maximum(0, x)

def softmax(x):
    exp_x = np.exp(x - np.max(x, axis=1, keepdims=True))
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)

# Simple 2-layer network
def forward_pass(X, W1, b1, W2, b2):
    # Layer 1
    Z1 = X @ W1 + b1
    A1 = relu(Z1)

    # Layer 2 (output)
    Z2 = A1 @ W2 + b2
    A2 = softmax(Z2)

    return A2, (Z1, A1, Z2)
```

Backpropagation

```
python
```

```

def backward_pass(X, y, W1, b1, W2, b2, cache, lr=0.01):
    Z1, A1, Z2 = cache
    m = X.shape[0]

    # Output layer gradient
    dZ2 = A2 - y # Softmax + cross-entropy derivative
    dW2 = (1/m) * A1.T @ dZ2
    db2 = (1/m) * np.sum(dZ2, axis=0)

    # Hidden layer gradient
    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * (Z1 > 0) # ReLU derivative
    dW1 = (1/m) * X.T @ dZ1
    db1 = (1/m) * np.sum(dZ1, axis=0)

    # Update weights
    W1 -= lr * dW1
    b1 -= lr * db1
    W2 -= lr * dW2
    b2 -= lr * db2

    return W1, b1, W2, b2

```

Principal Component Analysis (PCA)

python

```
from sklearn.decomposition import PCA

# Dimensionality reduction via eigendecomposition

def manual_pca(X, n_components):
    # Center data
    X_centered = X - np.mean(X, axis=0)

    # Covariance matrix
    cov_matrix = np.cov(X_centered.T)

    # Eigendecomposition
    eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

    # Sort by eigenvalues
    idx = eigenvalues.argsort()[::-1]
    eigenvalues = eigenvalues[idx]
    eigenvectors = eigenvectors[:, idx]

    # Select top n components
    components = eigenvectors[:, :n_components]

    # Transform data
    X_transformed = X_centered @ components

    return X_transformed, components
```

7. KEY FORMULAS QUICK REFERENCE

Activation Functions

```
python
```

```

# Sigmoid:  $\sigma(x) = 1/(1 + e^{(-x)})$ 
sigmoid = lambda x: 1 / (1 + np.exp(-x))

# Tanh:  $\tanh(x) = (e^x - e^{(-x)})/(e^x + e^{(-x)})$ 
tanh = lambda x: np.tanh(x)

# ReLU:  $\max(0, x)$ 
relu = lambda x: np.maximum(0, x)

# Leaky ReLU:  $\max(ax, x)$ 
leaky_relu = lambda x, alpha=0.01: np.where(x > 0, x, alpha * x)

# Softmax:  $e^{x_i} / \sum e^{x_j}$ 
def softmax(x):
    exp_x = np.exp(x - np.max(x))
    return exp_x / np.sum(exp_x)

```

Loss Functions

```

python

# Mean Squared Error
mse = lambda y_true, y_pred: np.mean((y_true - y_pred)**2)

# Mean Absolute Error
mae = lambda y_true, y_pred: np.mean(np.abs(y_true - y_pred))

# Binary Cross-Entropy
def binary_crossentropy(y_true, y_pred):
    epsilon = 1e-15
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

# Categorical Cross-Entropy
def categorical_crossentropy(y_true, y_pred):
    epsilon = 1e-15
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    return -np.sum(y_true * np.log(y_pred))

```

Regularization

```

python

```

```
# L1 Regularization (Lasso):  $\lambda \sum |w_i|$ 
```

```
l1_reg = lambda w, lambda_: lambda_ * np.sum(np.abs(w))
```

```
# L2 Regularization (Ridge):  $\lambda \sum w_i^2$ 
```

```
l2_reg = lambda w, lambda_: lambda_ * np.sum(w**2)
```

```
# Elastic Net:  $\lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$ 
```

```
elastic_net = lambda w, l1, l2: l1 * np.sum(np.abs(w)) + l2 * np.sum(w**2)
```

Distance Metrics

```
python
```

```
# Euclidean Distance:  $\sqrt{\sum (x_i - y_i)^2}$ 
```

```
euclidean = lambda x, y: np.sqrt(np.sum((x - y)**2))
```

```
# Manhattan Distance:  $\sum |x_i - y_i|$ 
```

```
manhattan = lambda x, y: np.sum(np.abs(x - y))
```

```
# Cosine Similarity:  $(x \cdot y) / (||x|| \cdot ||y||)$ 
```

```
cosine_sim = lambda x, y: np.dot(x, y) / (np.linalg.norm(x) * np.linalg.norm(y))
```

8. PRACTICAL TIPS

Feature Scaling

```
python
```

```
# Standardization (Z-score):  $(x - \mu) / \sigma$ 
```

```
def standardize(X):
```

```
    return (X - np.mean(X, axis=0)) / np.std(X, axis=0)
```

```
# Min-Max Normalization:  $(x - \min) / (\max - \min)$ 
```

```
def normalize(X):
```

```
    return (X - np.min(X, axis=0)) / (np.max(X, axis=0) - np.min(X, axis=0))
```

Train-Test Split

```
python
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Performance Metrics

```
python

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Classification
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)

# Regression
from sklearn.metrics import mean_squared_error, r2_score
mse = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_true, y_pred)
```

Summary of When to Use What

Concept	When to Use	Example
Linear Algebra	Data representation, transformations	Image processing, PCA
Calculus	Optimization, backpropagation	Training neural networks
Probability	Uncertainty modeling	Bayesian inference, GANs
Information Theory	Loss functions	Classification tasks
Optimization	Model training	Gradient descent variants
Statistics	Data analysis, feature selection	EDA, hypothesis testing

9. ADVANCED LINEAR ALGEBRA CONCEPTS

Matrix Rank & Null Space

Rank: Number of linearly independent rows/columns

```
python

A = np.array([[1, 2, 3], [2, 4, 6], [3, 6, 9]])
rank = np.linalg.matrix_rank(A) # 1 (all rows are linearly dependent)

# Full rank matrix
B = np.array([[1, 2], [3, 4]])
rank_B = np.linalg.matrix_rank(B) # 2 (full rank)
```

ML Applications:

- Feature redundancy detection
- Determining if a system of equations has unique solution
- Dimensionality analysis in neural networks

Orthogonality & Projection

Orthogonal vectors: $v \cdot w = 0$

```
python
```

```

def is_orthogonal(v, w):
    return np.abs(np.dot(v, w)) < 1e-10

# Gram-Schmidt Orthogonalization
def gram_schmidt(vectors):
    """Convert vectors to orthonormal basis"""
    basis = []
    for v in vectors:
        # Subtract projections onto previous basis vectors
        w = v.copy()
        for b in basis:
            w -= np.dot(v, b) * b
        # Normalize
        if np.linalg.norm(w) > 1e-10:
            basis.append(w / np.linalg.norm(w))
    return np.array(basis)

# Projection of v onto w
def project(v, w):
    return (np.dot(v, w) / np.dot(w, w)) * w

```

ML Applications:

- QR decomposition in optimization
- Orthogonal regularization in neural networks
- Feature decorrelation

Trace & Frobenius Norm

```

python

A = np.array([[1, 2], [3, 4]])

# Trace: sum of diagonal elements
trace = np.trace(A) # 1 + 4 = 5

# Frobenius Norm: ||A||_F = sqrt(sum(a_ij^2))
frobenius = np.linalg.norm(A, 'fro') # sqrt(1^2 + 2^2 + 3^2 + 4^2) = sqrt(30)

# Properties:
# tr(A + B) = tr(A) + tr(B)
# tr(AB) = tr(BA)
# tr(A^T) = tr(A)

```

ML Applications:

- Nuclear norm regularization
- Matrix completion
- Similarity measures between matrices

Kronecker Product & Vectorization

```
python

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Kronecker product
kron = np.kron(A, B)

# Vectorization (column-wise flattening)
vec_A = A.flatten('F') # Fortran-style (column-major)

# Property:  $\text{vec}(ABC) = (C^T \otimes A)\text{vec}(B)$ 
```

ML Applications:

- Convolutional neural networks (CNN operations)
 - Tensor operations
 - Graph neural networks
-

10. ADVANCED CALCULUS & OPTIMIZATION

Hessian Matrix

Definition: Matrix of second-order partial derivatives

```
python
```

```

import sympy as sp

x, y = sp.symbols('x y')
f = x**3 + y**3 - 3*x*y

# Hessian matrix:  $H[i,j] = \partial^2 f / \partial x_i \partial x_j$ 
hessian = sp.hessian(f, (x, y))
# [[6x, -3],
# [-3, 6y]]

# Numerical Hessian
def numerical_hessian(f, x, eps=1e-5):
    n = len(x)
    H = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            x_pp = x.copy()
            x_pp[i] += eps
            x_pp[j] += eps

            x_pm = x.copy()
            x_pm[i] += eps
            x_pm[j] -= eps

            x_mp = x.copy()
            x_mp[i] -= eps
            x_mp[j] += eps

            x_mm = x.copy()
            x_mm[i] -= eps
            x_mm[j] -= eps

            H[i,j] = (f(x_pp) - f(x_pm) - f(x_mp) + f(x_mm)) / (4 * eps**2)
    return H

```

ML Applications:

- Newton's method optimization
- Determining local minima/maxima (convexity analysis)
- Second-order optimization methods (L-BFGS)

Lagrange Multipliers

Constrained Optimization: Optimize $f(x)$ subject to $g(x) = 0$

$\nabla f = \lambda \nabla g$ (at optimal point)

python

*# Example: Maximize $f(x,y) = x*y$ subject to $x^2 + y^2 = 1$*

```
def lagrangian(x, y, lambda_):  
    return x*y - lambda_ * (x**2 + y**2 - 1)
```

Solve: $\nabla L = 0$

$y - 2\lambda x = 0$

$x - 2\lambda y = 0$

$x^2 + y^2 - 1 = 0$

Solution: $x = y = \pm 1/\sqrt{2}$, $\lambda = \pm 1/2$

ML Applications:

- Support Vector Machines (SVM)
- Constrained neural network training
- Dual formulations

Taylor Series Approximation

python

$f(x) \approx f(a) + f'(a)(x-a) + f''(a)(x-a)^2/2! + \dots$

```
def taylor_approximation(f, df, ddf, a, x, order=2):  
    """Taylor series approximation around point a"""  
    result = f(a)  
    if order >= 1:  
        result += df(a) * (x - a)  
    if order >= 2:  
        result += 0.5 * ddf(a) * (x - a)**2  
    return result
```

Used in Newton's method:

$x_{new} = x - f'(x) / f''(x)$

ML Applications:

- Function approximation
- Understanding gradient descent convergence
- Neural network analysis

Jacobian Matrix

Definition: Matrix of first-order partial derivatives for vector functions

```
python

# For  $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ , Jacobian  $J[i,j] = \partial f_i / \partial x_j$ 

def jacobian(func, x, eps=1e-7):
    """Numerical Jacobian computation"""
    x = np.array(x, dtype=float)
    f0 = func(x)
    n = len(x)
    m = len(f0) if hasattr(f0, '__len__') else 1
    J = np.zeros((m, n))

    for j in range(n):
        x_plus = x.copy()
        x_plus[j] += eps
        f_plus = func(x_plus)
        J[:, j] = (f_plus - f0) / eps

    return J

# Example: Neural network layer
def layer(x):
    W = np.array([[1, 2], [3, 4], [5, 6]])
    b = np.array([1, 1, 1])
    return W @ x + b

x = np.array([1.0, 2.0])
J = jacobian(layer, x)
# J equals W (the weight matrix)
```

ML Applications:

- Backpropagation in neural networks
 - Sensitivity analysis
 - Coordinate transformations
-

11. PROBABILITY THEORY DEEP DIVE

Conditional Probability & Independence

```
python

# Joint probability
def joint_prob(p_a, p_b_given_a):
    return p_a * p_b_given_a

# Marginal probability
def marginal_prob(joint_probs):
    return np.sum(joint_probs, axis=1)

# Independence test:  $P(A \cap B) = P(A) \cdot P(B)$ 
def is_independent(p_a, p_b, p_ab):
    return np.abs(p_ab - p_a * p_b) < 1e-10
```

Maximum Likelihood Estimation (MLE)

```
python

# Example: Estimating mean and variance of Gaussian

def gaussian_mle(data):
    """MLE for Gaussian parameters"""
    mu_hat = np.mean(data)
    sigma_hat = np.std(data, ddof=0) # MLE uses N, not N-1
    return mu_hat, sigma_hat

# Log-likelihood for Gaussian
def log_likelihood_gaussian(data, mu, sigma):
    n = len(data)
    ll = -n/2 * np.log(2 * np.pi * sigma**2)
    ll -= np.sum((data - mu)**2) / (2 * sigma**2)
    return ll

# Example data
data = np.random.normal(5, 2, 1000)
mu_mle, sigma_mle = gaussian_mle(data)
```

ML Applications:

- Parameter estimation in probabilistic models
- Training generative models
- Logistic regression

Maximum A Posteriori (MAP) Estimation

Combines prior knowledge with data

$$\theta_{\text{MAP}} = \operatorname{argmax} P(\theta|\mathbf{D}) = \operatorname{argmax} P(\mathbf{D}|\theta) \cdot P(\theta)$$

```
python

def map_estimation(data, prior_mean, prior_var, likelihood_var):
    """MAP estimation for Gaussian with Gaussian prior"""
    n = len(data)
    data_mean = np.mean(data)

    # Posterior mean (MAP estimate)
    posterior_precision = 1/prior_var + n/likelihood_var
    posterior_mean = (prior_mean/prior_var + n*data_mean/likelihood_var) / posterior_precision

    return posterior_mean

# Example: Prior belief  $\mu=0$ , observe data with mean 5
data = np.random.normal(5, 1, 100)
map_estimate = map_estimation(data, prior_mean=0, prior_var=10, likelihood_var=1)
# MAP estimate is between 0 and 5, weighted by certainties
```

ML Applications:

- Bayesian neural networks
- Regularization (L2 = Gaussian prior)
- Hyperparameter tuning

Covariance & Correlation

```
python
```


Covariance: $Cov(X, Y) = E[(X - \mu_x)(Y - \mu_y)]$

```
def covariance(x, y):  
    return np.mean((x - np.mean(x)) * (y - np.mean(y)))
```

Covariance matrix

```
def covariance_matrix(X):  
    """X is n_samples × n_features"""  
    X_centered = X - np.mean(X, axis=0)  
    return (X_centered.T @ X_centered) / len(X)
```

Correlation: $\rho(X, Y) = Cov(X, Y) / (\sigma_x \cdot \sigma_y)$

```
def correlation(x, y):  
    return covariance(x, y) / (np.std(x) * np.std(y))
```

Using numpy

```
X = np.random.randn(100, 3)  
cov_matrix = np.cov(X.T)  
corr_matrix = np.corrcoef(X.T)
```

ML Applications:

- Feature correlation analysis
- PCA computation
- Portfolio optimization

Multivariate Gaussian Distribution

python

```

from scipy.stats import multivariate_normal

# PDF:  $f(x) = (1/\sqrt{(2\pi)^k |\Sigma|}) \exp(-\frac{1}{2}(x-\mu)^T \Sigma^{-1} (x-\mu))$ 

mu = np.array([0, 0])
cov = np.array([[1, 0.5], [0.5, 1]])

# Sample from multivariate Gaussian
samples = np.random.multivariate_normal(mu, cov, 1000)

# Evaluate PDF
mvn = multivariate_normal(mean=mu, cov=cov)
pdf_value = mvn.pdf([0, 0])

# Mahalanobis distance:  $d = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)}$ 
def mahalanobis_distance(x, mu, cov_inv):
    diff = x - mu
    return np.sqrt(diff.T @ cov_inv @ diff)

```

ML Applications:

- Gaussian Mixture Models (GMM)
- Anomaly detection
- Multivariate regression

12. INFORMATION THEORY APPLICATIONS

Mutual Information

Measures dependency between variables

$$I(X;Y) = H(X) + H(Y) - H(X,Y)$$

python

```

def mutual_information(x, y, bins=10):
    """Estimate mutual information"""
    # Create 2D histogram
    hist_2d, _, _ = np.histogram2d(x, y, bins=bins)
    hist_2d = hist_2d / np.sum(hist_2d) # Normalize

    # Marginal distributions
    p_x = np.sum(hist_2d, axis=1)
    p_y = np.sum(hist_2d, axis=0)

    # Mutual information
    mi = 0
    for i in range(bins):
        for j in range(bins):
            if hist_2d[i, j] > 0:
                mi += hist_2d[i, j] * np.log(hist_2d[i, j] / (p_x[i] * p_y[j]))

    return mi

# Example
x = np.random.randn(1000)
y = x + np.random.randn(1000) * 0.5 # Correlated
mi = mutual_information(x, y)

```

ML Applications:

- Feature selection
- Image registration
- Information bottleneck theory in deep learning

Jensen-Shannon Divergence

Symmetric version of KL divergence

$JSD(P||Q) = \frac{1}{2}D_{KL}(P||M) + \frac{1}{2}D_{KL}(Q||M)$ where $M = \frac{1}{2}(P + Q)$

python

```
def js_divergence(p, q):
    """Jensen-Shannon divergence"""
    p = np.array(p)
    q = np.array(q)
    m = 0.5 * (p + q)

    return 0.5 * kl_divergence(p, m) + 0.5 * kl_divergence(q, m)

# Properties:  $0 \leq JSD \leq 1$ , symmetric
```

ML Applications:

- GANs (Generative Adversarial Networks)
- Model comparison
- Clustering evaluation

Perplexity

Measures how well a probability model predicts a sample

Perplexity = $2^{H(p)}$ = $\exp(H(p))$

```
python

def perplexity(probabilities):
    """Lower is better"""
    h = entropy(probabilities)
    return 2 ** h

# For language models
def language_model_perplexity(model_probs, actual_sequence):
    """
    model_probs: predicted probability for each word
    Lower perplexity = better model
    """
    log_probs = np.log2(model_probs)
    avg_log_prob = np.mean(log_probs)
    return 2 ** (-avg_log_prob)
```

ML Applications:

- Language model evaluation
- Topic modeling
- t-SNE visualization

13. ADVANCED OPTIMIZATION TECHNIQUES

Momentum

Accelerates gradient descent in relevant direction

```
python

def sgd_with_momentum(grad_fn, w, lr=0.01, momentum=0.9, iterations=1000):
    v = np.zeros_like(w) # Velocity

    for t in range(iterations):
        grad = grad_fn(w)
        v = momentum * v - lr * grad
        w = w + v

    return w

# Physical intuition: ball rolling down hill
#  $v_t = \beta v_{t-1} - \alpha \nabla L$ 
#  $w_t = w_{t-1} + v_t$ 
```

Nesterov Accelerated Gradient

```
python

def nesterov_momentum(grad_fn, w, lr=0.01, momentum=0.9, iterations=1000):
    v = np.zeros_like(w)

    for t in range(iterations):
        # Look ahead
        w_ahead = w + momentum * v
        grad = grad_fn(w_ahead)
        v = momentum * v - lr * grad
        w = w + v

    return w
```

RMSprop

Adapts learning rate for each parameter

```
python
```

```
def rmsprop(grad_fn, w, lr=0.001, decay=0.9, eps=1e-8, iterations=1000):  
    cache = np.zeros_like(w)  
  
    for t in range(iterations):  
        grad = grad_fn(w)  
        cache = decay * cache + (1 - decay) * grad**2  
        w = w - lr * grad / (np.sqrt(cache) + eps)  
  
    return w
```

AdaGrad

```
python  
  
def adagrad(grad_fn, w, lr=0.01, eps=1e-8, iterations=1000):  
    cache = np.zeros_like(w)  
  
    for t in range(iterations):  
        grad = grad_fn(w)  
        cache += grad**2  
        w = w - lr * grad / (np.sqrt(cache) + eps)  
  
    return w
```

Learning Rate Schedules

```
python
```

```

# Step decay
def step_decay(epoch, initial_lr=0.1, drop=0.5, epochs_drop=10):
    return initial_lr * (drop ** np.floor(epoch / epochs_drop))

# Exponential decay
def exponential_decay(epoch, initial_lr=0.1, k=0.1):
    return initial_lr * np.exp(-k * epoch)

# Cosine annealing
def cosine_annealing(epoch, initial_lr=0.1, T_max=100):
    return initial_lr * (1 + np.cos(np.pi * epoch / T_max)) / 2

# Warm restart
def cosine_annealing_warm_restart(epoch, initial_lr=0.1, T_0=10, T_mult=2):
    T_cur = epoch
    T_i = T_0
    while T_cur >= T_i:
        T_cur -= T_i
        T_i *= T_mult
    return initial_lr * (1 + np.cos(np.pi * T_cur / T_i)) / 2

```

ML Applications: Training stability, faster convergence, escaping local minima

14. DIMENSIONALITY REDUCTION TECHNIQUES

t-SNE (t-Distributed Stochastic Neighbor Embedding)

```

python

from sklearn.manifold import TSNE

# Non-linear dimensionality reduction for visualization
tsne = TSNE(n_components=2, perplexity=30, n_iter=1000)
X_embedded = tsne.fit_transform(X)

# Key concepts:
# - Preserves local structure
# - Converts distances to probabilities
# - Minimizes KL divergence between high and low dimensional distributions

```

Autoencoders

```

python

```

Neural network for dimensionality reduction

class Autoencoder:

def __init__(self, input_dim, encoding_dim):

self.input_dim = input_dim

self.encoding_dim = encoding_dim

def encode(self, x, W_enc, b_enc):

return sigmoid(x @ W_enc + b_enc)

def decode(self, h, W_dec, b_dec):

return sigmoid(h @ W_dec + b_dec)

def reconstruct(self, x, W_enc, b_enc, W_dec, b_dec):

h = self.encode(x, W_enc, b_enc)

return self.decode(h, W_dec, b_dec)

Loss: reconstruction error

$L = ||x - \text{decoder}(\text{encoder}(x))||^2$

Linear Discriminant Analysis (LDA)

python

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

Supervised dimensionality reduction

Maximizes class separability

lda = LinearDiscriminantAnalysis(n_components=2)

X_lda = lda.fit_transform(X, y)

Maximizes: (between-class variance) / (within-class variance)

Independent Component Analysis (ICA)

python

from sklearn.decomposition import FastICA

Separates multivariate signal into independent sources

ica = FastICA(n_components=3)

S = ica.fit_transform(X) *# Sources*

A = ica.mixing_ *# Mixing matrix*

Applications: Blind source separation, feature extraction

15. KERNEL METHODS

Kernel Trick

Implicit mapping to high-dimensional space

```
python

# Linear kernel:  $K(x, y) = x^T y$ 
def linear_kernel(x, y):
    return np.dot(x, y)

# Polynomial kernel:  $K(x, y) = (x^T y + c)^d$ 
def polynomial_kernel(x, y, degree=3, c=1):
    return (np.dot(x, y) + c) ** degree

# RBF/Gaussian kernel:  $K(x, y) = \exp(-\gamma ||x - y||^2)$ 
def rbf_kernel(x, y, gamma=0.1):
    return np.exp(-gamma * np.linalg.norm(x - y)**2)

# Sigmoid kernel:  $K(x, y) = \tanh(\alpha x^T y + c)$ 
def sigmoid_kernel(x, y, alpha=0.1, c=1):
    return np.tanh(alpha * np.dot(x, y) + c)
```

Kernel Matrix (Gram Matrix)

```
python

def compute_kernel_matrix(X, kernel_fn):
    """Compute pairwise kernel values"""
    n = len(X)
    K = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            K[i, j] = kernel_fn(X[i], X[j])
    return K

# Properties of valid kernels (Mercer's theorem):
# 1. Symmetric:  $K(x, y) = K(y, x)$ 
# 2. Positive semi-definite
```

ML Applications:

- Support Vector Machines
- Kernel PCA

- Gaussian Processes
-

16. SAMPLING & MONTE CARLO METHODS

Monte Carlo Integration

python

```
def monte_carlo_integration(f, a, b, n_samples=10000):  
    """Estimate integral of f from a to b"""  
    samples = np.random.uniform(a, b, n_samples)  
    estimate = (b - a) * np.mean(f(samples))  
    return estimate
```

Example: Estimate π

```
def estimate_pi(n_samples=100000):  
    x = np.random.uniform(-1, 1, n_samples)  
    y = np.random.uniform(-1, 1, n_samples)  
    inside = (x**2 + y**2) <= 1  
    return 4 * np.sum(inside) / n_samples
```

Markov Chain Monte Carlo (MCMC)

python

```

def metropolis_hastings(target_pdf, proposal_std=0.5, n_samples=10000):
    """MCMC sampling from arbitrary distribution"""
    samples = [0] # Initial state

    for _ in range(n_samples - 1):
        current = samples[-1]

        # Propose new state
        proposed = current + np.random.normal(0, proposal_std)

        # Acceptance probability
        acceptance = min(1, target_pdf(proposed) / target_pdf(current))

        if np.random.rand() < acceptance:
            samples.append(proposed)
        else:
            samples.append(current)

    return np.array(samples)

# Example: Sample from Gaussian
target = lambda x: np.exp(-0.5 * x**2)
samples = metropolis_hastings(target)

```

Gibbs Sampling

python

```
def gibbs_sampling(n_samples=1000):
    """Sample from bivariate Gaussian"""
    x = 0
    y = 0
    samples = []

    rho = 0.8 # Correlation

    for _ in range(n_samples):
        # Sample x | y
        x = np.random.normal(rho * y, np.sqrt(1 - rho**2))

        # Sample y | x
        y = np.random.normal(rho * x, np.sqrt(1 - rho**2))

        samples.append([x, y])

    return np.array(samples)
```

ML Applications:

- Bayesian inference
 - Training generative models
 - Uncertainty quantification
-

17. TIME SERIES & SEQUENTIAL DATA

Autocorrelation

python

```
def autocorrelation(x, lag):
    """Correlation of signal with itself at different lags"""
    n = len(x)
    x_mean = np.mean(x)
    c0 = np.sum((x - x_mean)**2) / n

    if lag == 0:
        return 1.0

    c_lag = np.sum((x[:-lag] - x_mean) * (x[lag:] - x_mean)) / n
    return c_lag / c0

# Compute for multiple lags
def acf(x, max_lag=20):
    return [autocorrelation(x, lag) for lag in range(max_lag + 1)]
```

Moving Average & Exponential Smoothing

```
python

def moving_average(x, window=5):
    """Simple moving average"""
    return np.convolve(x, np.ones(window)/window, mode='valid')

def exponential_smoothing(x, alpha=0.3):
    """Exponentially weighted moving average"""
    result = [x[0]]
    for i in range(1, len(x)):
        result.append(alpha * x[i] + (1 - alpha) * result[-1])
    return np.array(result)

# Double exponential smoothing (Holt's method)
def double_exponential_smoothing(x, alpha=0.3, beta=0.1):
    level = [x[0]]
    trend = [x[1] - x[0]]
    result = [x[0]]

    for i in range(1, len(x)):
        level.append(alpha * x[i] + (1 - alpha) * (level[-1] + trend[-1]))
        trend.append(beta * (level[-1] - level[-2]) + (1 - beta) * trend[-1])
        result.append(level[-1] + trend[-1])

    return np.array(result)
```

Recurrent Neural Network Math

python

RNN forward pass

```
def rnn_forward(x_sequence, W_xh, W_hh, W_hy, b_h, b_y):
```

```
    """
```

```
    x_sequence: (time_steps, input_dim)
```

```
    h_t = tanh(W_xh @ x_t + W_hh @ h_{t-1} + b_h)
```

```
    y_t = W_hy @ h_t + b_y
```

```
    """
```

```
    h = np.zeros(W_hh.shape[0])
```

```
    outputs = []
```

```
    for x_t in x_sequence:
```

```
        h = np.tanh(W_xh @ x_t + W_hh @ h + b_h)
```

```
        y = W_hy @ h + b_y
```

```
        outputs.append(y)
```

```
    return np.array(outputs), h
```

LSTM cell

```
def lstm_cell(x_t, h_prev, c_prev, W_f, W_i, W_o, W_c, b_f, b_i, b_o, b_c):
```

```
    """Long Short-Term Memory cell"""
```

Forget gate

```
f_t = sigmoid(W_f @ np.concatenate([h_prev, x_t]) + b_f)
```

Input gate

```
i_t = sigmoid(W_i @ np.concatenate([h_prev, x_t]) + b_i)
```

Cell update

```
c_tilde = np.tanh(W_c @ np.concatenate([h_prev, x_t]) + b_c)
```

```
c_t = f_t * c_prev + i_t * c_tilde
```

Output gate

```
o_t = sigmoid(W_o @ np.concatenate([h_prev, x_t]) + b_o)
```

```
h_t = o_t * np.tanh(c_t)
```

```
return h_t, c_t
```

ML Applications: Natural language processing, stock prediction, speech recognition

18. GRAPH THEORY FOR ML

Graph Representations

python

Adjacency Matrix

```
def create_adjacency_matrix(edges, n_nodes):
```

```
    A = np.zeros((n_nodes, n_nodes))
```

```
    for i, j in edges:
```

```
        A[i, j] = 1
```

```
        A[j, i] = 1 # Undirected
```

```
    return A
```

Degree Matrix

```
def degree_matrix(A):
```

```
    return np.diag(np.sum(A, axis=1))
```

Laplacian Matrix: $L = D - A$

```
def laplacian_matrix(A):
```

```
    D = degree_matrix(A)
```

```
    return D - A
```

Normalized Laplacian: $L_{norm} = D^{-1/2} L D^{-1/2}$

```
def normalized_laplacian(A):
```

```
    D = degree_matrix(A)
```

```
    D_inv_sqrt = np.diag(1.0 / np.sqrt(np.diag(D)))
```

```
    L = laplacian_matrix(A)
```

```
    return D_inv_sqrt @ L @ D_inv_sqrt
```

Graph Convolutional Networks (GCN)

python

```
def gcn_layer(X, A, W):
    """
    X: Node features (n_nodes × feature_dim)
    A: Adjacency matrix (n_nodes × n_nodes)
    W: Weight matrix (feature_dim × output_dim)
    """
    # Add self-loops
    A_tilde = A + np.eye(A.shape[0])

    # Degree matrix
    D_tilde = degree_matrix(A_tilde)
    D_inv_sqrt = np.diag(1.0 / np.sqrt(np.diag(D_tilde)))

    # Normalized adjacency
    A_hat = D_inv_sqrt @ A_tilde @ D_inv_sqrt

    # Propagation:  $H = \sigma(A\_hat @ X @ W)$ 
    return relu(A_hat @ X @ W)
```

PageRank Algorithm

python


```

def pagerank(A, damping=0.85, max_iter=100, tol=1e-6):
    """
    Compute PageRank scores
     $PR(i) = (1-d)/N + d \cdot \sum (PR(j)/L(j))$  for all j linking to i
    """
    n = A.shape[0]

    # Normalize adjacency matrix (column stochastic)
    out_degree = np.sum(A, axis=0)
    out_degree[out_degree == 0] = 1 # Avoid division by zero
    M = A / out_degree

    # Initialize PageRank scores
    pr = np.ones(n) / n

    for _ in range(max_iter):
        pr_new = (1 - damping) / n + damping * M @ pr

        if np.linalg.norm(pr_new - pr) < tol:
            break

        pr = pr_new

    return pr

```

ML Applications: Node importance, recommendation systems, graph neural networks

19. NUMERICAL METHODS & STABILITY

Condition Number

Measures sensitivity of function output to input perturbations

```
python
```

```

def condition_number(A):
    """
     $\kappa(A) = \|A\| \cdot \|A^{-1}\|$ 
    High condition number = ill-conditioned (numerical instability)
    """
    return np.linalg.cond(A)

# Example
A_good = np.array([[1, 0], [0, 1]])
A_bad = np.array([[1, 1], [1, 1.0001]])

print(condition_number(A_good)) #  $\approx 1$  (well-conditioned)
print(condition_number(A_bad)) #  $\gg 1$  (ill-conditioned)

```

ML Applications: Training stability, gradient checking, matrix inversions

Numerical Gradient Checking

```

python

def gradient_check(f, grad_f, x, eps=1e-5, tolerance=1e-7):
    """
    Verify gradient implementation
    Numerical:  $(f(x+\epsilon) - f(x-\epsilon)) / (2\epsilon)$ 
    """
    analytical_grad = grad_f(x)
    numerical_grad = np.zeros_like(x)

    for i in range(len(x)):
        x_plus = x.copy()
        x_plus[i] += eps
        x_minus = x.copy()
        x_minus[i] -= eps

        numerical_grad[i] = (f(x_plus) - f(x_minus)) / (2 * eps)

    # Relative error
    relative_error = np.linalg.norm(analytical_grad - numerical_grad) / \
        (np.linalg.norm(analytical_grad) + np.linalg.norm(numerical_grad))

    if relative_error < tolerance:
        print("✓ Gradient check passed!")
    else:
        print(f"✗ Gradient check failed! Relative error: {relative_error}")

    return relative_error

```

Numerical Integration Methods

python

Trapezoidal rule

```
def trapezoidal(f, a, b, n=1000):
```

```
    x = np.linspace(a, b, n)
```

```
    y = f(x)
```

```
    return (b - a) * (y[0] + 2*np.sum(y[1:-1]) + y[-1]) / (2*n)
```

Simpson's rule

```
def simpsons(f, a, b, n=1000):
```

```
    if n % 2 == 1:
```

```
        n += 1
```

```
    x = np.linspace(a, b, n+1)
```

```
    y = f(x)
```

```
    h = (b - a) / n
```

```
    return h/3 * (y[0] + 4*np.sum(y[1:-1:2]) + 2*np.sum(y[2:-1:2]) + y[-1])
```

20. FOURIER ANALYSIS

Discrete Fourier Transform (DFT)

python

```
# Manual DFT implementation
```

```
def dft(x):
```

```
    """
```

```
     $X[k] = \sum x[n] \cdot e^{(-2\pi i k n / N)}$ 
```

```
    Converts time domain to frequency domain
```

```
    """
```

```
    N = len(x)
```

```
    X = np.zeros(N, dtype=complex)
```

```
    for k in range(N):
```

```
        for n in range(N):
```

```
            X[k] += x[n] * np.exp(-2j * np.pi * k * n / N)
```

```
    return X
```

```
# Fast Fourier Transform ( $O(N \log N)$ )
```

```
def apply_fft(signal):
```

```
    return np.fft.fft(signal)
```

```
def apply_ifft(spectrum):
```

```
    return np.fft.ifft(spectrum)
```

```
# Example: Detect frequency components
```

```
t = np.linspace(0, 1, 500)
```

```
signal = np.sin(2 * np.pi * 5 * t) + 0.5 * np.sin(2 * np.pi * 10 * t)
```

```
spectrum = np.fft.fft(signal)
```

```
frequencies = np.fft.fftfreq(len(t), t[1] - t[0])
```

Convolution Theorem

Convolution in time $\leftrightarrow$ Multiplication in frequency

```
python
```

```
def convolve_fft(signal, kernel):
    """Fast convolution using FFT"""
    # Pad to avoid circular convolution
    n = len(signal) + len(kernel) - 1
    signal_padded = np.pad(signal, (0, n - len(signal)))
    kernel_padded = np.pad(kernel, (0, n - len(kernel)))

    # Convolution via FFT
    return np.fft.ifft(np.fft.fft(signal_padded) * np.fft.fft(kernel_padded)).real

# Standard convolution (slower)
def convolve_direct(signal, kernel):
    return np.convolve(signal, kernel, mode='full')
```

ML Applications:

- Convolutional Neural Networks
- Audio processing
- Image filtering

Wavelet Transform

```
python

import pywt

# Discrete Wavelet Transform
def dwt_decompose(signal, wavelet='db4', level=3):
    """Multi-level wavelet decomposition"""
    coeffs = pywt.wavedec(signal, wavelet, level=level)
    return coeffs

# Reconstruction
def dwt_reconstruct(coeffs, wavelet='db4'):
    return pywt.waverec(coeffs, wavelet)

# Example
signal = np.sin(2 * np.pi * np.linspace(0, 1, 1024))
coeffs = dwt_decompose(signal)
reconstructed = dwt_reconstruct(coeffs)
```

ML Applications: Feature extraction, denoising, compression

21. ATTENTION MECHANISMS

Scaled Dot-Product Attention

```
python

def scaled_dot_product_attention(Q, K, V, mask=None):
    """
    Attention(Q, K, V) = softmax(QK^T /  $\sqrt{d_k}$ )V

    Q: Query ( $n_{\text{queries}} \times d_k$ )
    K: Key ( $n_{\text{keys}} \times d_k$ )
    V: Value ( $n_{\text{keys}} \times d_v$ )
    """
    d_k = Q.shape[-1]

    # Compute attention scores
    scores = Q @ K.T / np.sqrt(d_k)

    # Apply mask if provided (for padding or causality)
    if mask is not None:
        scores = scores + mask * -1e9

    # Softmax to get attention weights
    attention_weights = softmax(scores)

    # Weighted sum of values
    output = attention_weights @ V

    return output, attention_weights

# Example usage
d_k, d_v = 64, 64
n_queries, n_keys = 10, 20

Q = np.random.randn(n_queries, d_k)
K = np.random.randn(n_keys, d_k)
V = np.random.randn(n_keys, d_v)

output, weights = scaled_dot_product_attention(Q, K, V)
```

Multi-Head Attention

```
python
```

```

def multi_head_attention(Q, K, V, num_heads=8):
    """
    Split into multiple heads, apply attention, concatenate
    """
    d_model = Q.shape[-1]
    d_k = d_model // num_heads

    # Split into heads
    Q_heads = Q.reshape(Q.shape[0], num_heads, d_k)
    K_heads = K.reshape(K.shape[0], num_heads, d_k)
    V_heads = V.reshape(V.shape[0], num_heads, d_k)

    # Apply attention to each head
    outputs = []
    for i in range(num_heads):
        out, _ = scaled_dot_product_attention(
            Q_heads[:, i, :],
            K_heads[:, i, :],
            V_heads[:, i, :]
        )
        outputs.append(out)

    # Concatenate heads
    concat = np.concatenate(outputs, axis=-1)

    return concat

```

Self-Attention

```

python

def self_attention(X, W_q, W_k, W_v):
    """
    Self-attention: Q = K = V = X
    Used in Transformers
    """
    Q = X @ W_q
    K = X @ W_k
    V = X @ W_v

    return scaled_dot_product_attention(Q, K, V)

```

ML Applications: Transformers, BERT, GPT, Vision Transformers

22. MATRIX CALCULUS

Vector-by-Vector Derivatives

```
python

#  $\partial(Ax)/\partial x = A^T$ 
def jacobian_linear(A):
    return A.T

#  $\partial(x^T A x)/\partial x = (A + A^T)x$ 
def gradient_quadratic(x, A):
    return (A + A.T) @ x

# If A is symmetric:  $\partial(x^T A x)/\partial x = 2Ax$ 
def gradient_quadratic_symmetric(x, A):
    return 2 * A @ x
```

Matrix-by-Scalar Derivatives

```
python

# Chain rule for matrices
#  $\partial L/\partial W = \partial L/\partial Y \cdot \partial Y/\partial W$ 

# Example: Linear layer
#  $Y = XW + b$ 
#  $\partial L/\partial W = X^T \cdot \partial L/\partial Y$ 
#  $\partial L/\partial b = \sum \partial L/\partial Y$  (sum over batch)
#  $\partial L/\partial X = \partial L/\partial Y \cdot W^T$ 

def linear_backward(dL_dY, X, W):
    """Backprop through linear layer"""
    dL_dW = X.T @ dL_dY
    dL_db = np.sum(dL_dY, axis=0)
    dL_dX = dL_dY @ W.T

    return dL_dX, dL_dW, dL_db
```

Common Derivatives in ML

```
python
```



```
# Softmax derivative
```

```
def softmax_derivative(s):
```

```
    """
```

```
    s: softmax output
```

```
    Returns Jacobian matrix
```

```
    """
```

```
    return np.diag(s) - np.outer(s, s)
```

```
# Batch normalization derivative
```

```
def batchnorm_backward(dout, x, gamma, eps=1e-5):
```

```
    """
```

```
    Backprop through batch normalization
```

```
    """
```

```
    N, D = x.shape
```

```
# Forward pass values (should be cached)
```

```
    mu = np.mean(x, axis=0)
```

```
    var = np.var(x, axis=0)
```

```
    std = np.sqrt(var + eps)
```

```
    x_norm = (x - mu) / std
```

```
# Gradients
```

```
    dgamma = np.sum(dout * x_norm, axis=0)
```

```
    dbeta = np.sum(dout, axis=0)
```

```
    dx_norm = dout * gamma
```

```
    dvar = np.sum(dx_norm * (x - mu) * -0.5 * (var + eps)**(-1.5), axis=0)
```

```
    dmu = np.sum(dx_norm * -1/std, axis=0) + dvar * np.sum(-2*(x - mu), axis=0) / N
```

```
    dx = dx_norm / std + dvar * 2 * (x - mu) / N + dmu / N
```

```
    return dx, dgamma, dbeta
```

23. ENSEMBLE METHODS MATHEMATICS

Bagging (Bootstrap Aggregating)

```
python
```

```

def bootstrap_sample(X, y, n_samples=None):
    """Sample with replacement"""
    if n_samples is None:
        n_samples = len(X)

    indices = np.random.choice(len(X), n_samples, replace=True)
    return X[indices], y[indices]

def bagging_predict(models, X):
    """Average predictions from multiple models"""
    predictions = np.array([model.predict(X) for model in models])
    return np.mean(predictions, axis=0)

# Variance reduction:  $Var(avg) = Var(model) / n\_models$ 

```

Boosting Weight Updates

```

python

# AdaBoost weight update
def adaboost_weights(y_true, y_pred, weights):
    """
    Update sample weights based on errors
    """
    # Error rate
    errors = (y_true != y_pred).astype(float)
    error_rate = np.sum(weights * errors) / np.sum(weights)

    # Model weight
    alpha = 0.5 * np.log((1 - error_rate) / (error_rate + 1e-10))

    # Update sample weights
    new_weights = weights * np.exp(-alpha * y_true * y_pred)
    new_weights /= np.sum(new_weights) # Normalize

    return new_weights, alpha

# Gradient Boosting residual
def gradient_boosting_residual(y_true, y_pred):
    """
    Negative gradient of loss function
    For MSE: residual = y_true - y_pred
    """
    return y_true - y_pred

```

Random Forest Information Gain

python

```
def gini_impurity(y):
    """
    Gini = 1 - Σ p_i^2
    Lower is better (more pure)
    """
    classes, counts = np.unique(y, return_counts=True)
    probabilities = counts / len(y)
    return 1 - np.sum(probabilities**2)

def information_gain(y_parent, y_left, y_right):
    """
    IG = Gini(parent) - weighted_average(Gini(children))
    """
    n = len(y_parent)
    n_left, n_right = len(y_left), len(y_right)

    gini_parent = gini_impurity(y_parent)
    gini_left = gini_impurity(y_left)
    gini_right = gini_impurity(y_right)

    return gini_parent - (n_left/n * gini_left + n_right/n * gini_right)
```

24. BAYESIAN METHODS

Conjugate Priors

python

```
# Beta-Bernoulli conjugate pair
```

```
def beta_bernoulli_posterior(alpha_prior, beta_prior, successes, failures):
```

```
    """
```

```
    Prior: Beta( $\alpha$ ,  $\beta$ )
```

```
    Likelihood: Bernoulli
```

```
    Posterior: Beta( $\alpha + \text{successes}$ ,  $\beta + \text{failures}$ )
```

```
    """
```

```
    alpha_post = alpha_prior + successes
```

```
    beta_post = beta_prior + failures
```

```
# Posterior mean (MAP estimate)
```

```
    posterior_mean = alpha_post / (alpha_post + beta_post)
```

```
    return alpha_post, beta_post, posterior_mean
```

```
# Gaussian-Gaussian conjugate pair
```

```
def gaussian_gaussian_posterior(prior_mu, prior_var, data, likelihood_var):
```

```
    """
```

```
    Prior: N( $\mu_0$ ,  $\sigma^2$ )
```

```
    Likelihood: N( $x \mid \mu$ ,  $\sigma^2$ )
```

```
    Posterior: N( $\mu_{\text{post}}$ ,  $\sigma_{\text{post}}^2$ )
```

```
    """
```

```
    n = len(data)
```

```
    data_mean = np.mean(data)
```

```
# Posterior parameters
```

```
    posterior_precision = 1/prior_var + n/likelihood_var
```

```
    posterior_var = 1 / posterior_precision
```

```
    posterior_mu = posterior_var * (prior_mu/prior_var + n*data_mean/likelihood_var)
```

```
    return posterior_mu, posterior_var
```

Variational Inference

```
python
```

```

def mean_field_vi(data, n_iterations=1000):
    """
    Approximate posterior with factorized distribution
    Minimize KL(q||p)
    """
    # Initialize variational parameters
    mu_q = 0
    sigma_q = 1

    for _ in range(n_iterations):
        # ELBO (Evidence Lower BOund)
        #  $L = E_q[\log p(x,z)] - E_q[\log q(z)]$ 

        # Update variational parameters (example for Gaussian)
        mu_q = np.mean(data) # Simplified update
        sigma_q = np.std(data)

    return mu_q, sigma_q

# ELBO computation
def elbo(data, mu_q, sigma_q, mu_prior, sigma_prior):
    """Evidence Lower Bound"""
    # Reconstruction term
    recon = -0.5 * np.sum((data - mu_q)**2) / sigma_q**2

    # KL divergence term
    kl = 0.5 * (sigma_q**2 / sigma_prior**2 +
               (mu_q - mu_prior)**2 / sigma_prior**2 -
               1 + np.log(sigma_prior**2 / sigma_q**2))

    return recon - kl

```

Gaussian Processes

python

```

def gaussian_process_regression(X_train, y_train, X_test, kernel_fn, noise=1e-8):
    """
    GP regression:  $f(x) \sim \text{GP}(m(x), k(x, x'))$ 
    Posterior:  $p(f^*|X, y, X^*) = N(\mu^*, \Sigma^*)$ 
    """

    # Compute kernel matrices
    K = compute_kernel_matrix(X_train, kernel_fn)
    K_star = np.array([[kernel_fn(x_test, x_train)
                        for x_train in X_train]
                       for x_test in X_test])
    K_star_star = compute_kernel_matrix(X_test, kernel_fn)

    # Add noise to diagonal
    K_noisy = K + noise * np.eye(len(K))

    # Posterior mean:  $\mu^* = K^* K^{(-1)} y$ 
    K_inv = np.linalg.inv(K_noisy)
    mu_star = K_star @ K_inv @ y_train

    # Posterior covariance:  $\Sigma^* = K^{**} - K^* K^{(-1)} K^{*T}$ 
    cov_star = K_star_star - K_star @ K_inv @ K_star.T

    # Posterior standard deviation
    std_star = np.sqrt(np.diag(cov_star))

    return mu_star, std_star

```

ML Applications: Hyperparameter optimization, Bayesian neural networks, uncertainty quantification

25. ADVANCED APPLICATIONS

Generative Adversarial Networks (GANs)

```
python
```

GAN loss functions

```
def discriminator_loss(real_output, fake_output):  
    """  
     $L_D = -E[\log D(x)] - E[\log(1 - D(G(z)))]$   
    Maximize probability of correct classification  
    """  
    real_loss = -np.mean(np.log(real_output + 1e-8))  
    fake_loss = -np.mean(np.log(1 - fake_output + 1e-8))  
    return real_loss + fake_loss
```

```
def generator_loss(fake_output):  
    """  
     $L_G = -E[\log D(G(z))]$   
    Maximize probability of fooling discriminator  
    """  
    return -np.mean(np.log(fake_output + 1e-8))
```

Wasserstein GAN (WGAN)

```
def wasserstein_discriminator_loss(real_output, fake_output):  
    """  
     $L_D = E[D(G(z))] - E[D(x)]$   
    Earth Mover's Distance  
    """  
    return np.mean(fake_output) - np.mean(real_output)
```

```
def wasserstein_generator_loss(fake_output):  
    """  
     $L_G = -E[D(G(z))]$   
    """  
    return -np.mean(fake_output)
```

Variational Autoencoders (VAE)

python

```

def vae_loss(x, x_recon, mu, log_var):
    """
    L = Reconstruction Loss + KL Divergence
    """
    # Reconstruction loss (binary cross-entropy or MSE)
    recon_loss = np.sum((x - x_recon)**2)

    # KL divergence:  $KL(q(z|x) || p(z))$ 
    # For  $q = N(\mu, \sigma^2)$  and  $p = N(0, 1)$ :
    #  $KL = -0.5 * \Sigma(1 + \log(\sigma^2) - \mu^2 - \sigma^2)$ 
    kl_loss = -0.5 * np.sum(1 + log_var - mu**2 - np.exp(log_var))

    return recon_loss + kl_loss

# Reparameterization trick
def reparameterize(mu, log_var):
    """
     $z = \mu + \sigma \cdot \epsilon$ , where  $\epsilon \sim N(0, 1)$ 
    Makes sampling differentiable
    """
    std = np.exp(0.5 * log_var)
    eps = np.random.randn(*mu.shape)
    return mu + std * eps

```

Reinforcement Learning Value Functions

python

Bellman equation: $V(s) = \max_a [R(s,a) + \gamma V(s')]$

```
def value_iteration(rewards, transitions, gamma=0.9, theta=1e-6):
```

```
    """
```

```
    Dynamic programming to find optimal value function
```

```
    """
```

```
    n_states = rewards.shape[0]
```

```
    V = np.zeros(n_states)
```

```
    while True:
```

```
        delta = 0
```

```
        for s in range(n_states):
```

```
            v = V[s]
```

```
            # Bellman update
```

```
            V[s] = np.max([rewards[s, a] + gamma * np.sum(transitions[s, a] * V)
```

```
                           for a in range(rewards.shape[1])])
```

```
            delta = max(delta, abs(v - V[s]))
```

```
    if delta < theta:
```

```
        break
```

```
    return V
```

```
# Q-learning update
```

```
def q_learning_update(Q, s, a, r, s_next, alpha=0.1, gamma=0.9):
```

```
    """
```

```
     $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ 
```

```
    """
```

```
    target = r + gamma * np.max(Q[s_next])
```

```
    Q[s, a] = Q[s, a] + alpha * (target - Q[s, a])
```

```
    return Q
```

Metric Learning

python

```

# Triplet loss
def triplet_loss(anchor, positive, negative, margin=0.2):
    """
    L = max(||f(a) - f(p)||² - ||f(a) - f(n)||² + margin, 0)
    Encourages: d(anchor, positive) < d(anchor, negative)
    """
    pos_dist = np.sum((anchor - positive)**2)
    neg_dist = np.sum((anchor - negative)**2)
    loss = np.maximum(pos_dist - neg_dist + margin, 0)
    return loss

# Contrastive loss
def contrastive_loss(x1, x2, y, margin=1.0):
    """
    y=1 (similar): L = ||x1 - x2||²
    y=0 (dissimilar): L = max(margin - ||x1 - x2||, 0)²
    """
    distance = np.linalg.norm(x1 - x2)
    if y == 1:
        return distance**2
    else:
        return max(margin - distance, 0)**2

```

26. COMPUTATIONAL COMPLEXITY

Big O Notation for ML Algorithms

Algorithm	Training	Prediction	Space
Linear Regression	$O(nd^2 + d^3)$	$O(d)$	$O(d)$
Logistic Regression	$O(ndi)$	$O(d)$	$O(d)$
k-NN	$O(1)$	$O(nd)$	$O(nd)$
Decision Tree	$O(n \cdot d \cdot \log n)$	$O(\log n)$	$O(n)$
Random Forest	$O(k \cdot n \cdot d \cdot \log n)$	$O(k \cdot \log n)$	$O(k \cdot n)$
SVM	$O(n^2d)$ to $O(n^3d)$	$O(sv \cdot d)$	$O(sv \cdot d)$
Neural Network	$O(w \cdot i)$	$O(w)$	$O(w)$
PCA	$O(nd^2 + d^3)$	$O(d^2)$	$O(d^2)$

Algorithm	Training	Prediction	Space
k-Means	$O(k \cdot n \cdot d \cdot i)$	$O(k \cdot d)$	$O(k \cdot d)$

n=samples, d=features, i=iterations, k=trees/clusters, w=weights, sv=support vectors

This comprehensive cheat sheet provides mathematical foundations with practical implementations for real-world machine learning applications.

PyTorch & TensorFlow: Complete Theoretical Guide

Table of Contents

1. Tensors: The Foundation
 2. Neural Network Layers
 3. Activation Functions
 4. Loss Functions
 5. Optimizers
 6. Regularization Techniques
 7. Architectures Explained
 8. Training Concepts
-

Tensors: The Foundation

What are Tensors?

Definition: A tensor is a multi-dimensional array that generalizes scalars, vectors, and matrices to higher dimensions.

Mathematical Hierarchy:

- **Scalar (0D):** Single number $\rightarrow (5)$
- **Vector (1D):** Array of numbers $\rightarrow ([1, 2, 3])$
- **Matrix (2D):** 2D array $\rightarrow ([[1, 2], [3, 4]])$
- **Tensor (3D+):** Multi-dimensional array $\rightarrow ([[[...]]])$

Why Tensors?

Neural networks process data in batches and require efficient parallel computation. Tensors allow:

1. **Batch Processing:** Process multiple samples simultaneously
 - Example: Shape $(32, 784) = 32$ images of 784 pixels each
2. **GPU Acceleration:** GPUs are optimized for tensor operations
 - Matrix multiplication on GPU is 100-1000x faster than CPU
3. **Automatic Differentiation:** Track computational graphs for backpropagation
 - Frameworks automatically compute gradients

When to Use Different Tensor Operations?

Reshape vs View (PyTorch):

- **View:** Fast but requires contiguous memory. Use for simple reshaping
- **Reshape:** More flexible, creates copy if needed. Use when unsure about memory layout
- **Practical:** Use `reshape` by default, it's safer

Element-wise vs Matrix Operations:

- **Element-wise** (`(*)`): Multiplies corresponding elements. Use for scaling/masking
- **Matrix multiplication** (`(@)`): Dot product. Use for linear transformations in neural networks

Example:

```
python

# Element-wise: scaling features
scaled = features * weights # [batch, features] * [batch, features]

# Matrix multiplication: linear layer
output = features @ weights.T # [batch, in_features] @ [out_features, in_features]
```

Neural Network Layers

1. Linear/Dense Layer

Mathematical Definition:

```
y = xW^T + b
where:
- x: input [batch_size, in_features]
- W: weights [out_features, in_features]
- b: bias [out_features]
- y: output [batch_size, out_features]
```

What it does: Performs a linear transformation - every input neuron connects to every output neuron.

When to use:

- **Transformers and modern architectures:** Preferred over Adam
- **Better generalization:** Separates weight decay from optimization
- **Same as Adam otherwise:** Easy drop-in replacement

Adam vs AdamW:

- Adam: Weight decay via L2 regularization (couples with adaptive learning rate)
- AdamW: Decoupled weight decay (more effective)
- Use **AdamW** for better results, especially large models

4. RMSprop

Mathematical Definition:

$$v_t = \beta * v_{t-1} + (1-\beta) * \nabla L^2$$
$$\theta_{t+1} = \theta_t - \eta * \nabla L / (\sqrt{v_t} + \epsilon)$$

Similar to Adam but without momentum term

When to use:

- **RNNs:** Historically good for recurrent networks
 - **Alternative to Adam:** Sometimes works when Adam doesn't
 - **Simpler than Adam:** Fewer hyperparameters
-

Regularization Techniques

Regularization prevents overfitting by constraining model complexity.

1. L2 Regularization (Weight Decay)

Mathematical Definition:

$$\text{Loss_total} = \text{Loss_original} + \lambda * \Sigma(w^2)$$

$$\text{Gradient: } \nabla L_{\text{total}} = \nabla L + 2\lambda w$$

$$\text{Update: } w \leftarrow w - \eta(\nabla L + 2\lambda w) = (1-2\eta\lambda)w - \eta\nabla L$$

Effect: Weights decay toward zero

When to use:

- **Overfitting detected:** Large weights, poor generalization
- **Many parameters:** Penalizes model complexity
- **Default regularization:** Common choice with dropout

Why it works:

- **Ockham's razor:** Simpler models generalize better
- **Smaller weights:** Less sensitive to individual features
- **Smooth predictions:** Prevents over-reliance on single features

Practical example:

```
python

#  $\lambda$  (weight_decay) values:
# 0.0001 - 0.001: Light regularization
# 0.01 - 0.1: Heavy regularization

optimizer = Adam(lr=0.001, weight_decay=0.01)

# Effect on large weight:  $w=10$ 
# Without L2:  $w$  might grow unbounded
# With L2:  $w$  penalized, shrinks toward 0
```

2. L1 Regularization

Mathematical Definition:

$$\text{Loss_total} = \text{Loss_original} + \lambda * \sum |w|$$

Effect: Drives weights exactly to zero (sparse)

When to use:

- **Feature selection:** Want to know which features important
- **Sparse models:** Many weights should be zero
- **Interpretability:** Smaller number of active features

L1 vs L2:

- **L1:** Creates sparse weights (many exactly 0)
- **L2:** Creates small weights (none exactly 0)
- **Choose L1 when:** Need interpretability, feature selection
- **Choose L2 when:** Want all features with small influence

3. Dropout (Covered Earlier)

Main points:

- Randomly drops neurons during training
- Rate 0.5 for dense layers, 0.1-0.2 for conv layers
- Prevents co-adaptation

4. Early Stopping

Definition: Monitor validation loss during training. Stop when it stops improving.

When to use:

- **Always:** Simple, effective regularization
- **Unknown optimal epochs:** Let validation decide
- **Resource constraints:** Don't waste computation

How it works:

```
python

patience = 10 # Wait 10 epochs for improvement
best_val_loss = infinity

for epoch in range(max_epochs):
    train(...)
    val_loss = validate(...)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        save_model()
        patience_counter = 0
    else:
        patience_counter += 1

    if patience_counter >= patience:
        break # Stop training

load_best_model() # Use best, not last
```

5. Data Augmentation

Definition: Create variations of training data (rotation, flip, crop, noise).

When to use:

- **Limited data:** < 1000 samples per class
- **Images:** Always beneficial
- **Overfitting:** Training acc >> validation acc

Common augmentations:

Images:

- Horizontal flip (for natural images)
- Rotation ($\pm 15^\circ$)
- Color jitter (brightness, contrast)
- Random crop
- Gaussian noise

Text:

- Synonym replacement
- Back-translation
- Random insertion/deletion

Why it works:

- Artificially increases dataset size
 - Forces model to learn invariances
 - Reduces overfitting without reducing capacity
-

Architectures Explained

1. Feedforward Neural Network (MLP)

Architecture:

Input $\rightarrow$ Dense $\rightarrow$ Activation $\rightarrow$ Dense $\rightarrow$ Activation $\rightarrow$... $\rightarrow$ Output

When to use:

- **Tabular data:** Features in rows, samples in columns
- **Structured data:** No spatial/temporal structure
- **Baseline model:** Start here before complex architectures
- **Small datasets:** $< 10,000$ samples

Examples:

- House price prediction (features: size, location, age)

- Credit scoring (features: income, debt, history)
- Simple classification (iris flowers)

Why it might fail:

- No spatial awareness (use CNN for images)
- No temporal awareness (use RNN for sequences)
- Requires all features at once

2. Convolutional Neural Network (CNN)

Architecture:

Input → Conv → Activation → Pool → Conv → Pool → Flatten → Dense → Output

Standard pattern:

- Increase filters: $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$
- Decrease spatial size: $224 \rightarrow 112 \rightarrow 56 \rightarrow 28$
- End with global pooling or flatten

When to use:

- **Images:** Primary choice
- **Spatial data:** Any grid-like structure
- **Local patterns matter:** Edges, textures, shapes
- **Translation invariance:** Object anywhere in image

Examples:

- Image classification (cats vs dogs)
- Object detection (find cars in image)
- Semantic segmentation (pixel-wise labeling)
- Medical imaging (X-ray diagnosis)

Key design choices:

1. Depth (number of layers):

- Shallow (3-5 layers): Simple tasks, small images
- Medium (10-20): Standard (ResNet-18, VGG-16)
- Deep (50-150): Complex tasks, large datasets

2. Filter sizes:

- Start: 7×7 or 5×5 (capture larger patterns)
- Middle/End: 3×3 (standard, efficient)
- 1×1 : Channel mixing, dimension reduction

3. Architecture patterns:

- **VGG**: Stack 3×3 convs, simple but deep
- **ResNet**: Skip connections, very deep (50-152 layers)
- **Inception**: Multiple kernel sizes in parallel
- **EfficientNet**: Compound scaling (width, depth, resolution)

3. Recurrent Neural Network (RNN/LSTM/GRU)

Architecture:

Input sequence $\rightarrow$ Embedding $\rightarrow$ LSTM $\rightarrow$ LSTM $\rightarrow$ Dense $\rightarrow$ Output

When to use:

- **Sequential data**: Order matters
- **Variable length**: Different sequence lengths
- **Temporal dependencies**: Past influences future

Examples:

- Language modeling (predict next word)
- Machine translation (English $\rightarrow$ French)
- Speech recognition (audio $\rightarrow$ text)
- Time series forecasting (stock prices)
- Video analysis (action recognition)

Sequence types:

1. **One-to-One**: Standard feedforward (not RNN)
2. **One-to-Many**: Image captioning (image $\rightarrow$ sentence)
3. **Many-to-One**: Sentiment analysis (sentence $\rightarrow$ positive/negative)
4. **Many-to-Many (same length)**: Video classification (frame-by-frame)
5. **Many-to-Many (different length)**: Translation (English $\rightarrow$ French)

4. Encoder-Decoder (Seq2Seq)

Architecture:

Encoder: Input sequence $\rightarrow$ LSTM $\rightarrow$ Context vector

Decoder: Context vector $\rightarrow$ LSTM $\rightarrow$ Output sequence

When to use:

- **Different length input/output:** Translation, summarization
- **Sequence-to-sequence:** Transform one sequence to another

With attention:

- Decoder can "attend" to different encoder positions
- Handles long sequences better
- Foundation for Transformers

5. Autoencoder

Architecture:

Encoder: Input $\rightarrow$ Compress $\rightarrow$ Latent representation

Decoder: Latent $\rightarrow$ Reconstruct $\rightarrow$ Output

Loss: Reconstruction error (MSE between input and output)

When to use:

- **Dimensionality reduction:** Alternative to PCA
- **Anomaly detection:** High reconstruction error = anomaly
- **Denoising:** Train on noisy input, clean target
- **Feature learning:** Use latent representation

Variants:

- **Variational Autoencoder (VAE):** Probabilistic latent space
- **Denoising Autoencoder:** Robust feature learning
- **Sparse Autoencoder:** L1 regularization on latent

6. Transfer Learning

Concept: Use model pretrained on large dataset, adapt to your task.

When to use:

- **Small dataset:** < 1000 samples per class
- **Similar task:** Pretrained domain relevant to yours
- **Limited compute:** Training from scratch expensive

Approaches:

1. Feature extraction (freeze base):

```
python

base_model.trainable = False # Freeze weights
# Only train new layers
```

- **When:** Very small dataset (< 1000 samples)
- **Fast:** Only train small head

2. Fine-tuning (unfreeze some layers):

```
python

# Train new layers first
base_model.trainable = False
model.fit(...) # Train head

# Then fine-tune base
base_model.trainable = True
model.fit(..., lr=1e-5) # Small learning rate
```

- **When:** Medium dataset (1000-10,000 samples)
- **Better performance:** Adapts features to your domain

3. Full training (unfreeze all):

- **When:** Large dataset (> 10,000 samples)
- **Best performance:** If enough data

Popular pretrained models:

- **Images:** ResNet, VGG, EfficientNet, Vision Transformer
- **Text:** BERT, GPT, RoBERTa, T5
- **Source:** ImageNet (images), Wikipedia+BookCorpus (text)

Training Concepts

1. Batch Size

Definition: Number of samples processed before updating weights.

Types:

- **Batch Gradient Descent:** Entire dataset (slow, memory intensive)
- **Stochastic Gradient Descent:** Single sample (noisy, unstable)
- **Mini-batch:** 16-512 samples (standard)

When to use different sizes:

Small batches (16-32):

- **Limited memory:** Large images, large models
- **Regularization effect:** Noise helps generalization
- **Faster iteration:** See more updates per epoch

Large batches (128-512):

- **Stable gradients:** Less noise in updates
- **Better hardware utilization:** GPUs prefer large batches
- **Faster training:** Fewer gradient computations

Practical guidelines:

- Start with 32 or 64
- Increase if training unstable
- Decrease if out of memory
- Powers of 2 for GPU efficiency

2. Learning Rate

Definition: Step size for parameter updates.

Impact:

- **Too high:** Loss explodes, unstable training
- **Too low:** Slow convergence, gets stuck
- **Just right:** Smooth, fast convergence

Finding good learning rate:

1. Learning rate finder:

```
python

# Start small, increase exponentially
lrs = [1e-7, 1e-6, ..., 1e-1]
for lr in lrs:
    loss = train_one_batch(lr)
    plot(lr, loss)

# Choose lr where loss decreasing fastest
```

2. Common starting points:

- Adam: 1e-3 or 3e-4
- SGD: 1e-2 or 1e-1
- Fine-tuning: 1e-5 or 1e-4

3. Learning rate schedules:

Step decay:

```
python

# Decrease by factor every N epochs
lr = 0.1
epochs [0-30]: lr = 0.1
epochs [31-60]: lr = 0.01
epochs [61+]: lr = 0.001
```

Exponential decay:

```
python

lr_t = lr_0 * e(-λt)
# Smooth decrease over time
```

Cosine annealing:

```
python

lr_t = lr_min + 0.5(lr_max - lr_min)(1 + cos(πt/T))
# Decreases then increases (helps escape local minima)
```

Reduce on plateau:

```
python

# If validation loss doesn't improve for N epochs:
lr = lr * 0.1 # Reduce by factor
```

3. Gradient Clipping

Definition: Limit gradient magnitude to prevent explosion.

Mathematical:

```
If ||gradient|| > threshold:
    gradient = threshold * gradient / ||gradient||
```

When to use:

- **RNNs:** Very prone to exploding gradients
- **Very deep networks:** Gradients can grow exponentially
- **Unstable training:** Loss suddenly spikes

Typical values:

- Clip norm: 1.0 or 5.0
- Clip value: 0.5

4. Validation Strategy

Purpose: Estimate model performance on unseen data.

Train/Validation/Test Split:

```
Total data: 100%
├── Training: 70-80% (Train model)
├── Validation: 10-15% (Tune hyperparameters)
└── Test: 10-15% (Final evaluation)
```

K-Fold Cross-Validation:

```
Split data into K folds (e.g., 5)
For each fold:
    Train on K-1 folds
    Validate on 1 fold
Average performance across folds
```


When to use K-fold:

- Small datasets (< 10,000 samples)
- Want robust estimate
- Have computation time (trains K models)

5. Metrics

Classification:

Accuracy:

Accuracy = Correct predictions / Total predictions

Use when: Classes balanced

Don't use when: Imbalanced (99% negative, 1% positive)

Precision & Recall:

Precision = True Positives / (True Positives + False Positives)

"Of predicted positives, how many correct?"

Recall = True Positives / (True Positives + False Negatives)

"Of actual positives, how many found?"

F1-Score = $2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$

Harmonic mean of precision and recall

When to use:

- **High Precision:** Spam detection (false positives annoying)
- **High Recall:** Disease detection (false negatives dangerous)
- **F1-Score:** Balance both

Regression:

MAE (Mean Absolute Error):

Interpretable, same units as target

Use when: Want interpretable error

RMSE (Root Mean Squared Error):

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Penalizes large errors more

Use when: Large errors particularly bad

R² (Coefficient of Determination):

$$R^2 = 1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$$

Range: $(-\infty, 1]$, 1 = perfect, 0 = baseline

Use when: Want relative performance measure

6. Overfitting vs Underfitting

Underfitting:

- **Symptom:** Both train and val accuracy low
- **Cause:** Model too simple, not enough training
- **Solution:**
 - More complex model (more layers/neurons)
 - Train longer
 - Remove regularization
 - Better features

Overfitting:

- **Symptom:** Train accuracy high, val accuracy low
- **Cause:** Model memorizes training data
- **Solution:**
 - More data
 - Data augmentation
 - Regularization (dropout, L2)
 - Simpler model
 - Early stopping

Sweet spot:

- Train and validation accuracy similar
- Both reasonably high
- Model generalizes well

7. Mixed Precision Training

Definition: Use 16-bit floats (half precision) instead of 32-bit for speed.

Benefits:

- **2x faster:** Half the memory bandwidth
- **2x memory:** Can use larger batch sizes
- **Minimal accuracy loss:** Careful handling maintains precision

When to use:

- **Large models:** Memory constrained
- **Modern GPUs:** V100, A100, RTX series with Tensor Cores
- **Production:** Faster inference

How it works:

- **Forward pass:** 16-bit (FP16)
 - **Gradients:** 16-bit
 - **Master weights:** 32-bit (FP32)
 - **Loss scaling:** Prevent underflow
-

Practical Workflow

Complete Training Pipeline

1. Data Preparation:

```
python
- Load data
- Split train/val/test
- Normalize/standardize
- Create DataLoader
- Setup augmentation
```

2. Model Architecture:

```
python
```

- Choose architecture (MLP/CNN/RNN)
- Number of layers
- Hidden sizes
- Activation functions
- Regularization (dropout)

3. Training Setup:

python

- Loss function (task-appropriate)
- Optimizer (start with Adam)
- Learning rate (1e-3 for Adam)
- Batch size (32-64)
- Number of epochs

4. Training Loop:

python

- Forward pass
- Compute loss
- Backward pass
- Update weights
- Log metrics

5. Monitoring:

python

- Plot train/val loss
- Check for overfitting
- Monitor metrics
- Save best model

6. Hyperparameter Tuning:

python

- Adjust learning rate
- Try different architectures
- Tune regularization
- Optimize batch size

7. Evaluation:

python

- Load best model
- Evaluate on test set
- Compute final metrics
- Analyze errors

Common Mistakes to Avoid

1. **Not shuffling data:** Model learns order
 2. **Not normalizing:** Features on different scales
 3. **Testing on training data:** Overly optimistic results
 4. **Ignoring class imbalance:** Accuracy misleading
 5. **Not using validation set:** Can't detect overfitting
 6. **Too complex model first:** Start simple
 7. **Not checking for data leakage:** Information from test in train
 8. **Forgetting model.eval():** Dropout/BatchNorm behave differently
-

Quick Decision Tree

What task?

Classification (categorical output):

- Tabular data → MLP
- Images → CNN
- Text → RNN/LSTM or Transformer
- Time series → LSTM/GRU

Regression (continuous output):

- Tabular data → MLP
- Images → CNN
- Sequential → LSTM/GRU

Architecture depth:

- < 1K samples → Shallow (3-5 layers)

- 1K-100K samples → Medium (10-20 layers)
-  100K samples → Deep (20-50+ layers) or pretrained

Regularization:

- Overfitting → Dropout (0.3-0.5), L2 (0.001-0.01)
- Small data → Data augmentation
- Always → Early stopping

Optimizer:

- Default → Adam (lr=1e-3)
- Not working → Try AdamW or SGD with momentum
- RNN → Try RMSprop

Loss function:

- Binary classification → BCELoss + Sigmoid
- Multi-class → CrossEntropyLoss + Softmax
- Regression → MSE or MAE
- Regression with outliers → Huber Loss

This guide should give you the theoretical foundation to understand when, why, and how to use each component effectively!

- **Classification tasks:** Final layer to produce class probabilities
- **Regression:** Output layer for continuous predictions
- **Feature transformation:** Hidden layers to learn representations
- **After flattening:** Convert CNN features to predictions

Why it works:

- **Universal function approximation:** Multiple linear layers with non-linearities can approximate any function
- **Feature combinations:** Learns weighted combinations of input features

Practical example:

```
python
```

Image classification: 28x28 image $\rightarrow$ 10 classes

Flatten: 784 inputs

Hidden: 128 neurons (learns representations)

Output: 10 classes

```
model = nn.Sequential(  
    nn.Linear(784, 128), # Learn 128 features from pixels  
    nn.ReLU(),  
    nn.Linear(128, 10) # Combine features for classification  
)
```

Parameters: $(in_features \times out_features + out_features)$ (weights + biases)

2. Convolutional Layers (Conv2D)

Mathematical Definition:

For each output position (i, j):

$$y[i,j] = \sum \sum x[i+m, j+n] * kernel[m, n] + bias$$

The kernel "slides" over the input, computing dot products.

What it does: Applies filters (kernels) that slide over the input to detect local patterns.

When to use:

- **Images:** Natural choice for spatial data
- **Time series:** Conv1D for temporal patterns
- **Any grid-like data:** Where local patterns matter

Why it works:

- **Translation invariance:** Same pattern detected anywhere in image
- **Parameter sharing:** Same kernel used across entire input (fewer parameters)
- **Hierarchy:** Early layers detect edges, later layers detect complex patterns

Key Concepts:

1. Kernel/Filter Size:

- 3×3 : Standard, captures local patterns (edges, textures)
- 5×5 or 7×7 : Larger receptive field, captures broader patterns
- 1×1 : Channel-wise mixing, reduces dimensions

2. Stride:

- Stride=1: Output nearly same size as input
- Stride=2: Downsamples by 2x (common alternative to pooling)
- Use **larger strides** to reduce computation and spatial dimensions

3. Padding:

- `padding='same'`: Output size = input size (adds zeros around border)
- `padding='valid'`: No padding, output shrinks
- Use **'same'** to preserve spatial dimensions

4. Number of Filters:

- Controls how many different patterns to detect
- Common: $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ (increases with depth)
- More filters = more capacity but more computation

Practical example:

```
python

# Image classification CNN
Conv2D(3, 32, kernel_size=3, padding=1) # 3 RGB → 32 features
# Detects edges, colors, simple textures

Conv2D(32, 64, kernel_size=3, padding=1) # 32 → 64 features
# Combines edges into shapes, patterns

Conv2D(64, 128, kernel_size=3, padding=1) # 64 → 128 features
# Detects complex objects, parts
```

Parameters: $\text{kernel_size}^2 \times \text{in_channels} \times \text{out_channels} + \text{out_channels}$

- Example: 3×3 kernel, $32 \rightarrow 64$ channels $= 3 \times 3 \times 32 \times 64 + 64 = 18,496$ parameters

Why fewer parameters than Dense?

- Dense layer with same input ($32 \times 32 \times 32 = 32,768$ inputs) to 64 outputs: 2+ million parameters
- Conv layer: Only ~18K parameters but achieves spatial understanding

3. Pooling Layers

Mathematical Definition:

Max Pooling:

For each 2×2 region:
output = $\max(x[i:i+2, j:j+2])$

Average Pooling:

For each 2×2 region:
output = $\text{mean}(x[i:i+2, j:j+2])$

What it does: Downsamples spatial dimensions by taking maximum or average over regions.

When to use:

- **After Conv layers:** Reduce spatial dimensions progressively
- **Reduce computation:** Fewer pixels = faster training
- **Prevent overfitting:** Adds translation invariance
- **Control output size:** Before dense layers

Max vs Average Pooling:

- **MaxPool:** Keeps strongest activations, better for textures/patterns
 - Use for: Classification, feature detection
- **AvgPool:** Smooths activations, better for general features
 - Use for: When preserving all information matters
- **Global Average/Max Pooling:** Reduce entire spatial dimension to 1×1
 - Use before final classification layer (replaces flatten)

Practical example:

```
python

# Input: 224×224 image
Conv2D(3, 64)    # 224×224×64
MaxPool2D(2)     # 112×112×64 (halved spatial dimensions)

Conv2D(64, 128)  # 112×112×128
MaxPool2D(2)     # 56×56×128

# Reduces computation by 75% at each step
# Final spatial size manageable for dense layers
```

Why use pooling?

- **Computational efficiency:** 2×2 pooling reduces dimensions by $4 \times$
 - **Invariance:** Small translations don't change output
 - **Receptive field:** Each neuron "sees" larger input region
-

4. Recurrent Layers (RNN, LSTM, GRU)

Mathematical Definitions:

Simple RNN:

$$h_t = \tanh(W_{ih} * x_t + b_{ih} + W_{hh} * h_{t-1} + b_{hh})$$

where:

- h_t : hidden state at time t
- x_t : input at time t
- h_{t-1} : previous hidden state

LSTM (Long Short-Term Memory):

$$\text{Forget gate: } f_t = \sigma(W_f * [h_{t-1}, x_t] + b_f)$$

$$\text{Input gate: } i_t = \sigma(W_i * [h_{t-1}, x_t] + b_i)$$

$$\text{Output gate: } o_t = \sigma(W_o * [h_{t-1}, x_t] + b_o)$$

$$\text{Cell state: } C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_c * [h_{t-1}, x_t] + b_c)$$

$$\text{Hidden state: } h_t = o_t \odot \tanh(C_t)$$

where σ = sigmoid, $\odot$ = element-wise multiplication

GRU (Gated Recurrent Unit):

$$\text{Reset gate: } r_t = \sigma(W_r * [h_{t-1}, x_t])$$

$$\text{Update gate: } z_t = \sigma(W_z * [h_{t-1}, x_t])$$

$$\text{Hidden state: } h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W * [r_t \odot h_{t-1}, x_t])$$

When to use:

RNN (Simple):

- **Short sequences:** < 20 timesteps
- **Simple patterns:** When long-term dependencies not critical
- **Fast training needed:** Fewer parameters than LSTM/GRU
- **Example:** Sentiment analysis on short sentences

LSTM:

- **Long sequences:** 100s to 1000s of timesteps
- **Long-term dependencies:** When context from far back matters
- **Language modeling:** Text generation, translation
- **Time series:** Stock prices, weather prediction
- **Example:** Machine translation (need to remember beginning of sentence)

GRU:

- **Middle ground:** Shorter than LSTM sequences but need memory
- **Faster than LSTM:** 25-30% fewer parameters
- **Similar performance:** Often matches LSTM with less computation
- **Example:** Speech recognition, music generation

Key Concepts:

1. Hidden State ($\mathbf{h_t}$):

- The "memory" of the network
- Carries information from previous timesteps
- Size determines capacity (common: 128, 256, 512)

2. return_sequences:

- `True`: Return output for every timestep [batch, seq_len, hidden]
- `False`: Return only last output [batch, hidden]
- **Use True** for: Sequence-to-sequence tasks
- **Use False** for: Classification from sequences

3. Bidirectional:

- Process sequence forward AND backward
- Captures context from both directions
- **Use for:** When entire sequence available (not streaming)
- Doubles parameters and computation

Practical example:

```
python
```

```
# Text classification (sentiment analysis)
Embedding(vocab_size=10000, embedding_dim=128) # Words → vectors
LSTM(128, return_sequences=False) # Process sequence, return final state
Dense(1, activation='sigmoid') # Binary classification

# Sequence-to-sequence (translation)
# Encoder
LSTM(256, return_sequences=True) # Process source language
LSTM(256, return_sequences=False) # Final encoder state

# Decoder
LSTM(256, return_sequences=True) # Generate target language
Dense(vocab_size, activation='softmax') # Predict words
```

Why LSTM over RNN?

- **Vanishing gradient problem:** Simple RNN gradients disappear over long sequences
- **LSTM gates:** Control information flow, preserve gradients
- **Long-term memory:** LSTM cell state acts as "highway" for information

Parameters comparison:

- RNN: $4 \times (\text{hidden_size} \times \text{input_size} + \text{hidden_size}^2)$
- LSTM: $4 \times \text{RNN}$ (4 gates)
- GRU: $3 \times \text{RNN}$ (3 gates) - 25% fewer than LSTM

5. Embedding Layer

Mathematical Definition:

Given integer input i (word index):
 $\text{embedding}[i] \rightarrow \mathbb{R}^d$ (d-dimensional vector)

Essentially a lookup table: $\text{matrix}[\text{vocab_size}, \text{embedding_dim}]$

What it does: Converts discrete tokens (words, items) into continuous dense vectors.

When to use:

- **Natural Language Processing:** Convert words to vectors
- **Recommendation systems:** Convert user/item IDs to embeddings
- **Categorical features:** Any discrete input with many categories

- **Graph neural networks:** Node embeddings

Why it works:

- **Semantic similarity:** Similar words get similar vectors
- **Dimensionality reduction:** 10,000 words $\rightarrow$ 300 dimensions
- **Learnable:** Embeddings trained specifically for your task

Key Concepts:

1. Vocabulary Size (**input_dim**):

- How many unique tokens (words, items)
- Example: 10,000 most common words

2. Embedding Dimension (**output_dim**):

- Vector size for each token
- Common choices:
 - 50-100: Small datasets, simple tasks
 - 128-256: Standard choice
 - 300-768: Large models (BERT uses 768)
- Trade-off: Larger = more expressiveness but more parameters

3. Pre-trained Embeddings:

- Word2Vec, GloVe, FastText: Trained on large corpora
- Transfer learning for NLP
- **Use when:** Limited training data
- **Fine-tune when:** Your domain is specific

Practical example:

```
python

# Movie recommendation
# 10,000 movies  $\rightarrow$  128-dim vectors
# 50,000 users  $\rightarrow$  128-dim vectors

movie_embedding = Embedding(10000, 128)
user_embedding = Embedding(50000, 128)

# Predict rating: dot product of embeddings
rating = user_vec · movie_vec

# Similar users/movies have high dot product
```

Why not one-hot encoding?

- One-hot: [0,0,0,1,0,...,0] (size = vocab_size)
 - 10,000 words = 10,000 dimensions
 - No similarity information
 - Sparse, inefficient
- Embedding: [0.2, -0.5, 0.8, ...] (size = embedding_dim)
 - 10,000 words = 128 dimensions (78x smaller)
 - Similar words have similar vectors
 - Dense, efficient

Parameters: $\text{vocab_size} \times \text{embedding_dim}$

- 10,000 words $\times$ 128 dim = 1.28M parameters
-

6. Normalization Layers

BatchNorm Mathematical Definition:

For each feature across batch:

$$\mu = (1/m) \sum x_i \quad (\text{mean})$$

$$\sigma^2 = (1/m) \sum (x_i - \mu)^2 \quad (\text{variance})$$

$$y = \gamma * (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$$

where:

- γ, β : learnable parameters
- ϵ : small constant for numerical stability (10^{-5})

What it does: Normalizes layer inputs to have mean=0, variance=1, then applies learnable scale and shift.

When to use:

BatchNorm:

- **Deep networks:** Essential for 10+ layers
- **Convolutional networks:** After conv layers (before or after activation)
- **When:** Batch size ≥ 16 (needs statistics from batch)
- **Not for:** Very small batches, online learning, or RNNs

LayerNorm:

- **RNNs/Transformers:** Batch size independent
- **Small batches:** Works with batch_size=1
- **Variable length sequences:** Better for NLP
- **Normalizes:** Across features, not batch

Why it works:

1. **Faster training:** Can use higher learning rates (10-100x)
2. **Reduces internal covariate shift:** Stabilizes input distributions
3. **Regularization effect:** Adds noise (each sample normalized differently)
4. **Gradient flow:** Prevents vanishing/exploding gradients

Practical example:

```
python

# CNN with BatchNorm
Conv2D(64, 3)
BatchNorm2D(64)  # Normalize 64 channels
ReLU()

# Why this order? Debated, but common:
# Conv → BN → Activation (most common)
# Conv → Activation → BN (also works)

# Without BatchNorm: may need learning_rate = 0.001
# With BatchNorm: can use learning_rate = 0.01 or higher
```

BatchNorm vs LayerNorm:

BatchNorm: Normalizes across batch dimension
 Input: [batch=32, channels=64, height=28, width=28]
 Normalizes: For each of 64 channels, across 32 samples

LayerNorm: Normalizes across feature dimension
 Input: [batch=32, features=512]
 Normalizes: For each sample, across 512 features

Training vs Inference:

- **Training:** Uses batch statistics (mean, variance)
- **Inference:** Uses running statistics (exponential moving average)
- **Why:** Single sample at inference, no batch to compute statistics

7. Dropout

Mathematical Definition:

Training:

$$y = x \odot m / (1-p)$$

where $m \sim \text{Bernoulli}(1-p)$ (random mask)

Inference:

$$y = x \text{ (no dropout)}$$

What it does: Randomly sets a fraction (p) of inputs to zero during training.

When to use:

- **Overfitting detected:** Training accuracy $\gg$ validation accuracy
- **After dense layers:** Most common placement
- **Large networks:** More parameters = more overfitting risk
- **Limited data:** Dropout acts as ensemble of sub-networks

When NOT to use:

- **Small models:** May hurt performance
- **Already underfitting:** Will make it worse
- **Convolutional layers:** Less common, use sparingly (dropout2d exists)

Why it works:

- **Prevents co-adaptation:** Neurons can't rely on specific other neurons
- **Ensemble effect:** Each training step uses different sub-network
- **Implicit regularization:** Similar effect to L2 regularization

Key Concepts:

1. Dropout Rate (p) :

- $p=0.2$: Drop 20% of neurons (light regularization)
- $p=0.5$: Drop 50% (standard choice, heavy regularization)
- $p=0.8$: Drop 80% (very aggressive, rarely used)

2. Placement:


```
python
```

```
# Standard pattern
```

```
Dense(512)
```

```
ReLU()
```

```
Dropout(0.5) # After activation
```

```
Dense(256)
```

```
ReLU()
```

```
Dropout(0.5)
```

```
Dense(10) # No dropout on output layer
```

3. Scaling:

- During training: multiply by $1/(1-p)$ to maintain expected output
- Ensures same scale during training and inference

Practical example:

```
python
```

```
# Without dropout: overfitting
```

```
# Train accuracy: 99%, Val accuracy: 75%
```

```
# With dropout: better generalization
```

```
# Train accuracy: 92%, Val accuracy: 88%
```

```
model = Sequential([
```

```
    Dense(512, activation='relu'),
```

```
    Dropout(0.5), # Regularizes
```

```
    Dense(256, activation='relu'),
```

```
    Dropout(0.3), # Less aggressive in deeper layers
```

```
    Dense(10, activation='softmax')
```

```
])
```

Activation Functions

Activation functions introduce non-linearity. Without them, stacked linear layers = single linear layer.

1. ReLU (Rectified Linear Unit)

Mathematical Definition:

$$f(x) = \max(0, x)$$

Derivative:

$$f'(x) = 1 \text{ if } x > 0$$
$$0 \text{ if } x \leq 0$$

When to use:

- **Default choice** for hidden layers
- **CNNs:** Almost always ReLU
- **MLPs:** Standard activation
- **Fast computation:** Just thresholding

Why it works:

- **Solves vanishing gradient:** Gradient is 1 for positive values
- **Sparse activations:** Many neurons output 0 (efficient)
- **Biological plausibility:** Neurons either fire or don't

Problems:

- **Dying ReLU:** If neuron outputs always negative, gradient = 0 forever
 - Solution: Use LeakyReLU or careful initialization

2. LeakyReLU

Mathematical Definition:

$$f(x) = x \text{ if } x > 0$$
$$\alpha x \text{ if } x \leq 0 \text{ (typically } \alpha = 0.01)$$

When to use:

- **Dying ReLU problem:** When many neurons become inactive
- **Alternative to ReLU:** Similar benefits, prevents death
- **Experimentation:** Try if ReLU doesn't work well

3. Sigmoid

Mathematical Definition:

$$f(x) = 1 / (1 + e^{(-x)})$$

Output range: (0, 1)

When to use:

- **Binary classification:** Output layer (single neuron)
- **Probabilities:** When need output in [0,1]
- **Gates in LSTM/GRU:** Controls information flow

When NOT to use:

- **Hidden layers:** Causes vanishing gradient problem
- **Multi-class:** Use softmax instead

Why output layer only:

- Gradients very small for $|x| > 3$
- Network learns slowly in hidden layers

4. Tanh

Mathematical Definition:

$$f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

Output range: (-1, 1)

When to use:

- **RNN hidden states:** Traditionally used (before LSTM/GRU)
- **When need negative outputs:** Unlike sigmoid [0,1]
- **Centered data:** Mean closer to 0 than sigmoid

Advantage over sigmoid:

- Zero-centered: Easier optimization
- Stronger gradients: Range [-1, 1] vs [0, 1]

5. Softmax

Mathematical Definition:

For vector $x = [x_1, \dots, x_n]$:

$$\text{softmax}(x)_i = e^{(x_i)} / \sum_j e^{(x_j)}$$

Properties:

- Output sums to 1
- Each output in (0, 1)
- Represents probability distribution

When to use:

- **Multi-class classification:** Always use for output layer
- **Probability distribution:** When need valid probabilities
- **Mutually exclusive classes:** Cat OR dog (not both)

Why it works:

- Converts logits (raw scores) to probabilities
- Differentiable: Can backpropagate through it
- Emphasizes largest values (exponential)

Practical example:

```
python

# 10-class classification
logits = [2.0, 1.0, 0.1, ...] # Raw network outputs

# Softmax
probs = softmax(logits)
# [0.68, 0.25, 0.03, ...] # Probabilities sum to 1

predicted_class = argmax(probs) # Class with highest probability
```

Loss Functions

Loss functions measure how wrong your predictions are. Training minimizes this loss.

1. Cross-Entropy Loss

Mathematical Definition:

Binary Cross-Entropy:

$$L = -[y * \log(\hat{y}) + (1-y) * \log(1-\hat{y})]$$

where:

- y: true label (0 or 1)
- $\hat{y}$: predicted probability

Categorical Cross-Entropy:

$$L = -\sum_i y_i * \log(\hat{y}_i)$$

where:

- y_i : true probability (one-hot: [0,0,1,0,...])
- $\hat{y}_i$: predicted probability from softmax

When to use:

- **Classification tasks:** Primary choice
- **Binary:** Sigmoid output + BCE loss
- **Multi-class:** Softmax output + CE loss
- **Probabilistic outputs:** When predictions are probabilities

Why it works:

- **Information theory:** Measures difference between distributions
- **Probabilistic interpretation:** Maximum likelihood estimation
- **Gradient behavior:** Large errors give large gradients (fast learning)

Practical example:

```
python
```

```
# Binary: Spam detection (0=not spam, 1=spam)
```

```
true = 1
```

```
predicted = 0.8 # Model 80% confident it's spam
```

```
loss = -(1*log(0.8)) = 0.22 # Small loss (correct)
```

```
predicted = 0.2 # Model only 20% confident
```

```
loss = -(1*log(0.2)) = 1.61 # Large loss (wrong)
```

```
# Multi-class: Image classification
```

```
true = [0, 0, 1, 0, 0] # Class 2
```

```
predicted = [0.1, 0.2, 0.6, 0.05, 0.05]
```

```
loss = -(0*log(0.1) + 0*log(0.2) + 1*log(0.6) + ...)
      = -log(0.6) = 0.51
```

2. Mean Squared Error (MSE)

Mathematical Definition:

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

When to use:

- **Regression tasks:** Predicting continuous values
- **Price prediction:** House prices, stock prices
- **Temperature, age, salary:** Any numerical output
- **When outliers matter:** MSE heavily penalizes large errors

Why it works:

- **Least squares:** Optimal under Gaussian noise assumption
- **Convex:** Single global minimum (easy optimization)
- **Interpretable:** Same units as target (squared)

Practical example:

```
python
```

House price prediction

true = 300,000

predicted = 320,000

error = 20,000

MSE = $20,000^2 = 400,000,000$

Temperature prediction

true = 25°C

predicted = 24°C

error = 1°C

MSE = $1^2 = 1$

3. Mean Absolute Error (MAE)

Mathematical Definition:

$$\text{MAE} = (1/n) \sum |y_i - \hat{y}_i|$$

When to use:

- **Outlier-robust regression:** When outliers shouldn't dominate
- **Interpretable errors:** Errors in same units as target
- **When all errors equal:** Don't want to heavily penalize large errors

MSE vs MAE:

- **MSE:** Penalizes large errors more (quadratic)
 - Use when: Large errors are particularly bad
- **MAE:** Treats all errors linearly
 - Use when: All errors equally important

Example:

python

Errors: [1, 2, 10]

$$\text{MAE} = (1 + 2 + 10) / 3 = 4.33$$

$$\text{MSE} = (1^2 + 2^2 + 10^2) / 3 = (1 + 4 + 100) / 3 = 35$$

MSE dominated by outlier (10), MAE treats all fairly

4. Huber Loss

Mathematical Definition:

$$L = 0.5 * (y - \hat{y})^2 \quad \text{if } |y - \hat{y}| \leq \delta$$
$$\delta * (|y - \hat{y}| - 0.5\delta) \quad \text{otherwise}$$

When to use:

- **Best of both worlds:** MSE for small errors, MAE for large
 - **Robust regression:** When some outliers exist
 - **Parameter δ :** Controls transition point (typically 1.0)
-

Optimizers

Optimizers update network weights to minimize loss. They implement gradient descent variants.

Basic Gradient Descent

Mathematical Definition:

$$\theta_{t+1} = \theta_t - \eta * \nabla L(\theta_t)$$

where:

- θ : parameters (weights)
- η : learning rate
- ∇L : gradient of loss

Problem: Updates all parameters with same learning rate. Slow convergence.

1. SGD (Stochastic Gradient Descent)

Mathematical Definition:

$$v_t = \gamma * v_{t-1} + \eta * \nabla L(\theta_t) \quad (\text{momentum})$$
$$\theta_{t+1} = \theta_t - v_t$$

where:

- γ : momentum (typically 0.9)
- Accumulates velocity in consistent directions

When to use:

- **Simple problems:** Small networks, simple data

- **Well-understood data:** When good hyperparameters known
- **Momentum helps:** Gets stuck in local minima less

Why momentum:

- Accelerates in consistent directions
- Dampens oscillations
- Escapes shallow local minima

Practical example:

```
python

# Without momentum: oscillates in ravines
# With momentum: accelerates downhill

optimizer = SGD(lr=0.01, momentum=0.9)
```

2. Adam (Adaptive Moment Estimation)

Mathematical Definition:

$$m_t = \beta_1 * m_{t-1} + (1-\beta_1) * \nabla L \quad (1st \text{ moment: mean})$$

$$v_t = \beta_2 * v_{t-1} + (1-\beta_2) * \nabla L^2 \quad (2nd \text{ moment: variance})$$

$$\hat{m}_t = m_t / (1 - \beta_1^t) \quad (\text{bias correction})$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta_{t+1} = \theta_t - \eta * \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

Defaults: $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$

When to use:

- **Default choice:** Works well for most problems
- **Deep networks:** Adapts learning rate per parameter
- **Sparse gradients:** NLP, embeddings
- **Start here:** If it doesn't work, try others

Why it works:

- **Adaptive learning rates:** Different rate for each parameter
- **Momentum:** Accelerates learning

- **Moving average of gradients:** Smooths noisy estimates
- **Parameter-wise:** Handles different scales automatically

Key Concepts:

1. Learning Rate (η):

- Default: 0.001 (works well often)
- Too high: Unstable, loss explodes
- Too low: Slow convergence
- Start with 0.001, decrease if unstable

2. β_1, β_2 :

- Rarely need to change
- β_1 controls momentum
- β_2 controls adaptive learning rate

Practical example:

```
python

# Parameter 1: gradients = [1, 1, 1, 1, ...] (consistent)
# → Adam increases learning rate for this parameter

# Parameter 2: gradients = [1, -1, 1, -1, ...] (noisy)
# → Adam decreases learning rate for this parameter

# Automatically handles different parameters!
```

3. AdamW

Mathematical Definition:

Same as Adam, but weight decay applied directly:

$$\theta_{t+1} = \theta_t - \eta * (\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda * \theta_t)$$

where λ is weight decay coefficient

When to use:

Mathematics for Machine Learning: Complete Cheat Sheet

1. Linear Algebra

Vectors

Concepts:

- Vector: ordered list of numbers $[x_1, x_2, \dots, x_n]$
- Magnitude: $\|v\| = \sqrt{(x_1^2 + x_2^2 + \dots + x_n^2)}$
- Dot Product: $v \cdot w = \sum (v_i w_i)$
- Unit Vector: $v / \|v\|$

Real Applications:

- Word embeddings in NLP (word2vec, GloVe) represent words as vectors
- Image pixels as vectors for computer vision
- User preferences in recommendation systems
- Feature vectors in any ML model

Matrices

Concepts:

- Matrix: 2D array of numbers ($m \times n$)
- Transpose: A^T flips rows and columns
- Matrix Multiplication: $(AB)_{ij} = \sum_k A_{ik} B_{kj}$
- Identity Matrix: I (diagonal of 1s)
- Inverse: A^{-1} where $AA^{-1} = I$
- Determinant: $\det(A)$ - scalar value

Real Applications:

- Neural network weight matrices transform inputs
- Image transformations (rotation, scaling, shearing)
- Data transformation in preprocessing
- Covariance matrices for PCA

Eigenvalues & Eigenvectors

Concepts:

- $Av = \lambda v$ (v is eigenvector, λ is eigenvalue)
- Eigendecomposition: $A = Q\Lambda Q^{-1}$
- Captures directions of maximum variance

Real Applications:

- Principal Component Analysis (PCA) for dimensionality reduction
- Google PageRank algorithm
- Stability analysis in neural networks
- Face recognition (Eigenfaces)

Singular Value Decomposition (SVD)

Concepts:

- $A = U\Sigma V^T$
- U : left singular vectors, Σ : singular values, V : right singular vectors
- Works for any matrix (not just square)

Real Applications:

- Recommender systems (Netflix, Amazon)
- Latent Semantic Analysis in NLP
- Image compression
- Noise reduction in data

Norms

Concepts:

- L1 norm: $\|x\|_1 = \sum |x_i|$
- L2 norm: $\|x\|_2 = \sqrt{\sum x_i^2}$
- L ∞ norm: $\|x\|_\infty = \max(|x_i|)$

Real Applications:

- L1 regularization (Lasso) for feature selection
 - L2 regularization (Ridge) to prevent overfitting
 - Distance metrics in clustering (K-means)
 - Loss function components
-

2. Calculus

Derivatives

Concepts:

- $f'(x) = \lim_{h \rightarrow 0} (f(x+h) - f(x))/h$
- Chain Rule: $(f \circ g)'(x) = f'(g(x)) \cdot g'(x)$
- Partial Derivatives: $\partial f / \partial x$ (derivative with respect to one variable)

Real Applications:

- Gradient descent optimization
- Backpropagation in neural networks
- Finding optimal parameters
- Sensitivity analysis

Gradients

Concepts:

- $\nabla f = [\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n]$
- Direction of steepest ascent
- Points perpendicular to level curves

Real Applications:

- Training all ML models (gradient descent)
- Computer vision edge detection
- Optimization algorithms (Adam, RMSprop)
- Neural network weight updates

Jacobian Matrix

Concepts:

- Matrix of all first-order partial derivatives
- $J = [\partial f_i / \partial x_j]$
- Maps how vector-valued function changes

Real Applications:

- Neural network layer transformations

- Robotics (velocity transformations)
- Automatic differentiation frameworks
- Sensitivity of model outputs

Hessian Matrix

Concepts:

- Matrix of second-order partial derivatives
- $H = [\partial^2 f / \partial x_i \partial x_j]$
- Indicates curvature

Real Applications:

- Second-order optimization methods
- Identifying saddle points in loss landscapes
- Newton's method for optimization
- Understanding model convergence

Integration

Concepts:

- $\int f(x) dx$ (area under curve)
- Definite integral: $\int_a^b f(x) dx$
- Fundamental Theorem: $\int f'(x) dx = f(x) + C$

Real Applications:

- Probability density functions (computing probabilities)
 - Expected values in reinforcement learning
 - Bayesian inference
 - Monte Carlo methods
-

3. Probability & Statistics

Probability Basics

Concepts:

- $P(A)$: probability of event A, range $[0,1]$

- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
- $P(A|B) = P(A \cap B)/P(B)$ (conditional probability)
- Bayes' Theorem: $P(A|B) = P(B|A)P(A)/P(B)$

Real Applications:

- Spam filtering (Naive Bayes)
- Medical diagnosis systems
- Weather prediction models
- Document classification

Random Variables

Concepts:

- Discrete: finite/countable outcomes
- Continuous: infinite outcomes in range
- Expected Value: $E[X] = \sum x \cdot P(X=x)$
- Variance: $\text{Var}(X) = E[(X-\mu)^2]$

Real Applications:

- Model uncertainty quantification
- A/B testing analysis
- Financial risk modeling
- Sensor noise modeling

Probability Distributions

Discrete:

- Bernoulli: single yes/no trial
- Binomial: n independent trials
- Poisson: count of events in interval

Continuous:

- Uniform: all values equally likely
- Normal/Gaussian: $N(\mu, \sigma^2)$, bell curve
- Exponential: time between events

Real Applications:

- Logistic regression (Bernoulli for binary classification)
- Image pixel intensities (Gaussian noise)
- Natural language word frequencies (Poisson)
- Generative Adversarial Networks (various distributions)

Central Limit Theorem

Concepts:

- Sum of independent random variables $\rightarrow$ Normal distribution
- Holds regardless of original distribution
- $n \geq 30$ typically sufficient

Real Applications:

- Confidence intervals for model predictions
- Bootstrap sampling methods
- A/B test validity
- Error analysis

Maximum Likelihood Estimation (MLE)

Concepts:

- Find parameters that maximize $P(\text{data}|\text{parameters})$
- $L(\theta) = \prod P(x_i|\theta)$
- Often use log-likelihood: $\log L(\theta) = \sum \log P(x_i|\theta)$

Real Applications:

- Training logistic regression
- Fitting Gaussian Mixture Models
- Parameter estimation in any probabilistic model
- Neural network optimization (cross-entropy loss)

Hypothesis Testing

Concepts:

- Null hypothesis H_0 vs alternative H_1
- p-value: probability of observing data if H_0 true
- Significance level α (typically 0.05)

- Type I error (false positive), Type II error (false negative)

Real Applications:

- A/B testing for product features
- Model comparison (is new model better?)
- Feature selection significance
- Clinical trial analysis

Covariance & Correlation

Concepts:

- $\text{Cov}(X, Y) = E[(X - \mu_x)(Y - \mu_y)]$
- Correlation: $\rho = \text{Cov}(X, Y) / (\sigma_x \sigma_y)$, range $[-1, 1]$
- Covariance Matrix: $\Sigma_{ij} = \text{Cov}(X_i, X_j)$

Real Applications:

- Feature correlation analysis
 - Portfolio optimization
 - Multivariate Gaussian distributions
 - Dimensionality reduction (PCA)
-

4. Optimization

Gradient Descent

Concepts:

- $\theta_{new} = \theta_{old} - \alpha \nabla f(\theta)$
- α : learning rate
- Iteratively move toward minimum
- Batch, Mini-batch, Stochastic variants

Real Applications:

- Training all neural networks
- Linear regression optimization
- Logistic regression training
- Deep learning model optimization

Convex Optimization

Concepts:

- Convex function: $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$
- Single global minimum
- Gradient descent guaranteed to converge

Real Applications:

- Support Vector Machines (SVM)
- Linear regression (closed-form solution)
- Lasso and Ridge regression
- Many traditional ML algorithms

Lagrange Multipliers

Concepts:

- Optimize $f(x)$ subject to $g(x) = 0$
- $\nabla f = \lambda \nabla g$ at optimum
- Handles constrained optimization

Real Applications:

- SVM margin optimization
- Constrained neural network training
- Resource allocation problems
- Portfolio optimization with constraints

Adam, RMSprop, Momentum

Concepts:

- Momentum: accumulates velocity
- RMSprop: adaptive learning rate
- Adam: combines momentum + RMSprop
- Faster convergence than vanilla gradient descent

Real Applications:

- Default optimizer for deep learning

- Training large language models
 - Computer vision models (ResNet, YOLO)
 - Faster training of complex models
-

5. Information Theory

Entropy

Concepts:

- $H(X) = -\sum P(x)\log P(x)$
- Measures uncertainty/information content
- Higher entropy = more uncertain

Real Applications:

- Decision tree splitting criteria
- Feature selection
- Compression algorithms
- Measuring model uncertainty

Cross-Entropy

Concepts:

- $H(P,Q) = -\sum P(x)\log Q(x)$
- Measures difference between distributions
- Used as loss function

Real Applications:

- Classification loss function
- Training neural networks
- Natural language processing models
- Image classification (softmax + cross-entropy)

KL Divergence

Concepts:

- $D_{KL}(P\|Q) = \sum P(x)\log(P(x)/Q(x))$

- Measures how Q differs from P
- Not symmetric: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$

Real Applications:

- Variational Autoencoders (VAE)
- Generative models evaluation
- Bayesian inference
- Model comparison

Mutual Information

Concepts:

- $I(X;Y) = H(X) + H(Y) - H(X,Y)$
- Measures dependence between variables
- $I(X;Y) = 0$ if X,Y independent

Real Applications:

- Feature selection algorithms
- Image registration
- Clustering evaluation
- Causal inference

6. Advanced Topics

Taylor Series

Concepts:

- $f(x) \approx f(a) + f'(a)(x-a) + \frac{f''(a)(x-a)^2}{2!} + \dots$
- Approximates functions as polynomials
- First-order: linear approximation

Real Applications:

- Activation function approximations
- Newton's method optimization
- Numerical gradient computation
- Understanding neural network training dynamics

Fourier Transform

Concepts:

- Decomposes signal into frequency components
- $F(\omega) = \int f(t)e^{-i\omega t}dt$
- Converts time domain $\leftrightarrow$ frequency domain

Real Applications:

- Audio processing (speech recognition)
- Image filtering and compression
- Signal denoising
- Convolutional Neural Networks (frequency domain convolutions)

Graph Theory Basics

Concepts:

- Graph: $G = (V, E)$ vertices and edges
- Adjacency matrix A : $A_{ij} = 1$ if edge exists
- Degree, paths, connectivity
- Laplacian matrix

Real Applications:

- Graph Neural Networks (GNN)
- Social network analysis
- Knowledge graphs
- Molecular structure prediction

Distance Metrics

Concepts:

- Euclidean: $d(x, y) = \sqrt{\sum (x_i - y_i)^2}$
- Manhattan: $d(x, y) = \sum |x_i - y_i|$
- Cosine Similarity: $\cos(\theta) = (x \cdot y) / (\|x\| \|y\|)$
- Hamming: count of differing positions

Real Applications:

- K-Nearest Neighbors (KNN)
- Clustering algorithms
- Similarity search
- Recommendation systems

Markov Chains

Concepts:

- State transitions with probabilities
- $P(X_{n+1}|X_n, X_{n-1}, \dots) = P(X_{n+1}|X_n)$
- Transition matrix, stationary distribution

Real Applications:

- Hidden Markov Models (speech recognition)
 - PageRank algorithm
 - Text generation models
 - Reinforcement learning (MDPs)
-

7. Practical ML Math Applications

Neural Network Forward Pass

$z = Wx + b$ (linear transformation)
 $a = \sigma(z)$ (activation function)
 Repeat for each layer

Math Used: Matrix multiplication, vector addition, function composition

Backpropagation

$\partial L / \partial W = \partial L / \partial a \cdot \partial a / \partial z \cdot \partial z / \partial W$ (chain rule)
 $W := W - \alpha (\partial L / \partial W)$ (gradient descent)

Math Used: Chain rule, partial derivatives, gradient descent

Principal Component Analysis (PCA)

1. Center data: $\tilde{X} = X - \text{mean}(X)$
2. Compute covariance: $C = (\tilde{X}^T \tilde{X}) / n$

3. Find eigenvectors of C
4. Project: $X_{\text{new}} = \tilde{X} \cdot V_k$

Math Used: Eigenvalues, eigenvectors, matrix multiplication

Linear Regression

$$\hat{y} = X\beta$$
$$\beta = (X^T X)^{-1} X^T y \text{ (closed form)}$$
$$\text{Loss: } L = \|y - X\beta\|^2$$

Math Used: Matrix inverse, linear algebra, L2 norm

Logistic Regression

$$\sigma(z) = 1/(1+e^{(-z)})$$
$$L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

Math Used: Sigmoid function, cross-entropy, gradient descent

Support Vector Machines

$$\text{Maximize margin: } 2/\|w\|$$
$$\text{Subject to: } y_i(w \cdot x_i + b) \geq 1$$

Math Used: Lagrange multipliers, convex optimization, dot products

K-Means Clustering

1. Assign: $c_i = \text{argmin}_k \|x_i - \mu_k\|^2$
2. Update: $\mu_k = \text{mean}(\text{points in cluster } k)$

Math Used: Euclidean distance, mean calculation, iterative optimization

Gaussian Mixture Models

$$P(x) = \sum_k \pi_k N(x|\mu_k, \Sigma_k)$$

EM algorithm for parameter estimation

Math Used: Multivariate Gaussian, probability, expectation-maximization

8. Key Formulas Quick Reference

Activation Functions:

- Sigmoid: $\sigma(x) = 1/(1+e^{(-x)})$
- Tanh: $\tanh(x) = (e^x - e^{(-x)})/(e^x + e^{(-x)})$
- ReLU: $\max(0, x)$
- Softmax: $\sigma(x)_i = e^{(x_i)}/\sum_j e^{(x_j)}$

Loss Functions:

- MSE: $(1/n)\sum (y - \hat{y})^2$
- Cross-Entropy: $-\sum y \cdot \log(\hat{y})$
- Hinge Loss: $\max(0, 1 - y \cdot \hat{y})$

Regularization:

- L1: $\lambda \sum |w_i|$
- L2: $\lambda \sum w_i^2$
- Elastic Net: $\lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$

Metrics:

- Accuracy: $(TP+TN)/(TP+TN+FP+FN)$
 - Precision: $TP/(TP+FP)$
 - Recall: $TP/(TP+FN)$
 - F1: $2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall})$
-

9. Study Path Recommendation

Foundation (2-3 weeks):

1. Linear algebra: vectors, matrices, operations
2. Basic calculus: derivatives, chain rule
3. Probability basics

Intermediate (3-4 weeks):

1. Matrix decompositions (SVD, eigenvalues)
2. Gradients, Jacobians

3. Probability distributions, MLE

Advanced (4-6 weeks):

1. Optimization algorithms
2. Information theory
3. Apply to real ML problems

Practice:

- Implement algorithms from scratch (numpy)
 - Derive backpropagation manually
 - Work through ML papers with focus on math
 - Use libraries (scikit-learn, PyTorch) to see math in action
-

10. Resources for Deep Dive

Books:

- "Mathematics for Machine Learning" by Deisenroth, Faisal, Ong
- "Pattern Recognition and Machine Learning" by Bishop
- "Deep Learning" by Goodfellow, Bengio, Courville

Online:

- 3Blue1Brown (YouTube) - visual explanations
- Khan Academy - fundamentals
- FastAI - practical applications
- MIT OpenCourseWare - rigorous foundations

Scikit-Learn: Complete Theoretical & Mathematical Guide

Table of Contents

1. Data Preprocessing Theory
 2. Classification Algorithms
 3. Regression Algorithms
 4. Unsupervised Learning
 5. Model Evaluation
 6. Cross-Validation & Tuning
-

1. Data Preprocessing Theory

Train-Test Split

When to Use: Always, before any model training

Why: To evaluate model performance on unseen data and detect overfitting

How It Works:

- Randomly divides data into training (typically 70-80%) and testing (20-30%)
- Stratified split maintains class distribution in both sets

Mathematical Concept:

- For dataset D with n samples: $D = D_{\text{train}} \cup D_{\text{test}}$
- $D_{\text{train}} \cap D_{\text{test}} = \emptyset$ (no overlap)

Key Rules:

- Never train on test data
 - Never use test data for any decision-making during training
 - Use stratification for imbalanced datasets
-

Feature Scaling

Standard Scaler (Z-score Normalization)

When to Use:

- Distance-based algorithms (KNN, SVM, K-Means)
- Gradient descent-based algorithms (Linear Regression, Neural Networks)
- When features have different units/scales

Why:

- Prevents features with larger scales from dominating
- Speeds up gradient descent convergence
- Makes features comparable

Formula:

$$z = (x - \mu) / \sigma$$

Where:

- x = original value
- μ = mean of feature
- σ = standard deviation
- z = standardized value

Properties:

- Mean = 0, Standard Deviation = 1
- Preserves outliers

Don't Use When:

- Tree-based algorithms (they're scale-invariant)
 - Features already on same scale
-

MinMax Scaler**When to Use:**

- Need bounded range [0,1] or custom range
- Neural networks (bound activation functions)
- Image data normalization

Why:

- Preserves relationships in data

- Useful when you need specific range

Formula:

$$x_scaled = (x - x_min) / (x_max - x_min)$$

For custom range [a, b]:

$$x_scaled = a + (x - x_min) \times (b - a) / (x_max - x_min)$$

Limitations:

- Sensitive to outliers (they compress the scale)
-

Robust Scaler

When to Use:

- Data has many outliers
- Need robust statistics

Why:

- Uses median and IQR instead of mean and std
- Resistant to outliers

Formula:

$$x_scaled = (x - median) / IQR$$

Where $IQR = Q3 - Q1$ (75th - 25th percentile)

Encoding Categorical Variables

Label Encoding

When to Use:

- Ordinal categorical variables (Low, Medium, High)
- Target variable encoding
- Binary categories

Why:

- Converts categories to numbers
- Preserves ordinal relationships

Formula:

Categories: [cat, dog, mouse] $\rightarrow$ [0, 1, 2]

Warning:

- Don't use for nominal variables (creates false ordering)
-

One-Hot Encoding**When to Use:**

- Nominal categorical variables (color, country)
- No natural ordering exists

Why:

- Prevents model from assuming ordinal relationships
- Each category becomes a binary feature

Formula:

For n categories, create n binary columns

Example: Color = [Red, Blue, Green]

Red $\rightarrow$ [1, 0, 0]

Blue $\rightarrow$ [0, 1, 0]

Green $\rightarrow$ [0, 0, 1]

Considerations:

- Creates n columns for n categories
 - Can cause dimensionality explosion
 - Use "drop_first=True" to avoid multicollinearity
-

Feature Selection

SelectKBest

When to Use:

- Too many features (curse of dimensionality)
- Need interpretability
- Reduce training time

Why:

- Removes irrelevant features
- Improves model performance
- Reduces overfitting

How It Works: Uses statistical tests:

- **f_classif**: ANOVA F-statistic for classification
- **chi2**: Chi-squared test (for non-negative features)
- **mutual_info_classif**: Mutual information

Mathematical Concept (F-statistic):

$$F = (\text{Between-group variability}) / (\text{Within-group variability})$$

Higher F → feature better discriminates classes

Recursive Feature Elimination (RFE)

When to Use:

- Need optimal feature subset
- Have trained model available

How It Works:

1. Train model on all features
2. Rank features by importance
3. Remove least important feature
4. Repeat until desired number remains

Why Better Than SelectKBest:

- Considers feature interactions
 - Model-based selection
-

Polynomial Features

When to Use:

- Linear model on non-linear data
- Capture feature interactions

Why:

- Allows linear models to fit curves
- Creates interaction terms

Formula:

For features $[x_1, x_2]$, degree=2:

Original: $[x_1, x_2]$

Polynomial: $[1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$

General: All combinations of features raised to powers $\leq$ degree

Warning:

- Features grow exponentially: n features, degree $d \rightarrow O(n^d)$ features
 - Can cause overfitting
-

2. Classification Algorithms

Logistic Regression

When to Use:

- Binary or multiclass classification
- Need probability estimates
- Linear decision boundary
- Baseline model

Why:

- Fast training and prediction
- Probabilistic interpretation
- Works well with linearly separable data

Mathematics:

For binary classification:

1. Linear combination:

$$z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = \mathbf{w} \cdot \mathbf{x} + b$$

2. Sigmoid function:

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

3. Prediction:

$$P(y=1|x) = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$$

4. Decision:

$$\hat{y} = 1 \text{ if } P(y=1|x) \geq 0.5, \text{ else } 0$$

Cost Function (Log Loss):

$$J(\mathbf{w}) = -1/m \sum [y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

Minimized using gradient descent

Key Parameters:

- **C**: Inverse regularization strength (smaller C = stronger regularization)
- **penalty**: 'l1' (sparse), 'l2' (default)

Assumptions:

- Linear relationship between features and log-odds
- Independent observations

Decision Tree Classifier

When to Use:

- Non-linear relationships

- Need interpretability
- Mixed data types (categorical + numerical)
- No feature scaling needed

Why:

- Easy to understand and visualize
- Handles non-linear data naturally
- No preprocessing required

How It Works:

1. Splitting Criterion (Gini Impurity):

$$\text{Gini} = 1 - \sum(p_i^2)$$

Where p_i = probability of class i

Example:

- Pure node [10 class A, 0 class B]: $\text{Gini} = 1 - (1^2 + 0^2) = 0$
- Mixed node [5 A, 5 B]: $\text{Gini} = 1 - (0.5^2 + 0.5^2) = 0.5$

2. Information Gain (Entropy):

$$\text{Entropy} = -\sum(p_i \cdot \log_2(p_i))$$

$$\text{Information Gain} = \text{Entropy}(\text{parent}) - \sum(\text{weighted_entropy}(\text{children}))$$

Algorithm:

1. Start with all data at root
2. For each feature, calculate best split (max information gain)
3. Choose feature with highest gain
4. Recursively split until stopping criterion

Key Parameters:

- **max_depth:** Maximum tree depth (prevents overfitting)
- **min_samples_split:** Minimum samples to split node
- **min_samples_leaf:** Minimum samples in leaf node
- **criterion:** 'gini' or 'entropy'

Pros:

- Interpretable
- No scaling needed
- Handles missing values

Cons:

- Prone to overfitting
 - Unstable (small data changes → different tree)
 - Biased toward features with more levels
-

Random Forest Classifier

When to Use:

- Need high accuracy
- Prevent overfitting of single tree
- Feature importance needed
- Works well "out of the box"

Why:

- Ensemble of trees reduces variance
- More robust than single tree
- Handles high-dimensional data

How It Works (Bagging + Random Subspace):

1. Bootstrap Aggregating:

For each tree:

- Randomly sample n data points WITH replacement
- Train decision tree on this sample

2. Random Feature Selection:

At each split:

- Randomly select k features ($k = \sqrt{n}$ for classification)
- Find best split among these k features only

3. Prediction:

Classification: Majority vote

$$\hat{y} = \text{mode}\{\text{tree}_1(x), \text{tree}_2(x), \dots, \text{tree}_n(x)\}$$

Mathematical Intuition:

Variance of average of N independent models:

$$\text{Var}(\text{average}) = \sigma^2/N$$

Random Forest reduces variance by averaging

Key Parameters:

- **n_estimators**: Number of trees (more = better, but slower)
- **max_features**: Features to consider per split ($\sqrt{n}$ default)
- **max_depth**: Maximum depth of each tree
- **min_samples_split**: Minimum samples to split

Pros:

- Reduced overfitting vs single tree
- Feature importance
- Parallel training

Cons:

- Less interpretable
- Slower than single tree
- More memory intensive

Support Vector Machine (SVM)

When to Use:

- High-dimensional data
- Clear margin of separation exists
- Small to medium datasets
- Need robust decision boundary

Why:

- Finds optimal separating hyperplane
- Works well in high dimensions
- Memory efficient (uses support vectors only)

Mathematics:

Linear SVM:

Goal: Find hyperplane $w \cdot x + b = 0$ that maximizes margin

$$\text{Margin} = 2/\|w\|$$

Optimization:

$$\text{minimize: } (1/2)\|w\|^2$$

$$\text{subject to: } y_i(w \cdot x_i + b) \geq 1 \text{ for all } i$$

Where $y_i \in \{-1, 1\}$

Soft Margin (C parameter):

$$\text{minimize: } (1/2)\|w\|^2 + C \cdot \sum \xi_i$$

Where ξ_i are slack variables allowing misclassification

- Large C: Small margin, fewer misclassifications (hard margin)
- Small C: Large margin, more misclassifications (soft margin)

Kernel Trick (Non-linear SVM):

Transform data to higher dimension where it's linearly separable

Common Kernels:

1. **Linear:** $K(x, x') = x \cdot x'$
 - Use when: Data is linearly separable
2. **RBF (Radial Basis Function):** $K(x, x') = \exp(-\gamma \|x - x'\|^2)$
 - Use when: Non-linear, don't know the structure
 - γ (gamma):
 - Large γ : Tight fit (overfitting risk)
 - Small γ : Loose fit (underfitting risk)
3. **Polynomial:** $K(x, x') = (\gamma \cdot x \cdot x' + r)^d$

- Use when: Features have interactions
- $d = \text{degree}$

Key Parameters:

- **C**: Regularization (1.0 default)
- **kernel**: 'linear', 'rbf', 'poly'
- **gamma**: Kernel coefficient ('scale' = $1/(n_{\text{features}} \times X.\text{var}())$)

Pros:

- Effective in high dimensions
- Memory efficient
- Versatile (different kernels)

Cons:

- Slow on large datasets ($O(n^2)$ to $O(n^3)$)
 - Requires feature scaling
 - Hard to interpret
 - Sensitive to parameters
-

K-Nearest Neighbors (KNN)

When to Use:

- Small to medium datasets
- Non-linear decision boundaries
- Instance-based learning needed

Why:

- Simple, intuitive
- No training phase (lazy learning)
- Naturally handles multiclass

How It Works:

1. For new point x :
 - Calculate distance to all training points
 - Select k nearest neighbors
2. Classification:
 - Majority vote among k neighbors
 - $\hat{y} = \text{mode}\{y_1, y_2, \dots, y_k\}$
3. Weighted voting (optional):
 - Closer neighbors have more influence
 - Weight = $1/\text{distance}$

Distance Metrics:

1. Euclidean (default):

$$d(x, y) = \sqrt{(\sum (x_i - y_i)^2)}$$

2. Manhattan:

$$d(x, y) = \sum |x_i - y_i|$$

Choosing k :

- Small k : Flexible, but noisy (overfitting)
- Large k : Smooth, but may miss local patterns (underfitting)
- Typical: $k = \sqrt{n}$ or use cross-validation
- Use odd k for binary classification (avoid ties)

Key Parameters:

- **n_neighbors**: Number of neighbors (k)
- **weights**: 'uniform' (all equal) or 'distance' (weighted)
- **metric**: Distance function

Pros:

- Simple, no training
- Works well with irregular decision boundaries
- Adapts to new data easily

Cons:

- Slow prediction ($O(n)$ per prediction)
 - Memory intensive (stores all training data)
 - Sensitive to irrelevant features
 - Requires feature scaling
 - Curse of dimensionality
-

Naive Bayes

When to Use:

- Text classification (spam detection, sentiment analysis)
- Fast training needed
- Baseline model
- High-dimensional data

Why:

- Extremely fast
- Works well with small datasets
- Probabilistic predictions

Mathematics (Bayes' Theorem):

$$P(y|x) = P(x|y) \cdot P(y) / P(x)$$

Where:

- $P(y|x)$ = Posterior probability
- $P(x|y)$ = Likelihood
- $P(y)$ = Prior probability
- $P(x)$ = Evidence

Prediction:

$$\hat{y} = \underset{y}{\operatorname{argmax}} P(y) \cdot \prod P(x_i|y)$$

"Naive" Assumption: Features are independent given the class:

$$P(x_1, x_2, \dots, x_n|y) = P(x_1|y) \cdot P(x_2|y) \cdot \dots \cdot P(x_n|y)$$

Types:

1. Gaussian Naive Bayes:

- Use when: Continuous features
- Assumes: Features follow normal distribution

$$P(x_i|y) = (1/\sqrt{2\pi\sigma^2}) \cdot \exp(-(x_i-\mu)^2/(2\sigma^2))$$

2. Multinomial Naive Bayes:

- Use when: Count data (word frequencies, tf-idf)
- Text classification

$$P(x_i|y) = (\text{count}(x_i, y) + \alpha) / (\text{count}(y) + \alpha \cdot n)$$

where α = smoothing parameter

3. Bernoulli Naive Bayes:

- Use when: Binary features
- Document classification (word present/absent)

Pros:

- Very fast training and prediction
- Works well with high-dimensional data
- Requires small training data
- Handles missing data well

Cons:

- Independence assumption rarely holds
- Can't learn feature interactions
- Zero-frequency problem (solved with smoothing)

Gradient Boosting

When to Use:

- Need highest accuracy
- Structured/tabular data

- Willing to tune hyperparameters
- Kaggle competitions

Why:

- Often best performance on tabular data
- Handles mixed data types
- Robust to outliers

How It Works (Boosting):

Unlike Random Forest (parallel), builds trees sequentially:

1. Start with initial prediction:

$$F_0(x) = \underset{\gamma}{\operatorname{argmin}} \sum L(y_i, \gamma) \quad (\text{often mean for regression})$$

2. For $m = 1$ to M :

a. Compute pseudo-residuals:

$$r_{im} = -[\partial L(y_i, F(x_i)) / \partial F(x_i)] \text{ at } F = F_{m-1}$$

b. Fit tree h_m to residuals

c. Update model:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

3. Final prediction:

$$F(x) = F_0(x) + \eta \cdot \sum h_m(x)$$

Intuition: Each tree corrects errors of previous trees

Key Parameters:

- **n_estimators:** Number of boosting stages (more = better, but slower)
- **learning_rate** (η): Shrinkage (0.01-0.1 typical)
 - Small η : Better generalization, needs more trees
 - Large η : Faster training, overfitting risk
- **max_depth:** Tree depth (3-10 typical, shallow trees work well)
- **subsample:** Fraction of samples for each tree (0.8 typical)

Relationship:

$$n_estimators \times learning_rate \approx \text{constant for similar performance}$$

Pros:

- State-of-art accuracy on tabular data
- Handles mixed data types
- Feature importance
- Robust to outliers

Cons:

- Slow training (sequential)
- Sensitive to hyperparameters
- Can overfit easily
- Requires careful tuning

Advanced Versions:

- **XGBoost**: Regularization, parallel processing
 - **LightGBM**: Fast, efficient, handles large data
 - **CatBoost**: Handles categorical features automatically
-

3. Regression Algorithms

Linear Regression**When to Use:**

- Linear relationship exists
- Need interpretability
- Baseline model
- Understand feature impact

Why:

- Simple, fast, interpretable
- Coefficients show feature importance
- Well-understood assumptions

Mathematics:**Model:**

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon$$

Matrix form: $y = X\beta + \varepsilon$

Where:

- y = target variable
- X = feature matrix
- β = coefficients
- ε = error term

Ordinary Least Squares (OLS) Solution:

Minimize: $RSS = \sum (y_i - \hat{y}_i)^2$

Closed-form solution:

$$\beta = (X^T X)^{-1} X^T y$$

Interpretation:

β_j = change in y for 1 unit change in x_j (holding others constant)

Assumptions:

1. **Linearity**: Relationship is linear
2. **Independence**: Observations are independent
3. **Homoscedasticity**: Constant variance of errors
4. **Normality**: Errors follow normal distribution
5. **No multicollinearity**: Features aren't highly correlated

Pros:

- Fast, simple
- Interpretable coefficients
- Works well when assumptions hold

Cons:

- Assumes linearity
- Sensitive to outliers
- Fails with multicollinearity
- Can overfit with many features

Ridge Regression (L2 Regularization)

When to Use:

- Multicollinearity exists
- Prevent overfitting
- Many correlated features
- Feature values should shrink but not zero

Why:

- Handles multicollinearity
- Reduces model variance
- All features retained (coefficients shrink)

Mathematics:

Minimize: $RSS + \alpha \cdot \sum \beta_j^2$

$$= \sum (y_i - \hat{y}_i)^2 + \alpha \cdot \|\beta\|_2^2$$

Where α = regularization strength

Closed-form solution:

$$\beta_{\text{ridge}} = (X^T X + \alpha I)^{-1} X^T y$$

Note: $(X^T X + \alpha I)$ is always invertible

Effect of α :

- $\alpha = 0$: Ordinary linear regression
- $\alpha \rightarrow \infty$: $\beta \rightarrow 0$ (model becomes constant)
- Typical: Use cross-validation to find optimal α

Pros:

- Solves multicollinearity
- Prevents overfitting
- Stable solution

Cons:

- All features retained (not sparse)
 - Less interpretable than linear regression
 - Need to choose α
-

Lasso Regression (L1 Regularization)

When to Use:

- Feature selection needed
- Sparse models desired
- Many irrelevant features
- Interpretability important

Why:

- Automatic feature selection
- Creates sparse models (many $\beta = 0$)
- Good for high-dimensional data

Mathematics:

Minimize: $\text{RSS} + \alpha \cdot \sum |\beta_j|$

$$= \sum (y_i - \hat{y}_i)^2 + \alpha \cdot \|\beta\|_1$$

Where α = regularization strength

No closed-form solution (requires iterative methods like coordinate descent)

Feature Selection Property:

As α increases:

- More coefficients become exactly zero
- Effectively performs feature selection

Comparison with Ridge:

Ridge: $\beta_j \rightarrow 0$ but never exactly 0

Lasso: β_j can be exactly 0

Pros:

- Automatic feature selection
- Sparse, interpretable models
- Works well with many features

Cons:

- Arbitrary selection among correlated features
 - Can select at most n features (when $n < p$)
 - Less stable than Ridge with correlated features
-

ElasticNet

When to Use:

- Many correlated features
- Need both regularization and feature selection
- Lasso is too unstable

Why:

- Combines L1 and L2 penalties
- Gets benefits of both Ridge and Lasso
- More stable than Lasso

Mathematics:

Minimize: $RSS + \alpha \cdot [(1-r) \cdot \sum \beta_j^2 + r \cdot \sum |\beta_j|]$

$= RSS + \alpha \cdot [(1-r) \cdot \|\beta\|_2^2 + r \cdot \|\beta\|_1]$

Where:

- α = regularization strength
- r = $l1_ratio$ (mix between L1 and L2)

Parameter Effects:

- $r = 0$: Pure Ridge
- $r = 1$: Pure Lasso
- $0 < r < 1$: ElasticNet

Pros:

- Handles correlated features better than Lasso
- Feature selection like Lasso
- Stability like Ridge

Cons:

- Two hyperparameters to tune
 - More complex than Ridge or Lasso alone
-

Decision Tree & Random Forest Regressor

Same principles as classification, but:

Splitting Criterion:

MSE (Mean Squared Error):

$$\text{MSE} = (1/n) \cdot \sum (y_i - \hat{y})^2$$

Variance Reduction:

Split to minimize weighted MSE of children nodes

Leaf Prediction:

$$\hat{y} = \text{mean}(y \text{ values in leaf})$$

Support Vector Regression (SVR)

When to Use:

- Non-linear relationships
- Robust to outliers needed
- High-dimensional data

Why:

- Tolerates errors within margin (ϵ -tube)
- Focuses on support vectors

Mathematics:

Minimize: $(1/2)||w||^2 + C \cdot \sum(\xi_i + \xi_i^*)$

Subject to:

- $y_i - (w \cdot x_i + b) \leq \epsilon + \xi_i$
- $(w \cdot x_i + b) - y_i \leq \epsilon + \xi_i^*$

Where:

- ϵ = epsilon (width of tube where errors are ignored)
- ξ_i, ξ_i^* = slack variables
- C = penalty parameter

ϵ -insensitive loss:

Loss = 0 if $|y - \hat{y}| \leq \epsilon$

Loss = $|y - \hat{y}| - \epsilon$ otherwise

Intuition: Only penalize predictions outside the ϵ -tube

Gradient Boosting Regressor

Same as classification, but:

Loss Function:

$L(y, F(x)) = (1/2)(y - F(x))^2$

Pseudo-residuals:

$r_i = y_i - F_{m-1}(x_i)$

Each tree fits to residuals of previous ensemble

4. Unsupervised Learning

K-Means Clustering

When to Use:

- Know number of clusters (k)
- Spherical clusters expected
- Fast clustering needed

Why:

- Simple, fast ($O(nkt)$ where t = iterations)
- Scales well to large datasets

Algorithm:

1. Initialize k centroids randomly
2. Repeat until convergence:
 - a. Assign each point to nearest centroid:
$$c_i = \underset{j}{\operatorname{argmin}} \|x_i - \mu_j\|^2$$
 - b. Update centroids:
$$\mu_j = (1/|C_j|) \cdot \sum(x_i) \text{ for all } x_i \text{ in cluster } j$$
3. Converges when assignments don't change

Objective Function:

$$\text{Minimize: } J = \sum_{j=1} \sum_{i \in C_j} \|x_i - \mu_j\|^2$$

(Within-cluster sum of squares - WCSS)

Choosing k (Elbow Method):

Plot WCSS vs k

Look for "elbow" where WCSS decrease slows

Silhouette Score:

$$s(i) = (b(i) - a(i)) / \max\{a(i), b(i)\}$$

Where:

- $a(i)$ = avg distance to points in same cluster
- $b(i)$ = avg distance to points in nearest cluster

Range: $[-1, 1]$

- Close to 1: Well clustered
- Close to 0: On border
- Negative: Likely wrong cluster

Pros:

- Fast, simple
- Scales to large datasets
- Works well with spherical clusters

Cons:

- Must specify k
 - Sensitive to initialization (use k-means++)
 - Assumes spherical clusters of similar size
 - Sensitive to outliers
-

DBSCAN (Density-Based Spatial Clustering)**When to Use:**

- Don't know number of clusters
- Arbitrary-shaped clusters
- Noise/outliers present
- Clusters have varying densities

Why:

- Finds clusters of arbitrary shape
- Automatically detects outliers
- No need to specify k

Algorithm:

Parameters:

- ϵ (eps): Maximum distance between points
- min_samples: Minimum points to form dense region

Point Types:

1. Core point: $\geq$ min_samples points within ϵ
2. Border point: $<$ min_samples within ϵ , but in neighborhood of core
3. Noise point: Neither core nor border

Algorithm:

1. For each unvisited point p:
 - Find all points within ϵ (ϵ -neighborhood)
 - If $|\text{neighborhood}| \geq \text{min_samples}$:
 - Start new cluster
 - Add all reachable points to cluster
 - Else: mark as noise (may change later)

Density Reachability:

Point q is density-reachable from p if:

- There's chain $p_1, \dots, p_n$ where:
 - $p_1 = p, p_n = q$
 - $p_{i+1} \in N_\epsilon(p_i)$ (ϵ -neighborhood)
 - p_i is core point (except possibly p_n)

Choosing Parameters:

- ϵ : Plot k-distance graph, look for elbow
- min_samples: Use $\geq$ dimensions + 1 (often 4-5 for 2D)

Pros:

- Finds arbitrary-shaped clusters
- Robust to outliers
- No need to specify number of clusters

Cons:

- Struggles with varying density clusters
 - Sensitive to ϵ and min_samples
 - Not fully deterministic (border points)
 - Slow on large datasets
-

Hierarchical Clustering

When to Use:

- Need cluster hierarchy
- Don't know optimal k
- Small to medium datasets
- Want dendrogram visualization

Why:

- No need to specify k initially
- Provides hierarchy (dendrogram)
- Deterministic

Types:

1. Agglomerative (Bottom-up):

1. Start: Each point is a cluster
2. Repeat:
 - Merge two closest clusters
 - Until: One cluster remains

2. Divisive (Top-down):

1. Start: All points in one cluster
2. Repeat:
 - Split cluster
 - Until: Each point is a cluster

Linkage Methods:

1. Single Linkage:

$$d(C_1, C_2) = \min\{d(x, y) : x \in C_1, y \in C_2\}$$

Tends to create long chains

2. Complete Linkage:

$$d(C_1, C_2) = \max \{d(x, y) : x \in C_1, y \in C_2\}$$

Tends to create compact clusters

3. Average Linkage:

$$d(C_1, C_2) = (1/|C_1||C_2|) \cdot \sum_{x \in C_1} \sum_{y \in C_2} d(x, y)$$

4. Ward's Method (Minimizes variance):

Merge clusters that minimize increase in total within-cluster variance
Most commonly used

Pros:

- Hierarchical structure
- No need to specify k upfront
- Deterministic

Cons:

- Computationally expensive $O(n^3)$
- Doesn't scale to large datasets
- Can't undo merges

Principal Component Analysis (PCA)

When to Use:

- Dimensionality reduction
- Visualization (reduce to 2D/3D)
- Remove multicollinearity
- Data compression
- Noise reduction

Why:

- Reduces dimensions while preserving variance

- Removes correlated features
- Speeds up algorithms

Mathematics:

Goal: Find directions (principal components) of maximum variance

1. Standardize data:

$$X_{\text{std}} = (X - \mu) / \sigma$$

2. Compute covariance matrix:

$$C = (1/n) \cdot X^T \cdot X$$

3. Eigendecomposition:

$$C \cdot v = \lambda \cdot v$$

Where:

- v = eigenvectors (principal components)
- λ = eigenvalues (variance explained)

4. Sort eigenvectors by eigenvalue (descending)

5. Select top k eigenvectors: $W = [v_1, v_2, \dots, v_k]$

6. Transform data:

$$X_{\text{pca}} = X \cdot W$$

Variance Explained:

Variance explained by $PC_i = \lambda_i / \sum \lambda_j$

Cumulative variance = $\sum(\lambda_i) / \sum(\lambda_j)$ for $i=1$ to k

Choosing Number of Components:

1. Keep components with cumulative variance $\geq 95\%$
2. Scree plot: Look for elbow
3. Kaiser criterion: Keep components with $\lambda > 1$

Interpretation:

$$PC_1 = w_{11}X_1 + w_{12}X_2 + \dots + w_{1n}X_n$$

Where w_{ij} are loadings (contribution of feature j to PC i)

Pros:

- Removes multicollinearity
- Reduces overfitting
- Speeds up algorithms
- Visualizes high-dimensional data

Cons:

- Components hard to interpret
 - Assumes linear relationships
 - Sensitive to scaling (must standardize)
 - Information loss
-

t-SNE (t-Distributed Stochastic Neighbor Embedding)**When to Use:**

- Visualization only (2D/3D)
- Explore high-dimensional data
- Find patterns, clusters visually

Why:

- Preserves local structure better than PCA
- Reveals clusters and patterns
- Non-linear dimensionality reduction

Mathematics:

Goal: Preserve pairwise similarities in low dimensions

1. High-dimensional similarities (Gaussian):

$$p_{j|i} = \exp(-\|x_i - x_j\|^2 / (2\sigma_i^2)) / \sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / (2\sigma_i^2))$$

$$p_{ij} = (p_{j|i} + p_{i|j}) / (2n)$$

2. Low-dimensional similarities (Student's t):

$$q_{ij} = (1 + \|y_i - y_j\|^2)^{-1} / \sum_{k \neq i} (1 + \|y_i - y_k\|^2)^{-1}$$

3. Minimize KL divergence:

$$\text{Cost} = \text{KL}(P||Q) = \sum_i \sum_j p_{ij} \cdot \log(p_{ij}/q_{ij})$$

Using gradient descent

Why t-distribution?

- Heavy tails → distant points can be farther apart
- Prevents "crowding problem" of high dimensions

Key Parameters:

- **perplexity**: Balance between local and global structure (5-50 typical)
 - Low perplexity: Focus on local structure
 - High perplexity: Focus on global structure
- **n_iter**: Iterations (≥ 1000 recommended)
- **learning_rate**: Step size (10-1000 typical)

Pros:

- Excellent for visualization
- Reveals clusters naturally
- Handles non-linear structures

Cons:

- Only for visualization (can't transform new data)
- Computationally expensive $O(n^2)$
- Different runs give different results
- Distances in t-SNE plot not meaningful
- Perplexity choice affects results

Best Practices:

1. Run multiple times with different random seeds
 2. Try different perplexity values (5, 30, 50)
 3. Use PCA first if > 50 dimensions
 4. Don't over-interpret cluster sizes/distances
-

5. Model Evaluation

Classification Metrics

Confusion Matrix

Actual	Predicted		
	Positive		Negative
	Pos	TP	FN
	Neg	FP	TN

Where:

- TP = True Positives
 - TN = True Negatives
 - FP = False Positives (Type I error)
 - FN = False Negatives (Type II error)
-

Accuracy

When to Use:

- Balanced datasets
- Equal cost for all errors

Formula:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Proportion of correct predictions

Limitation: Misleading for imbalanced datasets

Example:

Dataset: 95% class A, 5% class B

Model predicting all A: 95% accuracy (but useless!)

Precision

When to Use:

- Cost of False Positives is high
- Spam detection (don't want to mark real emails as spam)
- Medical diagnosis (don't want false alarms)

Formula:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

"Of all predicted positives, how many are actually positive?"

Also called: Positive Predictive Value (PPV)

Interpretation: High precision → Few false alarms

Recall (Sensitivity)

When to Use:

- Cost of False Negatives is high
- Disease screening (don't want to miss sick patients)
- Fraud detection (don't want to miss fraud cases)

Formula:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

"Of all actual positives, how many did we catch?"

Also called: Sensitivity, True Positive Rate (TPR)

Interpretation: High recall → Few missed positives

F1-Score

When to Use:

- Need balance between precision and recall
- Imbalanced datasets
- Single metric needed

Formula:

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Harmonic mean of precision and recall

Why Harmonic Mean?

- Penalizes extreme values
- F1 is high only if both precision and recall are high

Variants:

$$F\beta = (1 + \beta^2) \times (\text{Precision} \times \text{Recall}) / (\beta^2 \times \text{Precision} + \text{Recall})$$

- $\beta < 1$: Favor precision
- $\beta > 1$: Favor recall
- $\beta = 1$: F1 (balanced)
- $\beta = 2$: F2 (favor recall 2x)

ROC Curve and AUC

When to Use:

- Compare classifiers
- Choose threshold
- Evaluate across all thresholds

ROC Curve (Receiver Operating Characteristic):

X-axis: $FPR = FP / (FP + TN) = 1 - \text{Specificity}$

Y-axis: $TPR = TP / (TP + FN) = \text{Recall}$

Plot TPR vs FPR at various threshold settings

AUC (Area Under Curve):

Range: [0, 1]

- AUC = 1.0: Perfect classifier
- AUC = 0.9-1.0: Excellent
- AUC = 0.8-0.9: Good
- AUC = 0.7-0.8: Fair
- AUC = 0.5: Random classifier (diagonal line)
- AUC < 0.5: Worse than random

Interpretation:

AUC = Probability that classifier ranks random positive higher than random negative

Advantage:

- Threshold-independent
 - Good for imbalanced datasets
-

Regression Metrics

Mean Squared Error (MSE)

Formula:

$$MSE = (1/n) \cdot \sum (y_i - \hat{y}_i)^2$$

Properties:

- Penalizes large errors heavily (squared term)
- Not in original units
- Always positive
- Lower is better

When to Use:

- Want to penalize large errors
 - Optimization (differentiable)
-

Root Mean Squared Error (RMSE)

Formula:

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{[(1/n) \cdot \sum (y_i - \hat{y}_i)^2]}$$

Properties:

- Same units as target variable
- Interpretable
- Sensitive to outliers

When to Use:

- Need interpretable metric in original units
-

Mean Absolute Error (MAE)

Formula:

$$\text{MAE} = (1/n) \cdot \sum |y_i - \hat{y}_i|$$

Properties:

- Same units as target
- Less sensitive to outliers than MSE
- All errors weighted equally

Comparison:

MSE vs MAE:

- MSE: Penalizes large errors more (squared)
- MAE: All errors treated equally (linear)

Choose MAE if outliers should have less influence

R² Score (Coefficient of Determination)

Formula:

$$R^2 = 1 - (SS_{\text{res}} / SS_{\text{tot}})$$

Where:

- $SS_{\text{res}} = \sum (y_i - \hat{y}_i)^2$ (residual sum of squares)
- $SS_{\text{tot}} = \sum (y_i - \bar{y})^2$ (total sum of squares)

Alternative:

$$R^2 = 1 - \text{MSE}(\text{model}) / \text{MSE}(\text{baseline})$$

Interpretation:

$R^2 = 1.0$: Perfect predictions

$R^2 = 0.0$: Model as good as predicting mean

$R^2 < 0.0$: Model worse than baseline (overfitting on train, bad on test)

$R^2 = 0.7$: Model explains 70% of variance

Properties:

- Range: $(-\infty, 1]$
- Not always increasing with more features (use adjusted R^2)

Adjusted R²:

$$R^2_{\text{adj}} = 1 - [(1 - R^2) \cdot (n - 1) / (n - p - 1)]$$

Where:

- n = number of samples
- p = number of features

Penalizes adding features that don't improve fit

Mean Absolute Percentage Error (MAPE)

Formula:

$$\text{MAPE} = (100/n) \cdot \sum |y_i - \hat{y}_i| / |y_i|$$

Properties:

- Percentage $\rightarrow$ scale-independent
- Interpretable
- Undefined when $y_i = 0$

When to Use:

- Compare across different scales
- Need percentage interpretation

Limitation:

- Asymmetric (penalizes under-prediction more)
 - Undefined for zero values
-

6. Cross-Validation & Tuning

Cross-Validation**Why:**

- Get robust performance estimate
 - Use all data for both training and validation
 - Reduce variance in performance estimates
-

K-Fold Cross-Validation**How It Works:**

1. Split data into k equal folds
2. For each fold i:
 - Train on k-1 folds
 - Test on fold i
 - Record performance
3. Average k performance scores

$$\text{Final score} = (1/k) \cdot \sum \text{score}_i$$

Variance:

$$\text{var}(\text{score}) \propto 1/k$$

More folds $\rightarrow$ lower variance, but more computation

Choosing k:

- $k = 5$ or 10 (typical)
- $k = n$: Leave-One-Out (LOOCV) - high variance, expensive
- Larger k : More data per fold, but more computation
- Smaller k : Less computation, but higher variance

Pros:

- All data used for both training and testing
- More reliable than single split
- Less variance than holdout method

Cons:

- Computationally expensive (k model trainings)
 - Not appropriate for time-series (see TimeSeriesSplit)
-

Stratified K-Fold

When to Use:

- Classification with imbalanced classes
- Ensure each fold has same class distribution

How:

Each fold maintains the same class ratio as original dataset

Example: 80% class A, 20% class B

Each fold will also be 80% A, 20% B

Why Better:

- Prevents folds with only one class

- More reliable estimates for imbalanced data
-

Leave-One-Out Cross-Validation (LOOCV)

How:

$k = n$ (number of samples)

Each sample is test set once

n iterations, training on $n-1$ samples each time

Pros:

- Maximum training data ($n-1$)
- Deterministic (no randomness)
- Good for small datasets

Cons:

- Computationally expensive (n iterations)
 - High variance
 - Models highly correlated
-

Hyperparameter Tuning

Grid Search

How It Works:

1. Define parameter grid:

```
param_grid = {  
    'param1': [val1, val2, val3],  
    'param2': [valA, valB]  
}
```

2. Try all combinations:

Total combinations = $|\text{param1}| \times |\text{param2}| \times \dots$

3. For each combination:

- Perform k-fold CV
- Record average score

4. Select best combination

Complexity:

If grid has d dimensions with n values each:

Total models = $n^d \times k$ (folds)

Example: 3 params, 10 values each, 5-fold CV

= $10^3 \times 5 = 5000$ model trainings

Pros:

- Exhaustive search
- Guaranteed to find best combination in grid
- Parallelizable

Cons:

- Exponentially expensive
- Curse of dimensionality
- Might miss optimal values between grid points

Best Practices:

1. Start with coarse grid
 2. Refine around best region
 3. Use domain knowledge for ranges
-

Randomized Search

How It Works:

1. Define parameter distributions:

```
param_dist = {  
    'param1': uniform(0, 10),  
    'param2': randint(1, 100)  
}
```

2. Sample `n_iter` random combinations

3. Evaluate with cross-validation

4. Select best

Advantages over Grid Search:

Probability of finding near-optimal in budget:

Grid: Depends on grid spacing

Random: Depends on `n_iter` (independent of dimensions!)

For same budget:

- Random explores more parameter space
- Better for high dimensions

Mathematical Insight:

For k important parameters out of n :

- Grid: Must have fine spacing in all n dimensions
- Random: Samples effectively in k dimensions regardless of n

Pros:

- Efficient in high dimensions
- Can specify budget (`n_iter`)
- Often finds better results than grid with same budget
- Parallelizable

Cons:

- Not exhaustive
- May miss optimal combination

- Results vary with random seed

When to Use:

- Many hyperparameters (>3)
 - Continuous parameters
 - Limited computational budget
 - Don't know good ranges
-

Model Selection Strategy

Recommended Workflow:

1. Baseline Model:
 - Simple model with default parameters
 - Establishes lower bound
 2. Coarse Random Search:
 - Wide parameter ranges
 - $n_iter = 50-100$
 - Identify promising regions
 3. Fine Grid Search:
 - Narrow ranges around best from step 2
 - More granular grid
 4. Final Validation:
 - Retrain on full training set
 - Test on held-out test set (untouched until now)
-

Bias-Variance Tradeoff

Fundamental Equation:

Expected Error = Bias² + Variance + Irreducible Error

Total Error = $\sum (y_i - \hat{y}_i)^2$

Bias:

$$\text{Bias} = E[\hat{y}] - y_{\text{true}}$$

Error from wrong assumptions in learning algorithm

High Bias $\rightarrow$ Underfitting:

- Model too simple
- Doesn't capture patterns
- Poor on both train and test

Variance:

$$\text{Variance} = E[(\hat{y} - E[\hat{y}])^2]$$

Error from sensitivity to training data fluctuations

High Variance $\rightarrow$ Overfitting:

- Model too complex
- Captures noise
- Great on train, poor on test

Tradeoff:

As model complexity increases:

- Bias decreases (fits training data better)
- Variance increases (more sensitive to training data)

Optimal complexity: Minimize total error

Symptoms and Solutions:

Underfitting (High Bias):

- Train error high, test error high
- Both errors similar

Solutions:

- More complex model
- More features
- Less regularization
- Train longer

Overfitting (High Variance):

- Train error low, test error high
- Large gap between train and test

Solutions:

- Simpler model
- More training data
- Regularization
- Feature selection
- Early stopping
- Cross-validation

Good Fit:

- Both errors low
- Small gap between train and test

Algorithm Selection Guide

Quick Decision Tree

What's your goal?

- └─ Predict category
 - | └─ Interpretability needed → Logistic Regression, Decision Tree
 - | └─ Best accuracy → Random Forest, Gradient Boosting
 - | └─ High dimensions → SVM, Naive Bayes
 - | └─ Text data → Naive Bayes, Logistic Regression
 - | └─ Fast prediction → Naive Bayes, KNN
- └─ Predict number
 - | └─ Linear relationship → Linear, Ridge, Lasso
 - | └─ Feature selection → Lasso, ElasticNet
 - | └─ Multicollinearity → Ridge, ElasticNet
 - | └─ Best accuracy → Random Forest, Gradient Boosting

- └ Find groups
 - └ Know $k \rightarrow$ K-Means
 - └ Don't know $k \rightarrow$ DBSCAN, Hierarchical
 - └ Arbitrary shapes $\rightarrow$ DBSCAN
- └ Reduce dimensions
 - └ Linear $\rightarrow$ PCA
 - └ Visualization $\rightarrow$ t-SNE

Data Size Considerations

Small Data (< 1000 samples):

- Avoid complex models (overfitting risk)
- Use: Naive Bayes, Logistic Regression, Decision Tree
- Careful with: Random Forest, Gradient Boosting
- Essential: Cross-validation

Medium Data (1000 - 100k):

- Most algorithms work well
- Use: Random Forest, SVM, Gradient Boosting
- Tune hyperparameters

Large Data (> 100k):

- Avoid: KNN (slow prediction), SVM (slow training)
- Use: Linear models, Random Forest, Gradient Boosting
- Consider: SGDClassifier, MiniBatch algorithms

Feature Characteristics

Categorical Features:

- Tree-based: Handle naturally
- Linear models: Need encoding (one-hot)
- SVM/KNN: Need encoding + scaling

Mixed Types (numerical + categorical):

- Best: Tree-based algorithms
- OK: Linear (with preprocessing)
- Avoid: Naive Bayes (assumes continuous)

High Dimensions ($p > n$):

- Good: Lasso, ElasticNet, Naive Bayes

- OK: SVM with linear kernel
- Avoid: Linear Regression (underdetermined)

Common Pitfalls

1. Data Leakage:

WRONG: Scale before split

```
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test = split(X_scaled)
```

RIGHT: Split first, then scale

```
X_train, X_test = split(X)
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

2. Using Test Set Multiple Times:

Test set should be touched only once for final evaluation

Use validation set or cross-validation for tuning

3. Ignoring Class Imbalance:

Solutions:

- Stratified split
- Class weights
- Over/under sampling (SMOTE)
- Appropriate metrics (F1, not accuracy)

4. Not Scaling:

Always scale for: SVM, KNN, Linear models, Neural Networks

Never needed for: Tree-based algorithms

5. Overfitting to Validation Set:

Too many tuning iterations → overfitting to validation

Solution: Hold out final test set completely

Final Recommendations

Start Simple:

1. Baseline model (mean, mode, or simple model)
2. Linear model (Logistic/Linear Regression)
3. If needed, increase complexity

Always:

- Split your data properly
- Use cross-validation
- Scale when needed
- Check for overfitting
- Use appropriate metrics

Model Complexity Ladder:

Simple → Complex:

Linear → Polynomial → Tree → Random Forest → Gradient Boosting

Start simple, increase only if needed

Remember:

"Better data beats better algorithms"

Focus on:

- Quality features
- Proper preprocessing
- Understanding your problem

Before:

- Complex models
- Extensive tuning

Complete NumPy & Pandas Theory Guide

NumPy: Numerical Python

What is NumPy?

NumPy is the foundational package for numerical computing in Python. It provides a high-performance multidimensional array object and tools for working with these arrays.

Why NumPy exists:

- Python lists are slow for numerical operations (interpreted, dynamically typed)
- NumPy arrays are stored in contiguous memory blocks (C-like performance)
- Vectorized operations eliminate slow Python loops
- Foundation for scientific computing ecosystem (Pandas, SciPy, scikit-learn)

Performance difference:

Python list loop: $O(n)$ operations in Python interpreter
NumPy vectorized: $O(n)$ operations in compiled C code
Speed improvement: 10-100x faster for numerical operations

NumPy Arrays (ndarray)

Core Concept: N-Dimensional Arrays

An ndarray is a grid of values, all of the same type, indexed by a tuple of non-negative integers.

Mathematical representation:

- 0D array (scalar): $a = 5$
- 1D array (vector): $a = [1, 2, 3]$
- 2D array (matrix): $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$
- 3D+ array (tensor): Higher dimensional grids

Memory layout:

Array: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

Stored in memory: [1, 2, 3, 4, 5, 6] (row-major, C-style)

When to Use NumPy Arrays vs Python Lists

Use NumPy when:

1. Performing mathematical operations on entire arrays
2. Need memory efficiency (homogeneous data)
3. Require multi-dimensional data structures
4. Need linear algebra operations
5. Working with large datasets (>1000 elements)

Use Python lists when:

1. Data is heterogeneous (mixed types)
 2. Frequently inserting/deleting elements
 3. Small datasets where performance doesn't matter
 4. Need built-in Python methods (append, pop)
-

NumPy Array Creation

Why Different Creation Methods?

`np.zeros()`, `np.ones()`, `np.empty()`

- **When:** Initialize arrays with specific values
- **Why:** Memory pre-allocation is faster than growing arrays
- **Use case:** Creating placeholder arrays for computations

`np.arange()` vs `np.linspace()`

- `np.arange(start, stop, step)`: Specifies step size
 - Use when you know the interval between values
 - Example: `np.arange(0, 10, 2)` → [0, 2, 4, 6, 8]
- `np.linspace(start, stop, num)`: Specifies number of points
 - Use when you need exact number of evenly spaced values
 - Example: `np.linspace(0, 1, 5)` → [0, 0.25, 0.5, 0.75, 1.0]
 - Common in plotting, signal processing

`np.random` functions

- `np.random.rand()`: Uniform distribution [0, 1)

- `np.random.randn()`: Standard normal distribution ($\mu=0$, $\sigma=1$)
 - `np.random.randint()`: Discrete uniform distribution
 - **When:** Testing algorithms, simulations, initializing neural networks
-

Array Indexing and Slicing

Basic Indexing

Mathematical notation:

For 2D array $A[i, j]$:

- i : row index (0 to rows-1)
- j : column index (0 to cols-1)
- $A[i, j]$ accesses element at row i , column j

Why zero-based indexing:

- Memory offset calculation: $\text{address} = \text{base} + (\text{index} \times \text{element_size})$
- Matches pointer arithmetic in C/C++
- Simplifies slice notation

Boolean Indexing (Masking)

Concept: Use boolean arrays to select elements

How it works:

```
python

arr = np.array([1, 2, 3, 4, 5])
mask = arr > 3 # [False, False, False, True, True]
result = arr[mask] # [4, 5]
```

When to use:

- Filtering data based on conditions
- Replacing values conditionally
- Creating subsets for analysis

Mathematical interpretation:

- Boolean mask is an indicator function: $I(x > 3)$
- Returns 1 (True) where condition holds, 0 (False) otherwise

Fancy Indexing

Concept: Use arrays of indices to select elements

```
python

arr = np.array([10, 20, 30, 40, 50])
indices = [0, 2, 4]
result = arr[indices] # [10, 30, 50]
```

When to use:

- Selecting non-contiguous elements
 - Reordering data
 - Gathering results from computed indices
-

Broadcasting

What is Broadcasting?

Broadcasting allows NumPy to perform operations on arrays of different shapes by automatically expanding smaller arrays.

Mathematical concept: Extend smaller array to match larger array's shape without copying data.

Broadcasting rules:

1. If arrays have different numbers of dimensions, pad the smaller array's shape with 1s on the left
2. Arrays are compatible if, for each dimension, sizes are equal or one is 1
3. The dimension with size 1 is stretched to match the other

Example:

```
python

A: (3, 4)  # 3 rows, 4 columns
B: (4,)    # 1D array with 4 elements

Step 1: B becomes (1, 4)
Step 2: B is broadcast to (3, 4) by repeating
Result: A + B works element-wise
```

Visual representation:

```
A = [[1, 2, 3, 4],    B = [10, 20, 30, 40]
     [5, 6, 7, 8],
     [9, 10, 11, 12]]
```

Broadcasting B:

```
[[10, 20, 30, 40],
 [10, 20, 30, 40],
 [10, 20, 30, 40]]
```

Result A + B:

```
[[11, 22, 33, 44],
 [15, 26, 37, 48],
 [19, 30, 41, 52]]
```

Why broadcasting is powerful:

- Avoids explicit loops
- Saves memory (no actual copying)
- Vectorized operations remain fast
- More readable code

Common use cases:

1. Normalizing data: $(X - \text{mean}) / \text{std}$
2. Adding bias in neural networks: $\text{output} + \text{bias}$
3. Distance calculations: $\text{points} - \text{center}$

Universal Functions (ufuncs)

What are ufuncs?

Functions that operate element-wise on arrays, supporting broadcasting and type casting.

Mathematical operations:

- **Arithmetic:** $+$, $-$, $*$, $/$, $**$, $\%$
- **Trigonometric:** $\sin$, $\cos$, $\tan$, $\arcsin$, $\arccos$, $\arctan$
- **Exponential/Log:** $\exp$, $\log$, $\log_{10}$, $\log_2$
- **Comparison:** $>$, $<$, $>=$, $<=$, $==$, $!=$

Why ufuncs are fast:

1. Implemented in compiled C code
2. Operate on entire arrays at once (SIMD instructions)
3. Avoid Python interpreter overhead

Example: Speed comparison

```
python

# Slow: Python loop
result = [math.sqrt(x) for x in arr] # O(n) in Python

# Fast: NumPy ufunc
result = np.sqrt(arr) # O(n) in C, 10-100x faster
```

Aggregation Functions

Statistical Aggregations

Concept: Reduce array to summary statistics

Common aggregations:

1. `sum()`: $\sum x_i$ - Total of all elements
2. `mean()`: $\mu = (\sum x_i)/n$ - Average value
3. `std()`: $\sigma = \sqrt{(\sum (x_i - \mu)^2)/n}$ - Spread of data
4. `var()`: $\sigma^2 = \sum (x_i - \mu)^2/n$ - Variance
5. `min()`/`max()`: Extreme values
6. `median()`: Middle value (50th percentile)
7. `percentile()`: Value below which p% of data falls

Axis parameter explained:

For 2D array (matrix):

- `axis=None` (default): Aggregate entire array
- `axis=0`: Aggregate down columns (result has shape of columns)
- `axis=1`: Aggregate across rows (result has shape of rows)

Visual example:

```
Array: [[1, 2, 3],  
        [4, 5, 6]]
```

```
sum(axis=0): [5, 7, 9] # Column sums: [1+4, 2+5, 3+6]
```

```
sum(axis=1): [6, 15] # Row sums: [1+2+3, 4+5+6]
```

```
sum(): 21 # Total sum
```

Mathematical interpretation:

- `axis=0`: Apply operation along dimension 0 (collapse rows)
- `axis=1`: Apply operation along dimension 1 (collapse columns)

When to use:

- **Exploratory data analysis:** Understand data distribution
 - **Feature engineering:** Create summary features
 - **Validation:** Check for outliers, anomalies
 - **Reporting:** Generate statistics
-

Array Manipulation

Reshaping

Concept: Change array dimensions without changing data

Mathematical constraint:

Original size = New size

If `arr.shape = (m, n)`, then `arr.reshape(p, q)` requires $m \times n = p \times q$

`reshape()` vs `flatten()` vs `ravel()`:

1. `reshape(new_shape)`:
 - Returns view if possible (no copy)
 - Must satisfy size constraint
 - Use when you need specific dimensions
2. `flatten()`:
 - Always returns copy
 - Converts to 1D
 - Safe but slower

3. `ravel()`:

- Returns view if possible
- Converts to 1D
- Faster but changes affect original

When to reshape:

- Preparing data for machine learning (e.g., 28×28 images $\rightarrow$ 784 vector)
- Matrix operations requiring specific dimensions
- Broadcasting compatibility

Transpose

Mathematical concept:

If A is $m \times n$ matrix, then A^T is $n \times m$ matrix
where $(A^T)_{ij} = A_{ji}$

Visual example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

When to use:

- Linear algebra operations ($A^T \times A$)
- Switching row/column orientation
- Dot product calculations: `x.T @ y`

Stacking and Concatenation

`vstack` (vertical stack):

- Stack arrays vertically (row-wise)
- Arrays must have same number of columns
- Use when adding more rows (observations)

`hstack` (horizontal stack):

- Stack arrays horizontally (column-wise)
- Arrays must have same number of rows

- Use when adding more features (variables)

concatenate:

- General purpose concatenation
- Specify axis for direction
- More flexible than vstack/hstack

When to use:

- Combining datasets from multiple sources
 - Adding features or observations
 - Batch processing results
-

Linear Algebra

Matrix Multiplication

Three types of products:

1. **Element-wise (Hadamard product):** $A * B$

$$\begin{aligned} [1, 2] * [5, 6] &= [1 \times 5, 2 \times 6] = [5, 12] \\ [3, 4] \quad [7, 8] \quad [3 \times 7, 4 \times 8] \quad [21, 32] \end{aligned}$$

- Arrays must have same shape
- Use for scaling, masking

2. **Dot product:** $\text{np.dot}(A, B)$ or $A @ B$

$$\text{Mathematical: } (A @ B)_{ij} = \sum_k A_{ik} \times B_{kj}$$

$$\begin{aligned} [1, 2] @ [5, 6] &= [1 \times 5 + 2 \times 7, 1 \times 6 + 2 \times 8] = [19, 22] \\ [3, 4] \quad [7, 8] \quad [3 \times 5 + 4 \times 7, 3 \times 6 + 4 \times 8] \quad [43, 50] \end{aligned}$$

- A is (m, n), B is (n, p) $\rightarrow$ Result is (m, p)
- Inner dimensions must match
- Use for linear transformations, projections

3. **Matrix multiplication:** $\text{np.matmul}(A, B)$

- Similar to dot but handles higher dimensions differently
- Broadcasting for batch operations
- Recommended for modern code

When to use matrix multiplication:

- Linear regression: `X @ weights`
- Neural networks: `activation(W @ input + bias)`
- Coordinate transformations
- Solving systems of equations

Key Linear Algebra Operations

1. Inverse: `np.linalg.inv(A)`

- **Mathematical:** A^{-1} such that $A @ A^{-1} = I$
- **When:** Solving $Ax = b$ as $x = A^{-1}b$
- **Caution:** Only for square, non-singular matrices
- **Numerical stability:** Prefer `solve()` over explicit inverse

2. Determinant: `np.linalg.det(A)`

- **Mathematical:** Scalar value $\det(A)$
- **Interpretation:**
 - $\det(A) = 0 \rightarrow$ Matrix is singular (not invertible)
 - $|\det(A)| =$ volume scaling factor of transformation
- **When:** Checking invertibility, calculating volumes

3. Eigenvalues/Eigenvectors: `np.linalg.eig(A)`

- **Mathematical:** $Av = \lambda v$ where λ is eigenvalue, v is eigenvector
- **Interpretation:** Directions that are only scaled (not rotated) by A
- **When:**
 - Principal Component Analysis (PCA)
 - Stability analysis
 - Google PageRank algorithm
 - Quantum mechanics

4. Solve: `np.linalg.solve(A, b)`

- **Mathematical:** Solve $Ax = b$ for x

- **When:** Systems of linear equations
 - **Why better than inverse:** More numerically stable and faster
-

Pandas: Panel Data

What is Pandas?

Pandas provides data structures for efficiently storing and manipulating labeled, tabular data.

Why Pandas exists:

- NumPy lacks labels (row/column names)
- Need for heterogeneous data (mixed types)
- Database-like operations (group, join, merge)
- Time series functionality
- Missing data handling

Core philosophy:

- **NumPy:** Fast numerical operations on homogeneous arrays
 - **Pandas:** Flexible data manipulation with labels and mixed types
-

Pandas Data Structures

Series: 1D Labeled Array

Mathematical concept:

Series = Vector + Index
Values: [1, 2, 3]
Index: ['a', 'b', 'c']
Result: $a \rightarrow 1, b \rightarrow 2, c \rightarrow 3$

When to use Series:

- Single variable/feature
- Time series data
- Result of aggregations
- Column from DataFrame

Series vs NumPy array:

- Series has index (labels)
- Series allows mixed operations with labels
- Series is built on NumPy (has `.values` attribute)

DataFrame: 2D Labeled Table

Mathematical concept:

DataFrame = Matrix + Row Index + Column Names

	Col1	Col2	Col3
Row1	1	4	7
Row2	2	5	8
Row3	3	6	9

Structure:

- Each column is a Series
- All Series share the same index
- Different columns can have different types

When to use DataFrame:

- Tabular data (like Excel, SQL table)
 - Multiple features per observation
 - Mixed data types
 - Real-world datasets
-

Data Selection: loc vs iloc

The Fundamental Difference

`loc`: Label-based indexing

- Uses row/column labels (names)
- Includes the end point in slices
- Example: `df.loc['row1':'row3']` includes row3

`iloc`: Position-based indexing

- Uses integer positions (0, 1, 2, ...)
- Excludes the end point (Python style)
- Example: `df.iloc[0:3]` includes positions 0, 1, 2

Why two systems?

- Labels are meaningful but can be non-numeric
- Positions are universal but less intuitive
- Need both for flexibility

When to use:

- `loc`: When you know row/column names, time-based slicing
- `iloc`: When position matters, working with unknown labels

Speed comparison:

- `at/iat` for single values: 10x faster than `loc/iloc`
 - Use `at/iat` in loops (though vectorization is better)
-

Handling Missing Data

Types of Missing Data (Statistical Theory)

1. MCAR (Missing Completely at Random):

- Missingness unrelated to any variable
- Safe to drop or impute
- Example: Sensor randomly fails

2. MAR (Missing at Random):

- Missingness related to observed variables
- Can impute using related variables
- Example: Young people don't report income

3. MNAR (Missing Not at Random):

- Missingness related to unobserved value
- Dangerous to impute naively
- Example: High earners don't report income

Strategies for Handling Missing Data

1. Deletion:

```
python

df.dropna()          # Listwise deletion
df.dropna(thresh=0.8*len(df.columns)) # Threshold
```

- **When:** MCAR, small proportion missing (<5%)
- **Pros:** Simple, no bias if MCAR
- **Cons:** Loses information, reduces sample size

2. Imputation:

Mean/Median imputation:

```
python

df.fillna(df.mean())
df.fillna(df.median())
```

- **When:** Numeric data, roughly symmetric distribution
- **Pros:** Preserves sample size
- **Cons:** Underestimates variance, ignores relationships

Forward/Backward fill:

```
python

df.fillna(method='ffill') # Carry last observation forward
df.fillna(method='bfill') # Use next observation
```

- **When:** Time series, ordered data
- **Rationale:** Values change slowly over time
- **Cons:** Not suitable for cross-sectional data

Interpolation:

```
python

df.interpolate(method='linear')
df.interpolate(method='polynomial', order=2)
```

- **When:** Time series, continuous numeric data
- **Mathematical:** Fit function between known points

- **Linear:** $y = y_1 + (y_2 - y_1) \times (x - x_1) / (x_2 - x_1)$

3. Advanced methods:

- Multiple imputation (statistical)
 - KNN imputation (use similar records)
 - Model-based imputation (predict missing values)
-

Data Types and Memory Optimization

Why Data Types Matter

Memory usage example:

int64: 8 bytes per value

int32: 4 bytes per value

int8: 1 byte per value

For 1 million rows:

int64: 8 MB

int32: 4 MB (50% savings)

int8: 1 MB (87.5% savings)

Type selection guide:

Integers:

- `int8`: -128 to 127 (e.g., age, small counts)
- `int16`: -32,768 to 32,767 (e.g., year, medium counts)
- `int32`: ± 2 billion (default choice)
- `int64`: Very large numbers (often overkill)

Floats:

- `float32`: 7 decimal digits precision
- `float64`: 16 decimal digits precision (default)

Strings:

- `object`: Default, flexible but memory-heavy
- `category`: For repeated strings (huge savings)

Category type:

```
python
```

```
df['color'].astype('category')
```

- **When:** Column with repeated values
 - **How it works:** Maps strings to integers internally
 - **Memory saving:** $O(\text{unique_values})$ instead of $O(\text{total_values})$
 - **Example:** 1M rows, 10 unique colors $\rightarrow$ 99% memory reduction
-

GroupBy: Split-Apply-Combine

The GroupBy Paradigm

Conceptual model:

1. **Split:** Divide data into groups based on key(s)
2. **Apply:** Compute function on each group independently
3. **Combine:** Aggregate results back together

Mathematical representation:

For groups $g_1, g_2, \dots, g_k$:

Result = $[f(g_1), f(g_2), \dots, f(g_k)]$

where f could be: sum, mean, count, custom function

Example:

```
python
```

```
df.groupby('category')['sales'].sum()
```

Split:

Category A: [100, 200, 150]

Category B: [300, 250]

Category C: [400, 350, 300]

Apply (sum):

Category A: 450

Category B: 550

Category C: 1050

Combine:

category

A 450

B 550

C 1050

Aggregation vs Transform vs Filter

1. Aggregation:

```
python
```

```
df.groupby('group')['value'].mean()
```

- Returns one row per group
- Reduces data size
- Use for summary statistics

2. Transform:

```
python
```

```
df.groupby('group')['value'].transform('mean')
```

- Returns same shape as input
- Broadcasts result back to original rows
- Use for group-wise normalization: $(x - \text{group_mean}) / \text{group_std}$

3. Filter:

```
python
```

```
df.groupby('group').filter(lambda x: len(x) > 5)
```

- Returns subset of original data
- Keeps entire groups that meet condition
- Use for removing small groups

When to use each:

- **Aggregation:** Creating summary reports, statistics
 - **Transform:** Feature engineering, normalization
 - **Filter:** Data quality checks, outlier removal
-

Merging and Joining

Types of Joins (Set Theory)

Mathematical interpretation using sets:

Given two tables A and B on key column:

- $A = \{a_1, a_2, a_3\}$ (keys in A)
- $B = \{b_1, b_2, b_3\}$ (keys in B)

1. Inner Join: $A \cap B$

- Only matching keys
- Most restrictive
- Use when you need complete information from both sources

2. Left Join: $A \cup (A \cap B) = A$

- All keys from A, matching from B
- Missing B values become NaN
- Use when A is primary dataset

3. Right Join: $(A \cap B) \cup B = B$

- All keys from B, matching from A
- Rarely used (just swap and use left join)

4. Outer Join: $A \cup B$

- All keys from both
- Most inclusive
- Use when you want to preserve all information

Visual example:

```

A:      B:
ID Value_A  ID Value_B
1  100    2  200
2  150    3  250
3  175    4  300

Inner (1∩2): Left (keep A): Right (keep B): Outer (1∪2):
ID Val_A Val_B ID Val_A Val_B ID Val_A Val_B ID Val_A Val_B
2 150  200  1 100  NaN  2 150  200  1 100  NaN
3 175  250  2 150  200  3 175  250  2 150  200
          3 175  250  4 NaN  300  3 175  250
                    4 NaN  300

```

Join Performance Considerations

Complexity:

- Hash join: $O(n + m)$ average case
- Merge join (sorted): $O(n + m)$
- Nested loop: $O(n \times m)$ - avoid!

Best practices:

1. Set indices on join keys if joining repeatedly
2. Use categorical types for join columns with repeated values
3. Sort data before merge join
4. Prefer inner join when possible (smaller result)

Pivot Tables and Reshaping

Pivot Table Theory

Concept: Restructure data from long to wide format with aggregation

Mathematical operation:

Input (long format):

Date	Category	Value
2024-01-01	A	100
2024-01-01	B	200
2024-01-02	A	150
2024-01-02	B	250

Output (wide format after pivot):

	A	B
2024-01-01	100	200
2024-01-02	150	250

When to use:

- Creating cross-tabulations
- Converting from database format to analysis format
- Calculating summary statistics across dimensions
- Preparing data for visualization

Melt (Unpivot): Wide to Long

Inverse operation of pivot

Why melt:

1. **Tidy data principle:** Each variable is a column, each observation is a row
2. **Easier groupby:** When categories are in columns, hard to group
3. **Better for plotting:** Most plotting libraries prefer long format
4. **Database normalization:** Long format is normalized

Example:

Input (wide):

ID	Q1_Sales	Q2_Sales	Q3_Sales
1	100	150	200
2	120	140	180

Output (long after melt):

ID	Quarter	Sales
1	Q1	100
1	Q2	150
1	Q3	200
2	Q1	120
2	Q2	140
2	Q3	180

Time Series Analysis

Why Time Series is Special

Temporal dependencies:

- Today's value depends on yesterday's
- Violates i.i.d. assumption (independent and identically distributed)
- Need special methods: autocorrelation, stationarity

Key Time Series Operations

1. Resampling:

```
python
```

```
df.resample('M').mean() # Upsampling or downsampling
```

Upsampling (increase frequency):

- Daily → Hourly
- Fills missing values with NaN or interpolation
- Use when you need finer granularity

Downsampling (decrease frequency):

- Hourly → Daily, Daily → Monthly
- Requires aggregation function (mean, sum, last)

- Use for summarization, reducing noise

Common frequencies:

- `D`: Daily
- `W`: Weekly
- `M`: Month end
- `MS`: Month start
- `Q`: Quarter end
- `Y`: Year end
- `H`: Hourly
- `T` or `min`: Minute

2. Rolling Windows:

```
python

df.rolling(window=7).mean() # Moving average
```

Mathematical definition:

For window size k :

$$MA(t) = (1/k) \sum_{i=0}^{k-1} x(t-i)$$

Example with $k=3$:

Data: [1, 2, 3, 4, 5]

MA: [-, -, 2, 3, 4]

where 2 = (1+2+3)/3

When to use:

- **Smoothing:** Remove noise from data
- **Trend detection:** See underlying patterns
- **Feature engineering:** Create lagged features
- **Forecasting:** Simple prediction method

Types of rolling statistics:

- `rolling().mean()`: Smooth noise
- `rolling().std()`: Detect volatility changes
- `rolling().sum()`: Cumulative effects
- `rolling().max()`/`min()`: Range over window

3. Shifting:

python

```
df.shift(1)  # Lag by 1 period  
df.shift(-1) # Lead by 1 period
```

When to use:

- Creating lagged features: $df['value_lag1'] = df['value'].shift(1)$
- Calculating returns: $df['return'] = df['price'] / df['price'].shift(1) - 1$
- Comparing periods: $df['change'] = df['value'] - df['value'].shift(1)$

4. Differencing:

python

```
df.diff()  # First difference  
df.diff(2) # Second difference
```

Mathematical:

First difference: $\nabla x(t) = x(t) - x(t-1)$
Second difference: $\nabla^2 x(t) = \nabla x(t) - \nabla x(t-1)$

Purpose:

- Remove trend (make stationary)
- Calculate change/growth rate
- Required for many forecasting models (ARIMA)

5. Percentage Change:

python

```
df.pct_change() #  $(x(t) - x(t-1)) / x(t-1)$ 
```

When to use:

- Stock returns
 - Growth rates
 - Normalizes values (comparable across scales)
-

Advanced Pandas Concepts

Apply, Map, and Applymap

Confusion matrix:

Function	Works On	Returns	Use Case
<code>apply()</code>	DataFrame or Series	Series or DataFrame	Row/column-wise operations
<code>map()</code>	Series	Series	Element-wise transformations
<code>applymap()</code>	DataFrame	DataFrame	Element-wise (deprecated)

Performance hierarchy (fast to slow):

- 1. **Vectorized operations:** `df['A'] + df['B']` (fastest, always prefer)
- 2. **Built-in functions:** `df.sum()`, `df.mean()`
- 3. **NumPy ufuncs:** `np.sqrt(df)`
- 4. **apply() with NumPy:** `df.apply(np.sum)`
- 5. **apply() with Python:** `df.apply(lambda x: sum(x))`
- 6. **iterrows():** Avoid if possible (slowest)

When to use apply:

- Complex transformations not available as vectorized operations
- Custom functions requiring multiple columns
- Conditional logic too complex for `np.where()`

MultiIndex (Hierarchical Indexing)

Concept: Multiple levels of indices for rows/columns

Example:

		Sales	Profit
Region	Store		
North	A	100	20
	B	150	30
South	A	120	25
	B	180	35

Why use MultiIndex:

1. **Natural representation:** Some data has inherent hierarchy
2. **Efficient groupby:** Pre-grouped structure
3. **Slicing:** Select data at different levels
4. **Reshaping:** Easy pivot/unstack operations

When to use:

- Panel data (multiple dimensions: time, region, product)
- Results from groupby with multiple keys
- Time series with multiple instruments
- Repeated measures data

Method Chaining

Concept: Consecutive operations on DataFrame

```
python

(df
 .query('sales > 100')
 .groupby('category')
 .agg({'sales': 'sum', 'profit': 'mean'})
 .sort_values('sales', ascending=False)
 .head(10)
 )
```

Advantages:

- Readable pipeline
- No intermediate variables
- Easier to debug (comment out lines)
- Functional programming style

When to use:

- Data cleaning pipelines
 - Exploratory analysis
 - Reproducible workflows
-

Performance Optimization

Why Pandas Can Be Slow

Root causes:

1. **Python loops:** Interpreted, dynamically typed
2. **Object dtype:** Strings stored as Python objects (slow)
3. **Memory allocation:** Growing DataFrames requires reallocation
4. **Type checking:** Runtime type inference overhead

Optimization Strategies

1. Vectorization

Theory: Replace explicit loops with array operations

Mathematical complexity:

Python loop: n operations in Python interpreter = $O(n) \times \text{Python_overhead}$

Vectorized: n operations in C = $O(n) \times C_overhead$

Speed ratio: 10x - 100x faster

Example:

```
python

# Slow - Python loop
result = []
for x in df['A']:
    result.append(x * 2 + 5)

# Fast - Vectorized
result = df['A'] * 2 + 5
```

When vectorization is possible:

- Element-wise operations
- Mathematical transformations
- Boolean logic
- String operations (`.str` accessor)

2. Use Efficient Data Types

Memory and speed impact:

Operation on 1M elements:

int64 (8 bytes):

- Memory: 8 MB
- Speed: baseline

int32 (4 bytes):

- Memory: 4 MB (50% reduction)
- Speed: ~20% faster (better cache utilization)

category (string $\rightarrow$ int mapping):

- Memory: 0.1 MB for 100 unique values (98% reduction)
- Speed: 10x faster for groupby operations

Strategy:

- Use smallest integer type that fits range
- Convert repeated strings to category
- Use float32 for machine learning (acceptable precision)

3. Avoid DataFrame Growth

Why growing is slow:

Append to DataFrame n times:

- Each append creates new memory block
- Copy entire DataFrame each time
- Complexity: $O(n^2)$

Pre-allocate and fill:

- Allocate once
- Fill in place
- Complexity: $O(n)$

Best practice:

python

```
# Bad -  $O(n^2)$ 
df = pd.DataFrame()
for item in items:
    df = df.append(item)

# Good -  $O(n)$ 
results = []
for item in items:
    results.append(item)
df = pd.DataFrame(results)
```

4. Query vs Boolean Indexing

`query()` advantages:

- Optimized for large DataFrames (>15,000 rows)
- Uses numexpr library (faster evaluation)
- More readable for complex conditions

When to use:

```
python

# Small DataFrame - use boolean indexing
df[df['A'] > 5]

# Large DataFrame - use query
df.query('A > 5')

# Complex conditions - query is more readable
df.query('(A > 5) & (B < 10) & (C == "X")')
```

5. Chunking Large Files

Memory limitation problem:

```
File size: 10 GB
Available RAM: 8 GB
Loading entire file: MemoryError
```

Solution - process in chunks:

```
python
```

```
chunk_size = 10000
for chunk in pd.read_csv('large.csv', chunksize=chunk_size):
    # Process each chunk
    result = process(chunk)
    # Save or aggregate
```

When to use:

- File larger than RAM
- Only need aggregated results
- Streaming data processing

6. Use eval for Complex Expressions

Theory: Compile arithmetic expressions for faster evaluation

```
python

# Slower - standard pandas
df['result'] = df['A'] + df['B'] * df['C'] - df['D']

# Faster - compiled expression
df.eval('result = A + B * C - D')
```

Speed improvement:

- 2-3x faster for complex expressions
- Lower memory usage (no intermediate arrays)
- Works best with >10,000 rows

Statistical Concepts in Pandas

Descriptive Statistics

describe() output explained:

```
python

df.describe()
```

Returns these statistics:

- **count:** Number of non-null values (sample size n)

- **mean:** $\mu = (\sum x_i)/n$ - Central tendency
- **std:** $\sigma = \sqrt{(\sum (x_i - \mu)^2)/(n-1))}$ - Standard deviation (spread)
- **min/max:** Range of data
- **25%, 50%, 75%:** Quartiles (Q1, median, Q3)

Interpreting standard deviation:

- $\sigma \approx 0$: Data is clustered around mean
- Large σ : Data is spread out
- For normal distribution: $\sim 68\%$ within $\mu \pm \sigma$, $\sim 95\%$ within $\mu \pm 2\sigma$

Correlation

Mathematical definition:

Pearson correlation coefficient:

$$r = \frac{\sum((x_i - \bar{x})(y_i - \bar{y}))}{\sqrt{(\sum(x_i - \bar{x})^2 \times \sum(y_i - \bar{y})^2)}}$$

Range: $-1 \leq r \leq 1$

Interpretation:

- $r = 1$: Perfect positive correlation
- $r = -1$: Perfect negative correlation
- $r = 0$: No linear correlation
- $|r| > 0.7$: Strong correlation
- $|r| < 0.3$: Weak correlation

When to use:

python

`df.corr()` # All pairwise correlations

`df['A'].corr(df['B'])` # Specific pair

Use cases:

- Feature selection (remove highly correlated features)
- Identify relationships in data
- Validate hypotheses
- Detect multicollinearity

Caution:

- Correlation $\neq$ Causation
- Only measures linear relationships
- Sensitive to outliers

Covariance

Mathematical definition:

$$\text{Cov}(X, Y) = E[(X - \mu_x)(Y - \mu_y)] = \Sigma((x_i - \bar{x})(y_i - \bar{y})) / (n-1)$$

Difference from correlation:

- Covariance: Units depend on variables
- Correlation: Standardized (-1 to 1)
- Relation: $r = \text{Cov}(X, Y) / (\sigma_x \times \sigma_y)$

When to use:

- Portfolio theory (asset covariances)
 - Multivariate statistics
 - PCA preprocessing
-

Data Cleaning Theory

Tidy Data Principles

Hadley Wickham's definition:

1. Each variable forms a column
2. Each observation forms a row
3. Each type of observational unit forms a table

Why it matters:

- Standardized structure $\rightarrow$ easier analysis
- Works seamlessly with groupby, pivot, merge
- Reduces errors in analysis

Example of untidy $\rightarrow$ tidy:

Untidy:

Name	Q1_Sales	Q2_Sales	Q3_Sales
John	100	150	200
Jane	120	140	180

Tidy:

Name	Quarter	Sales
John	Q1	100
John	Q2	150
John	Q3	200
Jane	Q1	120
Jane	Q2	140
Jane	Q3	180

Outlier Detection

Statistical methods:

1. Z-score method:

$$z = (x - \mu) / \sigma$$

Outlier if $|z| > 3$

- Assumes normal distribution
- Threshold typically 2.5 or 3
- Sensitive to extreme outliers (they affect μ and σ)

2. IQR (Interquartile Range) method:

Q1 = 25th percentile

Q3 = 75th percentile

IQR = Q3 - Q1

Outlier if:

$x < Q1 - 1.5 \times IQR$ or $x > Q3 + 1.5 \times IQR$

- Robust to extreme values
- Doesn't assume distribution
- Used in box plots
- Generally preferred method

3. Modified Z-score (MAD - Median Absolute Deviation):

$MAD = \text{median}(|x_i - \text{median}(x)|)$

$\text{Modified z-score} = 0.6745 \times (x_i - \text{median}(x)) / MAD$

Outlier if $|\text{modified z-score}| > 3.5$

- Most robust method
- Resistant to extreme outliers
- Better for skewed distributions

When to remove outliers:

- Data entry errors (impossible values)
- Measurement errors
- Sample from different population

When to keep outliers:

- Genuine extreme values
 - Valuable information (fraud detection)
 - Small sample size (can't afford to lose data)
-

Feature Engineering with Pandas

Creating New Features

1. Mathematical transformations:

```
python

# Log transformation - reduce skewness
df['log_sales'] = np.log(df['sales'] + 1)

# Square root - moderate skewness
df['sqrt_age'] = np.sqrt(df['age'])

# Polynomial features
df['sales_squared'] = df['sales'] ** 2
```

Why transform:

- Linearize relationships
- Normalize distributions

- Reduce impact of outliers
- Meet model assumptions

2. Binning (Discretization):

```
python

# Equal-width bins
pd.cut(df['age'], bins=5)

# Equal-frequency bins (quantiles)
pd.qcut(df['income'], q=4)

# Custom bins
pd.cut(df['age'], bins=[0, 18, 65, 100],
      labels=['minor', 'adult', 'senior'])
```

When to bin:

- Capture non-linear relationships
- Reduce noise in continuous variables
- Create interpretable categories
- Handle outliers

3. Interaction features:

```
python

df['price_per_sqft'] = df['price'] / df['sqft']
df['total_rooms'] = df['bedrooms'] + df['bathrooms']
df['interaction'] = df['A'] * df['B']
```

Purpose:

- Capture combined effects
- Improve model performance
- Domain knowledge incorporation

Encoding Categorical Variables

1. Label Encoding:

```
python

df['category_encoded'] = df['category'].astype('category').cat.codes
```

When to use:

- Ordinal variables (small < medium < large)
- Tree-based models (decision trees, random forests)

Problems:

- Implies ordering (A=0, B=1, C=2 suggests $C > B > A$)
- Not suitable for non-ordinal categories

2. One-Hot Encoding:

```
python  
  
pd.get_dummies(df['category'])
```

Mathematical representation:

Original:	One-Hot Encoded:		
category	cat_A	cat_B	cat_C
A	1	0	0
B	0	1	0
C	0	0	1

When to use:

- Nominal variables (no order)
- Linear models, neural networks
- Few unique categories (<10)

Problems:

- Curse of dimensionality (many categories $\rightarrow$ many columns)
- Dummy variable trap (perfect multicollinearity)
- Solution: Drop one category (`drop_first=True`)

3. Frequency Encoding:

```
python  
  
freq = df['category'].value_counts()  
df['category_freq'] = df['category'].map(freq)
```

When to use:

- Many categories
- Frequency is informative
- Reduce dimensionality

4. Target Encoding:

```
python

mean_target = df.groupby('category')['target'].mean()
df['category_encoded'] = df['category'].map(mean_target)
```

When to use:

- Categorical with many levels
 - Strong relationship with target
 - Careful: Risk of overfitting (use cross-validation)
-

Window Functions Theory

Cumulative Operations

Cumulative sum:

Mathematical: $S(n) = \sum_{i=1}^n x_i$

Data: [1, 2, 3, 4, 5]

Cumsum: [1, 3, 6, 10, 15]

When to use:

- Running totals (accumulated sales)
- Cumulative statistics
- Ranking over time

Cumulative product:

$P(n) = \prod_{i=1}^n x_i$

Example - compound returns:

Returns: [1.05, 1.03, 1.02]

Cumulative: [1.05, 1.0815, 1.10313]

Use cases:

- Compound interest
- Growth factors
- Probability chains

Ranking

Ranking methods:

```
python  
  
df['rank'] = df['score'].rank()
```

Methods (for ties):

1. **average:** $(1+2)/2 = 1.5$ for both
2. **min:** Both get 1
3. **max:** Both get 2
4. **first:** Order of appearance
5. **dense:** 1, 1, 2 (no gaps)

When to use:

- Percentile calculation
- Competition results
- Creating ordinal features

Expanding Windows

Concept: Window starts at first row and grows

```
python  
  
df['expanding_mean'] = df['value'].expanding().mean()
```

Mathematical:

For time t:

Expanding mean = $(1/t) \times \sum_{i=1}^t x_i$

t=1: x_1

t=2: $(x_1 + x_2) / 2$

t=3: $(x_1 + x_2 + x_3) / 3$

When to use:

- All-time statistics
- Historical averages
- Baseline comparison

Difference from rolling:

- Rolling: Fixed window size
 - Expanding: Growing window size
-

Query Optimization

Index Theory

What is an index:

- Data structure (usually B-tree or hash table)
- Maps values to row locations
- Trades space for speed

Complexity:

Without index:

- Search: $O(n)$ - scan entire DataFrame
- Filter: $O(n)$

With index:

- Search: $O(\log n)$ - binary search
- Filter: $O(\log n + k)$ where k = result size

When to index:

- Frequent filtering on column
- Merging/joining on column

- Groupby on column
- Time series (datetime index)

Cost:

- Additional memory (~20% overhead)
- Slower inserts/updates
- Only helps for indexed columns

Best practices:

```
python

# Index before repeated operations
df = df.set_index('id')
result1 = df.loc[123] # Fast
result2 = df.loc[456] # Fast

# Reset index when no longer needed
df = df.reset_index()
```

Filter Order Matters

Optimization principle: Apply most selective filters first

```
python

# Less efficient
df[(df['B'] < 100) & (df['A'] == 'rare_value')]

# More efficient (if 'A' is very selective)
df[(df['A'] == 'rare_value') & (df['B'] < 100)]
```

Why:

- Short-circuit evaluation
- Smaller intermediate results
- Better memory usage

Rule of thumb:

1. Equality filters first (most selective)
2. Range filters second
3. Expensive operations last (string matching, regex)

Memory Management

View vs Copy

Critical concept: Some operations return views (reference), others return copies

View (shares memory):

```
python

subset = df[['A', 'B']] # Copy
row_slice = df[10:20]  # View (usually)
column = df['A']       # View
```

Why it matters:

```
python

# Modifying view affects original
row_slice = df[10:20]
row_slice['A'] = 99 # Changes df!

# Solution: Explicit copy
row_slice = df[10:20].copy()
row_slice['A'] = 99 # df unchanged
```

Checking:

```
python

subset._is_view # Check if view
```

Best practice:

- Use `.copy()` when creating subset for modification
- Understand view vs copy behavior
- Watch for `SettingWithCopyWarning`

Memory Profiling

Check memory usage:

```
python
```

```
df.info(memory_usage='deep') # Actual memory usage
df.memory_usage(deep=True).sum() # Total bytes
```

Optimization workflow:

```
python

# 1. Check current usage
initial = df.memory_usage(deep=True).sum()

# 2. Optimize types
df['int_col'] = df['int_col'].astype('int32')
df['cat_col'] = df['cat_col'].astype('category')

# 3. Check improvement
final = df.memory_usage(deep=True).sum()
reduction = (initial - final) / initial * 100
print(f'Reduced by {reduction:.1f}%')
```

Best Practices Summary

Code Organization

1. Readability:

```
python

# Bad
x=df[df.A>5&df.B<10].groupby('C').D.sum()

# Good
filtered = df[(df['A'] > 5) & (df['B'] < 10)]
result = filtered.groupby('C')['D'].sum()

# Best - method chaining
result = (df
    .query('A > 5 and B < 10')
    .groupby('C')['D']
    .sum()
)
```

2. Avoid magic numbers:

```
python
```

```
# Bad
```

```
df = df[df['age'] < 120]
```

```
# Good
```

```
MAX_REASONABLE_AGE = 120
```

```
df = df[df['age'] < MAX_REASONABLE_AGE]
```

3. Validate assumptions:

```
python
```

```
assert df['price'].min() > 0, "Negative prices found"
```

```
assert df['date'].is_monotonic_increasing, "Dates not sorted"
```

```
assert df.index.is_unique, "Duplicate indices"
```

Performance Checklist

1. **Use vectorized operations** instead of loops
2. **Choose appropriate data types** (int32, category)
3. **Index columns** used for frequent filtering/joining
4. **Pre-allocate DataFrames** instead of growing
5. **Use query()** for complex filters on large data
6. **Process in chunks** for files larger than RAM
7. **Profile before optimizing** - measure, don't guess

Data Quality Checks

Standard workflow:

```
python
```

1. Shape and types

`df.shape`

`df.dtypes`

`df.info()`

2. Missing data

`df.isnull().sum()`

`df.isnull().mean()` # Percentage missing

3. Duplicates

`df.duplicated().sum()`

4. Value ranges

`df.describe()`

`df['numeric_col'].quantile([0.01, 0.99])` # Check extremes

5. Cardinality

`df.nunique()`

6. Sample inspection

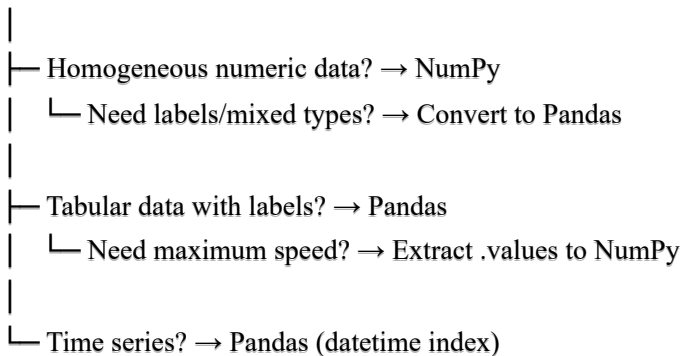
`df.sample(10)`

`df.head()`

When to Use What: Decision Trees

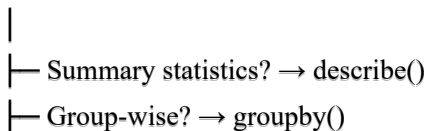
Choosing Between NumPy and Pandas

Start



Choosing Aggregation Method

Aggregation Task



- └─ Time-based? → `resample()`
- └─ Cross-tabulation? → `pivot_table()` or `crosstab()`
- └─ Rolling statistics? → `rolling()`

Choosing Join Method

Merging DataFrames

- |
- └─ Need all from both? → `how='outer'`
- └─ Keep all from left? → `how='left'`
- └─ Only matching? → `how='inner'`
- └─ Simple concat? → `pd.concat()`
- └─ Join on index? → `df1.join(df2)`

This guide provides the theoretical foundation for understanding when, why, and how to use NumPy and Pandas operations effectively in real-world data analysis scenarios.

Complete Pandas & NumPy Cheat Sheet

NumPy Fundamentals

Importing and Basic Array Creation

```
python

import numpy as np

# Creating arrays
a = np.array([1, 2, 3])          # 1D array
b = np.array([[1, 2, 3], [4, 5, 6]]) # 2D array
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]]) # 3D array

# Special arrays
np.zeros((3, 4))                # 3x4 array of zeros
np.ones((2, 3, 4))              # 2x3x4 array of ones
np.empty((2, 2))                # Uninitialized array
np.full((3, 3), 7)              # 3x3 array filled with 7
np.eye(4)                       # 4x4 identity matrix
np.arange(0, 10, 2)             # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5)            # 5 evenly spaced values between 0 and 1
np.random.rand(3, 4)            # Random values [0, 1) in 3x4 shape
np.random.randn(3, 4)           # Random normal distribution
np.random.randint(0, 10, (3, 4)) # Random integers
```

Array Attributes

```
python

arr = np.array([[1, 2, 3], [4, 5, 6]])

arr.shape    # (2, 3) - dimensions
arr.size     # 6 - total elements
arr.ndim     # 2 - number of dimensions
arr.dtype    # data type (int64, float64, etc.)
arr.itemsize # bytes per element
arr.nbytes   # total bytes consumed
```

Array Indexing and Slicing

```
python
```

```
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
```

```
# Basic indexing
```

```
arr[0]      # First row: [1, 2, 3, 4]
```

```
arr[1, 2]   # Element at row 1, col 2: 7
```

```
arr[-1]     # Last row: [9, 10, 11, 12]
```

```
# Slicing
```

```
arr[0:2]    # First two rows
```

```
arr[:, 1]   # All rows, second column: [2, 6, 10]
```

```
arr[1:, 2:] # Rows 1+, columns 2+
```

```
arr[::2, ::2] # Every other row and column
```

```
# Boolean indexing
```

```
arr[arr > 5] # Elements greater than 5
```

```
arr[(arr > 5) & (arr < 10)] # Multiple conditions
```

```
arr[arr % 2 == 0] # Even numbers
```

```
# Fancy indexing
```

```
arr[[0, 2]]  # Rows 0 and 2
```

```
arr[[0, 1], [1, 2]] # Elements at (0,1) and (1,2)
```

Array Operations

```
python
```

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

```
# Arithmetic operations (element-wise)
```

```
a + b      # [5, 7, 9]
```

```
a - b      # [-3, -3, -3]
```

```
a * b      # [4, 10, 18]
```

```
a / b      # [0.25, 0.4, 0.5]
```

```
a ** 2     # [1, 4, 9]
```

```
a % 2      # [1, 0, 1]
```

```
# Universal functions
```

```
np.sqrt(a) # Square root
```

```
np.exp(a)  # Exponential
```

```
np.log(a)  # Natural log
```

```
np.sin(a)  # Sine
```

```
np.cos(a)  # Cosine
```

```
np.abs(a)  # Absolute value
```

```
np.ceil(a) # Ceiling
```

```
np.floor(a) # Floor
```

```
np.round(a, 2) # Round to 2 decimals
```

```
# Comparison operations
```

```
a > 2      # [False, False, True]
```

```
a == b     # Element-wise comparison
```

```
np.allclose(a, b) # Check if arrays are close (with tolerance)
```

Array Manipulation

```
python
```



```
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Reshaping
arr.reshape(3, 2)    # Reshape to 3x2
arr.flatten()        # Flatten to 1D
arr.ravel()          # Flatten (returns view if possible)
arr.T                # Transpose
arr.transpose()      # Transpose (explicit)

# Stacking and splitting
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
np.vstack([a, b])    # Stack vertically
np.hstack([a, b])    # Stack horizontally
np.concatenate([a, b]) # Concatenate
np.stack([a, b])     # Stack along new axis
np.split(arr, 2)     # Split into 2 equal arrays

# Adding/removing elements
np.append(a, [7, 8]) # Append elements
np.insert(a, 1, 99)  # Insert 99 at index 1
np.delete(a, 1)      # Delete element at index 1
np.unique(a)         # Unique elements
```

Aggregation Functions

```
python
```

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
# Basic aggregations
```

```
arr.sum()      # Sum of all elements
```

```
arr.mean()     # Mean
```

```
arr.std()      # Standard deviation
```

```
arr.var()      # Variance
```

```
arr.min()      # Minimum
```

```
arr.max()      # Maximum
```

```
arr.prod()     # Product of all elements
```

```
# Axis-specific operations
```

```
arr.sum(axis=0) # Sum along columns: [5, 7, 9]
```

```
arr.sum(axis=1) # Sum along rows: [6, 15]
```

```
arr.mean(axis=0) # Mean of each column
```

```
arr.cumsum()    # Cumulative sum
```

```
arr.cumprod()   # Cumulative product
```

```
# Position functions
```

```
arr.argmin()    # Index of minimum
```

```
arr.argmax()    # Index of maximum
```

```
np.where(arr > 3) # Indices where condition is true
```

```
np.percentile(arr, 50) # 50th percentile (median)
```

```
np.median(arr)   # Median
```

```
np.quantile(arr, 0.75) # 75th quantile
```

Linear Algebra

```
python
```

```
a = np.array([[1, 2], [3, 4]])
```

```
b = np.array([[5, 6], [7, 8]])
```

```
# Matrix operations
```

```
np.dot(a, b)    # Matrix multiplication
```

```
a @ b           # Matrix multiplication (Python 3.5+)
```

```
np.matmul(a, b) # Matrix multiplication
```

```
np.linalg.inv(a) # Matrix inverse
```

```
np.linalg.det(a) # Determinant
```

```
np.linalg.eig(a) # Eigenvalues and eigenvectors
```

```
np.linalg.solve(a, b) # Solve linear system
```

```
np.trace(a)      # Trace (sum of diagonal)
```

```
np.diag(a)       # Extract diagonal
```

Broadcasting

```
python

# Broadcasting allows operations on arrays of different shapes
a = np.array([[1, 2, 3], [4, 5, 6]]) # (2, 3)
b = np.array([10, 20, 30])          # (3,)
result = a + b # b is broadcast to (2, 3): [[11, 22, 33], [14, 25, 36]]

# More examples
a = np.array([[1], [2], [3]]) # (3, 1)
b = np.array([10, 20, 30])    # (3,)
result = a + b # Result is (3, 3)
```

Pandas Fundamentals

Importing and Basic Structures

```
python

import pandas as pd

# Series - 1D labeled array
s = pd.Series([1, 2, 3, 4])
s = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s = pd.Series({'a': 1, 'b': 2, 'c': 3})

# DataFrame - 2D labeled data structure
df = pd.DataFrame({
    'col1': [1, 2, 3],
    'col2': ['a', 'b', 'c'],
    'col3': [1.5, 2.5, 3.5]
})

# From various sources
df = pd.DataFrame(np.random.randn(4, 3), columns=['A', 'B', 'C'])
df = pd.DataFrame([[1, 2], [3, 4]], columns=['A', 'B'])
```

Reading and Writing Data

```
python
```

```
# CSV files
df = pd.read_csv('file.csv')
df = pd.read_csv('file.csv', sep=';', encoding='utf-8')
df = pd.read_csv('file.csv', index_col=0, parse_dates=['date_col'])
df.to_csv('output.csv', index=False)

# Excel files
df = pd.read_excel('file.xlsx', sheet_name='Sheet1')
df.to_excel('output.xlsx', sheet_name='Data', index=False)

# JSON files
df = pd.read_json('file.json')
df.to_json('output.json', orient='records')

# SQL databases
df = pd.read_sql('SELECT * FROM table', connection)
df.to_sql('table_name', connection, if_exists='replace')

# Other formats
df = pd.read_parquet('file.parquet')
df = pd.read_html('url_or_html_string')[0] # Returns list of DataFrames
df.to_pickle('file.pkl')
df = pd.read_pickle('file.pkl')
```

DataFrame Inspection

```
python

df.head(10)      # First 10 rows (default 5)
df.tail(10)      # Last 10 rows
df.sample(5)     # Random 5 rows
df.shape         # (rows, columns)
df.info()        # Column types, non-null counts, memory usage
df.describe()    # Statistical summary
df.dtypes        # Data types of each column
df.columns       # Column names
df.index         # Index (row labels)
df.values        # NumPy array of values
df.nunique()     # Number of unique values per column
df.memory_usage() # Memory usage per column
```

Indexing and Selection

```
python
```

Column selection

`df['col1']` *# Single column (Series)*

`df[['col1', 'col2']]` *# Multiple columns (DataFrame)*

Row selection

`df[0:3]` *# Rows 0-2 by position*

`df['2020-01-01':'2020-12-31']` *# Slice by index labels*

loc - label-based indexing

`df.loc[0]` *# Row with label 0*

`df.loc[0:3]` *# Rows 0 to 3 (inclusive)*

`df.loc[:, 'col1']` *# All rows, column 'col1'*

`df.loc[0:3, ['col1', 'col2']]` *# Rows 0-3, specific columns*

`df.loc[df['col1'] > 5]` *# Boolean indexing*

iloc - position-based indexing

`df.iloc[0]` *# First row*

`df.iloc[0:3]` *# Rows 0-2*

`df.iloc[:, 0]` *# All rows, first column*

`df.iloc[0:3, 0:2]` *# Rows 0-2, columns 0-1*

`df.iloc[[0, 2, 4]]` *# Specific rows by position*

at and iat - fast scalar access

`df.at[0, 'col1']` *# Value at row 0, column 'col1'*

`df.iat[0, 0]` *# Value at row 0, column 0 (by position)*

Boolean indexing

`df[df['col1'] > 5]`

`df[(df['col1'] > 5) & (df['col2'] < 10)]`

`df[df['col1'].isin([1, 2, 3])]`

`df[df['col1'].between(5, 10)]`

`df[~df['col1'].isin([1, 2, 3])]` *# NOT in list*

Data Cleaning

python

Handling missing data

```
df.isnull()          # Boolean mask of null values
df.notnull()         # Boolean mask of non-null values
df.isnull().sum()    # Count nulls per column
df.dropna()          # Drop rows with any null
df.dropna(axis=1)     # Drop columns with any null
df.dropna(how='all')  # Drop rows where all values are null
df.dropna(thresh=2)   # Keep rows with at least 2 non-null values
df.fillna(0)         # Fill nulls with 0
df.fillna(method='ffill') # Forward fill
df.fillna(method='bfill') # Backward fill
df.fillna(df.mean())  # Fill with column mean
df.interpolate()     # Interpolate missing values
```

Removing duplicates

```
df.duplicated()      # Boolean mask of duplicate rows
df.drop_duplicates()  # Remove duplicate rows
df.drop_duplicates(subset=['col1']) # Based on specific columns
df.drop_duplicates(keep='last') # Keep last occurrence
```

Replacing values

```
df.replace(0, np.nan) # Replace 0 with NaN
df.replace({'col1': {0: 100}}) # Replace in specific column
df.replace([0, 1], [100, 200]) # Multiple replacements
df['col1'].replace({0: 100, 1: 200}) # Dictionary mapping
```

Renaming

```
df.rename(columns={'old': 'new'})
df.rename(index={0: 'first'})
df.columns = ['new1', 'new2', 'new3']
df.rename(str.lower, axis='columns') # Lowercase all columns
```

Data types

```
df['col1'].astype(int)
df['col1'].astype('category')
pd.to_numeric(df['col1'], errors='coerce') # Convert to numeric, invalid -> NaN
pd.to_datetime(df['date_col'])
```

Data Manipulation

python

Adding/removing columns

```
df['new_col'] = df['col1'] + df['col2']  
df['new_col'] = df.apply(lambda row: row['col1'] * 2, axis=1)  
df.insert(1, 'new_col', [1, 2, 3]) # Insert at position 1  
df.drop('col1', axis=1) # Drop column  
df.drop(['col1', 'col2'], axis=1) # Drop multiple columns  
df.pop('col1') # Remove and return column
```

Adding/removing rows

```
df.loc[len(df)] = [1, 2, 3] # Add row at end  
df.drop(0, axis=0) # Drop row by label  
df.drop([0, 1, 2]) # Drop multiple rows
```

Sorting

```
df.sort_values('col1') # Sort by column  
df.sort_values(['col1', 'col2'], ascending=[True, False])  
df.sort_index() # Sort by index  
df.nlargest(5, 'col1') # Top 5 values  
df.nsmallest(5, 'col1') # Bottom 5 values
```

Filtering

```
df.query('col1 > 5')  
df.query('col1 > 5 and col2 < 10')  
df.query('col1 in [1, 2, 3]')
```

Apply functions

```
df['col1'].apply(lambda x: x * 2)  
df.apply(lambda x: x.max() - x.min()) # Apply to each column  
df.applymap(lambda x: x * 2) # Apply to every element (deprecated, use map)  
df.map(lambda x: x * 2) # Apply to every element
```

String operations

```
df['col1'].str.lower()  
df['col1'].str.upper()  
df['col1'].str.strip()  
df['col1'].str.replace('old', 'new')  
df['col1'].str.contains('pattern')  
df['col1'].str.split(',')  
df['col1'].str.extract(r'(\d+)') # Extract with regex  
df['col1'].str.len()  
df['col1'].str.startswith('A')  
df['col1'].str.endswith('Z')
```

GroupBy Operations

Basic grouping

```
grouped = df.groupby('col1')
grouped.size()      # Count of each group
grouped.groups      # Dictionary of groups
grouped.get_group('A') # Get specific group
```

Aggregations

```
df.groupby('col1').sum()
df.groupby('col1').mean()
df.groupby('col1').agg(['sum', 'mean', 'count'])
df.groupby('col1')['col2'].sum() # Aggregate specific column
df.groupby(['col1', 'col2']).sum() # Multiple groupby columns
```

Custom aggregations

```
df.groupby('col1').agg({
    'col2': 'sum',
    'col3': 'mean',
    'col4': ['min', 'max']
})
```

Transform and filter

```
df.groupby('col1')['col2'].transform('sum') # Broadcast result back
df.groupby('col1').filter(lambda x: len(x) > 3) # Keep groups with >3 rows
```

Iterating groups

```
for name, group in df.groupby('col1'):
    print(name, group)
```

Merging and Joining

python

Concatenation

```
pd.concat([df1, df2])      # Vertical stack
pd.concat([df1, df2], axis=1) # Horizontal stack
pd.concat([df1, df2], ignore_index=True) # Reset index
```

Merging (SQL-style joins)

```
pd.merge(df1, df2, on='key')      # Inner join
pd.merge(df1, df2, on='key', how='left') # Left join
pd.merge(df1, df2, on='key', how='right') # Right join
pd.merge(df1, df2, on='key', how='outer') # Outer join
pd.merge(df1, df2, left_on='key1', right_on='key2') # Different key names
pd.merge(df1, df2, left_index=True, right_index=True) # Join on index
```

Join (index-based merge)

```
df1.join(df2)
df1.join(df2, how='left', lsuffix='_left', rsuffix='_right')
```

Append (deprecated, use concat)

```
pd.concat([df1, df2], ignore_index=True)
```

Pivot Tables and Reshaping

python

Pivot table

```
df.pivot(index='row_col', columns='col_col', values='value_col')
df.pivot_table(values='value_col', index='row_col',
               columns='col_col', aggfunc='mean')
df.pivot_table(values='value_col', index='row_col',
               columns='col_col', aggfunc=['sum', 'mean'])
```

Melt (unpivot)

```
pd.melt(df, id_vars=['id'], value_vars=['col1', 'col2'])
pd.melt(df, id_vars=['id'], var_name='variable', value_name='value')
```

Stack and unstack

```
df.stack()      # Pivot columns to rows
df.unstack()    # Pivot rows to columns
df.unstack(level=0) # Unstack specific level
```

Crosstab

```
pd.crosstab(df['col1'], df['col2'])
pd.crosstab(df['col1'], df['col2'], normalize='all') # Show percentages
```

Time Series

```
python

# Creating datetime index
df['date'] = pd.to_datetime(df['date'])
df.set_index('date', inplace=True)
date_range = pd.date_range('2020-01-01', '2020-12-31', freq='D')

# Accessing datetime components
df['date'].dt.year
df['date'].dt.month
df['date'].dt.day
df['date'].dt.dayofweek
df['date'].dt.hour
df['date'].dt.date
df['date'].dt.time

# Resampling
df.resample('M').sum() # Monthly sum
df.resample('W').mean() # Weekly mean
df.resample('Q').last() # Quarterly last value
df.resample('2H').mean() # Every 2 hours

# Rolling windows
df.rolling(window=7).mean() # 7-day moving average
df.rolling(window=7).sum()
df.rolling(window=7, min_periods=1).mean() # Handle edge cases
df.expanding().mean() # Expanding window

# Shifting
df.shift(1) # Shift down by 1
df.shift(-1) # Shift up by 1
df.diff() # Difference from previous row
df.pct_change() # Percentage change

# Time zone operations
df.tz_localize('UTC')
df.tz_convert('US/Eastern')
```

Advanced Operations

```
python
```

```

# Multi-indexing
df.set_index(['col1', 'col2'])
df.reset_index()
df.xs('key', level='level_name') # Cross-section

# Categorical data
df['col'] = df['col'].astype('category')
df['col'].cat.categories
df['col'].cat.codes
df['col'].cat.add_categories(['new'])
df['col'].cat.remove_categories(['old'])

# Window functions
df['rank'] = df['col1'].rank()
df['cumsum'] = df['col1'].cumsum()
df['cummax'] = df['col1'].cummax()
df['cummin'] = df['col1'].cummin()

# Binning
pd.cut(df['col1'], bins=5) # Equal-width bins
pd.qcut(df['col1'], q=4) # Equal-frequency bins (quartiles)
pd.cut(df['col1'], bins=[0, 10, 20, 30], labels=['low', 'med', 'high'])

# One-hot encoding
pd.get_dummies(df['col1'])
pd.get_dummies(df, columns=['col1', 'col2'])

# Correlation and covariance
df.corr() # Correlation matrix
df.cov() # Covariance matrix
df['col1'].corr(df['col2']) # Correlation between two columns

# Value counts
df['col1'].value_counts()
df['col1'].value_counts(normalize=True) # Percentages
df['col1'].value_counts(bins=5)

# Sample data
df.sample(n=10) # 10 random rows
df.sample(frac=0.1) # 10% random sample
df.sample(n=10, replace=True) # With replacement

```

Performance Optimization

python

```
# Vectorized operations (avoid loops)
df['result'] = df['col1'] * df['col2'] # Fast
# Avoid: df['result'] = df.apply(lambda x: x['col1'] * x['col2'], axis=1)

# Use categorical for repeated strings
df['col'] = df['col'].astype('category')

# Read chunks for large files
for chunk in pd.read_csv('large_file.csv', chunksize=10000):
    process(chunk)

# Use efficient data types
df['int_col'] = df['int_col'].astype('int32') # Instead of int64
df['float_col'] = df['float_col'].astype('float32')

# Query instead of boolean indexing for large DataFrames
df.query('col1 > 5') # Faster than df[df['col1'] > 5]

# Use eval for complex expressions
df.eval('result = col1 + col2 * col3')

# Avoid chained indexing
df.loc[df['col1'] > 5, 'col2'] = 0 # Good
# Avoid: df[df['col1'] > 5]['col2'] = 0 # SettingWithCopyWarning
```

Common Real-World Patterns

Data Cleaning Pipeline

```
python
```

```
# Complete cleaning workflow
```

```
df = pd.read_csv('data.csv')
```

```
# 1. Handle missing values
```

```
df = df.dropna(thresh=len(df.columns)*0.5) # Drop rows with >50% missing
```

```
df = df.fillna(df.mean(numeric_only=True)) # Fill numeric with mean
```

```
# 2. Remove duplicates
```

```
df = df.drop_duplicates()
```

```
# 3. Fix data types
```

```
df['date'] = pd.to_datetime(df['date'])
```

```
df['category'] = df['category'].astype('category')
```

```
# 4. Handle outliers
```

```
Q1 = df['numeric_col'].quantile(0.25)
```

```
Q3 = df['numeric_col'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
df = df[~((df['numeric_col'] < (Q1 - 1.5 * IQR)) |  
          (df['numeric_col'] > (Q3 + 1.5 * IQR)))]
```

```
# 5. Standardize text
```

```
df['text_col'] = df['text_col'].str.lower().str.strip()
```

Data Aggregation and Reporting

```
python
```

```
# Create summary report
```

```
summary = df.groupby('category').agg({  
    'sales': ['sum', 'mean', 'count'],  
    'profit': 'sum',  
    'customer_id': 'nunique'  
}).round(2)
```

```
# Rename columns
```

```
summary.columns = ['total_sales', 'avg_sales', 'num_transactions',  
                   'total_profit', 'unique_customers']
```

```
# Add percentage columns
```

```
summary['pct_of_total'] = summary['total_sales'] / summary['total_sales'].sum() * 100
```

Time Series Analysis

```
python
```

```
# Prepare time series data
df['date'] = pd.to_datetime(df['date'])
df = df.set_index('date').sort_index()

# Calculate metrics
df['7day_ma'] = df['value'].rolling(7).mean()
df['pct_change'] = df['value'].pct_change()
df['ytd'] = df['value'].cumsum()

# Resample to monthly
monthly = df.resample('M').agg({
    'value': 'sum',
    'transactions': 'count',
    'customer_id': 'nunique'
})
```

Data Transformation for ML

```
python

# Prepare data for machine learning
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Handle categorical variables
df_encoded = pd.get_dummies(df, columns=['category', 'region'])

# Separate features and target
X = df_encoded.drop('target', axis=1)
y = df_encoded['target']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Quick Reference

NumPy vs Pandas

Operation	NumPy	Pandas
Create	<code>np.array([1,2,3])</code>	<code>pd.Series([1,2,3])</code>
Indexing	<code>arr[0]</code>	<code>df.iloc[0]</code> or <code>df.loc[0]</code>
Slicing	<code>arr[1:3]</code>	<code>df[1:3]</code> or <code>df.iloc[1:3]</code>
Boolean	<code>arr[arr > 5]</code>	<code>df[df['col'] > 5]</code>
Aggregation	<code>arr.sum()</code>	<code>df.sum()</code> or <code>df['col'].sum()</code>

Common Gotchas

```
python

# 1. Chained indexing - causes SettingWithCopyWarning
df[df['A'] > 0]['B'] = 1 # Bad
df.loc[df['A'] > 0, 'B'] = 1 # Good

# 2. Modifying during iteration
for idx, row in df.iterrows():
    df.at[idx, 'new'] = row['old'] * 2 # Slow but works
# Better: df['new'] = df['old'] * 2

# 3. Comparing arrays
(arr1 == arr2).all() # Check if all elements equal
np.array_equal(arr1, arr2) # Better way

# 4. Integer division
np.array([1, 2, 3]) / np.array([2, 2, 2]) # Returns float
np.array([1, 2, 3]) // np.array([2, 2, 2]) # Integer division
```

This cheat sheet covers the essential operations you'll use in real-world data analysis and manipulation tasks.

PyTorch & TensorFlow Complete Cheat Sheet

Table of Contents

1. [Installation & Setup](#)
 2. [Tensor Operations](#)
 3. [Neural Network Layers](#)
 4. [Model Building](#)
 5. [Training Loop](#)
 6. [Loss Functions & Optimizers](#)
 7. [Data Loading](#)
 8. [Model Saving & Loading](#)
 9. [Common Patterns](#)
-

Installation & Setup

PyTorch

```
bash

pip install torch torchvision torchaudio
```

```
python

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import torchvision
import torchvision.transforms as transforms

# Check CUDA availability
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

TensorFlow

```
bash

pip install tensorflow
```



```
python
```

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
import numpy as np

# Check GPU availability
print("GPUs Available: ", len(tf.config.list_physical_devices('GPU')))
```

Tensor Operations

Creating Tensors

PyTorch:

```
python

# From data
x = torch.tensor([1, 2, 3])
x = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)

# Zeros, ones, random
zeros = torch.zeros(3, 4)
ones = torch.ones(2, 3)
rand = torch.rand(3, 3) # Uniform [0, 1)
randn = torch.randn(3, 3) # Normal distribution
empty = torch.empty(2, 2)

# From numpy
import numpy as np
np_array = np.array([1, 2, 3])
tensor = torch.from_numpy(np_array)

# Like another tensor
x_like = torch.zeros_like(x)
x_like = torch.ones_like(x)
```

TensorFlow:

```
python
```

```
# From data
```

```
x = tf.constant([1, 2, 3])
```

```
x = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
```

```
# Zeros, ones, random
```

```
zeros = tf.zeros([3, 4])
```

```
ones = tf.ones([2, 3])
```

```
rand = tf.random.uniform([3, 3]) # Uniform [0, 1)
```

```
randn = tf.random.normal([3, 3]) # Normal distribution
```

```
# From numpy
```

```
np_array = np.array([1, 2, 3])
```

```
tensor = tf.constant(np_array)
```

```
# Like another tensor
```

```
x_like = tf.zeros_like(x)
```

```
x_like = tf.ones_like(x)
```

Tensor Attributes & Operations

PyTorch:

```
python
```

```
x = torch.randn(3, 4)
```

```
# Attributes
```

```
print(x.shape) # torch.Size([3, 4])
```

```
print(x.dtype) # torch.float32
```

```
print(x.device) # cpu or cuda
```

```
# Move to device
```

```
x = x.to(device)
```

```
x = x.cuda() # Move to GPU
```

```
x = x.cpu() # Move to CPU
```

```
# Reshape
```

```
x = x.view(12) # Must be contiguous
```

```
x = x.reshape(2, 6) # More flexible
```

```
x = x.flatten()
```

```
# Transpose
```

```
x = x.t() # 2D transpose
```

```
x = x.transpose(0, 1) # Swap dimensions
```

```
x = x.permute(1, 0, 2) # Rearrange dimensions
```

```
# Type conversion
```

```
x = x.float()
```

```
x = x.long()
```

```
x = x.int()
```

TensorFlow:

```
python
```

```
x = tf.random.normal([3, 4])
```

```
# Attributes
```

```
print(x.shape) # (3, 4)
```

```
print(x.dtype) # float32
```

```
# Reshape
```

```
x = tf.reshape(x, [12])
```

```
x = tf.reshape(x, [2, 6])
```

```
x = tf.reshape(x, [-1]) # Flatten
```

```
# Transpose
```

```
x = tf.transpose(x)
```

```
x = tf.transpose(x, perm=[1, 0, 2])
```

```
# Type conversion
```

```
x = tf.cast(x, tf.float32)
```

```
x = tf.cast(x, tf.int32)
```

Mathematical Operations

PyTorch:

```
python
```

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
```

Element-wise operations

```
c = a + b
c = a - b
c = a * b
c = a / b
c = a ** 2
c = torch.sqrt(a)
c = torch.exp(a)
c = torch.log(a)
```

Matrix operations

```
A = torch.randn(3, 4)
B = torch.randn(4, 5)
C = torch.mm(A, B) # Matrix multiplication
C = A @ B # Same as above
```

Batch matrix multiplication

```
A = torch.randn(10, 3, 4)
B = torch.randn(10, 4, 5)
C = torch.bmm(A, B) # (10, 3, 5)
```

Reduction operations

```
mean = a.mean()
sum_val = a.sum()
max_val = a.max()
min_val = a.min()
argmax = a.argmax()
```

TensorFlow:

```
python
```

```
a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])

# Element-wise operations
c = a + b
c = a - b
c = a * b
c = a / b
c = tf.pow(a, 2)
c = tf.sqrt(tf.cast(a, tf.float32))
c = tf.exp(tf.cast(a, tf.float32))
c = tf.math.log(tf.cast(a, tf.float32))
```

```
# Matrix operations
A = tf.random.normal([3, 4])
B = tf.random.normal([4, 5])
C = tf.matmul(A, B)
C = A @ B # Same as above
```

```
# Reduction operations
mean = tf.reduce_mean(a)
sum_val = tf.reduce_sum(a)
max_val = tf.reduce_max(a)
min_val = tf.reduce_min(a)
argmax = tf.argmax(a)
```

Indexing & Slicing

PyTorch:

```
python

x = torch.randn(4, 5)

# Basic indexing
first_row = x[0]
first_col = x[:, 0]
subset = x[1:3, 2:4]

# Boolean indexing
mask = x > 0
positive = x[mask]

# Advanced indexing
indices = torch.tensor([0, 2])
selected = x[indices]
```

TensorFlow:

```
python
```

```
x = tf.random.normal([4, 5])
```

```
# Basic indexing
```

```
first_row = x[0]
```

```
first_col = x[:, 0]
```

```
subset = x[1:3, 2:4]
```

```
# Boolean indexing
```

```
mask = x > 0
```

```
positive = tf.boolean_mask(x, mask)
```

```
# Advanced indexing
```

```
indices = [0, 2]
```

```
selected = tf.gather(x, indices)
```

Neural Network Layers

Common Layers

PyTorch:

```
python
```

```
import torch.nn as nn
```

```
# Linear (Dense) layer
```

```
linear = nn.Linear(in_features=10, out_features=5)
```

```
# Convolutional layers
```

```
conv1d = nn.Conv1d(in_channels=3, out_channels=16, kernel_size=3)
```

```
conv2d = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride=1, padding=1)
```

```
conv3d = nn.Conv3d(in_channels=3, out_channels=16, kernel_size=3)
```

```
# Pooling layers
```

```
maxpool = nn.MaxPool2d(kernel_size=2, stride=2)
```

```
avgpool = nn.AvgPool2d(kernel_size=2)
```

```
adaptivepool = nn.AdaptiveAvgPool2d((1, 1))
```

```
# Normalization layers
```

```
batchnorm = nn.BatchNorm2d(num_features=16)
```

```
layernorm = nn.LayerNorm(normalized_shape=10)
```

```
# Dropout
```

```
dropout = nn.Dropout(p=0.5)
```

```
dropout2d = nn.Dropout2d(p=0.5)
```

```
# Activation functions
```

```
relu = nn.ReLU()
```

```
sigmoid = nn.Sigmoid()
```

```
tanh = nn.Tanh()
```

```
leaky_relu = nn.LeakyReLU(0.01)
```

```
softmax = nn.Softmax(dim=1)
```

```
# Recurrent layers
```

```
rnn = nn.RNN(input_size=10, hidden_size=20, num_layers=2, batch_first=True)
```

```
lstm = nn.LSTM(input_size=10, hidden_size=20, num_layers=2, batch_first=True)
```

```
gru = nn.GRU(input_size=10, hidden_size=20, num_layers=2, batch_first=True)
```

```
# Embedding
```

```
embedding = nn.Embedding(num_embeddings=1000, embedding_dim=128)
```

TensorFlow/Keras:

```
python
```



```
from tensorflow.keras import layers

# Dense layer
dense = layers.Dense(units=5, activation='relu')

# Convolutional layers
conv1d = layers.Conv1D(filters=16, kernel_size=3, activation='relu')
conv2d = layers.Conv2D(filters=16, kernel_size=3, strides=1, padding='same', activation='relu')
conv3d = layers.Conv3D(filters=16, kernel_size=3, activation='relu')

# Pooling layers
maxpool = layers.MaxPooling2D(pool_size=2, strides=2)
avgpool = layers.AveragePooling2D(pool_size=2)
globalavgpool = layers.GlobalAveragePooling2D()

# Normalization layers
batchnorm = layers.BatchNormalization()
layernorm = layers.LayerNormalization()

# Dropout
dropout = layers.Dropout(rate=0.5)

# Activation functions (as layers)
relu = layers.ReLU()
# Or use activation parameter in layers
dense_with_activation = layers.Dense(10, activation='relu')

# Recurrent layers
simple_rnn = layers.SimpleRNN(units=20, return_sequences=True)
lstm = layers.LSTM(units=20, return_sequences=True)
gru = layers.GRU(units=20, return_sequences=True)

# Embedding
embedding = layers.Embedding(input_dim=1000, output_dim=128)

# Flatten
flatten = layers.Flatten()
```

Model Building

Sequential Models

PyTorch:

```
python
```

```
# Method 1: Using nn.Sequential
```

```
model = nn.Sequential(  
    nn.Linear(784, 256),  
    nn.ReLU(),  
    nn.Dropout(0.2),  
    nn.Linear(256, 128),  
    nn.ReLU(),  
    nn.Dropout(0.2),  
    nn.Linear(128, 10)  
)
```

```
# Method 2: Using OrderedDict
```

```
from collections import OrderedDict  
model = nn.Sequential(OrderedDict([  
    ('fc1', nn.Linear(784, 256)),  
    ('relu1', nn.ReLU()),  
    ('dropout1', nn.Dropout(0.2)),  
    ('fc2', nn.Linear(256, 10))  
]))
```

TensorFlow/Keras:

```
python
```

```
# Method 1: Sequential API
```

```
model = keras.Sequential([  
    layers.Dense(256, activation='relu', input_shape=(784,)),  
    layers.Dropout(0.2),  
    layers.Dense(128, activation='relu'),  
    layers.Dropout(0.2),  
    layers.Dense(10, activation='softmax')  
])
```

```
# Method 2: Adding layers one by one
```

```
model = keras.Sequential()  
model.add(layers.Dense(256, activation='relu', input_shape=(784,)))  
model.add(layers.Dropout(0.2))  
model.add(layers.Dense(10, activation='softmax'))
```

Custom Models

PyTorch:

```
python
```

```

class CustomModel(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(CustomModel, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.2)
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x

model = CustomModel(784, 256, 10).to(device)

```

TensorFlow/Keras:

```

python

class CustomModel(keras.Model):
    def __init__(self, hidden_size, num_classes):
        super(CustomModel, self).__init__()
        self.fc1 = layers.Dense(hidden_size, activation='relu')
        self.dropout = layers.Dropout(0.2)
        self.fc2 = layers.Dense(num_classes, activation='softmax')

    def call(self, inputs, training=False):
        x = self.fc1(inputs)
        x = self.dropout(x, training=training)
        x = self.fc2(x)
        return x

model = CustomModel(256, 10)

```

CNN Example

PyTorch:

```

python

```

```

class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(64 * 8 * 8, 128)
        self.fc2 = nn.Linear(128, 10)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = self.pool(torch.relu(self.conv2(x)))
        x = x.view(-1, 64 * 8 * 8) # Flatten
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

model = CNN().to(device)

```

TensorFlow/Keras:

```

python

model = keras.Sequential([
    layers.Conv2D(32, 3, padding='same', activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D(2),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(2),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

```

RNN/LSTM Example

PyTorch:

```

python

```

```

class RNNModel(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, num_classes):
        super(RNNModel, self).__init__()
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(x.device)
        c0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(x.device)

        out, _ = self.lstm(x, (h0, c0))
        out = self.fc(out[:, -1, :]) # Last time step
        return out

model = RNNModel(input_size=28, hidden_size=128, num_layers=2, num_classes=10).to(device)

```

TensorFlow/Keras:

```

python

model = keras.Sequential([
    layers.LSTM(128, return_sequences=True, input_shape=(None, 28)),
    layers.LSTM(128),
    layers.Dense(10, activation='softmax')
])

```

Training Loop

Basic Training Loop

PyTorch:

```

python

```

Setup

```
model = CustomModel(784, 256, 10).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Training loop

```
num_epochs = 10
for epoch in range(num_epochs):
    model.train() # Set to training mode
    running_loss = 0.0

    for i, (inputs, labels) in enumerate(train_loader):
        inputs = inputs.to(device)
        labels = labels.to(device)

        # Zero the gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(inputs)
        loss = criterion(outputs, labels)

        # Backward pass
        loss.backward()

        # Update weights
        optimizer.step()

    running_loss += loss.item()

epoch_loss = running_loss / len(train_loader)
print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {epoch_loss:.4f}')
```

Evaluation

```
model.eval() # Set to evaluation mode
with torch.no_grad():
    correct = 0
    total = 0
    for inputs, labels in test_loader:
        inputs = inputs.to(device)
        labels = labels.to(device)
        outputs = model(inputs)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
```

```
accuracy = 100 * correct / total
print(f'Test Accuracy: {accuracy:.2f}%')
```

TensorFlow/Keras:

```
python

# Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Training
history = model.fit(
    train_dataset,
    epochs=10,
    validation_data=test_dataset,
    verbose=1
)

# Evaluation
test_loss, test_acc = model.evaluate(test_dataset)
print(f'Test Accuracy: {test_acc:.4f}')
```

Custom training loop (advanced)

```
optimizer = keras.optimizers.Adam()
loss_fn = keras.losses.SparseCategoricalCrossentropy()
```

@tf.function

```
def train_step(x, y):
    with tf.GradientTape() as tape:
        predictions = model(x, training=True)
        loss = loss_fn(y, predictions)
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))
    return loss
```

for epoch in range(10):

```
    for x_batch, y_batch in train_dataset:
        loss = train_step(x_batch, y_batch)
    print(f'Epoch {epoch+1}, Loss: {loss.numpy():.4f}')
```

Loss Functions & Optimizers

Loss Functions

PyTorch:

```
python

import torch.nn as nn

# Classification
criterion = nn.CrossEntropyLoss() # For multi-class
criterion = nn.BCELoss() # Binary cross-entropy
criterion = nn.BCEWithLogitsLoss() # BCE with logits (more stable)
criterion = nn.NLLLoss() # Negative log likelihood

# Regression
criterion = nn.MSELoss() # Mean squared error
criterion = nn.L1Loss() # Mean absolute error
criterion = nn.SmoothL1Loss() # Huber loss

# Custom weights
weights = torch.tensor([1.0, 2.0, 3.0])
criterion = nn.CrossEntropyLoss(weight=weights)
```

TensorFlow/Keras:

```
python
```



```
from tensorflow.keras import losses
```

```
# Classification
```

```
loss = losses.SparseCategoricalCrossentropy()
```

```
loss = losses.CategoricalCrossentropy()
```

```
loss = losses.BinaryCrossentropy()
```

```
# Regression
```

```
loss = losses.MeanSquaredError()
```

```
loss = losses.MeanAbsoluteError()
```

```
loss = losses.Huber()
```

```
# In model.compile
```

```
model.compile(
```

```
    loss='sparse_categorical_crossentropy',
```

```
    # or
```

```
    loss=losses.SparseCategoricalCrossentropy()
```

```
)
```

Optimizers

PyTorch:

```
python
```

```
import torch.optim as optim
```

```
# Basic optimizers
```

```
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001, betas=(0.9, 0.999))
```

```
optimizer = optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)
```

```
optimizer = optim.RMSprop(model.parameters(), lr=0.01, alpha=0.99)
```

```
# Different learning rates for different layers
```

```
optimizer = optim.Adam([  
    {'params': model.conv_layers.parameters(), 'lr': 1e-4},  
    {'params': model.fc_layers.parameters(), 'lr': 1e-3}  
])
```

```
# Learning rate scheduler
```

```
from torch.optim.lr_scheduler import StepLR, ReduceLROnPlateau
```

```
scheduler = StepLR(optimizer, step_size=30, gamma=0.1)
```

```
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=10)
```

```
# Usage in training loop
```

```
for epoch in range(num_epochs):
```

```
    # Training code...
```

```
    scheduler.step() # Update learning rate
```

TensorFlow/Keras:

```
python
```

```
from tensorflow.keras import optimizers

# Basic optimizers
optimizer = optimizers.SGD(learning_rate=0.01, momentum=0.9)
optimizer = optimizers.Adam(learning_rate=0.001)
optimizer = optimizers.RMSprop(learning_rate=0.01)

# Learning rate schedules
lr_schedule = keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=1e-3,
    decay_steps=10000,
    decay_rate=0.9
)
optimizer = optimizers.Adam(learning_rate=lr_schedule)

# Callbacks for learning rate
from tensorflow.keras.callbacks import ReduceLROnPlateau

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.1,
    patience=10,
    min_lr=1e-7
)

model.fit(train_dataset, epochs=10, callbacks=[reduce_lr])
```

Data Loading

Custom Dataset

PyTorch:

```
python
```

```
from torch.utils.data import Dataset, DataLoader
```

```
class CustomDataset(Dataset):
```

```
    def __init__(self, data, labels, transform=None):
```

```
        self.data = data
```

```
        self.labels = labels
```

```
        self.transform = transform
```

```
    def __len__(self):
```

```
        return len(self.data)
```

```
    def __getitem__(self, idx):
```

```
        sample = self.data[idx]
```

```
        label = self.labels[idx]
```

```
        if self.transform:
```

```
            sample = self.transform(sample)
```

```
        return sample, label
```

```
# Create dataset and dataloader
```

```
dataset = CustomDataset(data, labels)
```

```
dataloader = DataLoader(
```

```
    dataset,
```

```
    batch_size=32,
```

```
    shuffle=True,
```

```
    num_workers=4,
```

```
    pin_memory=True # For GPU
```

```
)
```

```
# Usage
```

```
for batch_data, batch_labels in dataloader:
```

```
    # Training code
```

```
    pass
```

TensorFlow:

```
python
```

```

# From numpy arrays
train_dataset = tf.data.Dataset.from_tensor_slices((train_data, train_labels))
train_dataset = train_dataset.shuffle(10000).batch(32).prefetch(tf.data.AUTOTUNE)

# From generator
def data_generator():
    for i in range(len(data)):
        yield data[i], labels[i]

dataset = tf.data.Dataset.from_generator(
    data_generator,
    output_signature=(
        tf.TensorSpec(shape=(28, 28), dtype=tf.float32),
        tf.TensorSpec(shape=(), dtype=tf.int32)
    )
)
dataset = dataset.batch(32).prefetch(tf.data.AUTOTUNE)

```

Image Data Augmentation

PyTorch:

```

python

from torchvision import transforms

# Define transforms
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

# Apply to dataset
from torchvision.datasets import ImageFolder

dataset = ImageFolder(root='path/to/data', transform=transform)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)

```

TensorFlow:

```
python
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Data augmentation
datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True,
    zoom_range=0.2
)

# From directory
train_generator = datagen.flow_from_directory(
    'path/to/data',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
)

# Using tf.data
def augment(image, label):
    image = tf.image.resize(image, [224, 224])
    image = tf.image.random_flip_left_right(image)
    image = tf.image.random_brightness(image, 0.2)
    return image, label

dataset = dataset.map(augment, num_parallel_calls=tf.data.AUTOTUNE)
```

Model Saving & Loading

PyTorch:

```
python
```

Save entire model

```
torch.save(model, 'model.pth')
```

Load entire model

```
model = torch.load('model.pth')
```

Save only state dict (recommended)

```
torch.save(model.state_dict(), 'model_weights.pth')
```

Load state dict

```
model = CustomModel(784, 256, 10)
```

```
model.load_state_dict(torch.load('model_weights.pth'))
```

```
model.eval()
```

Save checkpoint (with optimizer state)

```
checkpoint = {
```

```
    'epoch': epoch,
```

```
    'model_state_dict': model.state_dict(),
```

```
    'optimizer_state_dict': optimizer.state_dict(),
```

```
    'loss': loss
```

```
}
```

```
torch.save(checkpoint, 'checkpoint.pth')
```

Load checkpoint

```
checkpoint = torch.load('checkpoint.pth')
```

```
model.load_state_dict(checkpoint['model_state_dict'])
```

```
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
```

```
epoch = checkpoint['epoch']
```

```
loss = checkpoint['loss']
```

TensorFlow/Keras:

python

```
# Save entire model (architecture + weights + optimizer)
model.save('my_model.h5')
model.save('my_model') # SavedModel format (recommended)

# Load model
model = keras.models.load_model('my_model.h5')

# Save only weights
model.save_weights('model_weights.h5')

# Load weights
model.load_weights('model_weights.h5')

# Save with checkpoints during training
checkpoint_callback = keras.callbacks.ModelCheckpoint(
    filepath='checkpoint.h5',
    save_weights_only=True,
    save_best_only=True,
    monitor='val_loss',
    mode='min'
)

model.fit(train_dataset, epochs=10, callbacks=[checkpoint_callback])
```

Common Patterns

Transfer Learning

PyTorch:

```
python
```



```

import torchvision.models as models

# Load pretrained model
model = models.resnet50(pretrained=True)

# Freeze all layers
for param in model.parameters():
    param.requires_grad = False

# Replace final layer
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, num_classes)

# Only train final layer
optimizer = optim.Adam(model.fc.parameters(), lr=0.001)

```

TensorFlow/Keras:

```

python

# Load pretrained model
base_model = keras.applications.ResNet50(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)

# Freeze base model
base_model.trainable = False

# Add custom layers
model = keras.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

```

Mixed Precision Training

PyTorch:

```

python

```

```
from torch.cuda.amp import autocast, GradScaler
```

```
scaler = GradScaler()
```

```
for inputs, labels in train_loader:
```

```
    optimizer.zero_grad()
```

```
    with autocast():
```

```
        outputs = model(inputs)
```

```
        loss = criterion(outputs, labels)
```

```
    scaler.scale(loss).backward()
```

```
    scaler.step(optimizer)
```

```
    scaler.update()
```

TensorFlow:

```
python
```

```
from tensorflow.keras import mixed_precision
```

```
# Enable mixed precision
```

```
mixed_precision.set_global_policy('mixed_float16')
```

```
# Model automatically uses mixed precision
```

```
model = keras.Sequential(...)
```

```
model.compile(optimizer='adam', loss='categorical_crossentropy')
```

Gradient Clipping

PyTorch:

```
python
```

```
# Clip by norm
```

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

```
# Clip by value
```

```
torch.nn.utils.clip_grad_value_(model.parameters(), clip_value=0.5)
```

```
# In training loop
```

```
optimizer.zero_grad()
```

```
loss.backward()
```

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

```
optimizer.step()
```

TensorFlow:

```
python
```

```
# In optimizer
```

```
optimizer = keras.optimizers.Adam(clipnorm=1.0) # Clip by norm
```

```
optimizer = keras.optimizers.Adam(clip
```

Complete Scikit-Learn Cheat Sheet

Table of Contents

1. Installation & Imports
 2. Data Loading & Preprocessing
 3. Train-Test Split
 4. Feature Scaling
 5. Encoding Categorical Variables
 6. Feature Engineering
 7. Supervised Learning - Classification
 8. Supervised Learning - Regression
 9. Unsupervised Learning
 10. Model Evaluation Metrics
 11. Cross-Validation
 12. Hyperparameter Tuning
 13. Pipelines
 14. Model Persistence
-

1. Installation & Imports

```
python

# Installation
pip install scikit-learn

# Core imports
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder, OneHotEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

2. Data Loading & Preprocessing

Loading Built-in Datasets

```
python
```

```
# Load sample datasets
```

```
from sklearn.datasets import load_iris, load_boston, load_digits, make_classification
```

```
# Classification dataset
```

```
iris = load_iris()
```

```
X, y = iris.data, iris.target
```

```
# Regression dataset
```

```
california = fetch_california_housing()
```

```
X, y = california.data, california.target
```

```
# Generate synthetic data
```

```
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)
```

Handling Missing Values

```
python
```

```
from sklearn.impute import SimpleImputer
```

```
# Impute with mean
```

```
imputer = SimpleImputer(strategy='mean') # Options: 'mean', 'median', 'most_frequent', 'constant'
```

```
X_imputed = imputer.fit_transform(X)
```

```
# Impute with specific value
```

```
imputer = SimpleImputer(strategy='constant', fill_value=0)
```

```
X_imputed = imputer.fit_transform(X)
```

3. Train-Test Split

```
python
```

```
from sklearn.model_selection import train_test_split

# Basic split (80-20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Stratified split (maintains class distribution)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

# Train-Validation-Test split
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)
```

4. Feature Scaling

```
python

from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

# StandardScaler (mean=0, std=1)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # Use same scaler, don't fit again!

# MinMaxScaler (scale to range [0,1])
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X_train)

# MinMaxScaler with custom range
scaler = MinMaxScaler(feature_range=(0, 10))
X_scaled = scaler.fit_transform(X_train)

# RobustScaler (robust to outliers)
scaler = RobustScaler()
X_scaled = scaler.fit_transform(X_train)
```

5. Encoding Categorical Variables

```
python
```

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
```

```
# LabelEncoder (for target variable or ordinal features)
```

```
le = LabelEncoder()
```

```
y_encoded = le.fit_transform(y) # ['cat', 'dog', 'cat'] -> [0, 1, 0]
```

```
y_original = le.inverse_transform(y_encoded) # Decode back
```

```
# OneHotEncoder (for nominal features)
```

```
ohe = OneHotEncoder(sparse_output=False)
```

```
X_encoded = ohe.fit_transform(X_categorical)
```

```
# Get feature names
```

```
feature_names = ohe.get_feature_names_out(['color'])
```

```
# Using pandas get_dummies (alternative)
```

```
df_encoded = pd.get_dummies(df, columns=['color', 'size'])
```

6. Feature Engineering

Feature Selection

```
python
```

```
from sklearn.feature_selection import SelectKBest, chi2, f_classif, RFE
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
# SelectKBest - Select top k features
```

```
selector = SelectKBest(f_classif, k=10)
```

```
X_new = selector.fit_transform(X, y)
```

```
selected_features = selector.get_support(indices=True)
```

```
# Recursive Feature Elimination (RFE)
```

```
estimator = RandomForestClassifier()
```

```
rfe = RFE(estimator, n_features_to_select=10)
```

```
X_new = rfe.fit_transform(X, y)
```

Polynomial Features

```
python
```

```
from sklearn.preprocessing import PolynomialFeatures

# Create polynomial features
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
```

7. Supervised Learning - Classification

Logistic Regression

```
python

from sklearn.linear_model import LogisticRegression

# Create and train
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
y_pred_proba = model.predict_proba(X_test) # Probability estimates

# Evaluate
accuracy = model.score(X_test, y_test)
```

Decision Tree Classifier

```
python

from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=5, min_samples_split=10, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Feature importance
importances = model.feature_importances_
```

Random Forest Classifier

```
python
```



```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Feature importance
importances = model.feature_importances_
```

Support Vector Machine (SVM)

```
python

from sklearn.svm import SVC

# Linear kernel
model = SVC(kernel='linear', C=1.0, random_state=42)

# RBF kernel
model = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

K-Nearest Neighbors (KNN)

```
python

from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5, weights='uniform') # weights: 'uniform', 'distance'
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Naive Bayes

```
python
```

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB
```

```
# Gaussian Naive Bayes
```

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

```
# Multinomial Naive Bayes (for count data)
```

```
model = MultinomialNB()
```

```
model.fit(X_train, y_train)
```

Gradient Boosting

```
python
```

```
from sklearn.ensemble import GradientBoostingClassifier
```

```
model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

8. Supervised Learning - Regression

Linear Regression

```
python
```

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
# Coefficients and intercept
```

```
coefficients = model.coef_
```

```
intercept = model.intercept_
```

Ridge Regression (L2 Regularization)

```
python
```

```
from sklearn.linear_model import Ridge

model = Ridge(alpha=1.0) # Higher alpha = more regularization
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Lasso Regression (L1 Regularization)

```
python

from sklearn.linear_model import Lasso

model = Lasso(alpha=0.1)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

ElasticNet (L1 + L2)

```
python

from sklearn.linear_model import ElasticNet

model = ElasticNet(alpha=0.1, l1_ratio=0.5) # l1_ratio: mix of L1 and L2
model.fit(X_train, y_train)
```

Decision Tree Regressor

```
python

from sklearn.tree import DecisionTreeRegressor

model = DecisionTreeRegressor(max_depth=5, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Random Forest Regressor

```
python

from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Support Vector Regression (SVR)

```
python

from sklearn.svm import SVR

model = SVR(kernel='rbf', C=1.0, gamma='scale')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Gradient Boosting Regressor

```
python

from sklearn.ensemble import GradientBoostingRegressor

model = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

9. Unsupervised Learning

K-Means Clustering

```
python

from sklearn.cluster import KMeans

model = KMeans(n_clusters=3, random_state=42)
clusters = model.fit_predict(X)

# Cluster centers
centers = model.cluster_centers_

# Inertia (sum of squared distances to nearest cluster center)
inertia = model.inertia_
```

DBSCAN

```
python
```

```
from sklearn.cluster import DBSCAN
```

```
model = DBSCAN(eps=0.5, min_samples=5)
```

```
clusters = model.fit_predict(X)
```

Hierarchical Clustering

```
python
```

```
from sklearn.cluster import AgglomerativeClustering
```

```
model = AgglomerativeClustering(n_clusters=3, linkage='ward') # linkage: 'ward', 'complete', 'average'
```

```
clusters = model.fit_predict(X)
```

Principal Component Analysis (PCA)

```
python
```

```
from sklearn.decomposition import PCA
```

```
# Reduce to 2 components
```

```
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X)
```

```
# Variance explained
```

```
explained_variance = pca.explained_variance_ratio_
```

```
# Keep components that explain 95% variance
```

```
pca = PCA(n_components=0.95)
```

```
X_pca = pca.fit_transform(X)
```

t-SNE

```
python
```

```
from sklearn.manifold import TSNE
```

```
tsne = TSNE(n_components=2, random_state=42)
```

```
X_tsne = tsne.fit_transform(X)
```

10. Model Evaluation Metrics

Classification Metrics

```
python

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score, roc_curve
)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

# Precision, Recall, F1-Score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Classification Report (all metrics)
report = classification_report(y_test, y_pred)
print(report)

# ROC-AUC Score (binary classification)
roc_auc = roc_auc_score(y_test, y_pred_proba[:, 1])

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba[:, 1])
```

Regression Metrics

```
python
```

```
from sklearn.metrics import (
    mean_squared_error, mean_absolute_error, r2_score,
    mean_absolute_percentage_error
)

# Mean Squared Error
mse = mean_squared_error(y_test, y_pred)

# Root Mean Squared Error
rmse = mean_squared_error(y_test, y_pred, squared=False)

# Mean Absolute Error
mae = mean_absolute_error(y_test, y_pred)

# R-squared Score
r2 = r2_score(y_test, y_pred)

# Mean Absolute Percentage Error
mape = mean_absolute_percentage_error(y_test, y_pred)
```

11. Cross-Validation

```
python
```

```
from sklearn.model_selection import cross_val_score, cross_validate, KFold, StratifiedKFold

# Simple cross-validation (default 5-fold)
scores = cross_val_score(model, X, y, cv=5)
print(f'Accuracy: {scores.mean():.3f} (+/- {scores.std():.3f})')

# Cross-validation with specific metric
scores = cross_val_score(model, X, y, cv=5, scoring='f1_weighted')

# Multiple metrics
scoring = ['accuracy', 'precision_weighted', 'recall_weighted']
scores = cross_validate(model, X, y, cv=5, scoring=scoring)

# K-Fold
kfold = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kfold)

# Stratified K-Fold (maintains class distribution)
skfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=skfold)

# Leave-One-Out Cross-Validation
from sklearn.model_selection import LeaveOneOut
loo = LeaveOneOut()
scores = cross_val_score(model, X, y, cv=loo)
```

12. Hyperparameter Tuning

Grid Search

```
python
```



```
from sklearn.model_selection import GridSearchCV

# Define parameter grid
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15, None],
    'min_samples_split': [2, 5, 10]
}

# Create GridSearchCV
grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1, # Use all processors
    verbose=1
)

# Fit
grid_search.fit(X_train, y_train)

# Best parameters and score
print(f'Best parameters: {grid_search.best_params_}')
print(f'Best score: {grid_search.best_score_:.3f}')

# Best model
best_model = grid_search.best_estimator_

# Predict with best model
y_pred = best_model.predict(X_test)
```

Randomized Search

```
python
```

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

# Define parameter distributions
param_distributions = {
    'n_estimators': randint(50, 500),
    'max_depth': randint(5, 50),
    'min_samples_split': randint(2, 20),
    'min_samples_leaf': randint(1, 10),
    'max_features': uniform(0.1, 0.9)
}

# Create RandomizedSearchCV
random_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_distributions,
    n_iter=100, # Number of parameter settings sampled
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    random_state=42,
    verbose=1
)

# Fit
random_search.fit(X_train, y_train)

# Best parameters
print(f"Best parameters: {random_search.best_params_}")
best_model = random_search.best_estimator_
```

13. Pipelines

Basic Pipeline

```
python
```

```
from sklearn.pipeline import Pipeline

# Create pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(random_state=42))
])

# Fit and predict
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
accuracy = pipeline.score(X_test, y_test)
```

Pipeline with Preprocessing

```
python
```

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Define column types
numeric_features = ['age', 'income']
categorical_features = ['gender', 'city']

# Create transformers
numeric_transformer = Pipeline([
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline([
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Combine transformers
preprocessor = ColumnTransformer([
    ('num', numeric_transformer, numeric_features),
    ('cat', categorical_transformer, categorical_features)
])

# Full pipeline
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression())
])

pipeline.fit(X_train, y_train)
```

Pipeline with GridSearch

```
python
```

```
# Create pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', SVC())
])

# Define parameters (use double underscore for pipeline steps)
param_grid = {
    'classifier__C': [0.1, 1, 10],
    'classifier__kernel': ['linear', 'rbf'],
    'classifier__gamma': ['scale', 'auto']
}

# GridSearch with pipeline
grid_search = GridSearchCV(pipeline, param_grid, cv=5)
grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_
```

14. Model Persistence

Save and Load Models

```
python

import pickle
import joblib

# Using joblib (recommended for scikit-learn)
joblib.dump(model, 'model.pkl')
loaded_model = joblib.load('model.pkl')

# Using pickle
with open('model.pkl', 'wb') as f:
    pickle.dump(model, f)

with open('model.pkl', 'rb') as f:
    loaded_model = pickle.load(f)

# Verify loaded model
y_pred = loaded_model.predict(X_test)
```

Common Workflow Example

python

1. Import libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
import joblib
```

2. Load data

```
df = pd.read_csv('data.csv')
X = df.drop('target', axis=1)
y = df['target']
```

3. Split data

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

4. Scale features

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

5. Create and train model

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train_scaled, y_train)
```

6. Make predictions

```
y_pred = model.predict(X_test_scaled)
```

7. Evaluate

```
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

8. Hyperparameter tuning

```
param_grid = {'n_estimators': [50, 100, 200], 'max_depth': [5, 10, None]}
grid_search = GridSearchCV(model, param_grid, cv=5)
grid_search.fit(X_train_scaled, y_train)
```

9. Save model

```
joblib.dump(grid_search.best_estimator_, 'best_model.pkl')
```

10. Load and use

```
loaded_model = joblib.load('best_model.pkl')
new_predictions = loaded_model.predict(new_data_scaled)
```

Quick Reference: Algorithm Selection

Classification:

- Small dataset + need interpretability → Logistic Regression, Decision Tree
- Large dataset + high accuracy → Random Forest, Gradient Boosting
- Text classification → Naive Bayes
- Non-linear boundaries → SVM with RBF kernel

Regression:

- Linear relationships → Linear Regression
- Prevent overfitting → Ridge/Lasso
- Non-linear relationships → Random Forest Regressor, Gradient Boosting

Clustering:

- Know number of clusters → K-Means
- Don't know number + arbitrary shapes → DBSCAN
- Hierarchical structure → Agglomerative Clustering

Dimensionality Reduction:

- Linear reduction → PCA
 - Non-linear visualization → t-SNE
-

Tips and Best Practices

1. **Always scale features** for distance-based algorithms (KNN, SVM, Linear models)
2. **Use stratified split** for imbalanced datasets
3. **Never fit scalers on test data** - only transform
4. **Use cross-validation** to get robust performance estimates
5. **Start simple** then increase complexity
6. **Use pipelines** to prevent data leakage
7. **Set random_state** for reproducibility
8. **Check for overfitting** using train vs validation performance
9. **Handle class imbalance** with stratification, SMOTE, or class weights
10. **Save preprocessing objects** along with models

